

Formalizing Dana Scott’s 1982 Information Systems in Lean 4

Lars Warren Ericson¹

¹ORCID: 0000-0001-8299-9361

¹Catskills Research Company

¹lars.ericson@catskillsresearch.com

September 4, 2026

Github: <https://github.com/catskillsresearch/scott1982>

Palomar Registration:

<https://palomar-registry.org/entry?id=PALOMAR-2026-08-24-000005&version=1>

Abstract

In 1982 Dana Scott published *Domains for Denotational Semantics* (ICALP, LNCS 140), presenting domains via **information systems**: finite consistency and entailment on data objects (tokens), with domain elements recovered as consistent, deductively closed sets. This is the third of Scott’s major presentations of domain theory—after continuous lattices [Sco72] and neighbourhood systems (PRG-19, [Sco81]) — and is the most explicitly constructive of the three. Companion Lean formalizations of those earlier presentations are [scott1972](#) [SR72] and [scott1980](#) [ER80].

This Lean 4 formalization targets the **entire paper** (Sections 1–8). We strive to avoid the law of excluded middle. Every completed module is audited with `#print axioms`; the target footprint is `#print axioms` $\subseteq$ `{propext, Quot.sound}`. Choice-tainted mathlib `Finset` operations are replaced by the prelude in `Scott1982/Constructive.lean`.

1 Introduction

Scott’s 1982 ICALP paper reorganizes domain theory around four simple ideas:

1. **Data objects** (tokens / propositions) with a distinguished least-informative object Δ .
2. **Consistency** (Con): which finite sets of tokens can hold of a single element.
3. **Entailment** ($\vdash$): which tokens are forced by a consistent set.
4. **Elements** as consistent, deductively closed sets of tokens — the domain $|A|$.

From this substrate he builds approximable mappings (a category), products, separated sums, function spaces (cartesian closed structure), least fixed points, and recursive domain equations (trees / S-expressions, λ -calculus models, universal domains).

1.1 Paper section dependency

Edges follow Scott’s narrative dependencies. Lean import graphs for §§2–3 and §§5–8 are stable (see per-section diagrams below); §4 Factoids 4.1–4.6 are mechanized (the dashed §4→§5 edge remains advisory: later sections do not import §4).

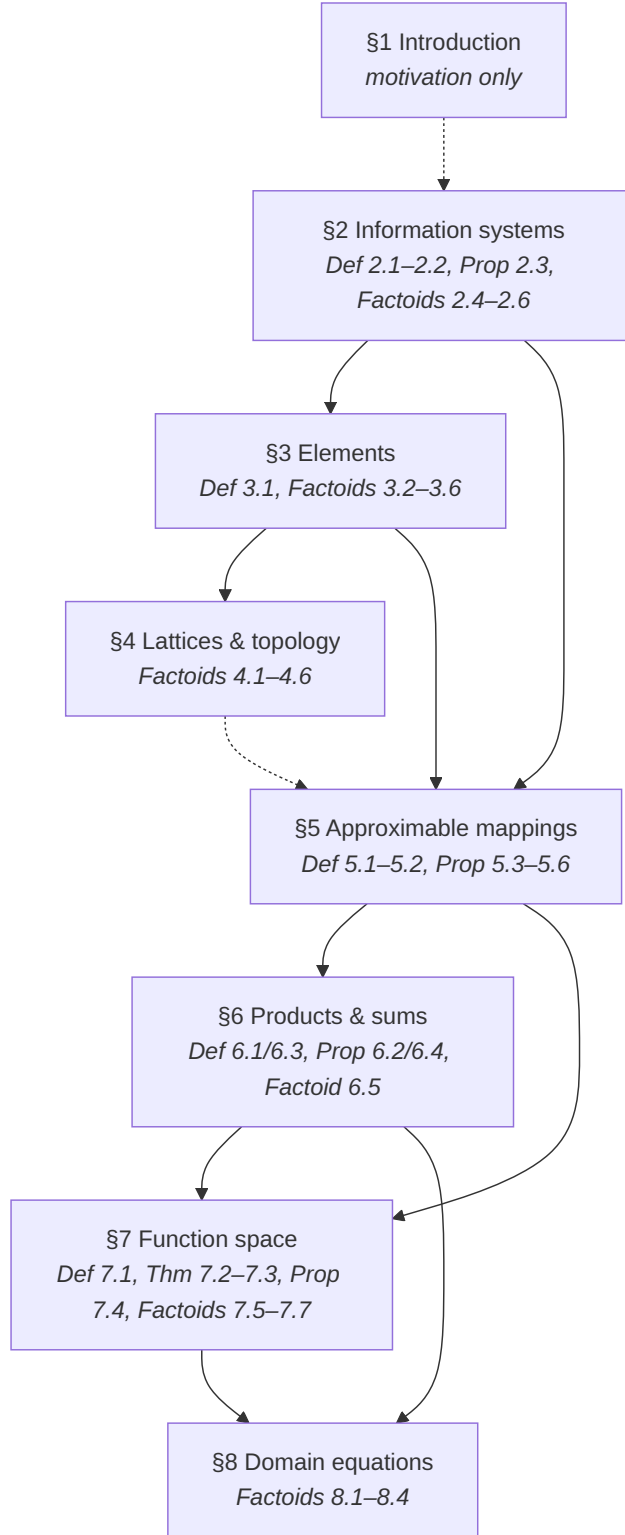


Figure 1: Paper section dependency.

1.2 Lean module map

Named concepts get section-style modules; unnamed numbered results get **DefinitionXY** / **PropositionXY** / **TheoremXY** files; invented claims use **Factoid** labels. Edges are Lean imports (coarse: **Factoid3*** stands for the §3 family). §8 factoids are **independent** constructions — they do not import each other; **DomainEquation.lean** only re-exports 8.1–8.2.

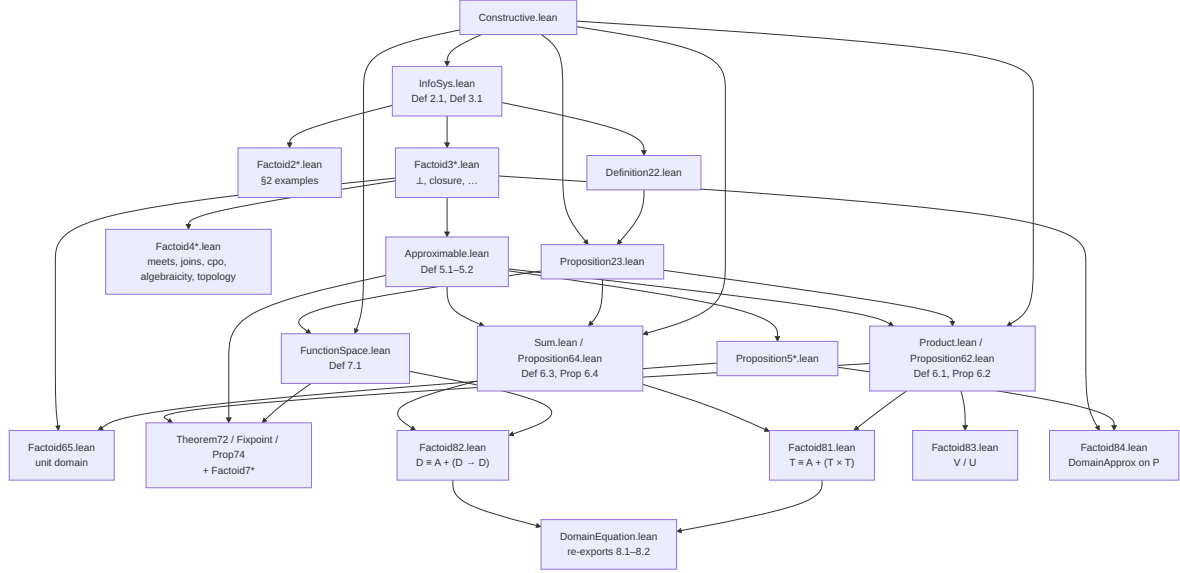


Figure 2: Lean module map.

2 Methodology

2.1 Source material

Primary source: Dana Scott, *Domains for Denotational Semantics*, ICALP 1982, LNCS 140.
Repository copy: [sources/Domains_for_Denotational_Semantics.pdf](#).

2.2 Numbering

- **Scott’s numbers** are used wherever present (Def/Prop/Thm share one counter per section).
- **Invented claims** (informal remarks we elevate to formal statements) use **Factoid** labels continuing the section counter, e.g. after Def 3.1 the next invented item is Factoid 3.2.
- Factoids are first-class inventory rows with Lean files and proof notes.

2.3 Constructivity

Target: `#print axioms ⊆ {propext, Quot.sound}`. Scott1982/**Constructive.lean** supplies choice-free **funion** ($\cup'$) and **insert_comm'**. Avoid **mathlib** ($\cdot \cup \cdot$), **Finset.image**, **tauto**, **aesop** unless audited.

2.4 Portable prior work

Where Scott 1972 [Sco72] / `scott1972` [SR72] and PRG-19 [Sco81] / `scott1980` [ER80] developed the *same* mathematics in a portable form, we **copy** the Lean into this repo (adapted to `InfoSys` / `Finset` as needed) rather than depending on sibling packages. Cross-presentation equivalence theorems remain in `scott_models`.

3 Chronological Formalization Narrative

3.1 §1 Introduction

Motivational prose only — no numbered mathematical claims. No Lean modules.

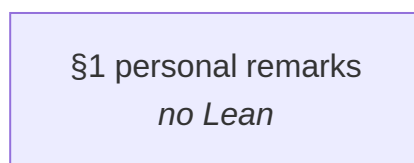


Figure 3: §1 Introduction.

3.2 §2 Information systems

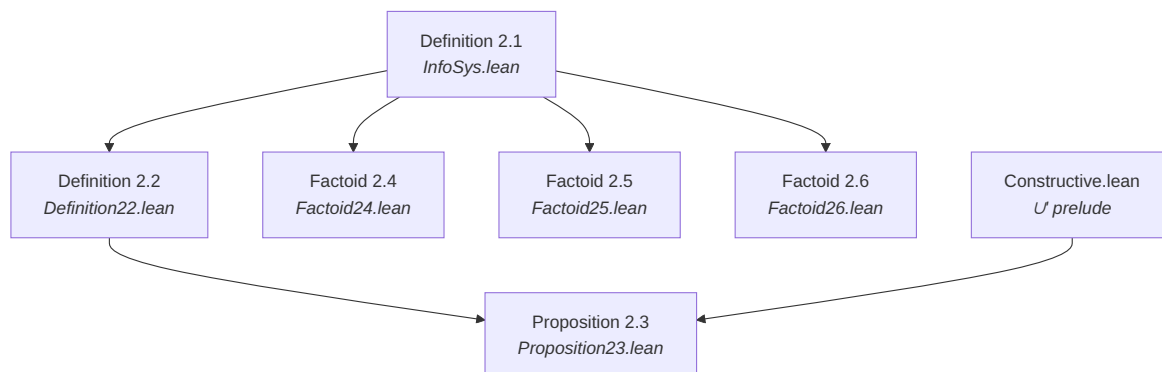


Figure 4: §2 Information systems.

Paper order: examples sit between Def 2.1 and Def 2.2; Lean matches that — each `Factoid2*.lean` imports only `InfoSys`. Prop 2.3 also needs `Constructive` for $\cup'$.

Definition 2.1

- **Mathematical Target:** Information system $(D, \Delta, \text{Con}, \vdash)$ with Scott's six axioms (i)–(vi).
- **Lean File:** `Scott1982/InfoSys.lean` (`structure InfoSys`)

- **Proof Notes:** structure fields `con_subset`, `con_sing`, `ent_con`, `ent_bot`, `ent_refl`, `ent_trans`. Uses `insert` (not `mathlib` `∪`) in `ent_con` for choice-freedom. Footprint target `{propext, Quot.sound}`.

Definition 2.2

- **Mathematical Target:** For $u, v \in \text{Con}$, write $u \vdash v$ to mean $u \vdash X$ for all $X \in v$.
- **Lean File:** `Scott1982/Definition22.lean`
- **Proof Notes:** `InfoSys.EntSet`.

Proposition 2.3

- **Mathematical Target:** For $u, v, w, u', v' \in \text{Con}$: (i) $\emptyset \vdash \{\Delta\}$; (ii) $u \vdash v \Rightarrow u \cup v \in \text{Con}$;
(iii) $u \vdash u$; (iv) transitivity; (v) monotonicity; (vi) $u \vdash v \wedge u \vdash v' \Rightarrow u \vdash v \cup v'$.
- **Lean File:** `Scott1982/Proposition23.lean`
- **Proof Notes:** uses `∪'` from `Constructive.lean` for (ii) and (vi).

Factoid 2.4

- **Mathematical Target:** First example: $D = \mathbb{N}$, $\Delta = 0$, all finite sets consistent, $\{n_i\} \vdash m$ iff $m = 0 \vee \exists i, m \leq n_i$.
- **Lean File:** `Scott1982/Factoid24.lean`
- **Proof Notes:** `example : InfoSys ℕ with lowerBoundEnt`; axioms (i)–(iii) trivial (`Con = univ`); (iv) 0-bot; (v) reflexivity via `le_refl`; (vi) cut by chaining $\leq$ through the witness in `u`. Imports only `InfoSys` (Def 2.1 apparatus). No `sorry`.

Factoid 2.5

- **Mathematical Target:** Second example: open intervals (n, m) with $n < m$, plus $(0, \infty)$.
- **Lean File:** `Scott1982/Factoid25.lean`
- **Proof Notes:** `Token = bot ⊕ strict intervals`; `satisfaction on ; ofSatisfaction` builds any Scott-style semantic `InfoSys` from `Sat + true bot + inhabited singletons`. `Ent` pairs consistency of the LHS with $\forall$ -entailment (so `ent_con` is not vacuous). Midpoint witness for singletons. No `sorry`.

Factoid 2.6

- **Mathematical Target:** Third example: partial functions $A \multimap B$ as graphs plus Δ .
- **Lean File:** `Scott1982/Factoid26.lean`
- **Proof Notes:** `Token A B = bot ⊕ (A × B)`; `Consistent = functional on pairs`; minimal `Ent` ($X = \Delta \vee X \in u$) with LHS consistency; `partialFunctionSystem A B` plus `example` on $\mathbb{N}/\text{Bool}$. No `sorry`.

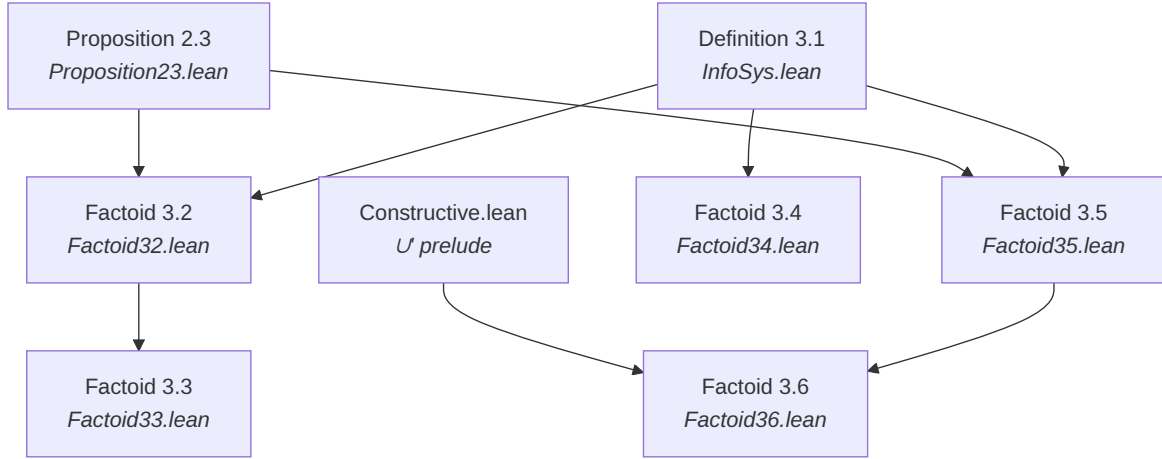


Figure 5: §3 The elements of a system.

3.3 §3 The elements of a system

Def 3.1 (Element) lives in `InfoSys.lean` with Def 2.1. Factoids 3.2/3.5 import `Proposition23` (for `EntSet /  $\cup$` lemmas); 3.3 builds on 3.2; 3.4 needs only `InfoSys`; 3.6 imports `Factoid35` and `Constructive` for directedness via `$\cup$` .

Definition 3.1

- **Mathematical Target:** Elements $|A|$: subsets $x \subseteq D$ with (i) every finite subset in `Con`,
(ii) closed under entailment. Total elements `Tot_A`.
- **Lean File:** `Scott1982/InfoSys.lean` (`InfoSys.Element`, `PartialOrder`)
- **Proof Notes:** `Element / PartialOrder`; `IsTotal / top` via Factoid 3.4.

Factoid 3.2

- **Mathematical Target:** Every element contains Δ .
- **Lean File:** `Scott1982/Factoid32.lean`

Factoid 3.3

- **Mathematical Target:** $\perp_A = \{X \mid \{\Delta\} \vdash X\}$ is the least element.
- **Lean File:** `Scott1982/Factoid33.lean`

Factoid 3.4

- **Mathematical Target:** Top $\top = D$ exists iff all finite subsets are consistent; then unique total.
- **Lean File:** `Scott1982/Factoid34.lean`
- **Proof Notes:** `IsTotal` (maximal); `AllConsistent`; `topElement` with carrier `Set.univ`; `exists_top_iff_allConsistent`; `uniqueness eq_topElement_of_isTotal / exists_unique_total_of_allConsistent`. No sorry.

Factoid 3.5

- **Mathematical Target:** $\text{Closure} = \{X \mid u \vdash X\}$ of $u \in \text{Con}$ is an element (finite element).
- **Lean File:** Scott1982/Factoid35.lean

Factoid 3.6

- **Mathematical Target:** Every element is the directed union of its finite approximations: $x = \bigcup \{u \subseteq x, u \in \text{Con}\}$.
 - **Lean File:** Scott1982/Factoid36.lean
 - **Proof Notes:** `approxUnion / element_eq_approxUnion` (via singleton witnesses and entailment closure); `closures_directed` using $\cup'$; helpers `closure_le_of_entSet`, `closure_le_of_subset`, `closure_le_element`. No sorry.
-

3.4 §4 Domains as lattices and as topological spaces

Scott keeps §4 informal (bridge to lattice/topology presentations). Elevated claims are Factoids; Lean edges below match imports. Later sections (§5–§8) do not depend on these modules.

Factoid 4.1

- **Mathematical Target:** $|A|$ is an inf-semilattice under $\cap$; $x \subseteq y \leftrightarrow x \cap y = x$.
- **Lean File:** Scott1982/Factoid41.lean
- **Proof Notes:** `inf` via carrier $\cap$; `inf_le_left/right`, `le_inf`, `le_iff_inf_eq`; idempotent/commutative/associative; `botElement_inf`. No sorry.

Factoid 4.2

- **Mathematical Target:** Nonempty families of elements have set-theoretic intersections that are elements.
- **Lean File:** Scott1982/Factoid42.lean
- **Proof Notes:** `familyInf / familyInf_le / le_familyInf`; agrees with binary `inf` via `familyInf_pair`. Axioms $\subseteq \{\text{proptext}, \text{Quot.sound}\}$. No sorry.

Factoid 4.3

- **Mathematical Target:** Join of a family exists in $|A|$ iff the union is consistent; then `join` = deductive closure of the union.
- **Lean File:** Scott1982/Factoid43.lean
- **Proof Notes:** `IsFinitelyConsistent / deductiveClosure / familySup / binary join`; `exists_isLUB_iff / exists_join_iff`. Axioms $\subseteq \{\text{proptext}, \text{Quot.sound}\}$. No sorry.

Factoid 4.4

- **Mathematical Target:** Directed (in particular chain) unions of elements are elements (cpo).

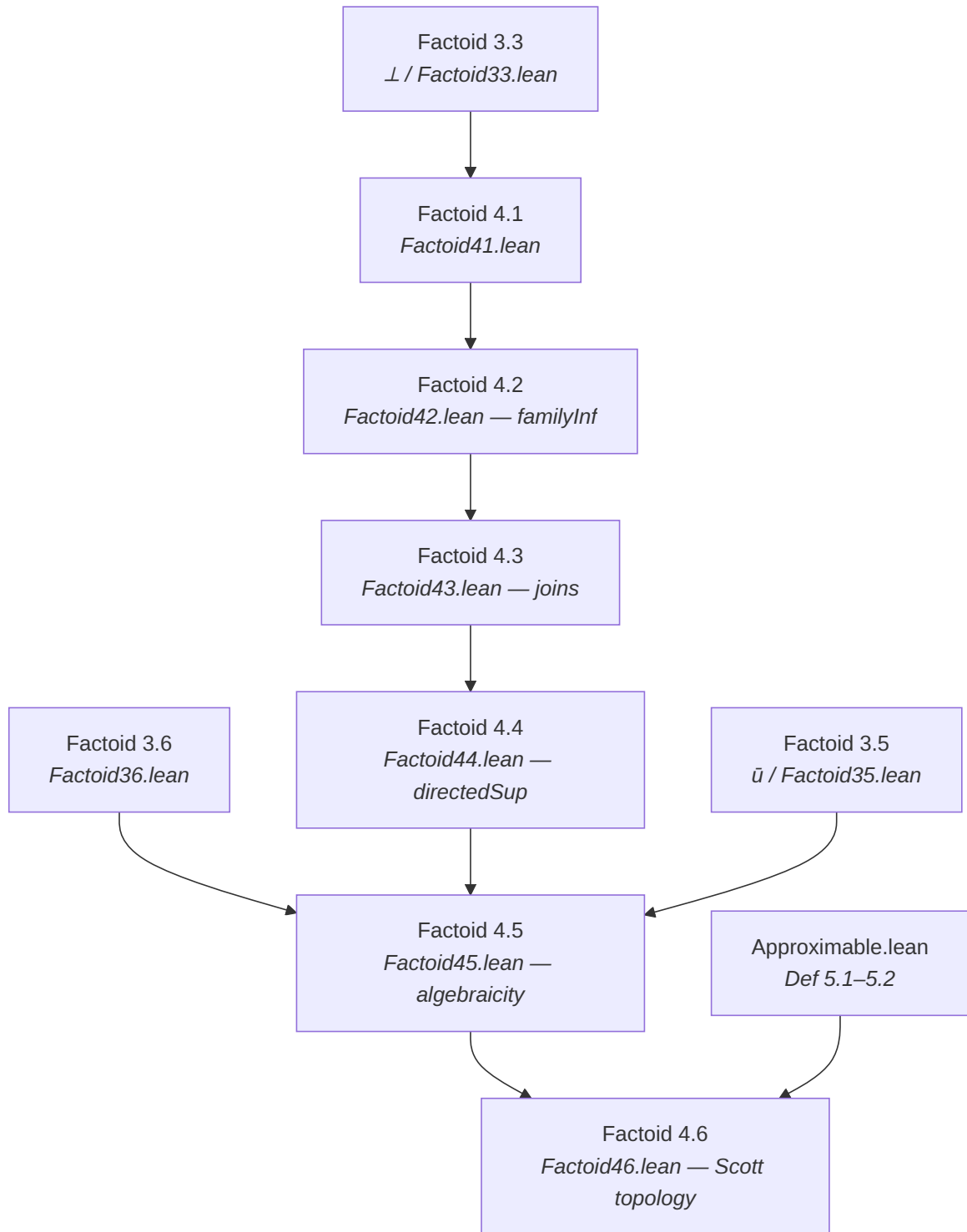


Figure 6: §4 Domains as lattices and as topological spaces.

- **Lean File:** Scott1982/Factoid44.lean
- **Proof Notes:** IsDirected / IsChain; directedSup with carrier = raw union; chainSup. Axioms $\subseteq \{\text{propext}, \text{Quot.sound}\}$. No sorry.

Factoid 4.5

- **Mathematical Target:** Finite elements are compact; every element is directed lub of finite elements below it (algebraicity).
- **Lean File:** Scott1982/Factoid45.lean
- **Proof Notes:** finiteApproximants directed; eq_directedSup_finiteApproximants; compact_closure. Axioms $\subseteq \{\text{propext}, \text{Quot.sound}\}$. No sorry.

Factoid 4.6

- **Mathematical Target:** Scott topology via basic opens $\{x \upharpoonright u \subseteq x\}$; approximable maps = Scott-continuous maps.
- **Lean File:** Scott1982/Factoid46.lean
- **Proof Notes:** basicOpen / basicOpen_inter / T₀; ScottContinuous; toScottContinuous / ofScottContinuous / toElement_ofScottContinuous. Axioms $\subseteq \{\text{propext}, \text{Quot.sound}\}$. No sorry.

3.5 §5 Approximable mappings between domains

Approximable.lean imports Constructive and Factoid35. Def 5.1–5.2 and 5.3(i)/(iv) live there; singleton reduction and 5.3(v)/(iii)/(ii) are in Proposition53.lean. Prop 5.5 imports Prop 5.4; Prop 5.6 imports Prop 5.3 (extensionality) and Prop 5.5 (composition).

Definition 5.1

- **Mathematical Target:** Approximable mapping $f : A \rightarrow B$ as relation on $\text{Con}_A \times \text{Con}_B$ with

$$(i) \emptyset f \emptyset; (ii) u f v \wedge u f v' \Rightarrow u f (v \cup v'); (iii) u' \vdash u, u f v, v \vdash v' \Rightarrow u' f v'.$$

- **Lean File:** Scott1982/Approximable.lean
- **Proof Notes:** Structure. Adapted from PRG-19 ApproximableMap pattern, rewritten for Finset/Con.

Definition 5.2

- **Mathematical Target:** $f(x) = \{Y \mid \exists u \subseteq x, u f \{Y\}\}$.
- **Lean File:** Scott1982/Approximable.lean
- **Proof Notes:** toElement; exists_rel_of_subset_image.

Proposition 5.3(i)

- **Mathematical Target:** f maps elements to elements under Def 5.2 ($f(x) \in |B|$).

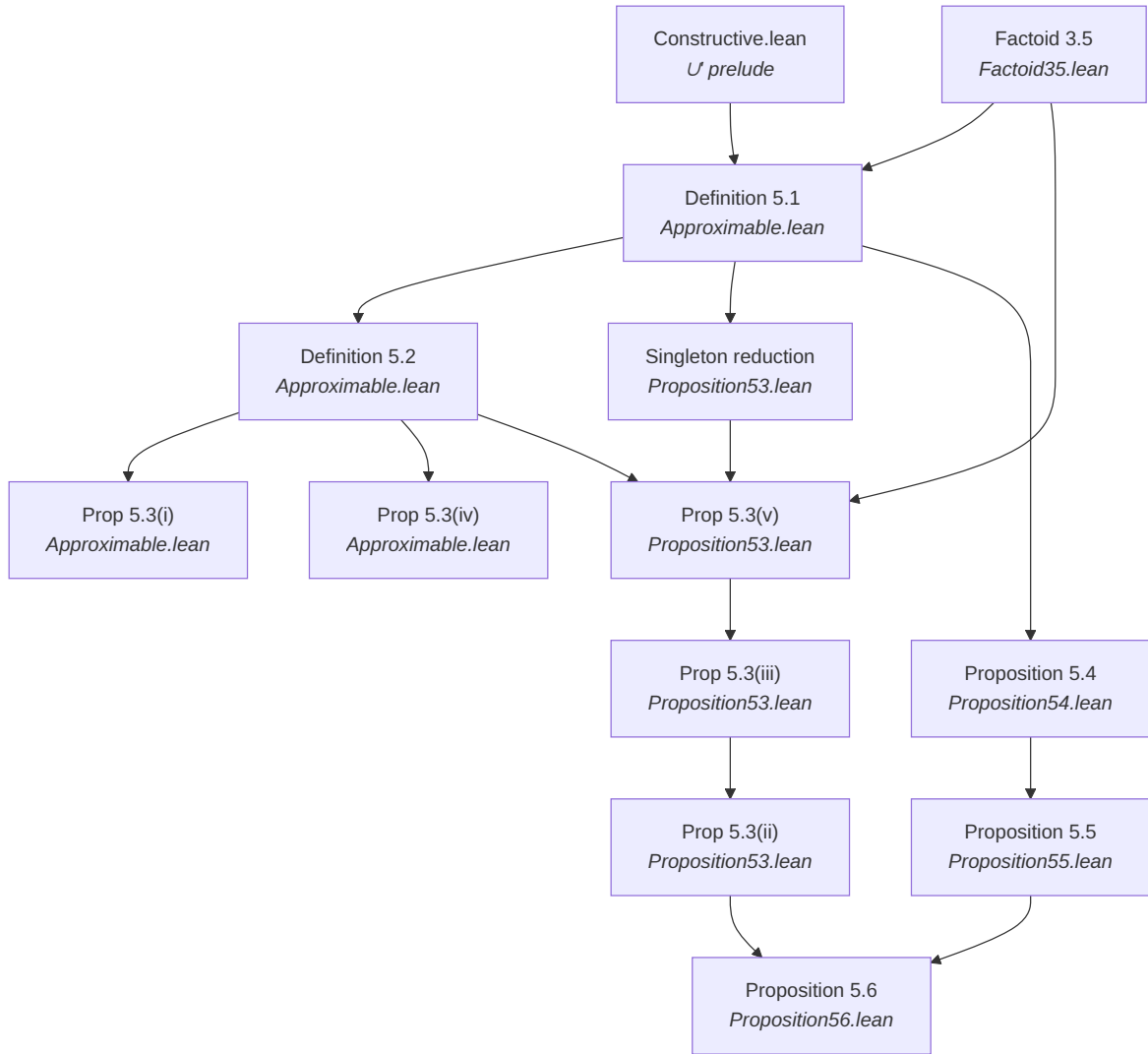


Figure 7: §5 Approximable mappings between domains.

- **Lean File:** Scott1982/Approximable.lean
- **Proof Notes:** toElement (consistency + closure).

Proposition 5.3(ii)

- **Mathematical Target:** Extensionality: $f = g$ iff $f(x) = g(x)$ for all $x \in |A|$.
- **Lean File:** Scott1982/Proposition53.lean
- **Proof Notes:** ext_iff_toElement via 5.3(iii) both ways + ApproximableMap.ext. No sorry.

Proposition 5.3(iii)

- **Mathematical Target:** Pointwise order: $f \subseteq g$ (as relations) iff $f(x) \subseteq g(x)$ for all $x \in |A|$.
- **Lean File:** Scott1982/Proposition53.lean
- **Proof Notes:** Le / le_iff_toElement_le; $\leftarrow$ uses 5.3(v). No sorry.

Proposition 5.3(iv)

- **Mathematical Target:** Monotonicity: $x \subseteq y$ in $|A|$ implies $f(x) \subseteq f(y)$ in $|B|$.
- **Lean File:** Scott1982/Approximable.lean
- **Proof Notes:** toElement_mono.

Singleton reduction

- **Mathematical Target:** Scott's remark before Def 5.2: $u f v \leftrightarrow \forall Y \in v, u f \{Y\}$ (for $u \in \text{Con}_A, v \in \text{Con}_B$).
- **Lean File:** Scott1982/Proposition53.lean
- **Proof Notes:** rel_iff_forall_singleton (mono + union_right / $\cup'$). Unnumbered inventory row. No sorry.

Proposition 5.3(v)

- **Mathematical Target:** Bridge lemma: $u f v \leftrightarrow v \subseteq f()$ for $u \in \text{Con}_A, v \in \text{Con}_B$.
- **Lean File:** Scott1982/Proposition53.lean
- **Proof Notes:** rel_iff_closure_le; $\rightarrow$ via subset_closure + mono; $\leftarrow$ via exists_rel_of_subset_image on then EntSet u u' + mono. No sorry.

Proposition 5.4

- **Mathematical Target:** Identity I_A given by $u I v \leftrightarrow u \vdash v; I(x) = x$.
- **Lean File:** Scott1982/Proposition54.lean

Proposition 5.5

- **Mathematical Target:** Composition $g \circ f$; $(g \circ f)(x) = g(f(x))$.
- **Lean File:** Scott1982/Proposition55.lean

Proposition 5.6

- **Mathematical Target:** Unique constant map $\text{const } b$ with $(\text{const } b)(x) = b$; also $f \circ (\text{const } b) = \text{const } (f(b))$ and $(\text{const } b) \circ g = \text{const } b$.
- **Lean File:** Scott1982/Proposition56.lean
- **Proof Notes:** constMap via $u (\text{const } b) v \leftrightarrow \uparrow v \subseteq b$; $\text{constMap_toElement}$; uniqueness via ext_iff_toElement ; composition laws via comp_toElement . No sorry.

3.6 §6 Products and sums of domains

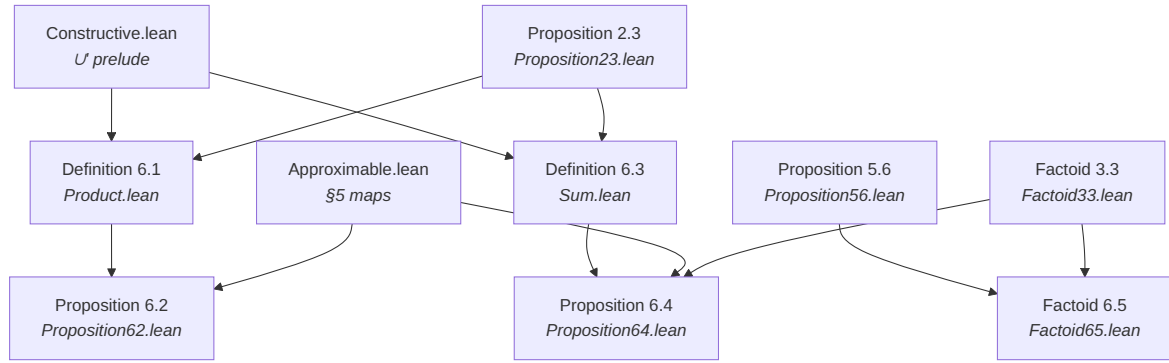


Figure 8: §6 Products and sums of domains.

`Product.lean` (Def 6.1 / `productSystem`) imports `Constructive` and `Proposition23`. `Proposition62.lean` adds approximable `fst/snd/⟨f,g⟩` on top of `Product` / Prop 5.3–5.5. `Sum.lean` (Def 6.3 / `sumSystem`) is parallel to `Product.lean`; `Proposition64.lean` adds `inl/inr/[f,g]` (also `Factoid 3.3` for $\perp$ on copairing). `Factoid65.lean` (unit domain) imports Prop 5.6 (`constMap`) and `Factoid 3.3` ($\perp$); Scott places it after the product remarks, before the sum construction.

Definition 6.1

- **Mathematical Target:** Product information system $A \times B$ on tagged tokens.
- **Lean File:** Scott1982/Product.lean
- **Proof Notes:** `ProdToken` / `IsProdToken`; `prodBot`; `fstFinset` / `sndFinset`; `ProdCon` / `ProdEnt`; `productSystem` : `InfoSys` _ with all six Def 2.1 axioms. No sorry.

Proposition 6.2

- **Mathematical Target:** Approximable `fst`, `snd`, pairing $\langle f, g \rangle$ with universal property ($A \times B$ as an `InfoSys` is already Def 6.1 / `productSystem`).
- **Lean File:** Scott1982/Proposition62.lean
- **Proof Notes:** `fstMap` / `sndMap` / `pairMap`; `comp_fstMap_pairMap` / `comp_sndMap_pairMap`; uniqueness via `element_eq_of_fst_snd` + Prop 5.3 extensionality. Axioms $\subseteq \{\text{propext}, \text{Quot.sound}\}$. No sorry.

Definition 6.3

- **Mathematical Target:** Separated sum $A + B$.
- **Lean File:** `Scott1982/Sum.lean`
- **Proof Notes:** `SumToken (left / right / bot); sumBot; lftFinset / rhtFinset; SumCon / SumEnt; sumSystem : InfoSys _` with all six Def 2.1 axioms. Axioms $\subseteq \{\text{propext}, \text{Quot.sound}\}$. No sorry.

Proposition 6.4

- **Mathematical Target:** Sum is an information system; injections and copairing.
- **Lean File:** `Scott1982/Proposition64.lean`
- **Proof Notes:** `inlMap / inrMap / copairMap; comp_copairMap_inlMap / comp_copairMap_inrMap; copairMap_botElement; uniqueness via relation extensionality (choice-free lft/rht emptiness dichotomy)`. Axioms $\subseteq \{\text{propext}, \text{Quot.sound}\}$. No sorry.

Factoid 6.5

- **Mathematical Target:** Unit domain 1 with unique element $\perp$; terminal/initial mapping facts Scott records at end of §6.
- **Lean File:** `Scott1982/Factoid65.lean`
- **Proof Notes:** `unitSystem on PUnit; unitElement_eq_bot; approxMap_from_unit_eq_const / approxMap_to_unit_eq_const; toElement_constMap_bot`. Axioms $\subseteq \{\text{propext}, \text{Quot.sound}\}$. No sorry.

3.7 §7 The function space as a domain

`FunctionSpace.lean` (Def 7.1) imports `Constructive` and `Proposition23` only. Theorem 7.2 adds Prop 5.3/5.5, Prop 6.2 (`apply/curry` pairing), and Factoid 3.2. Theorem 7.3 (`fix`) sits on Theorem 7.2 plus Factoid 3.3 ($\perp$); Prop 7.4 adds identity/composition and closure. Factoid 7.5 reuses Factoid 2.5 (`BOOL`) for the conditional iso. Factoid 7.6 curries Prop 5.6/`pair` composites. Factoid 7.7 packages the CCC as `Mathlib Category / CartesianMonoidalCategory / MonoidalClosed` (Scott data constructive; `Mathlib` packaging uses `Classical.choice`).

Definition 7.1

- **Mathematical Target:** Function-space information system $A \rightarrow B$ with Scott's `Con/|` on pairs of consistent sets.
- **Lean File:** `Scott1982/FunctionSpace.lean`
- **Proof Notes:** `FunToken / funBot; funInputUnion / funOutputUnion; FunCon / FunEnt` (witness form of 7.1(iv)); `functionSystem : InfoSys _` with all six Def 2.1 axioms. Choice-free `decidableEq_finset` for token equality. Axioms $\subseteq \{\text{propext}, \text{Quot.sound}\}$. No sorry.

Theorem 7.2

- **Mathematical Target:** Elements of $|A \rightarrow B|$ = approximable maps; `apply` and `curry`.

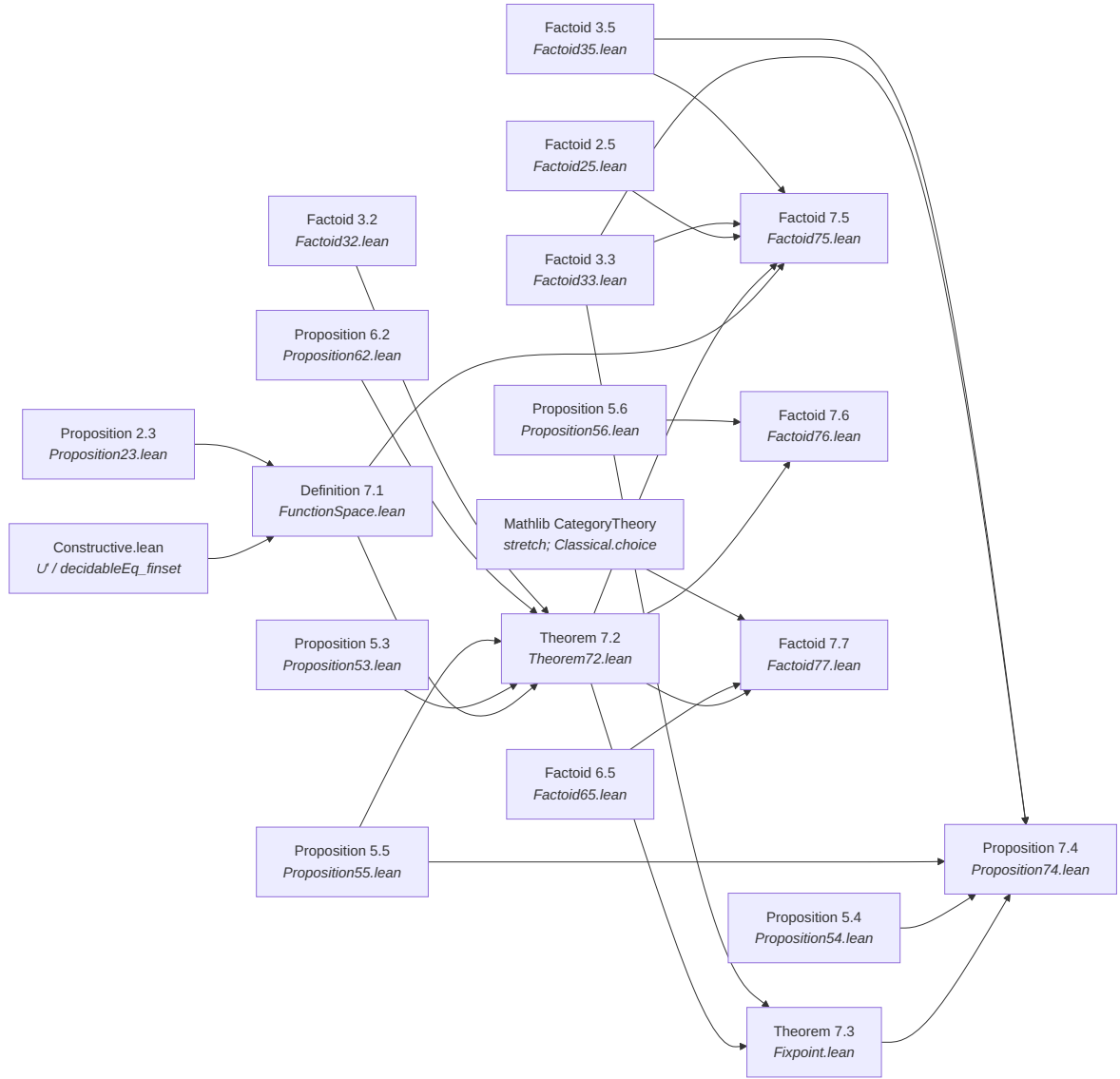


Figure 9: §7 The function space as a domain.

- **Lean File:** `Scott1982/Theorem72.lean`
- **Proof Notes:** bijection `approxMap_toElement` / `element_toApproxMap`; approximable `applyMap` and `curryMap`; `apply(g,y) = g(y)`; `uncurry ∘ curry = id` with uniqueness of `curry`. Axioms $\subseteq \{\text{propext}, \text{Quot.sound}\}$. No sorry.

Theorem 7.3

- **Mathematical Target:** Least fixed-point operator `fix` : $(A \rightarrow A) \rightarrow A$.
- **Lean File:** `Scott1982/Fixpoint.lean`
- **Proof Notes:** `FixChain` / `FixReach`; approximable `fixMap`; `fixElement` with `f(fix f) = fix f`, least among pre-fixed points, and operator uniqueness. Axioms $\subseteq \{\text{propext}, \text{Quot.sound}\}$. No sorry.

Proposition 7.4

- **Mathematical Target:** Plotkin-style equational characterization of `fix`.
- **Lean File:** `Scott1982/Proposition74.lean`
- **Proof Notes:** (ii) `fix(f) = f(fix(f))`; (iii) naturality for strict commuting maps; uniqueness via the Theorem 7.3 least-fixed-point property. Axioms $\subseteq \{\text{propext}, \text{Quot.sound}\}$. No sorry.

Factoid 7.5

- **Mathematical Target:** Strict function space $A \rightarrow B$ and `strict` operator; $A \times A \cong (B\text{OOL} \rightarrow A)$.
- **Lean File:** `Scott1982/Factoid75.lean`
- **Proof Notes:** `IsStrict`; `strictify` with `IsStrict_strictify`; `strictFunctionSystem` (`StrictFunToken` + empty-input `Con` clause); retract via `approxMap_toStrictElement` / `strictifyElement`; flat `boolSystem`; conditional `condMap` with `condMap_unique` / `conditional_iso` ($A \times A \cong (B\text{OOL} \rightarrow A)$ at the strict-map level, packaged into `|B\text{OOL} \rightarrow A|` by `condElement`). Axioms $\subseteq \{\text{propext}, \text{Quot.sound}\}$. No sorry.

Factoid 7.6

- **Mathematical Target:** Combinators `const`, `pair`, `comp` as approximable operators.
- **Lean File:** `Scott1982/Factoid76.lean`
- **Proof Notes:** `constOp` = `curry fst` with `constOp_toElement`; `pairOp` / `compOp` by currying `apply-fst/snd` composites; `pairOp_toElement` / `compOp_toElement` recover `\langle f,g \rangle` and `g ∘ f`. Axioms $\subseteq \{\text{propext}, \text{Quot.sound}\}$. No sorry.

Factoid 7.7

- **Mathematical Target:** Scott's CCC remark (*Categories again*): Props 6.2 and Theorem 7.2 (with unit 1 from Factoid 6.5) show the category of information systems and approximable maps is cartesian closed.
- **Lean File:** `Scott1982/Factoid77.lean`

- **Proof Notes:** bundled `InfoSysObj`; Mathlib `Category`, `CartesianMonoidalCategory` (terminal 1, binary products from Prop 6.2), and `MonoidalClosed` via `tensorExpEquiv / uncurryRight_comp_left` (Scott curry after product symmetry, left-natural in the domain). Bare `Category` axioms $\subseteq \{\text{propxt}, \text{Quot.sound}\}$; `CartesianMonoidalCategory / MonoidalClosed` pull `Classical.choice` through Mathlib’s chosen-products / adjunction constructors — an artifact of Lean’s category library, isolated to this stretch factoid, not of Scott’s argument. No sorry.

3.8 §8 Some domain equations

Scott presents Factoids 8.1–8.4 in narrative order, but the Lean modules are **independent** constructions (no `Factoid8i` $\rightarrow$ `Factoid8(i+1)` imports). Edges below are actual Lean imports. `DomainEquation.lean` only re-exports 8.1–8.2.

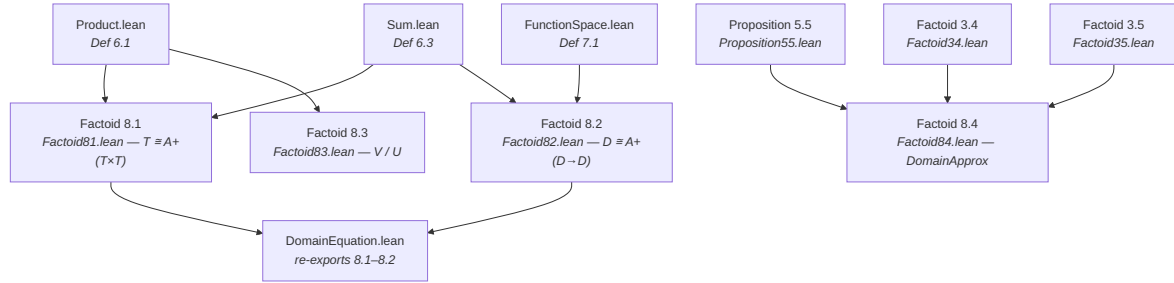


Figure 10: §8 Some domain equations.

Factoid 8.1

- **Mathematical Target:** Inductive construction of tree / S-expression domain $T \cong A + (T \times T)$.
- **Lean File:** `Scott1982/Factoid81.lean`
- **Proof Notes:** inductive `TreeToken` (Scott (1)–(3)); projection-based `TreeCon / TreeEnt` matching `sum×product` clauses (4)–(12); `treeSystem` : `InfoSys` _ with all six Def 2.1 axioms; unfolding `treeUnfold / treeRhs` into `sumSystem A` (`productSystem (treeSystem A)` (`treeSystem A`)). Axioms $\subseteq \{\text{propxt}, \text{Quot.sound}\}$. No sorry.

Factoid 8.2

- **Mathematical Target:** λ -calculus model $D \cong A + (D \rightarrow D)$ via mutual recursion of `D` and `Con`.
- **Lean File:** `Scott1982/Factoid82.lean`
- **Proof Notes:** choice-free well-founded `LamConDepth` / inductive `LamEntDepth` (Scott (8)–(11)); Def 2.1 `ent_refl / ent_con / ent_trans` (bot/atom + `funTok` via combined witness + depth IH); `lambdaSystem` : `InfoSys` (`LamDToken A`) on depth-WF tokens; unfolding `lamUnfold / lamRhs` into `sumSystem A` (`functionSystem (lambdaSystem A)` (`lambdaSystem A`)). Staged `LamConN` retained as scaffolding. Axioms $\subseteq \{\text{propxt}, \text{Quot.sound}\}$. No sorry.

Factoid 8.3

- **Mathematical Target:** Universal domain remarks (V , retract U).
- **Lean File:** `Scott1982/Factoid83.lean`
- **Proof Notes:** `VToken` (Scott (1)–(2): $\Delta // \text{pairL/pairR}$); all finite sets consistent; inductive `VEnt` (3)–(6); `vSystem` : `InfoSys VToken`; `vUnfold` / `vRhs` into `productSystem` `vSystem` `vSystem` ($V \cong V \times V$); `UToken` = $D_V \setminus \{\}$ with `UCon` = $\not\vdash_V$ and `UEnt` from `VEnt` (7)–(10); `uSystem` : `InfoSys UToken`. Scott’s retract/universality theorem (every countably based domain is a retract of U) is stated without proof in the source and left external. Axioms $\subseteq \{\text{propext}, \text{Quot.sound}\}$. No sorry.

Factoid 8.4

- **Mathematical Target:** Domain of domains via approximable maps on P satisfying (1)–(5).
- **Lean File:** `Scott1982/Factoid84.lean`
- **Proof Notes:** (Scott’s sketch, as far as stated) — powerset `InfoSys P` on $\mathbb{N}$ ($0 = \Delta$, all finite sets `Con`, minimal `Ent`); Scott’s (1)–(5) as `ReflApprox` / `IdemApprox` / `BotApprox` / `RogueApprox` / `ConsistentApprox` on `ApproximableMap P P`; package `DomainApprox`; consistency remark `scottCon`; inhabitant `flatDomainApprox` (1 as rogue); `I_P` fails (4). Token-level `InfoSys` for the domain of domains (U -from- V style on $P \rightarrow P$) and functors/powerdomains are deferred with Scott. Axioms $\subseteq \{\text{propext}, \text{Quot.sound}\}$. No sorry.

4 Build

```
lake exe cache get
lake build Scott1982
```

Pinned: Lean / mathlib **v4.33.0**.

Acknowledgments (Dana Scott [Sco82], AI tool cards, artifact URL) are injected before References when building `arxiv.tex` via `scripts/ai_model_cards.py` — they are not kept in this file.

Rebuild this document (PDF with complete Lean source appendix, one subsection per module):

```
bash scripts/build_arxiv_pdf.sh # expand Lean -> tex -> arxiv.pdf
bash scripts/build_arxiv_pdf.sh --pdf-only # PDF only when arxiv.tex already current
```

5 Acknowledgments

- **Dana Scott** — *Domains for Denotational Semantics* [Sco82], the paper this development formalizes.

5.1 AI-assisted development

The human author retains sole responsibility for the mathematical content, the choice of formalization route, and every formal claim in this work. Following standard publisher practice (e.g., COPE guidance on authorship and AI tools [COPE24]), **no large language model is listed as a co-author** — authorship implies an accountability that automated systems cannot bear.

Because this development may borrow Lean and proof patterns from the sibling formalizations `scott1972` [SR72] and `scott1980` [ER80], **every model in the registry is treated as used** for acknowledgement purposes. We gratefully acknowledge assistance from the following tools (auto-generated from `scripts/ai_model_cards.py` when building `arxiv.tex`):

- **Cursor** [Cur26] — agent-assisted editing in the Cursor IDE: formalizing Scott’s 1982 information-system layer in Lean 4 / `mathlib`, `lake build` repair, working from the Scott 1982 source PDF, drafting this narrative (`arxiv.md`), and tracking the formalized inventory. Generated Lean was provisional until it compiled under the pinned toolchain. Also used in the sibling formalizations whose portable work may be copied into this repo.
- **Cursor Composer 2.5 Fast** [Cmp25] — routine multi-step work: module scaffolding, dependency-ordered wiring of `Scott1982/`, documentation and Mermaid blueprints, medium proof obligations where the strategy was already fixed, and the triple-pass vision OCR pipeline (`composer-2.5`). Also used in sibling repos whose portable Lean may be adapted here.
- **Cursor Grok 4.5** [Grk45] — primary agent model for the `scott1982` information-systems formalization sessions: inventory design in `arxiv.md`, constructive `InfoSys / Approximable` Lean development, choice-free `Finset` discipline, and adaptation of portable patterns from prior Scott formalizations. Jointly trained by SpaceXAI and Cursor; used here via the Cursor agent environment.
- **Anthropic Claude Sonnet 5 (medium reasoning)** [Son26] — day-to-day formalization and proof-engineering in Cursor at the medium reasoning tier in sibling Scott formalizations (and available for this repo): inventory and narrative maintenance, module wiring, and medium-complexity Lean obligations where the proof strategy was already fixed. Listed because portable prior work may be copied into `scott1982`.
- **Anthropic Claude Opus 4.8 (high reasoning)** [Ant26] — selective use for the heaviest proof work in sibling formalizations (e.g. continuous lattices and PRG-19). Every emitted proof term was checked by the Lean kernel. Listed because portable prior work may be copied into `scott1982`.
- **Google Gemini 3.5 Flash** [Gem25] — exploratory passes in sibling formalizations (typographic conventions, scope decisions). Listed because portable prior work may be copied into `scott1982`.

All definitions, constructivity audits, and final prose were reviewed by the human author, who takes full responsibility for them.

5.2 Artifact availability

The development is at github.com/catskillsresearch/scott1982. Run `lake build Scott1982` for the formalization; `bash scripts/build_arxiv_tex.sh` builds `arxiv.tex` from `arxiv.md` (Lean Code appendix = GitHub hyperlinks).

List of Figures

1	Paper section dependency.	2
2	Lean module map.	3
3	§1 Introduction.	4
4	§2 Information systems.	4
5	§3 The elements of a system.	6
6	§4 Domains as lattices and as topological spaces.	8
7	§5 Approximable mappings between domains.	10
8	§6 Products and sums of domains.	12
9	§7 The function space as a domain.	14
10	§8 Some domain equations.	16

6 References

- [Sco82] D. Scott. *Domains for Denotational Semantics*. ICALP 1982, LNCS 140, pp. 577–613.
- [Sco81] D. Scott. *Lectures on a Mathematical Theory of Computation*. PRG-19, Oxford, 1981.
- [Sco72] D. Scott. *Continuous Lattices*. LNM 274, 1972.
- [SR72] Catskills Research. *scott1972*. <https://github.com/catskillsresearch/scott1972>.
- [ER80] Catskills Research. *scott1980*. <https://github.com/catskillsresearch/scott1980>.
- [Win93] G. Winskel. *The Formal Semantics of Programming Languages*. MIT Press, 1993.
- [COPE24] Committee on Publication Ethics (COPE). *Authorship and AI tools: COPE position statement*. 2024. <https://publicationethics.org/guidance/cope-position/authorship-and-ai-tools>
- [Cur26] Anyosphere, Inc. *Cursor: AI-native code editor and agent environment*. <https://cursor.com> (accessed 2026).
- [Cmp25] Anyosphere, Inc. *Composer 2.5*. Model announcement and documentation, <https://cursor.com/blog/composer-2-5>; model card as integrated in Cursor, <https://cursor.com/docs/models> (accessed 2026).
- [Grk45] SpaceXAI and Anyosphere, Inc. *Grok 4.5*. Model documentation, <https://docs.x.ai/developers/models/grok-4.5>; Cursor announcement, <https://cursor.com/blog/grok-4-5> (accessed 2026).
- [Son26] Anthropic. *Claude Sonnet 5* (medium reasoning variant). System card, <https://www.anthropic.com/claude-sonnet-5-system-card>; model documentation as integrated in Cursor, <https://cursor.com/docs/models> (accessed 2026).
- [Ant26] Anthropic. *Claude Opus 4.8* (high thinking/reasoning variant). System card and announcement, <https://www.anthropic.com/news/claude-opus-4-8>; model documentation as integrated in Cursor, <https://cursor.com/docs/models/claude-opus-4-8> (accessed 2026).
- [Gem25] Google DeepMind. *Gemini 3.5 Flash*. Technical documentation and model cards. <https://ai.google.dev/gemini-api/docs/models> (accessed 2026).

A Complete Lean source

Role	File	Lines
Root import graph	Scott1982.lean	47
Core infrastructure	Scott1982/Constructive.lean	124
Core infrastructure	Scott1982/InfoSys.lean	129
Definition	Scott1982/Definition22.lean	45
Proposition	Scott1982/Proposition23.lean	102
Factoid	Scott1982/Factoid24.lean	64
Factoid	Scott1982/Factoid25.lean	126
Factoid	Scott1982/Factoid26.lean	123
Factoid	Scott1982/Factoid32.lean	33
Factoid	Scott1982/Factoid33.lean	55
Factoid	Scott1982/Factoid34.lean	77
Factoid	Scott1982/Factoid35.lean	49
Factoid	Scott1982/Factoid36.lean	90
Factoid	Scott1982/Factoid41.lean	93
Factoid	Scott1982/Factoid42.lean	71
Factoid	Scott1982/Factoid43.lean	184
Factoid	Scott1982/Factoid44.lean	104
Factoid	Scott1982/Factoid45.lean	96
Factoid	Scott1982/Factoid46.lean	207
Core infrastructure	Scott1982/Approximable.lean	144
Proposition	Scott1982/Proposition53.lean	123
Proposition	Scott1982/Proposition54.lean	63
Proposition	Scott1982/Proposition55.lean	82
Proposition	Scott1982/Proposition56.lean	89
Core infrastructure	Scott1982/Product.lean	367
Proposition	Scott1982/Proposition62.lean	434
Core infrastructure	Scott1982/Sum.lean	516
Proposition	Scott1982/Proposition64.lean	837
Factoid	Scott1982/Factoid65.lean	82
Core infrastructure	Scott1982/FunctionSpace.lean	549
Theorem / Lemma	Scott1982/Theorem72.lean	1020
Core infrastructure	Scott1982/Fixpoint.lean	348
Proposition	Scott1982/Proposition74.lean	171
Factoid	Scott1982/Factoid75.lean	800
Factoid	Scott1982/Factoid76.lean	361
Factoid	Scott1982/Factoid77.lean	373
Factoid	Scott1982/Factoid81.lean	672
Factoid	Scott1982/Factoid82.lean	2406
Factoid	Scott1982/Factoid83.lean	612
Factoid	Scott1982/Factoid84.lean	317
Core infrastructure	Scott1982/DomainEquation.lean	33

Total: 41 files, 12218 lines of Lean.

Files appear in `Scott1982.lean` import order. Each block is a verbatim copy of the repository file at generation time.

A.1 Scott1982.lean

47 lines.

Lean 4 source

```
/-
Copyright (c) 2026  Lars Warren Ericson.  All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github:  https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Constructive
import Scott1982.InfoSys
import Scott1982.Definition22
import Scott1982.Proposition23
import Scott1982.Factoid24
import Scott1982.Factoid25
import Scott1982.Factoid26
import Scott1982.Factoid32
import Scott1982.Factoid33
import Scott1982.Factoid34
import Scott1982.Factoid35
import Scott1982.Factoid36
import Scott1982.Factoid41
import Scott1982.Factoid42
import Scott1982.Factoid43
import Scott1982.Factoid44
import Scott1982.Factoid45
import Scott1982.Factoid46
import Scott1982.Approximable
import Scott1982.Proposition53
import Scott1982.Proposition54
import Scott1982.Proposition55
import Scott1982.Proposition56
import Scott1982.Product
import Scott1982.Proposition62
import Scott1982.Sum
import Scott1982.Proposition64
import Scott1982.Factoid65
import Scott1982.FunctionSpace
import Scott1982.Theorem72
import Scott1982.Fixpoint
import Scott1982.Proposition74
import Scott1982.Factoid75
import Scott1982.Factoid76
import Scott1982.Factoid77
import Scott1982.Factoid81
import Scott1982.Factoid82
import Scott1982.Factoid83
import Scott1982.Factoid84
import Scott1982.DomainEquation
```

A.2 Scott1982/Constructive.lean

124 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Mathlib.Data.Finset.Basic

/-!
# A choice-free `Finset` prelude

One of the project's goals (Goal 3) is to certify that the information-system
presentation of Scott domains can be developed in a purely constructive fragment of
Lean: every result must have a #print axioms footprint contained in
[propext, Quot.sound], with no Classical.choice and no use of the law of excluded
middle.

This is harder than it looks, because several of mathlib's Finset operations and even
a few basic lemmas transitively depend on Classical.choice (through the
Multiset.dedup / quotient machinery), in version v4.30.0:

* tainted operations: (* U *), Finset.image, (* x ? *), Finset.biUnion, (* \ *),
  and mathlib's Finset.decidableEq (via Multiset quotients);
* tainted lemmas: e.g. Finset.insert_comm, Finset.singleton_subset_iff;
* tainted tactics: tauto, aesop (they close goals via classical reasoning).

By contrast the following are choice-free and form our working toolkit: insert,
(* I *), Finset.filter, Finset.fold, Multiset.foldr, the membership/subset lemmas
(Finset.mem_insert, Finset.mem_singleton, Finset.subset_iff, Finset.mem_coe,
Finset.coe_subset, Finset.mem_inter, Finset.ext), set-level unions/intersections,
and explicit term-mode/rintro/constructor proofs.

This file provides the finite-set operations the development needs but mathlib only
offers in choice-tainted form: a binary union of Finset's, built choice-free by
folding insert, and a decidable equality for Finset via subset antisymmetry.
Every declaration here is audited to depend only on
[propext, Quot.sound].
-/

namespace Scott1982.Constructive

variable {a : Type*} [DecidableEq a]

/-- Choice-free commutativity of insert (mathlib's Finset.insert_comm is choice-tainted).
Needed to fold insert over a Multiset. -/
theorem insert_comm' (a b : a) (s : Finset a) :
  insert a (insert b s) = insert b (insert a s) := by
  ext x
  simp only [Finset.mem_insert]
  constructor
  * rintro (h | h | h)
    exacts [Or.inr (Or.inl h), Or.inl h, Or.inr (Or.inr h)]
  * rintro (h | h | h)
    exacts [Or.inr (Or.inl h), Or.inl h, Or.inr (Or.inr h)]

instance instLeftCommutativeInsert :
  LeftCommutative (insert : a -> Finset a -> Finset a) := <insert_comm'>

/-- Choice-free binary union of finite sets, obtained by folding insert over the second
argument's underlying multiset. Definitionally equal in content to u U v, but -- unlike
mathlib's (* U *) -- free of any Classical.choice dependency. -/
def funion (u v : Finset a) : Finset a := Multiset.foldr insert u v.1

```

```

@[inherit_doc] infixl:65 " U ' " => funion

theorem mem_foldr_insert (a : a) (u : Finset a) (s : Multiset a) :
  a in Multiset.foldr insert u s <-> a in u \ / a in s := by
  refine Multiset.induction_on s ?_ ?_
  * simp
  * intro b t ih
    simp only [Multiset.foldr_cons, Finset.mem_insert, ih, Multiset.mem_cons]
    constructor
    * rintro (h | h | h)
      exacts [Or.inr (Or.inl h), Or.inl h, Or.inr (Or.inr h)]
    * rintro (h | h | h)
      exacts [Or.inr (Or.inl h), Or.inl h, Or.inr (Or.inr h)]

@[simp] theorem mem_funion {a : a} {u v : Finset a} :
  a in u U ' v <-> a in u \ / a in v := mem_foldr_insert a u v.1

/-- The coercion of `u U ' v` to a `Set` is the (choice-free) set union of the coercions. -/
theorem coe_funion (u v : Finset a) :
  ((u U ' v) : Set a) = (u : Set a) U v := by
  ext x
  simp only [Set.mem_union, Finset.mem_coe, mem_funion]

theorem subset_funion_left (u v : Finset a) : u subseq u U ' v := fun _ hx => mem_funion.2
  (Or.inl hx)

theorem subset_funion_right (u v : Finset a) : v subseq u U ' v := fun _ hx => mem_funion
  .2 (Or.inr hx)

/-- Universal property of the union: `u U ' v subseq w` iff both `u subseq w` and `v
  subseq w`. -/
theorem funion_subset_iff {u v w : Finset a} : u U ' v subseq w <-> u subseq w \ /
  v subseq w := by
  constructor
  * intro h
    exact <fun x hx => h (subset_funion_left u v hx),
      fun x hx => h (subset_funion_right u v hx)>
  * rintro <hu, hv> x hx
    rcases mem_funion.1 hx with h | h
    exacts [hu h, hv h]

omit [DecidableEq a] in
/-- If mutual subset holds, the finsets are equal. -/
theorem decidableEq_finset_eq_of_subset (s t : Finset a) (h : s subseq t \ / t subseq
  s) : s = t :=
  Finset.Subset.antisymm h.1 h.2

omit [DecidableEq a] in
/-- Mutual subset is required for finset equality in this decidable instance. -/
theorem decidableEq_finset_false_of_ne (s t : Finset a) (h : not (s subseq t \ / t
  subseq s)) (heq : s = t) :
  False := by
  subst heq
  exact h <Finset.Subset.refl _, Finset.Subset.refl _>

/-- Choice-free decidable equality for `Finset`.
  mathlib's `Finset.decidableEq` goes through `Multiset` quotients and pulls
  `Classical.choice`; this version uses only decidable membership and subset. -/
def decidableEq_finset {a : Type*} [DecidableEq a] : DecidableEq (Finset a) :=
  fun s t =>
    if h : s subseq t \ / t subseq s then
      isTrue (decidableEq_finset_eq_of_subset s t h)

```

```

    else
      isFalse (decidableEq_finset_false_of_ne s t h)
end Scott1982.Constructive

```

A.3 Scott1982/InfoSys.lean

129 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Mathlib.Data.Finset.Basic
import Mathlib.Data.Set.Basic

/-!
# Scott Information Systems

Following Dana Scott, "Domains for Denotational Semantics" (ICALP 1982) and the
compact presentation in Glynn Winskel, "The Formal Semantics of Programming
Languages", Chapter 8.

Following Scott's **Definition 2.1**, an information system is a structure
 $(P, \Delta, \text{Con}, \vdash)$  where

*  $P$  is a set of *data objects* / *propositions* (our token type  $a$ );
*  $\Delta$  in  $P$  is a distinguished *least informative* object (here the field  $\text{bot}$ );
*  $\text{Con}$  is a set of finite subsets of  $P$ , the *consistent* sets; and
*  $\vdash$  (entailment, here  $\text{Ent}$ ) relates a finite set to a token it forces.

Scott's six axioms (Def. 2.1) are, for finite  $u, v \subseteq P$  and  $X$  in  $P$ :

* (i)  $u$  in  $\text{Con}$  whenever  $u \subseteq v$  in  $\text{Con}$  --  $\text{con\_subset}$ 
* (ii)  $\{X\}$  in  $\text{Con}$  --  $\text{con\_sing}$ 
* (iii)  $u \cup \{X\}$  in  $\text{Con}$  whenever  $u \vdash X$  --  $\text{ent\_con}$ 
* (iv)  $u \vdash \Delta$  --  $\text{ent\_bot}$ 
* (v)  $u \vdash X$  whenever  $X$  in  $u$  --  $\text{ent\_refl}$ 
* (vi) if  $v \vdash Y$  for all  $Y$  in  $u$  and  $u \vdash X$  then  $v \vdash X$  --  $\text{ent\_trans}$ 

The **domain** determined by an information system is the poset of its *elements*
(a.k.a. *ideals*): sets of tokens that are consistent on every finite subset and
closed under entailment, ordered by inclusion. This file sets up the structure, the
notion of element, and the partial order; later files build the function, product,
and sum spaces.

This is the **1982** presentation; the development is kept choice-free (constructive),
matching Scott's emphasis on the constructive nature of the definitions.
-/

namespace Scott1982

universe u

/-- A Scott information system on a type of tokens  $a$ , following Scott's Definition 2.1
in "Domains for Denotational Semantics" (ICALP 1982).

```



```

`DecidableEq a` is required so that finite token sets support union ( $X \cup \{a\}$ ) and the
other `Finset` operations the axioms mention. -/
structure InfoSys (a : Type u) [DecidableEq a] where
  /-- The distinguished least-informative object `Delta`. -/
  bot : a
  /-- The consistent finite sets of tokens. -/
  Con : Set (Finset a)
  /-- Entailment: `Ent u a` means the consistent set `u` forces the token `a`. -/
  Ent : Finset a → a → Prop
  /-- (i) Consistency is downward closed under `subseq`. -/
  con_subset : forall {u v : Finset a}, u in Con → v subseq u → v in Con
  /-- (ii) Every singleton is consistent. -/
  con_sing : forall a : a, {a} in Con
  /-- (iii) A set entailing `a` stays consistent when `a` is added. Scott writes this as
  `u ∪ {a} in Con`; we use the definitionally identical `insert a u`, because mathlib's
  `Finset` union instance (unlike `insert`) depends on `Classical.choice`, which would
  break the constructive development. -/
  ent_con : forall {u : Finset a} {a : a}, Ent u a → insert a u in Con
  /-- (iv) The least token `Delta` is entailed by every consistent set. -/
  ent_bot : forall {u : Finset a}, u in Con → Ent u bot
  /-- (v) Entailment is reflexive on members of a consistent set. -/
  ent_refl : forall {u : Finset a} {a : a}, u in Con → a in u → Ent u a
  /-- (vi) Entailment is transitive (cut): if a consistent `u` entails every member of a
  consistent `v`, and `u ⊢ c`, then `v ⊢ c`. -/
  ent_trans : forall {u v : Finset a} {c : a},
    u in Con → v in Con → (forall y in u, Ent v y) → Ent u c → Ent v c

namespace InfoSys

variable {a : Type u} [DecidableEq a] (sys : InfoSys a)

/-- An *element* (ideal) of the domain: a set of tokens that is consistent on every
finite subset and closed under entailment. -/
structure Element where
  /-- The underlying set of tokens. -/
  carrier : Set a
  /-- Every finite subset of the element is consistent. -/
  consistent : forall Y : Finset a, (Y : Set a) subseq carrier → Y in sys.Con
  /-- The element is closed under entailment. -/
  closed : forall (Y : Finset a) (a : a), (Y : Set a) subseq carrier → sys.Ent Y a →
    a in carrier

/-- Extensional equality of elements. -/
theorem Element.ext {x y : sys.Element} (h : x.carrier = y.carrier) : x = y := by
  cases x
  cases y
  subst h
  rfl

/-- Reflexivity of the carrier-inclusion order. -/
theorem element_le_refl (x : sys.Element) : x.carrier subseq x.carrier :=
  Set.Subset.refl _

/-- Transitivity of the carrier-inclusion order. -/
theorem element_le_trans (x y z : sys.Element)
  (hxy : x.carrier subseq y.carrier) (hyz : y.carrier subseq z.carrier) :
  x.carrier subseq z.carrier :=
  Set.Subset.trans hxy hyz

/-- Antisymmetry of the carrier-inclusion order. -/
theorem element_le_antisymm (x y : sys.Element)
  (hxy : x.carrier subseq y.carrier) (hyx : y.carrier subseq x.carrier) :
  x = y :=

```

```

    Element.ext sys (Set.Subset.antisymm hxy hyx)

/-- Elements are ordered by inclusion of their carriers; this is the Scott ordering. -/
instance : PartialOrder sys.Element where
  le x y := x.carrier subseq y.carrier
  le_refl := element_le_refl sys
  le_trans := element_le_trans sys
  le_antisymm := element_le_antisymm sys

/-- Empty set is consistent (subset of any singleton). -/
theorem con_empty : ({ } : Finset a) in sys.Con :=
  sys.con_subset (sys.con_sing sys.bot) (Finset.empty_subset _)

end InfoSys

end Scott1982

```

A.4 Scott1982/Definition22.lean

45 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.InfoSys

/-!
# Definition 2.2 -- set-level entailment

**Scott 1982, Definition 2.2.** For  $u, v$  in  $\text{Con}$  we write  $u \mid\!-\! v$  to mean that
 $u \mid\!-\! X$  for all  $X$  in  $v$ .
-/

namespace Scott1982

namespace InfoSys

universe u

variable {a : Type u} [DecidableEq a] (sys : InfoSys a)

/-- **Definition 2.2 (Scott 1982).** Set-level entailment:  $\text{EntSet } u \ v$  means
 $u \mid\!-\! X$  for every  $X$  in  $v$ . -/
def EntSet (u v : Finset a) : Prop := forall X in v, sys.Ent u X

theorem entSet_empty (u : Finset a) : sys.EntSet u ({ } : Finset a) := by
  intro X hX
  exact False.elim (Finset.notMem_empty X hX)

theorem entSet_singleton {u : Finset a} {X : a} :
  sys.EntSet u {X} <=> sys.Ent u X := by
  constructor
  * intro h
    exact h X (Finset.mem_singleton_self X)
  * intro h Y hY
    rw [Finset.mem_singleton] at hY

```

```

      subst hY
      exact h

end InfoSys

end Scott1982

```

A.5 Scott1982/Proposition23.lean

102 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Constructive
import Scott1982.Definition22

/-!
# Proposition 2.3 -- elementary properties of set-level entailment

**Scott 1982, Proposition 2.3.** For all `u, v, w, u', v'` in `Con`:
(i) `{}` |- `{Delta}`; (ii) `u |- v => u U v` in `Con`; (iii) `u |- u`;
(iv) transitivity; (v) monotonicity; (vi) `u |- v /\ u' |- v' => u |- v U v'`.
-/

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- `u U ' insert a t = insert a (u U ' t)`.-/
theorem funion_insert (u : Finset a) (a : a) (t : Finset a) :
  u U ' insert a t = insert a (u U ' t) := by
  ext x
  constructor
  * intro hx
    rcases mem_funion.mp hx with hu | hins
    * exact Finset.mem_insert_of_mem (mem_funion.mpr (Or.inl hu))
    * rcases Finset.mem_insert.mp hins with ha | ht
      * exact Finset.mem_insert.mpr (Or.inl ha)
      * exact Finset.mem_insert_of_mem (mem_funion.mpr (Or.inr ht))
  * intro hx
    rcases Finset.mem_insert.mp hx with ha | h
    * exact mem_funion.mpr (Or.inr (Finset.mem_insert.mpr (Or.inl ha)))
    * rcases mem_funion.mp h with hu | ht
      * exact mem_funion.mpr (Or.inl hu)
      * exact mem_funion.mpr (Or.inr (Finset.mem_insert.mpr (Or.inr ht)))

/-- **Proposition 2.3(i).** `{}` |- `{Delta}`.-/
theorem proposition_2_3_i : sys.EntSet {} ({sys.bot} : Finset a) := by
  intro X hX
  rw [Finset.mem_singleton] at hX
  subst hX

```

```

exact sys.ent_bot sys.con_empty

/-- **Proposition 2.3(iii).** `u |- u`. -/
theorem proposition_2_3_iii {u : Finset a} (hu : u in sys.Con) : sys.EntSet u u :=
  fun _X hX => sys.ent_refl hu hX

/-- **Proposition 2.3(iv).** Transitivity of `EntSet`. -/
theorem proposition_2_3_iv {u v w : Finset a}
  (hu : u in sys.Con) (hv : v in sys.Con)
  (huv : sys.EntSet u v) (hvw : sys.EntSet v w) : sys.EntSet u w :=
  fun X hX => sys.ent_trans hu hv huv (hvw X hX)

/-- **Proposition 2.3(v).** Monotonicity: `u subseq u'`, `u |- v`, `v' subseq v` =>
  `u' |- v'`. -/
theorem proposition_2_3_v {u u' v v' : Finset a}
  (hu : u in sys.Con) (hu' : u' in sys.Con)
  (hsubu : u subseq u') (huv : sys.EntSet u v) (hsubv : v' subseq v) :
  sys.EntSet u' v' := by
  intro X hX
  have hEnt : sys.Ent u X := huv X (hsubv hX)
  refine sys.ent_trans hu' hu ?_ hEnt
  intro y hy
  exact sys.ent_refl hu' (hsubu hy)

/-- **Proposition 2.3(vi).** `u |- v` and `u |- v'` imply `u |- v U ' v'`. -/
theorem proposition_2_3_vi {u v v' : Finset a}
  (huv : sys.EntSet u v) (huv' : sys.EntSet u v') : sys.EntSet u (v U ' v') := by
  intro X hX
  rcases mem_funion.mp hX with h | h
  * exact huv X h
  * exact huv' X h

/-- **Proposition 2.3(ii).** `u |- v` implies `u U ' v in Con`. -/
theorem proposition_2_3_ii {u v : Finset a} (hu : u in sys.Con) (h : sys.EntSet u v) :
  u U ' v in sys.Con := by
  have : forall s : Finset a, (forall x in s, x in v) -> u U ' s in sys.Con := by
    intro s
    refine Finset.induction_on s ?_ ?_
  * intro _
    -- foldr insert u 0 = u
    simp [funion, Multiset.foldr] using hu
  * intro a t _ha ih hmem
    have hEnt_u_a : sys.Ent u a := h a (hmem a (Finset.mem_insert_self a t))
    have hut : u U ' t in sys.Con := ih fun x hx => hmem x (Finset.mem_insert_of_mem hx)
    have hEnt : sys.Ent (u U ' t) a :=
      sys.ent_trans hut hu (fun y hy => sys.ent_refl hut (subset_funion_left u t hy))
      hEnt_u_a
    have hins : insert a (u U ' t) in sys.Con := sys.ent_con hEnt
    simp [funion_insert] using hins
  exact this v fun _ hx => hx

end InfoSys

end Scott1982

```

A.6 Scott1982/Factoid24.lean

64 lines.

Lean 4 source

/-

Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: <https://github.com/catskillsresearch/scott1982>
- /

```
import Scott1982.InfoSys
```

```
/-!
```

```
# Factoid 2.4 -- first example: lower bounds on Nat
```

```
**Scott 1982, S2 (?A first example?).** Data objects are non-negative integers,  

read as propositions `n` <= `x`. Take `Delta = 0`, every finite set consistent, and
```

```
'''
```

```
{n_0, ..., n??_1} |- m iff m = 0 \ / exists i, m <= n_i.
```

```
'''
```

```
Scott: ?That |- is an entailment relation in the sense of our axioms is clear?  

We package this as an `example` that builds an `InfoSys Nat` and discharges Def 2.1.  

- /
```

```
namespace Scott1982
```

```
namespace Factoid24
```

```
/-- Entailment for the lower-bound system: `u` |- `m` iff `m = 0` or some `n` in `u` has `m` <=  

`n`. - /
```

```
def lowerBoundEnt (u : Finset Nat) (m : Nat) : Prop :=  

m = 0 \ / exists n in u, m <= n
```

```
/-- **Factoid 2.4.** The Nat lower-bound information system of Scott 1982, S2. - /  

example : InfoSys Nat where
```

```
bot := 0  

Con := Set.univ  

Ent := lowerBoundEnt  

con_subset := by  

  intro u v _ _  

  exact Set.mem_univ v  

con_sing := by  

  intro a  

  exact Set.mem_univ _  

ent_con := by  

  intro u a _  

  exact Set.mem_univ _  

ent_bot := by  

  intro u _  

  exact Or.inl rfl  

ent_refl := by  

  intro u a _ ha  

  exact Or.inr <a, ha, le_refl>  

ent_trans := by  

  intro u v c _ _ hvEnt huEnt  

  rcases huEnt with rfl | <n, hn, hcn>  

  * exact Or.inl rfl  

  * -- `u` |- `c` via witness `n` in `u` with `c` <= `n`; use `v` |- `n`  

  rcases hvEnt n hn with rfl | <k, hk, hnk>  

  * -- `n = 0`, so `c` <= `0` => `c = 0`  

  exact Or.inl (Nat.le_zero.mp hcn)  

  * exact Or.inr <k, hk, le_trans hcn hnk>
```

```
end Factoid24
```

end Scott1982

A.7 Scott1982/Factoid25.lean

126 lines.

Lean 4 source

```
/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Mathlib.Algebra.Order.Field.Rat
import Mathlib.Tactic.Linarith
import Mathlib.Tactic.NormNum
import Scott1982.InfoSys

/-!
# Factoid 2.5 -- second example: open intervals plus `(0, inf)`

**Scott 1982, S2 (?A second example?).** Data objects are pairs `(n, m)` of integers
with `n < m`, read as `n < x < m`, together with an artificial least-informative
object `(0, inf)` (our `bot`). Consistency: some point satisfies every token in the
finite set. Entailment: every point satisfying the left-hand set satisfies the
right-hand token.

We take the point space to be `ℚ` so every strict integer-endpoint interval is
inhabited (Def 2.1(ii)). The always-true reading of `bot` matches Scott's
?artificial? Delta.
-/

namespace Scott1982

namespace Factoid25

/-- Tokens: `bot` ? `(0, inf)`, or a strict integer interval `(lo, hi)`. -/
inductive Token : Type where
| bot
| interval (lo hi : Nat) (hlt : lo < hi)

namespace Token

/-- Decidable equality ignores the `<` proof (proof-irrelevant). -/
instance instDecidableEq : DecidableEq Token
| bot, bot => isTrue rfl
| bot, interval _ _ => isFalse Token.noConfusion
| interval _ _ , bot => isFalse Token.noConfusion
| interval n m _ , interval n' m' _ =>
  if h : n = n' /\ m = m' then
    isTrue (by
      rcases h with <rfl, rfl>
      rfl)
  else
    isFalse (by
      intro hEq
      injection hEq with hn hm
      exact h <hn, hm>)

end Token
```

```

/-- Satisfaction of a token at a rational point. -/
def Sat (x : ?) : Token -> Prop
| .bot => True
| .interval lo hi _ => (lo : ?) < x /\ x < (hi : ?)

/-- Midpoint witness: `lo < hi` (on `Nat`) implies `(lo+hi)/2` lies in `(lo, hi)` on `?`. -/
theorem sat_midpoint {lo hi : Nat} (hlt : lo < hi) :
  Sat (((lo : ?) + (hi : ?)) / 2) (.interval lo hi hlt) := by
  have h2 : (0 : ?) < 2 := by norm_num
  have _hlo : (lo : ?) < (hi : ?) := Nat.cast_lt.mpr hlt
  constructor
  * -- lo < (lo+hi)/2 <-> 2*lo < lo+hi <-> lo < hi
    rw [lt_div_iff_0 h2]
    linarith
  * rw [div_lt_iff_0 h2]
    linarith

/-- Every token is satisfiable. -/
theorem sat_sing (t : Token) : exists x : ?, Sat x t := by
  cases t with
  | bot => exact <0, trivial>
  | interval lo hi hlt => exact <_, sat_midpoint hlt>

/-- Generic information system from a satisfaction relation with a true `bot`
and inhabited singletons (Scott's ?obvious interpretation?).

`Ent` packages consistency of the left-hand set together with semantic entailment,
matching Scott's typing of `|-` as a relation on `Con x D` and making `ent_con`
hold (a purely vacuous `forall`-entailment from an inconsistent set would not). -/
def ofSatisfaction {tau : Type*} [DecidableEq tau] {Point : Type*}
  (Sat : Point -> tau -> Prop) (bot : tau)
  (sat_bot : forall p, Sat p bot)
  (sat_sing : forall t, exists p, Sat p t) : InfoSys tau where
bot := bot
Con := {u | exists p, forall t in u, Sat p t}
Ent u X := (exists p, forall t in u, Sat p t) /\ forall p, (forall t in u, Sat p t)
  -> Sat p X
con_subset := by
  rintro u v <p, hp> hsub
  exact <p, fun t ht => hp t (hsub ht)>
con_sing := by
  intro a
  obtain <p, hp> := sat_sing a
  exact <p, fun t ht => by
    rw [Finset.mem_singleton] at ht
    subst ht
    exact hp>
ent_con := by
  rintro u a <<p, hp>, hEnt>
  exact <p, fun t ht => by
    rcases Finset.mem_insert.mp ht with rfl | ht'
    * exact hEnt p hp
    * exact hp t ht'>
ent_bot := by
  intro u hu
  exact <hu, fun p _ => sat_bot p>
ent_refl := by
  intro u a hu ha
  exact <hu, fun p hp => hp a ha>
ent_trans := by
  intro u v c hv _hu hvu <_, huEnt>
  refine <hv, fun p hp => huEnt p ?_>

```

```

    intro y hy
    exact (hvu y hy).2 p hp

/-- **Factoid 2.5.** Interval information system of Scott 1982, S2. -/
example : InfoSys Token :=
  ofSatisfaction Sat Token.bot (fun _ => trivial) sat_sing

end Factoid25

end Scott1982

```

A.8 Scott1982/Factoid26.lean

123 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.InfoSys

/-!
# Factoid 2.6 -- third example: partial-function graphs plus `Delta`

**Scott 1982, S2 (?A third example?).** Fix sets `A` and `B`. Data objects are pairs
`(a, b)` in  $A \times B$  (meaning ?the function maps `a` to `b`?) together with a least-
informative
`Delta`. A finite set of pairs is consistent iff it is functional:

'''
{(a_i, b_i)} in Con iff (a_i = a? -> b_i = b? for all i, j);
'''

adjoining `Delta` preserves consistency. Entailment is the minimal one:

'''
u |- X iff X = Delta \ / X in u.
'''
-/

namespace Scott1982

namespace Factoid26

/-- Tokens: `bot` ? `Delta`, or a graph atom `pair a b`. -/
inductive Token (A B : Type*) where
| bot
| pair (a : A) (b : B)
deriving DecidableEq

/-- Scott?s consistency: the pairs in `u` form a partial function `A ? B`
(`bot` is ignored). -/
def Consistent {A B : Type*} (u : Finset (Token A B)) : Prop :=
  forall ?a : A? ?b b' : B?,
    Token.pair a b in u -> Token.pair a b' in u -> b = b'

theorem consistent_empty {A B : Type*} :

```



```

    Consistent ({t} : Finset (Token A B)) := by
intro _a _b _b' hb _
exact False.elim (Finset.notMem_empty _ hb)

theorem consistent_singleton {A B : Type*} [DecidableEq A] [DecidableEq B]
  (t : Token A B) : Consistent ({t} : Finset (Token A B)) := by
intro a b b' hb hb'
rw [Finset.mem_singleton] at hb hb'
-- both equal `t`; cases on `t`
cases t with
| bot =>
  cases hb
| pair a0 b0 =>
  cases hb
  cases hb'
  rfl

theorem consistent_subset {A B : Type*} {u v : Finset (Token A B)} (huv : v subseq u)
  (hu : Consistent u) : Consistent v :=
fun _a _b _b' hb hb' => hu (huv hb) (huv hb')

theorem consistent_insert_bot {A B : Type*} [DecidableEq A] [DecidableEq B]
  {u : Finset (Token A B)} (hu : Consistent u) :
  Consistent (insert (Token.bot : Token A B) u) := by
intro a b b' hb hb'
have hb1 : Token.pair a b in u := by
  rcases Finset.mem_insert.mp hb with h | h
  * cases h
  * exact h
have hb2 : Token.pair a b' in u := by
  rcases Finset.mem_insert.mp hb' with h | h
  * cases h
  * exact h
exact hu hb1 hb2

theorem consistent_insert_mem {A B : Type*} [DecidableEq A] [DecidableEq B]
  {u : Finset (Token A B)} {t : Token A B}
  (ht : t in u) (hu : Consistent u) : Consistent (insert t u) := by
simpa [Finset.insert_eq_of_mem ht] using hu

/-- Minimal entailment, with LHS consistency baked in (Scott types `|-` on `Con`). -/
def Ent {A B : Type*} (u : Finset (Token A B)) (X : Token A B) : Prop :=
  Consistent u /\ (X = Token.bot \/ X in u)

/-- **Factoid 2.6.** Partial-function information system of Scott 1982, S2. -/
def partialFunctionSystem (A B : Type*) [DecidableEq A] [DecidableEq B] :
  InfoSys (Token A B) where
  bot := Token.bot
  Con := {u | Consistent u}
  Ent := Ent
  con_subset := by
    intro u v hu hv
    exact consistent_subset hv hu
  con_sing := fun t => consistent_singleton t
  ent_con := by
    rintro u a <hu, hX>
    rcases hX with rfl | hmem
    * exact consistent_insert_bot hu
    * exact consistent_insert_mem hmem hu
  ent_bot := by
    intro u hu
    exact <hu, Or.inl rfl>
  ent_refl := by

```

```

    intro u a hu ha
    exact <hu, Or.inr ha>
ent_trans := by
  intro u v c hv _hu hvu <_, hc>
  refine <hv, ?_>
  rcases hc with rfl | hc
  * exact Or.inl rfl
  * have <_, hcv> := hvu c hc
    exact hcv

/-- Instantiation for concrete carrier sets (Scott: ?two fixed sets?). -/
example : InfoSys (Token Nat Bool) := partialFunctionSystem Nat Bool

end Factoid26

end Scott1982

```

A.9 Scott1982/Factoid32.lean

33 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Proposition23

/-!
# Factoid 3.2 -- every element contains Delta

**Factoid 3.2 (inventory).** Scott remarks after Def 3.1 that every element contains `Delta`,
because the least informative proposition is true of all elements.
-/

namespace Scott1982

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- **Factoid 3.2.** Every element contains `Delta`. -/
theorem factoid_3_2 (x : sys.Element) : sys.bot in x.carrier := by
  have hempty : (({} : Finset a) : Set a) subseteq x.carrier := by
    intro a ha
    exact False.elim (Finset.notMem_empty a (Finset.mem_coe.1 ha))
  have hEnt : sys.Ent {} sys.bot := sys.ent_bot sys.con_empty
  exact x.closed {} sys.bot hempty hEnt

end InfoSys

end Scott1982

```

A.10 Scott1982/Factoid33.lean

55 lines.

Lean 4 source

```
/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Factoid32

/-!
# Factoid 3.3 -- the bottom element False

**Factoid 3.3 (inventory).** Scott defines
`False_A = {X in D_A ? {Delta_A} |- _A X}` and notes it is the least element of `|A|`.
-/

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- **Factoid 3.3.** The bottom element `False = {X ? {Delta} |- X}`. -/
def botElement : sys.Element where
  carrier := {X | sys.Ent {sys.bot} X}
  consistent := by
    intro Y hY
    have hbot : ({sys.bot} : Finset a) in sys.Con := sys.con_sing sys.bot
    have hEnt : sys.EntSet {sys.bot} Y := fun X hX => hY (Finset.mem_coe.2 hX)
    have hunion : {sys.bot} U ' Y in sys.Con := proposition_2_3_ii sys hbot hEnt
    exact sys.con_subset hunion (subset_funion_right _ _)
  closed := by
    intro Y a hY hEnt
    have hbot : ({sys.bot} : Finset a) in sys.Con := sys.con_sing sys.bot
    have hYcon : Y in sys.Con := by
      have hEntY : sys.EntSet {sys.bot} Y := fun X hX => hY (Finset.mem_coe.2 hX)
      exact sys.con_subset (proposition_2_3_ii sys hbot hEntY) (subset_funion_right _ _)
    exact sys.ent_trans hbot hYcon (fun y hy => hY (Finset.mem_coe.2 hy)) hEnt

/-- `False` is least. -/
theorem botElement_le (x : sys.Element) : sys.botElement <= x := by
  intro a ha
  have hsub : (({sys.bot} : Finset a) : Set a) subseq x.carrier := by
    intro b hb
    have hb' : b in ({sys.bot} : Finset a) := Finset.mem_coe.1 hb
    rw [Finset.mem_singleton] at hb'
    subst hb'
    exact factoid_3_2 sys x
  exact x.closed {sys.bot} a hsub ha

end InfoSys

end Scott1982
```

A.11 Scott1982/Factoid34.lean

77 lines.

Lean 4 source

```
/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.InfoSys

/-!
# Factoid 3.4 -- top element and total elements

**Factoid 3.4 (inventory / Scott S3).** There need be no top element `top`. Such an
element exists if and only if every finite subset of the token set is consistent, in
which case `top = D` as a set. When it exists, it is the unique *total* (maximal)
element of the domain.
-/

namespace Scott1982

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- A **total** element is a maximal element of `|A|` under inclusion. -/
def IsTotal (x : sys.Element) : Prop := forall y, x <= y -> y <= x

/-- Every finite set of tokens is consistent. -/
def AllConsistent : Prop := forall u : Finset a, u in sys.Con

/-- **Factoid 3.4.** When every finite set is consistent, the top element is `D` itself. -/
def topElement (h : sys.AllConsistent) : sys.Element where
  carrier := Set.univ
  consistent := fun Y _ => h Y
  closed := fun _ _ _ => trivial

theorem le_topElement (h : sys.AllConsistent) (x : sys.Element) :
  x <= sys.topElement h := fun _ _ => trivial

theorem topElement_isTotal (h : sys.AllConsistent) :
  sys.IsTotal (sys.topElement h) := fun y _ => sys.le_topElement h y

/-- When `top` exists, it is the unique total element. -/
theorem eq_topElement_of_isTotal (h : sys.AllConsistent) {x : sys.Element}
  (hx : sys.IsTotal x) : x = sys.topElement h :=
  le_antisymm (sys.le_topElement h x) (hx _ (sys.le_topElement h x))

theorem isTotal_iff_eq_topElement (h : sys.AllConsistent) {x : sys.Element} :
  sys.IsTotal x <-> x = sys.topElement h :=
  <sys.eq_topElement_of_isTotal h, fun hx => hx |> sys.topElement_isTotal h>

theorem exists_unique_total_of_allConsistent (h : sys.AllConsistent) :
  exists ! x : sys.Element, sys.IsTotal x :=
  <sys.topElement h, <sys.topElement_isTotal h, fun _ hy =>
  sys.eq_topElement_of_isTotal h hy>>

/-- An element whose carrier is the whole token set is total (hence the greatest element). -/
theorem isTotal_of_carrier_univ {x : sys.Element} (hx : x.carrier = Set.univ) :
  sys.IsTotal x := fun _ _ _ => hx |> trivial

/-- Conversely: any total element equals an element with carrier `D`. -/
theorem eq_of_carrier_univ_of_isTotal {x t : sys.Element}
```

```

    (hx : x.carrier = Set.univ) (ht : sys.IsTotal t) : t = x :=
le_antisymm (fun _ => hx |> trivial) (ht x (fun _ => hx |> trivial))

/-- **Factoid 3.4.** Top `top` = D` exists as an element iff every finite subset is consistent
. -/
theorem exists_top_iff_allConsistent :
  (exists x : sys.Element, x.carrier = Set.univ) <=> sys.AllConsistent := by
  constructor
  * intro <x, hx> u
    exact x.consistent u (hx |> Set.subset_univ _)
  * intro h
    exact <sys.topElement h, rfl>

end InfoSys

end Scott1982

```

A.12 Scott1982/Factoid35.lean

49 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Proposition23

/-!
# Factoid 3.5 -- finite elements as entailment closures

**Factoid 3.5 (inventory).** For `u` in `Con`, the closure
`u` = {X ? u | X} is an element of `|A|` (a *finite element*).
-/

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- **Factoid 3.5.** Entailment closure of a consistent set. -/
def closure (u : Finset a) (hu : u in sys.Con) : sys.Element where
  carrier := {X | sys.Ent u X}
  consistent := by
    intro Y hY
    have hEnt : sys.EntSet u Y := fun X hX => hY (Finset.mem_coe.2 hX)
    exact sys.con_subset (proposition_2_3_ii sys hu hEnt) (subset_funion_right _ _)
  closed := by
    intro Y a hY hEnt
    have hYcon : Y in sys.Con := by
      have hEntY : sys.EntSet u Y := fun X hX => hY (Finset.mem_coe.2 hX)
      exact sys.con_subset (proposition_2_3_ii sys hu hEntY) (subset_funion_right _ _)
    exact sys.ent_trans hu hYcon (fun y hy => hY (Finset.mem_coe.2 hy)) hEnt

theorem mem_closure_iff {u : Finset a} (hu : u in sys.Con) {X : a} :

```

```

      X in (sys.closure u hu).carrier <=> sys.Ent u X := Iff.rfl

/-- `u` subseq u`. -/
theorem subset_closure {u : Finset a} (hu : u in sys.Con) :
  u subseq (sys.closure u hu).carrier :=
  fun _ hX => sys.ent_refl hu (Finset.mem_coe.1 hX)

end InfoSys

end Scott1982

```

A.13 Scott1982/Factoid36.lean

90 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Constructive
import Scott1982.Factoid35

/-!
# Factoid 3.6 -- every element is the directed union of its finite approximations

**Factoid 3.6 (Scott S3 / inventory).** For every element `x` in  $|A|$ ,
`x` equals the union of all `u` with `u` in  $Con$  and `u` subseq `x`.
Intuitively: every element is the limit of its finite approximations. The union is directed.
-/

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- If `w` entails every member of `u`, then `u` subseq `w`. -/
theorem closure_le_of_entSet {u w : Finset a} (hu : u in sys.Con) (hw : w in sys.Con)
  (h : sys.EntSet w u) : sys.closure u hu <= sys.closure w hw := by
  intro a ha
  exact sys.ent_trans hw hu h ha

/-- Monotonicity of closure under inclusion of consistent sets. -/
theorem closure_le_of_subset {u w : Finset a} (hu : u in sys.Con) (hw : w in sys.Con)
  (hsub : u subseq w) : sys.closure u hu <= sys.closure w hw :=
  sys.closure_le_of_entSet hu hw fun _ hy => sys.ent_refl hw (hsub hy)

/-- Finite approximations below `x` really do approximate `x`. -/
theorem closure_le_element (x : sys.Element) {u : Finset a} (hu : u in sys.Con)
  (hsub : u subseq x.carrier) : sys.closure u hu <= x := by
  intro a ha
  exact x.closed u a hsub ha

/-- The set-theoretic union of all finite approximations of `x`. -/
def approxUnion (x : sys.Element) : Set a :=

```

```

{a | exists (u : Finset a) (hu : u in sys.Con),
  u subseq x.carrier /\ a in (sys.closure u hu).carrier}

/-- **Factoid 3.6.** Membership in `x` iff membership in some finite approximation `u` subseq
  x`. -/
theorem mem_element_iff_mem_closure (x : sys.Element) (a : a) :
  a in x.carrier <=> a in sys.approxUnion x := by
  constructor
  * intro ha
    refine <{a}, sys.con_sing a, ?_, ?_>
  * intro b hb
    have hb' : b in ({a} : Finset a) := Finset.mem_coe.1 hb
    rw [Finset.mem_singleton] at hb'
    subst hb'
    exact ha
  * exact sys.ent_refl (sys.con_sing a) (Finset.mem_singleton_self a)
  * rintro <u, hu, hsub, ha>
    exact x.closed u a hsub ha

/-- **Factoid 3.6.** Carrier form: `x = ?{u | u in Con, u subseq x}`. -/
theorem element_eq_approxUnion (x : sys.Element) : x.carrier = sys.approxUnion x := by
  ext a
  exact sys.mem_element_iff_mem_closure x a

/-- The family of finite approximations to `x` is directed under `<=`. -/
theorem closures_directed (x : sys.Element) {u v : Finset a}
  (hu : u in sys.Con) (hv : v in sys.Con)
  (huX : u subseq x.carrier) (hvX : v subseq x.carrier) :
  exists (w : Finset a) (hw : w in sys.Con),
    w subseq x.carrier /\
    sys.closure u hu <= sys.closure w hw /\
    sys.closure v hv <= sys.closure w hw := by
  let w := u ∪ v
  have hwX : w subseq x.carrier := by
    intro a ha
    rcases mem_funion.1 (Finset.mem_coe.1 ha) with h | h
    * exact huX (Finset.mem_coe.2 h)
    * exact hvX (Finset.mem_coe.2 h)
  have hw : w in sys.Con := x.consistent w hwX
  refine <w, hw, hwX, ?_, ?_>
  * exact sys.closure_le_of_subset hu hw (subset_funion_left u v)
  * exact sys.closure_le_of_subset hv hw (subset_funion_right u v)

end InfoSys

end Scott1982

```

A.14 Scott1982/Factoid41.lean

93 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

```

```
import Scott1982.Factoid33
```

```

/-!
# Factoid 4.1 -- `|A|` is an inf-semilattice under intersection

**Factoid 4.1 (Scott S4 / inventory).** The intersection of two elements is again an
element, so `|A|` is an inf-semilattice under `I`. In particular
`x ⊆ y <-> x I y = x`. Scott also records that `I` is idempotent, commutative,
and
associative, and that `False` is a zero for `I`.
-/

namespace Scott1982

namespace InfoSys

variable {α : Type*} [DecidableEq α] (sys : InfoSys α)

/-- **Factoid 4.1.** Binary meet of elements is set-theoretic intersection. -/
def inf (x y : sys.Element) : sys.Element where
  carrier := x.carrier I y.carrier
  consistent := fun Y hY =>
    x.consistent Y (Set.Subset.trans hY Set.inter_subset_left)
  closed := fun Y a hY hEnt =>
    <x.closed Y a (Set.Subset.trans hY Set.inter_subset_left) hEnt,
    y.closed Y a (Set.Subset.trans hY Set.inter_subset_right) hEnt>

theorem inf_le_left (x y : sys.Element) : sys.inf x y <= x :=
  Set.inter_subset_left

theorem inf_le_right (x y : sys.Element) : sys.inf x y <= y :=
  Set.inter_subset_right

theorem le_inf {x y z : sys.Element} (hxy : x <= y) (hxz : x <= z) : x <= sys.inf y z :=
  fun _ ha => <hxy ha, hxz ha>

/-- Meet is the greatest lower bound. -/
theorem inf_le_iff {x y z : sys.Element} : z <= sys.inf x y <-> z <= x /\ z <= y :=
  <fun h => <le_trans h (sys.inf_le_left x y), le_trans h (sys.inf_le_right x y)>,
  fun <hx, hy> => sys.le_inf hx hy>

/-- **Factoid 4.1.** `x ⊆ y` iff `x I y = x`. -/
theorem le_iff_inf_eq {x y : sys.Element} : x <= y <-> sys.inf x y = x := by
  constructor
  * intro h
    refine le_antisymm (sys.inf_le_left x y) ?_
    intro a ha
    exact <ha, h ha>
  * intro heq
    have hc : x.carrier = x.carrier I y.carrier :=
      Eq.symm (congrArg Element.carrier heq)
    intro a ha
    exact ((Set.ext_iff.1 hc a).1 ha).2

theorem inf_idem (x : sys.Element) : sys.inf x x = x :=
  (sys.le_iff_inf_eq (y := x)).1 (le_refl x)

theorem inf_comm (x y : sys.Element) : sys.inf x y = sys.inf y x := by
  refine le_antisymm ?_ ?_
  * exact sys.le_inf (sys.inf_le_right x y) (sys.inf_le_left x y)
  * exact sys.le_inf (sys.inf_le_right y x) (sys.inf_le_left y x)

theorem inf_assoc (x y z : sys.Element) : sys.inf (sys.inf x y) z = sys.inf x (sys.inf y z) :=
  by
    refine le_antisymm ?_ ?_

```



```

* refine sys.le_inf ?_ (sys.le_inf ?_ ?_)
  * exact le_trans (sys.inf_le_left _ _) (sys.inf_le_left _ _)
  * exact le_trans (sys.inf_le_left _ _) (sys.inf_le_right _ _)
  * exact sys.inf_le_right _ _
* refine sys.le_inf (sys.le_inf ?_ ?_) ?_
  * exact sys.inf_le_left _ _
  * exact le_trans (sys.inf_le_right _ _) (sys.inf_le_left _ _)
  * exact le_trans (sys.inf_le_right _ _) (sys.inf_le_right _ _)

theorem inf_mono {x_1 x_2 y_1 y_2 : sys.Element} (hx : x_1 <= x_2) (hy : y_1 <= y_2) :
  sys.inf x_1 y_1 <= sys.inf x_2 y_2 :=
  sys.le_inf (le_trans (sys.inf_le_left _ _) hx) (le_trans (sys.inf_le_right _ _) hy)

/-- `False` is a zero for meet: `False I x = False`. -/
theorem botElement_inf (x : sys.Element) : sys.inf sys.botElement x = sys.botElement :=
  (sys.le_iff_inf_eq (x := sys.botElement) (y := x)).1 (sys.botElement_le x)

theorem inf_botElement (x : sys.Element) : sys.inf x sys.botElement = sys.botElement := by
  rw [sys.inf_comm, sys.botElement_inf]

end InfoSys

end Scott1982

```

A.15 Scott1982/Factoid42.lean

71 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Factoid41

/-!
# Factoid 4.2 -- conditional completeness for meets

**Factoid 4.2 (Scott S4).** For any *non-empty* family of elements, the set-theoretic
intersection of the family is again an element. Thus `|A|` is a conditionally complete
inf-semilattice.
-/

namespace Scott1982

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- Carrier intersection of a family of elements. -/
def familyInfCarrier (S : Set sys.Element) : Set a :=
  {a | forall x in S, a in x.carrier}

/-- **Factoid 4.2.** Meet of a nonempty family of elements (set-theoretic intersection). -/
def familyInf (S : Set sys.Element) (hne : S.Nonempty) : sys.Element where
  carrier := sys.familyInfCarrier S
  consistent := by
    intro Y hY

```

```

    obtain <x_0, hx_0> := hne
    have hYx : (Y : Set a) subseq x_0.carrier := fun a ha => (hY ha) x_0 hx_0
    exact x_0.consistent Y hYx
closed := by
  intro Y a hY hEnt x hx
  have hYx : (Y : Set a) subseq x.carrier := fun b hb => (hY hb) x hx
  exact x.closed Y a hYx hEnt

theorem familyInf_le (S : Set sys.Element) (hne : S.Nonempty) {x : sys.Element}
  (hx : x in S) : sys.familyInf S hne <= x :=
  fun _a ha => ha x hx

theorem le_familyInf (S : Set sys.Element) (hne : S.Nonempty) {z : sys.Element}
  (h : forall x in S, z <= x) : z <= sys.familyInf S hne :=
  fun _a ha x hx => h x hx ha

/-- `familyInf` is the greatest lower bound of the family. -/
theorem familyInf_le_iff (S : Set sys.Element) (hne : S.Nonempty) {z : sys.Element} :
  z <= sys.familyInf S hne <-> forall x in S, z <= x :=
  <fun hz _x hx => le_trans hz (sys.familyInf_le S hne hx), sys.le_familyInf S hne>

/-- Binary meet agrees with the two-element family meet. -/
theorem familyInf_pair (x y : sys.Element) :
  sys.familyInf ({x, y} : Set sys.Element) <x, Set.mem_insert x {y}> = sys.inf x y := by
  refine le_antisymm ?_ ?_
  * exact sys.le_inf
    (sys.familyInf_le _ _ (Set.mem_insert x {y}))
    (sys.familyInf_le _ _ (Set.mem_insert_of_mem _ rfl))
  * apply sys.le_familyInf
    intro z hz
    rcases Set.mem_insert_iff.mp hz with rfl | hy
    * exact sys.inf_le_left _ _
    * have : z = y := hy
      subst this
      exact sys.inf_le_right _ _

end InfoSys

end Scott1982

```

A.16 Scott1982/Factoid43.lean

184 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Constructive
import Scott1982.Factoid35
import Scott1982.Factoid42

/-!
# Factoid 4.3 -- consistent joins

**Factoid 4.3 (Scott S4).** The join of a family of elements exists in `|A|` iff the
union of their carriers is finitely consistent; in that case the join is the deductive

```

```

closure of the union. Binary case: `x sqsup y` exists iff some element bounds both.
-/

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- A (possibly infinite) set of tokens is *finitely consistent* when every finite subset
lies in `Con`. -/
def IsFinitelyConsistent (U : Set a) : Prop :=
  forall Y : Finset a, (Y : Set a) subseq U -> Y in sys.Con

/-- Witness: some finite `w subseq U` entails every member of `Y subseq
deductiveClosure U`. -/
theorem exists_entSet_of_subset_deductive
  (U : Set a) (hU : sys.IsFinitelyConsistent U) (Y : Finset a)
  (hY : (Y : Set a) subseq {a | exists u : Finset a, (u : Set a) subseq U /\ sys.
    Ent u a}) :
  exists w : Finset a, (w : Set a) subseq U /\ sys.EntSet w Y := by
  induction Y using Finset.induction_on with
  | empty =>
    exact <{}, fun _ ha => False.elim (Finset.notMem_empty _ (Finset.mem_coe.1 ha)),
      fun _ ha => False.elim (Finset.notMem_empty _ ha)>
  | insert a s _ha ih =>
    have ha' : exists u : Finset a, (u : Set a) subseq U /\ sys.Ent u a :=
      hY (Finset.mem_coe.2 (Finset.mem_insert_self a s))
    have hs' : (s : Set a) subseq {b | exists u : Finset a, (u : Set a) subseq U /\
      sys.Ent u b} := by
      intro b hb
      exact hY (Finset.mem_coe.2 (Finset.mem_insert_of_mem (Finset.mem_coe.1 hb)))
    obtain <ua, hua, hEnta> := ha'
    obtain <us, hus, hEnts> := ih hs'
    refine <ua U ' us, ?_, ?_>
    * intro z hz
      rcases mem_funion.mp (Finset.mem_coe.1 hz) with h | h
      * exact hua (Finset.mem_coe.2 h)
      * exact hus (Finset.mem_coe.2 h)
    * intro b hb
      have hwU : (ua U ' us : Set a) subseq U := by
        intro z hz
        rcases mem_funion.mp (Finset.mem_coe.1 hz) with h | h
        * exact hua (Finset.mem_coe.2 h)
        * exact hus (Finset.mem_coe.2 h)
      have hw : ua U ' us in sys.Con := hU _ hwU
      rcases Finset.mem_insert.mp hb with rfl | hb'
      * exact sys.ent_trans hw (hU ua hua)
        (fun y hy => sys.ent_refl hw (mem_funion.mpr (Or.inl hy))) hEnta
      * exact sys.ent_trans hw (hU us hus)
        (fun y hy => sys.ent_refl hw (mem_funion.mpr (Or.inr hy))) (hEnts b hb')

/-- Deductive closure of a finitely consistent set. -/
def deductiveClosure (U : Set a) (hU : sys.IsFinitelyConsistent U) : sys.Element where
  carrier := {a | exists u : Finset a, (u : Set a) subseq U /\ sys.Ent u a}
  consistent := by
    intro Y hY
    obtain <w, hwU, hEnt> := sys.exists_entSet_of_subset_deductive U hU Y hY
    have hw : w in sys.Con := hU w hwU
    exact sys.con_subset (proposition_2_3_ii sys hw hEnt) (subset_funion_right _ _)
  closed := by

```

```

    intro Y a hY hEnt
    obtain <w, hwU, hEntY> := sys.exists_entSet_of_subset_deductive U hU Y hY
    have hw : w in sys.Con := hU w hwU
    have hYcon : Y in sys.Con :=
      sys.con_subset (proposition_2_3_ii sys hw hEntY) (subset_funion_right _ _)
    exact <w, hwU, sys.ent_trans hw hYcon hEntY hEnt>

theorem subset_deductiveClosure (U : Set a) (hU : sys.IsFinitelyConsistent U) :
  U subseq (sys.deductiveClosure U hU).carrier := by
  intro a ha
  refine <{a}, ?, sys.ent_refl (hU {a} ?) (Finset.mem_singleton_self a)>
  * intro b hb
    have : b = a := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
    subst this
    exact ha
  * intro b hb
    have : b = a := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
    subst this
    exact ha

/-- Carrier union of a family of elements. -/
def familyUnionCarrier (S : Set sys.Element) : Set a :=
  {a | exists x in S, a in x.carrier}

/-- Join of a family whose carrier-union is finitely consistent. -/
def familySup (S : Set sys.Element)
  (hU : sys.IsFinitelyConsistent (sys.familyUnionCarrier S)) : sys.Element :=
  sys.deductiveClosure (sys.familyUnionCarrier S) hU

theorem le_familySup (S : Set sys.Element)
  (hU : sys.IsFinitelyConsistent (sys.familyUnionCarrier S)) {x : sys.Element}
  (hx : x in S) : x <= sys.familySup S hU := by
  intro a ha
  exact sys.subset_deductiveClosure _ hU <x, hx, ha>

theorem familySup_le (S : Set sys.Element)
  (hU : sys.IsFinitelyConsistent (sys.familyUnionCarrier S)) {z : sys.Element}
  (h : forall x in S, x <= z) : sys.familySup S hU <= z := by
  intro a ha
  obtain <u, huU, hEnt> := ha
  have huZ : (u : Set a) subseq z.carrier := by
    intro b hb
    obtain <x, hx, hb'> := huU hb
    exact h x hx hb'
  exact z.closed u a huZ hEnt

/-- An upper bound of `S` forces the carrier-union to be finitely consistent. -/
theorem finitelyConsistent_of_upperBound (S : Set sys.Element) {z : sys.Element}
  (h : forall x in S, x <= z) : sys.IsFinitelyConsistent (sys.familyUnionCarrier S) :=
  by
  intro Y hY
  have hYZ : (Y : Set a) subseq z.carrier := by
    intro a ha
    obtain <x, hx, ha'> := hY ha
    exact h x hx ha'
  exact z.consistent Y hYZ

/-- Join exists iff the union is finitely consistent. -/
theorem exists_isLUB_iff (S : Set sys.Element) :
  (exists z : sys.Element, (forall x in S, x <= z) /\ forall w, (forall x in S, x
    <= w) -> z <= w) <->
  Nonempty (sys.IsFinitelyConsistent (sys.familyUnionCarrier S)) := by
  constructor

```

```

* rintro <z, hub, _>
  exact <sys.finitelyConsistent_of_upperBound S hub>
* rintro <hU>
  refine <sys.familySup S hU, fun x hx => sys.le_familySup S hU hx,
    fun w hw => sys.familySup_le S hU hw>

/-- Binary join via deductive closure of `x.carrier U y.carrier`, when consistent. -/
def join (x y : sys.Element)
  (h : sys.IsFinitelyConsistent (x.carrier U y.carrier)) : sys.Element :=
  sys.deductiveClosure (x.carrier U y.carrier) h

theorem le_join_left (x y : sys.Element)
  (h : sys.IsFinitelyConsistent (x.carrier U y.carrier)) : x <= sys.join x y h :=
  fun _a ha => sys.subset_deductiveClosure _ h (Or.inl ha)

theorem le_join_right (x y : sys.Element)
  (h : sys.IsFinitelyConsistent (x.carrier U y.carrier)) : y <= sys.join x y h :=
  fun _a ha => sys.subset_deductiveClosure _ h (Or.inr ha)

theorem join_le (x y z : sys.Element)
  (h : sys.IsFinitelyConsistent (x.carrier U y.carrier))
  (hx : x <= z) (hy : y <= z) : sys.join x y h <= z := by
  intro a ha
  obtain <u, huU, hEnt> := ha
  have huZ : (u : Set a) subseq z.carrier := by
    intro b hb
    rcases huU hb with hb' | hb'
    * exact hx hb'
    * exact hy hb'
  exact z.closed u a huZ hEnt

/-- Binary join exists iff some element bounds both. -/
theorem exists_join_iff (x y : sys.Element) :
  (exists z : sys.Element, x <= z /\ y <= z) <=>
  Nonempty (sys.IsFinitelyConsistent (x.carrier U y.carrier)) := by
  constructor
* rintro <z, hx, hy>
  exact <fun Y hY => z.consistent Y (fun a ha => by
    rcases hY ha with h | h
    * exact hx h
    * exact hy h)>
* rintro <h>
  exact <sys.join x y h, sys.le_join_left x y h, sys.le_join_right x y h>

end InfoSys

end Scott1982

```

A.17 Scott1982/Factoid44.lean

104 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Constructive

```

```

import Scott1982.Factoid43

/-!
# Factoid 4.4 -- directed (and chain) unions are elements

**Factoid 4.4 (Scott S4).** The union of a nonempty directed family of elements is again
an element (hence a lub). In particular, unions of increasing chains are elements, so
`|A|` is a cpo under inclusion.
-/

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- Upward-directed family of elements: any two have a common upper bound in the family. -/
def IsDirected (S : Set sys.Element) : Prop :=
  forall (x y : sys.Element), x in S -> y in S -> exists z in S, x <= z /\ y
    <= z

/-- A chain: any two elements are comparable. -/
def IsChain (S : Set sys.Element) : Prop :=
  forall (x y : sys.Element), x in S -> y in S -> x <= y \/ y <= x

theorem IsDirected_of_IsChain {S : Set sys.Element} (h : sys.IsChain S) : sys.IsDirected S :=
  by
    intro x y hx hy
    rcases h x y hx hy with hxy | hyx
    * exact <y, hy, hxy, le_rfl>
    * exact <x, hx, le_rfl, hyx>

/-- Given `Y` whose tokens each lie in some member of a nonempty directed family, find a
single member containing all of `Y`. -/
theorem exists_mem_of_subset_directedUnion (S : Set sys.Element) (hne : S.Nonempty)
  (hdir : sys.IsDirected S) (Y : Finset a)
  (hY : (Y : Set a) subseq sys.familyUnionCarrier S) :
  exists z in S, (Y : Set a) subseq z.carrier := by
  induction Y using Finset.induction_on with
  | empty =>
    obtain <z, hz> := hne
    exact <z, hz, fun _ ha => False.elim (Finset.notMem_empty _ (Finset.mem_coe.1 ha))>
  | insert a s _ha ih =>
    have ha' : exists x in S, a in x.carrier :=
      hY (Finset.mem_coe.2 (Finset.mem_insert_self a s))
    have hs' : (s : Set a) subseq sys.familyUnionCarrier S := by
      intro b hb
      exact hY (Finset.mem_coe.2 (Finset.mem_insert_of_mem (Finset.mem_coe.1 hb)))
    obtain <xa, hxa, haS> := ha'
    obtain <xs, hxs, hsS> := ih hs'
    obtain <z, hz, hxa_le, hxs_le> := hdir xa xs hxa hxs
    refine <z, hz, ?_>
    intro b hb
    rcases Finset.mem_insert.mp (Finset.mem_coe.1 hb) with rfl | hb'
    * exact hxa_le haS
    * exact hxs_le (hsS (Finset.mem_coe.2 hb'))

/-- Directed families are finitely consistent on their carrier union. -/
theorem finitelyConsistent_of_directed (S : Set sys.Element) (hne : S.Nonempty)
  (hdir : sys.IsDirected S) :
  sys.IsFinitelyConsistent (sys.familyUnionCarrier S) := by

```

```

intro Y hY
obtain <z, _hz, hYZ> := sys.exists_mem_of_subset_directedUnion S hne hdir Y hY
exact z.consistent Y hYZ

/-- **Factoid 4.4.** Union of a nonempty directed family is an element (as `familySup`). -/
def directedSup (S : Set sys.Element) (hne : S.Nonempty) (hdir : sys.IsDirected S) :
  sys.Element :=
  sys.familySup S (sys.finitelyConsistent_of_directed S hne hdir)

theorem le_directedSup (S : Set sys.Element) (hne : S.Nonempty) (hdir : sys.IsDirected S)
  {x : sys.Element} (hx : x in S) : x <= sys.directedSup S hne hdir :=
  sys.le_familySup S (sys.finitelyConsistent_of_directed S hne hdir) hx

theorem directedSup_le (S : Set sys.Element) (hne : S.Nonempty) (hdir : sys.IsDirected S)
  {z : sys.Element} (h : forall x in S, x <= z) : sys.directedSup S hne hdir <= z :=
  sys.familySup_le S (sys.finitelyConsistent_of_directed S hne hdir) h

/-- For directed families, the lub carrier equals the raw union (already deductively closed).
-/
theorem directedSup_carrier_eq_union (S : Set sys.Element) (hne : S.Nonempty)
  (hdir : sys.IsDirected S) :
  (sys.directedSup S hne hdir).carrier = sys.familyUnionCarrier S := by
  let hU := sys.finitelyConsistent_of_directed S hne hdir
  refine Set.Subset.antisymm ?_ (sys.subset_deductiveClosure _ hU)
  intro a ha
  obtain <u, huU, hEnt> := (ha : a in (sys.deductiveClosure _ hU).carrier)
  obtain <z, hz, huZ> := sys.exists_mem_of_subset_directedUnion S hne hdir u huU
  exact <z, hz, z.closed u a huZ hEnt>

/-- Chain unions are elements. -/
def chainSup (S : Set sys.Element) (hne : S.Nonempty) (hchain : sys.IsChain S) : sys.Element :=
  sys.directedSup S hne (sys.IsDirected_of_IsChain hchain)

end InfoSys

end Scott1982

```

A.18 Scott1982/Factoid45.lean

96 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Factoid36
import Scott1982.Factoid44

/-!
# Factoid 4.5 -- algebraicity via finite elements

**Factoid 4.5 (Scott S4).** Finite elements `u` are compact, and every element is the
directed lub of the finite elements below it (algebraicity). The carrier identity
`x = ?{u | u subteq x}` is Factoid 3.6; here we package directedness, the lub form, and
compactness.
-/

```

```

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- `False = {}`. -/
theorem botElement_eq_closure_empty :
  sys.botElement = sys.closure ({} : Finset a) sys.con_empty := by
  refine le_antisymm ?_ ?_
  * intro a ha
    -- ha : Ent {bot} a; show Ent {} a
    exact sys.ent_trans sys.con_empty (sys.con_sing sys.bot)
      (fun y hy => by
        have : y = sys.bot := Finset.mem_singleton.mp hy
        subst this
        exact sys.ent_bot sys.con_empty)
    ha
  * intro a ha
    -- ha : Ent {} a; show Ent {bot} a
    exact sys.ent_trans (sys.con_sing sys.bot) sys.con_empty
      (fun _ hy => False.elim (Finset.notMem_empty _ hy)) ha

/-- Finite approximations of `x`: the set `{u | u in Con, u subseq x}`. -/
def finiteApproximants (x : sys.Element) : Set sys.Element :=
  {y | exists (u : Finset a) (hu : u in sys.Con), u subseq x.carrier /\ y = sys.
    closure u hu}

theorem nonempty_finiteApproximants (x : sys.Element) :
  (sys.finiteApproximants x).Nonempty := by
  refine <sys.botElement, {}, sys.con_empty, ?_, sys.botElement_eq_closure_empty>
  intro a ha
  exact False.elim (Finset.notMem_empty _ (Finset.mem_coe.1 ha))

theorem directed_finiteApproximants (x : sys.Element) :
  sys.IsDirected (sys.finiteApproximants x) := by
  intro y_1 y_2 hy_1 hy_2
  obtain <u, hu, huX, rfl> := hy_1
  obtain <v, hv, hvX, rfl> := hy_2
  obtain <w, hw, hwX, huw, hvw> := sys.closures_directed x hu hv huX hvX
  exact <sys.closure w hw, <w, hw, hwX, rfl>, huw, hvw>

/-- **Factoid 4.5 / algebraicity.** `x` is the directed lub of its finite approximants. -/
theorem eq_directedSup_finiteApproximants (x : sys.Element) :
  x = sys.directedSup (sys.finiteApproximants x)
  (sys.nonempty_finiteApproximants x) (sys.directed_finiteApproximants x) := by
  refine le_antisymm ?_ ?_
  * -- x <= directedSup: every token of x lies in some u subseq x
    intro a ha
    have : a in sys.approxUnion x := (sys.mem_element_iff_mem_closure x a).1 ha
    obtain <u, hu, huX, ha'> := this
    rw [sys.directedSup_carrier_eq_union]
    exact <sys.closure u hu, <u, hu, huX, rfl>, ha'>
  * -- directedSup <= x: each approxinant is <= x
    apply sys.directedSup_le
    intro y hy
    obtain <u, hu, huX, rfl> := hy
    exact sys.closure_le_element x hu huX

/-- Compactness of finite elements: if `u <= sqsup S` for directed nonempty `S`, then
`u <=` some member of `S`. -/

```



```

theorem compact_closure (S : Set sys.Element) (hne : S.Nonempty) (hdir : sys.IsDirected S)
  {u : Finset a} (hu : u in sys.Con)
  (hle : sys.closure u hu <= sys.directedSup S hne hdir) :
  exists z in S, sys.closure u hu <= z := by
  have huU : (u : Set a) subseq sys.familyUnionCarrier S := by
    intro a ha
    have : a in (sys.directedSup S hne hdir).carrier :=
      hle (sys.subset_closure hu (Finset.mem_coe.2 ha))
    rwa [sys.directedSup_carrier_eq_union S hne hdir] at this
  obtain <z, hz, huZ> := sys.exists_mem_of_subset_directedUnion S hne hdir u huU
  exact <z, hz, sys.closure_le_element z hu huZ>

end InfoSys

end Scott1982

```

A.19 Scott1982/Factoid46.lean

207 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Approximable
import Scott1982.Factoid45

/-!
# Factoid 4.6 -- Scott topology and continuous maps

**Factoid 4.6 (Scott S4).** Basic opens  $[u] = \{x \mid u \text{ subseq } x\}$  form a neighborhood basis;
 $|A|$  is  $T_0$ . Approximable maps induce Scott-continuous functions on elements, and
conversely every Scott-continuous map arises from an approximable relation
 $(u \text{ f } v \leftrightarrow v \text{ subseq } f(u))$ .
-/

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

/-- **Scott S4.** Basic open  $[u] = \{x \text{ in } |A| \mid u \text{ subseq } x\}$ . -/
def basicOpen (A : InfoSys a) (u : Finset a) : Set A.Element :=
  {x | u subseq x.carrier}

theorem mem_basicOpen {A : InfoSys a} {u : Finset a} {x : A.Element} :
  x in A.basicOpen u <-> u subseq x.carrier := Iff.rfl

/-- Intersection of basic opens:  $[u] \cap [v] = [u \cup v]$ . -/
theorem basicOpen_inter (A : InfoSys a) (u v : Finset a) :
  A.basicOpen u ∩ A.basicOpen v = A.basicOpen (u ∪ v) := by
  ext x
  constructor
  * intro <hu, hv> a ha

```

```

    rcases mem_funion.mp (Finset.mem_coe.1 ha) with h | h
    * exact hu (Finset.mem_coe.2 h)
    * exact hv (Finset.mem_coe.2 h)
  * intro huv
  refine <?_, ?_>
  * intro a ha
    exact huv (Finset.mem_coe.2 (mem_funion.mpr (Or.inl (Finset.mem_coe.1 ha))))
  * intro a ha
    exact huv (Finset.mem_coe.2 (mem_funion.mpr (Or.inr (Finset.mem_coe.1 ha))))

/-- **T_0:** elements with the same basic-open neighborhoods are equal. -/
theorem eq_of_basicOpen_eq (A : InfoSys a) {x y : A.Element}
  (h : forall u : Finset a, x in A.basicOpen u <-> y in A.basicOpen u) : x = y := by
  refine le_antisymm ?_ ?_
  * intro a ha
    have : x in A.basicOpen {a} := by
      intro b hb
      have : b = a := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
      subst this
      exact ha
    exact ((h {a}).1 this) (Finset.mem_coe.2 (Finset.mem_singleton_self a))
  * intro a ha
    have : y in A.basicOpen {a} := by
      intro b hb
      have : b = a := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
      subst this
      exact ha
    exact ((h {a}).2 this) (Finset.mem_coe.2 (Finset.mem_singleton_self a))

/-- Image of a nonempty set under a function is nonempty. -/
theorem nonempty_image {X Y : Type*} {S : Set X} (hne : S.Nonempty) (f : X -> Y) :
  (f '' S).Nonempty :=
  hne.image f

/-- Monotone images preserve directedness. -/
theorem directed_image {A : InfoSys a} {B : InfoSys b} {S : Set A.Element}
  (hdir : A.IsDirected S) (f : A.Element -> B.Element)
  (hmono : forall {x y}, x <= y -> f x <= f y) : B.IsDirected (f '' S) := by
  intro x y hx hy
  obtain <x_0, hx_0, rfl> := hx
  obtain <y_0, hy_0, rfl> := hy
  obtain <z_0, hz_0, hxz, hyz> := hdir x_0 y_0 hx_0 hy_0
  exact <f z_0, <z_0, hz_0, rfl>, hmono hxz, hmono hyz>

/-- Scott-continuous map of domains: monotone and preserves directed lubs. -/
structure ScottContinuous (A : InfoSys a) (B : InfoSys b) where
  toFun : A.Element -> B.Element
  mono' : forall {x y : A.Element}, x <= y -> toFun x <= toFun y
  map_directedSup :
    forall (S : Set A.Element) (hne : S.Nonempty) (hdir : A.IsDirected S),
      toFun (A.directedSup S hne hdir) =
        B.directedSup (toFun '' S) (nonempty_image hne toFun)
        (directed_image hdir toFun mono')

namespace ScottContinuous

variable {A : InfoSys a} {B : InfoSys b}

theorem mono (f : ScottContinuous A B) {x y : A.Element} (h : x <= y) :
  f.toFun x <= f.toFun y :=
  f.mono' h

end ScottContinuous

```

```

namespace ApproximableMap

variable {A : InfoSys a} {B : InfoSys b}

/-- Approximable maps preserve directed lubs (Scott continuity on elements). -/
theorem toElement_directedSup (f : ApproximableMap A B)
  (S : Set A.Element) (hne : S.Nonempty) (hdir : A.IsDirected S) :
  f.toElement (A.directedSup S hne hdir) =
    B.directedSup (f.toElement '' S) (nonempty_image hne f.toElement)
    (directed_image hdir f.toElement f.toElement_mono) := by
  refine le_antisymm ?_ ?_
  * intro Y <u, hu, hrel>
  have huU : (u : Set a) subseq A.familyUnionCarrier S := by
    intro a ha
    have : a in (A.directedSup S hne hdir).carrier := hu ha
    rwa [A.directedSup_carrier_eq_union S hne hdir] at this
  obtain <z, hz, huZ> := A.exists_mem_of_subset_directedUnion S hne hdir u huU
  have : Y in (f.toElement z).carrier := <u, huZ, hrel>
  rw [B.directedSup_carrier_eq_union]
  exact <f.toElement z, <z, hz, rfl>, this>
  * apply B.directedSup_le
  intro y hy
  obtain <x, hx, rfl> := hy
  exact f.toElement_mono (A.le_directedSup S hne hdir hx)

/-- **Factoid 4.6 ( -> ).** Every approximable map is Scott-continuous on elements. -/
def toScottContinuous (f : ApproximableMap A B) : ScottContinuous A B where
  toFun := f.toElement
  mono' := f.toElement_mono
  map_directedSup := f.toElement_directedSup

/-- Relation recovered from a Scott-continuous map: `u f v <-> v subseq g(u)`. -/
def ofScottContinuous (g : ScottContinuous A B) : ApproximableMap A B where
  rel u v := exists hu : u in A.Con, v in B.Con /\ v subseq (g.toFun (A.closure u hu
  )).carrier
  rel_dom := fun <hu, _, _> => hu
  rel_cod := fun <_, hv, _> => hv
  empty_rel := by
    refine <A.con_empty, B.con_empty, ?_>
    intro b hb
    exact False.elim (Finset.notMem_empty _ (Finset.mem_coe.1 hb))
  union_right := by
    intro u v v' <hu, hv, hsub> <_hu', hv', hsub'>
    refine <hu, ?_, ?_>
    * have hsubUV : (v U ' v') subseq (g.toFun (A.closure u hu)).carrier := by
      intro b hb
      rcases mem_funion.mp (Finset.mem_coe.1 hb) with h | h
      * exact hsub (Finset.mem_coe.2 h)
      * exact hsub' (Finset.mem_coe.2 h)
    exact (g.toFun (A.closure u hu)).consistent (v U ' v') hsubUV
  * intro b hb
  rcases mem_funion.mp (Finset.mem_coe.1 hb) with h | h
  * exact hsub (Finset.mem_coe.2 h)
  * exact hsub' (Finset.mem_coe.2 h)
  mono := by
    intro u u' v v' <hu, hv, hsub> hEntUU' hEntVV' hu' hv'
    refine <hu', hv', ?_>
    have hclos : A.closure u hu <= A.closure u' hu' :=
      A.closure_le_of_entSet hu hu' hEntUU'
    intro b hb
    have hEntb : B.Ent v b := hEntVV' b hb
    have hb_in : b in (g.toFun (A.closure u hu)).carrier :=

```

```

      (g.toFun (A.closure u hu)).closed v b hsub hEntb
    exact g.mono hclos hb_in

/-- **Factoid 4.6 ( <- ).** Scott-continuous maps recover as `toElement` of the induced
approximable relation. -/
theorem toElement_ofScottContinuous (g : ScottContinuous A B) (x : A.Element) :
  (ofScottContinuous g).toElement x = g.toFun x := by
  refine le_antisymm ?_ ?_
  * intro Y <u, hu, <huCon, _, hsub>>
    have : A.closure u huCon <= x := A.closure_le_element x huCon hu
    exact g.mono this (hsub (Finset.mem_coe.2 (Finset.mem_singleton_self Y)))
  * intro Y hY
    let S := A.finiteApproximants x
    have hne := A.nonempty_finiteApproximants x
    have hdir := A.directed_finiteApproximants x
    have hx : x = A.directedSup S hne hdir := A.eq_directedSup_finiteApproximants x
    have hdirImg := directed_image hdir g.toFun g.mono'
    have hneImg := nonempty_image hne g.toFun
    have hg : g.toFun x = B.directedSup (g.toFun '' S) hneImg hdirImg := by
      have h1 : g.toFun x = g.toFun (A.directedSup S hne hdir) := congrArg g.toFun hx
      have h2 : g.toFun (A.directedSup S hne hdir) =
        B.directedSup (g.toFun '' S) hneImg hdirImg :=
          g.map_directedSup S hne hdir
      exact h1.trans h2
    have hY' : Y in (B.directedSup (g.toFun '' S) hneImg hdirImg).carrier := by
      rw [ <- hg ]; exact hY
    have hY'' : Y in B.familyUnionCarrier (g.toFun '' S) := by
      rwa [ <- B.directedSup_carrier_eq_union (g.toFun '' S) hneImg hdirImg ]
    obtain <z, hz, hYz> := hY''
    obtain <y, hy, rfl> := hz
    obtain <u, hu, huX, rfl> := hy
    refine <u, huX, <hu, B.con_sing Y, ?_>>
    intro b hb
    have : b = Y := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
    subst this
    exact hYz

end ApproximableMap

end InfoSys

end Scott1982

```

A.20 Scott1982/Approximable.lean

144 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Constructive
import Scott1982.Factoid35

/-!
# Approximable mappings -- Definitions 5.1 and 5.2

```

Adapted from the PRG-19 approximable-map pattern, rewritten for Scott 1982
information systems (relations on $\text{Con} \times \text{Con}$).
-/

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

```
/-- **Definition 5.1 (Scott 1982).** Approximable mapping between information systems. -/
structure ApproximableMap (A : InfoSys a) (B : InfoSys b) where
  rel : Finset a -> Finset b -> Prop
  rel_dom : forall {u v}, rel u v -> u in A.Con
  rel_cod : forall {u v}, rel u v -> v in B.Con
  empty_rel : rel {} {}
  union_right : forall {u v v'}, rel u v -> rel u v' -> rel u (v U ' v')
  mono : forall {u u' v v'},
    rel u v -> A.EntSet u' u -> B.EntSet v v' -> u' in A.Con -> v' in B.Con ->
    rel u' v'
```

namespace ApproximableMap

variable {A : InfoSys a} {B : InfoSys b}

```
theorem ext {f g : ApproximableMap A B} (h : forall u v, f.rel u v <=> g.rel u v) : f = g :=
  by
  obtain <rf, _, _, _, _, _> := f
  obtain <rg, _, _, _, _, _> := g
  have : rf = rg := by
    funext u v
    exact propext (h u v)
  subst this
  rfl
```

```
private theorem singleton_funion_eq (a : b) (s : Finset b) :
  ({a} : Finset b) U ' s = insert a s := by
  ext x
  constructor
  * intro hx
    rcases mem_funion.mp hx with ha | hs
    * exact Finset.mem_insert.mpr (Or.inl (Finset.mem_singleton.mp ha))
    * exact Finset.mem_insert_of_mem hs
  * intro hx
    rcases Finset.mem_insert.mp hx with ha | hs
    * exact mem_funion.mpr (Or.inl (Finset.mem_singleton.mpr ha))
    * exact mem_funion.mpr (Or.inr hs)
```

```
/-- Given `Y subseteq f(x).carrier`, produce `u subseteq x` with `u f Y`. -/
theorem exists_rel_of_subset_image (f : ApproximableMap A B) (x : A.Element)
  (Y : Finset b)
  (hY : Y subseteq ({Y : b | exists u : Finset a, u subseteq x.carrier /\ f.rel u {Y}}
    : Set b)) :
  exists u : Finset a, u subseteq x.carrier /\ f.rel u Y := by
  induction Y using Finset.induction_on with
  | empty =>
    refine <{}, ?, ?_>
  * intro a ha
    exact False.elim (Finset.notMem_empty a (Finset.mem_coe.1 ha))
```

```

* exact f.mono f.empty_rel (A.entSet_empty {}) (B.entSet_empty {}) A.con_empty B.con_empty
| insert a s _ha ih =>
have ha' : exists u : Finset a, u subseq x.carrier /\ f.rel u {a} :=
  hY (Finset.mem_coe.2 (Finset.mem_insert_self a s))
have hs' :
  s subseq ({Y : b | exists u : Finset a, u subseq x.carrier /\ f.rel u {Y}} :
Set b) := by
  intro b hb
  exact hY (Finset.mem_coe.2 (Finset.mem_insert_of_mem (Finset.mem_coe.1 hb)))
obtain <ua, hua, hrela> := ha'
obtain <us, hus, hrels> := ih hs'
refine <ua U ' us, ?_, ?_>
* intro z hz
have hz' : z in ua U ' us := Finset.mem_coe.1 hz
rcases mem_funion.mp hz' with h | h
* exact hua (Finset.mem_coe.2 h)
* exact hus (Finset.mem_coe.2 h)
* have hUcon : ua U ' us in A.Con :=
  x.consistent (ua U ' us) fun z hz => by
    rcases mem_funion.mp hz with h | h
    * exact hua (Finset.mem_coe.2 h)
    * exact hus (Finset.mem_coe.2 h)
have hEnt_ua : A.EntSet (ua U ' us) ua :=
  fun y hy => A.ent_refl hUcon (subset_funion_left ua us hy)
have hEnt_us : A.EntSet (ua U ' us) us :=
  fun y hy => A.ent_refl hUcon (subset_funion_right ua us hy)
have h1 : f.rel (ua U ' us) {a} :=
  f.mono hrela hEnt_ua
  (fun z hz => by
    rw [Finset.mem_singleton] at hz
    rw [hz]
    exact B.ent_refl (B.con_sing a) (Finset.mem_singleton_self a))
have h2 : f.rel (ua U ' us) s :=
  f.mono hrels hEnt_us (proposition_2_3_iii B (f.rel_cod hrels))
  hUcon (f.rel_cod hrels)
have hU : f.rel (ua U ' us) ({a} U ' s) := f.union_right h1 h2
simpa [singleton_funion_eq] using hU

/-- **Definition 5.2.** Image of an element under an approximable mapping. -/
def toElement (f : ApproximableMap A B) (x : A.Element) : B.Element where
  carrier := {Y | exists u : Finset a, u subseq x.carrier /\ f.rel u {Y}}
  consistent := by
    intro Y hY
    obtain <u, _, hrel> := exists_rel_of_subset_image f x Y hY
    exact f.rel_cod hrel
closed := by
  intro Y a hY hEnt
  obtain <u, hu, hrel> := exists_rel_of_subset_image f x Y hY
  have harel : f.rel u {a} :=
    f.mono hrel (proposition_2_3_iii A (f.rel_dom hrel))
    (fun z hz => by
      rw [Finset.mem_singleton] at hz
      rw [hz]
      exact hEnt)
    (f.rel_dom hrel) (B.con_sing a)
  exact <u, hu, harel>

@[simp] theorem mem_toElement (f : ApproximableMap A B) (x : A.Element) {Y : b} :
  Y in (f.toElement x).carrier <=> exists u : Finset a, u subseq x.carrier /\ f.
  rel u {Y} :=
  Iff.rfl

```

```

theorem toElement_mono (f : ApproximableMap A B) {x y : A.Element} (hxy : x <= y) :
  f.toElement x <= f.toElement y := by
  intro Y <u, hu, hrel>
  exact <u, fun z hz => hxy (hu hz), hrel>

end ApproximableMap

end InfoSys

end Scott1982

```

A.21 Scott1982/Proposition53.lean

123 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Approximable

/-!
# Proposition 5.3 -- images, order, and the closure bridge

**Scott 1982, Proposition 5.3.** Remaining clauses after Def 5.2 / `toElement`:
singleton reduction (Scott's remark before Def 5.2); (v) the bridge
`u f v <-> v subseq f(u)`; (iii) pointwise order; (ii) extensionality via elements.
(i) and (iv) are already `toElement` / `toElement_mono` in `Approximable.lean`.
-/

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys
namespace ApproximableMap

variable {a b : Type*} [DecidableEq a] [DecidableEq b]
variable {A : InfoSys a} {B : InfoSys b}

private theorem singleton_funion_eq (a : b) (s : Finset b) :
  ({a} : Finset b) ∪ s = insert a s := by
  ext x
  constructor
  * intro hx
    rcases mem_funion.mp hx with ha | hs
    * exact Finset.mem_insert.mpr (Or.inl (Finset.mem_singleton.mp ha))
    * exact Finset.mem_insert_of_mem hs
  * intro hx
    rcases Finset.mem_insert.mp hx with ha | hs
    * exact mem_funion.mpr (Or.inl (Finset.mem_singleton.mpr ha))
    * exact mem_funion.mpr (Or.inr hs)

/-- **Singleton reduction** (Scott, before Def 5.2). -/
theorem rel_iff_forall_singleton (f : ApproximableMap A B)
  {u : Finset a} {v : Finset b} (hu : u in A.Con) (hv : v in B.Con) :
  f.rel u v <-> forall Y in v, f.rel u ({Y} : Finset b) := by

```

```

constructor
* intro hrel Y hY
  exact f.mono hrel (proposition_2_3_iii A hu)
    (fun Z hZ => by
      rw [Finset.mem_singleton] at hZ
      subst hZ
      exact B.ent_refl hv hY)
  hu (B.con_sing Y)
* intro hsing
  induction v using Finset.induction_on with
  | empty =>
    exact f.mono f.empty_rel (A.entSet_empty u) (B.entSet_empty {}) hu B.con_empty
  | insert a s ha ih =>
    have hv' : insert a s in B.Con := hv
    have hsCon : s in B.Con := B.con_subset hv' (Finset.subset_insert a s)
    have h1 : f.rel u {a} := hsing a (Finset.mem_insert_self a s)
    have h2 : f.rel u s :=
      ih hsCon fun Y hY => hsing Y (Finset.mem_insert_of_mem hY)
    have hU : f.rel u ({a} ∪ s) := f.union_right h1 h2
    simp [singleton_funion_eq] using hU

/-- **Proposition 5.3(v).** `u f v` iff `v subseq f(u)` . -/
theorem rel_iff_closure_le (f : ApproximableMap A B)
  {u : Finset a} {v : Finset b} (hu : u in A.Con) (hv : v in B.Con) :
  f.rel u v <=> B.closure v hv <= f.toElement (A.closure u hu) := by
  constructor
* intro hrel Y hY
  refine <u, A.subset_closure hu, ?_>
  exact f.mono hrel (proposition_2_3_iii A hu)
    (fun Z hZ => by
      rw [Finset.mem_singleton] at hZ
      subst hZ
      exact hY)
  hu (B.con_sing Y)
* intro hsub
  have hv_sub : v subseq (f.toElement (A.closure u hu)).carrier :=
    fun y hy => hsub (B.subset_closure hv hy)
  obtain <u', hu', hrel'> := exists_rel_of_subset_image f (A.closure u hu) v hv_sub
  have hEnt : A.EntSet u' := fun y hy => hu' (Finset.mem_coe.2 hy)
  exact f.mono hrel' hEnt (proposition_2_3_iii B hv) hu hv

/-- Relation inclusion of approximable maps. -/
def le (f g : ApproximableMap A B) : Prop := forall ?u v?, f.rel u v -> g.rel u v

/-- **Proposition 5.3(iii).** `f subseq g` iff `f(x) subseq g(x)` for all elements `x`. -/
theorem le_iff_toElement_le (f g : ApproximableMap A B) :
  le f g <=> forall x : A.Element, f.toElement x <= g.toElement x := by
  constructor
* intro hfg x Y <u, hu, hrel>
  exact <u, hu, hfg hrel>
* intro hpoint u v hrel
  have hu : u in A.Con := f.rel_dom hrel
  have hv : v in B.Con := f.rel_cod hrel
  have hbridge : B.closure v hv <= f.toElement (A.closure u hu) :=
    (f.rel_iff_closure_le hu hv).1 hrel
  have hsub : B.closure v hv <= g.toElement (A.closure u hu) :=
    le_trans hbridge (hpoint (A.closure u hu))
  exact (g.rel_iff_closure_le hu hv).2 hsub

/-- **Proposition 5.3(ii).** `f = g` iff `f(x) = g(x)` for all elements `x`. -/
theorem ext_iff_toElement (f g : ApproximableMap A B) :
  f = g <=> forall x : A.Element, f.toElement x = g.toElement x := by

```



```

    constructor
    * intro h x
      rw [h]
    * intro hpoint
      refine ApproximableMap.ext fun u v => ?_
    constructor
    * intro hrel
      exact (le_iff_toElement_le f g).2 (fun x => (hpoint x) |> le_rfl) hrel
    * intro hrel
      exact (le_iff_toElement_le g f).2 (fun x => (hpoint x).symm |> le_rfl) hrel

end ApproximableMap

end InfoSys

end Scott1982

```

A.22 Scott1982/Proposition54.lean

63 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Approximable

/-!
# Proposition 5.4 -- the identity approximable mapping

**Scott 1982, Proposition 5.4.** `u I_A v <-> u |- _A v`, and `I_A(x) = x`.
-/

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (A : InfoSys a)

/-- **Proposition 5.4(i).** Identity approximable mapping given by entailment. -/
def idMap : ApproximableMap A A where
  rel u v := u in A.Con /\ v in A.Con /\ A.EntSet u v
  rel_dom h := h.1
  rel_cod h := h.2.1
  empty_rel := <A.con_empty, A.con_empty, A.entSet_empty {}>
  union_right := by
    rintro u v v' <hu, _hv, huv> <_, _hv', huv'>
    have hEnt : A.EntSet u (v U ' v') := proposition_2_3_vi (sys := A) huv huv'
    have hBig : u U ' (v U ' v') in A.Con :=
      proposition_2_3_ii (sys := A) (u := u) (v := v U ' v') hu hEnt
    have hCon : v U ' v' in A.Con :=
      A.con_subset hBig (subset_funion_right u (v U ' v'))
    exact <hu, hCon, hEnt>
  mono := by
    rintro u u' v v' <hu, hv, huv> hEntu hEntv hu' hv'

```

```

    refine <hu', hv', ?_>
    exact proposition_2_3_iv A hu' hv
      (proposition_2_3_iv A hu' hu hEntu huv) hEntv

/-- **Proposition 5.4(ii).** `I_A(x) = x`. -/
theorem idMap_toElement (x : A.Element) : (idMap A).toElement x = x := by
  apply le_antisymm
  * intro Y <u, hu, hrel>
    -- hrel.2.2 : EntSet u {Y}, so Ent u Y
    have hEnt : A.Ent u Y := hrel.2.2 Y (Finset.mem_singleton_self Y)
    exact x.closed u Y hu hEnt
  * intro Y hY
    refine <{Y}, ?_, ?_>
    * intro z hz
      have hz' : z in ({Y} : Finset a) := Finset.mem_coe.1 hz
      rw [Finset.mem_singleton] at hz'
      rw [hz']
      exact hY
    * refine <A.con_sing Y, A.con_sing Y, ?_>
      exact proposition_2_3_iii A (A.con_sing Y)

end InfoSys

end Scott1982

```

A.23 Scott1982/Proposition55.lean

82 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Proposition54

/-!
# Proposition 5.5 -- composition of approximable mappings

**Scott 1982, Proposition 5.5.** `u (g comp f) w <-> exists v, u f v /\ v g w`, and
`(g comp f)(x) = g(f(x))`.
-/

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys
namespace ApproximableMap

variable {a b ? : Type*} [DecidableEq a] [DecidableEq b] [DecidableEq ?]
variable {A : InfoSys a} {B : InfoSys b} {C : InfoSys ?}

/-- **Proposition 5.5(i).** Composition of approximable mappings. -/
def comp (g : ApproximableMap B C) (f : ApproximableMap A B) : ApproximableMap A C where
  rel u w := exists v, f.rel u v /\ g.rel v w
  rel_dom := fun <_, hf, _> => f.rel_dom hf
  rel_cod := fun <_, _, hg> => g.rel_cod hg

```

```

empty_rel := <{}, f.empty_rel, g.empty_rel>
union_right := by
  rintro u w w' <v, hf, hg> <v', hf', hg'>
  -- Need common middle: use union_right on f to get u f (v U ' v'), then g.
  have hfU : f.rel u (v U ' v') := f.union_right hf hf'
  have hvU : v U ' v' in B.Con := f.rel_cod hfU
  have hEntv : B.EntSet (v U ' v') v :=
    fun y hy => B.ent_refl hvU (subset_funion_left v v' hy)
  have hEntv' : B.EntSet (v U ' v') v' :=
    fun y hy => B.ent_refl hvU (subset_funion_right v v' hy)
  have hg1 : g.rel (v U ' v') w :=
    g.mono hg hEntv (proposition_2_3_iii C (g.rel_cod hg)) hvU (g.rel_cod hg)
  have hg2 : g.rel (v U ' v') w' :=
    g.mono hg' hEntv' (proposition_2_3_iii C (g.rel_cod hg')) hvU (g.rel_cod hg')
  exact <v U ' v', hfU, g.union_right hg1 hg2>
mono := by
  rintro u u' w w' <v, hf, hg> hEntu hEntw hu' hw'
  refine <v, ?, ?_>
  * exact f.mono hf hEntu (proposition_2_3_iii B (f.rel_cod hf)) hu' (f.rel_cod hf)
  * exact g.mono hg (proposition_2_3_iii B (f.rel_cod hf)) hEntw (f.rel_cod hf) hw'

/-- **Proposition 5.5(ii).** `(g comp f)(x) = g(f(x))`. -/
theorem comp_toElement (g : ApproximableMap B C) (f : ApproximableMap A B) (x : A.Element) :
  (comp g f).toElement x = g.toElement (f.toElement x) := by
  apply le_antisymm
  * intro Z <u, hu, <v, hf, hg>>
    -- Z in g(f(x)): need exists v' subseq f(x), v' g {Z}. Have v with u f v and v g {Z}
    }?
    -- hg : g.rel v w where w should be {Z}. Our rel is to w = {Z} from toElement def.
    -- Actually <u, hu, hrel> where hrel : exists v, f.rel u v /\ g.rel v {Z}
    refine <v, ?, hg>
    intro b hb
    -- b in v => need b in f(x). Since u f v and u subseq x, use: u f {b} by mono, so
    b in f(x).
    have hb' : b in v := Finset.mem_coe.1 hb
    have hsing : f.rel u {b} :=
      f.mono hf (proposition_2_3_iii A (f.rel_dom hf))
      (fun z hz => by
        rw [Finset.mem_singleton] at hz; subst hz
        exact B.ent_refl (f.rel_cod hf) hb')
      (f.rel_dom hf) (B.con_sing b)
    exact <u, hu, hsing>
  * intro Z <v, hv, hg>
    -- Z in g(f(x)) with v subseq f(x), v g {Z}.
    -- Need u subseq x with u (g comp f) {Z}. From v subseq f(x), get u with u f v (
    exists_rel).
    obtain <u, hu, hf> := exists_rel_of_subset_image f x v hv
    exact <u, hu, <v, hf, hg>>

end ApproximableMap

end InfoSys

end Scott1982

```

A.24 Scott1982/Proposition56.lean

89 lines.

[Lean 4 source](#)

/-

Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: <https://github.com/catskillsresearch/scott1982>
-/-

```
import Scott1982.Proposition53
import Scott1982.Proposition55
```

```
/-!
```

```
# Proposition 5.6 -- constant approximable mappings
```

```
**Scott 1982, Proposition 5.6.** For a fixed element `b` in  $|B|$  there is a unique
approximable map  $\text{const } b : A \rightarrow B$  with  $(\text{const } b)(x) = b$  for all  $x$ . Moreover
 $f \circ \text{const } b = \text{const } (f(b))$  and  $(\text{const } b) \circ g = \text{const } b$ .
-/-
```

```
namespace Scott1982
```

```
open Scott1982.Constructive
```

```
namespace InfoSys
```

```
namespace ApproximableMap
```

```
variable {a b ? ? : Type*} [DecidableEq a] [DecidableEq b] [DecidableEq ?] [DecidableEq ?]
variable {A : InfoSys a} {B : InfoSys b} {C : InfoSys ?} {D : InfoSys ?}
```

```
/-- **Proposition 5.6.** Constant map:  $u(\text{const } b) v$  means  $v \subseteq b$ . -/
```

```
def constMap (b : B.Element) : ApproximableMap A B where
  rel u v := u in A.Con /\ (v : Set b) ⊆ b.carrier
  rel_dom h := h.1
  rel_cod h := b.consistent _ h.2
  empty_rel := <A.con_empty, fun _ h => False.elim (Finset.notMem_empty _ (Finset.mem_coe.1 h))
  >
  union_right := by
    rintro u v v' <hu, hv> <_, hv'>
    refine <hu, ?_>
    intro y hy
    rcases mem_funion.mp (Finset.mem_coe.1 hy) with h | h
    * exact hv (Finset.mem_coe.2 h)
    * exact hv' (Finset.mem_coe.2 h)
  mono := by
    rintro u u' v v' <_hu, hv> _hEntu hEntv hu' _hv'
    refine <hu', ?_>
    intro y hy
    have hEnt : B.Ent v y := hEntv y hy
    exact b.closed v y hv hEnt
```

```
/-- **Proposition 5.6(i).**  $(\text{const } b)(x) = b$ . -/
```

```
theorem constMap_toElement (b : B.Element) (x : A.Element) :
  (constMap (A := A) b).toElement x = b := by
  apply le_antisymm
  * intro Y <_u, _hu, <_, hsub>>
    exact hsub (Finset.mem_coe.2 (Finset.mem_singleton_self Y))
  * intro Y hY
    refine <{ }, ?_, ?_>
    * intro a ha
      exact False.elim (Finset.notMem_empty a (Finset.mem_coe.1 ha))
    * refine <A.con_empty, ?_>
      intro Z hZ
      have hZ' : Z in ({Y} : Finset b) := Finset.mem_coe.1 hZ
      rw [Finset.mem_singleton] at hZ'
      subst hZ'
```

```

    exact hY

/-- Uniqueness: any map with constant value `b` equals `const b`. -/
theorem constMap_unique (b : B.Element) (f : ApproximableMap A B)
  (h : forall x : A.Element, f.toElement x = b) :
  f = constMap (A := A) b :=
  (ext_iff_toElement f (constMap (A := A) b)).2 fun x => by
    rw [h x, constMap_toElement]

/-- **Proposition 5.6(ii).** `f comp (const b) = const (f(b))`. -/
theorem comp_constMap (f : ApproximableMap B C) (b : B.Element) :
  comp f (constMap (A := A) b) = constMap (A := A) (f.toElement b) :=
  (ext_iff_toElement _ _).2 fun x => by
    rw [comp_toElement, constMap_toElement, constMap_toElement]

/-- **Proposition 5.6(iii).** `(const b) comp g = const b`. -/
theorem constMap_comp (b : B.Element) (g : ApproximableMap D A) :
  comp (constMap (A := A) b) g = constMap (A := D) b :=
  (ext_iff_toElement _ _).2 fun x => by
    rw [comp_toElement, constMap_toElement, constMap_toElement]

end ApproximableMap

end InfoSys

end Scott1982

```

A.25 Scott1982/Product.lean

367 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Constructive
import Scott1982.Proposition23

/-!
# Product of information systems -- Definition 6.1

**Scott 1982, Definition 6.1.** Tokens of `A x B` are pairs of the form
`(X, Delta_B)` or `(Delta_A, Y)`, with `Delta = (Delta_A, Delta_B)`. Consistency and entailment
act
independently on the two projections `fst u` and `snd u`.
-/

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

/-- Scott 6.1(i): a product token is `(X, Delta_B)` or `(Delta_A, Y)`. -/
def IsProdToken (A : InfoSys a) (B : InfoSys b) (p : a x b) : Prop :=

```

```

p.1 = A.bot  \ /  p.2 = B.bot

instance (A : InfoSys a) (B : InfoSys b) (p : a x b) : Decidable (IsProdToken A B p) :=
  if h1 : p.1 = A.bot then isTrue (Or.inl h1)
  else if h2 : p.2 = B.bot then isTrue (Or.inr h2)
  else isFalse fun h => h.elim h1 h2

/-- Token type of the product system `A x B`. -/
def ProdToken (A : InfoSys a) (B : InfoSys b) : Type _ :=
  {p : a x b // IsProdToken A B p}

instance (A : InfoSys a) (B : InfoSys b) : DecidableEq (ProdToken A B) :=
  Subtype.instDecidableEq

variable (A : InfoSys a) (B : InfoSys b)

/-- Product bottom `Delta_{A x B} = (Delta_A, Delta_B)`. -/
def prodBot : ProdToken A B :=
  <(A.bot, B.bot), Or.inl rfl>

private instance instLeftCommutativeFstInsert :
  LeftCommutative fun p : ProdToken A B => (insert p.val.1 : Finset a -> Finset a) :=
  <fun p q s => insert_comm' p.val.1 q.val.1 s>

private instance instLeftCommutativeSndInsert :
  LeftCommutative fun p : ProdToken A B => (insert p.val.2 : Finset b -> Finset b) :=
  <fun p q s => insert_comm' p.val.2 q.val.2 s>

/-- Left projection of a finite set of product tokens (Scott 6.1). -/
def fstFinset (u : Finset (ProdToken A B)) : Finset a :=
  Multiset.foldr (fun p : ProdToken A B => insert p.val.1) ({}) : Finset a
  (u.filter (fun p => p.val.2 = B.bot)).1

/-- Right projection of a finite set of product tokens (Scott 6.1). -/
def sndFinset (u : Finset (ProdToken A B)) : Finset b :=
  Multiset.foldr (fun p : ProdToken A B => insert p.val.2) ({}) : Finset b
  (u.filter (fun p => p.val.1 = A.bot)).1

private theorem mem_foldr_fst (s : Multiset (ProdToken A B)) (x : a) :
  x in Multiset.foldr (fun p : ProdToken A B => insert p.val.1) ({}) : Finset a s <->
  exists p in s, p.val.1 = x := by
  refine Multiset.induction_on s ?_ ?_
  * constructor
  * intro hx
    exact False.elim (Finset.notMem_empty x hx)
  * rintro <p, hp, _>
    exact False.elim (by cases hp)
  * intro p t ih
    simp only [Multiset.foldr_cons, Finset.mem_insert, ih, Multiset.mem_cons]
    constructor
  * rintro (hx | <q, hq, hq1>)
    * exact <p, Or.inl rfl, hx.symm>
    * exact <q, Or.inr hq, hq1>
  * rintro <q, hq, hq1>
    rcases hq with rfl | hq
    * exact Or.inl hq1.symm
    * exact Or.inr <q, hq, hq1>

private theorem mem_foldr_snd (s : Multiset (ProdToken A B)) (y : b) :
  y in Multiset.foldr (fun p : ProdToken A B => insert p.val.2) ({}) : Finset b s <->
  exists p in s, p.val.2 = y := by
  refine Multiset.induction_on s ?_ ?_
  * constructor

```

```

* intro hy
  exact False.elim (Finset.notMem_empty y hy)
* rintro <p, hp, _>
  exact False.elim (by cases hp)
* intro p t ih
  simp only [Multiset.foldr_cons, Finset.mem_insert, ih, Multiset.mem_cons]
  constructor
* rintro (hy | <q, hq, hq2>)
  * exact <p, Or.inl rfl, hy.symm>
  * exact <q, Or.inr hq, hq2>
* rintro <q, hq, hq2>
  rcases hq with rfl | hq
  * exact Or.inl hq2.symm
  * exact Or.inr <q, hq, hq2>

theorem mem_fstFinset {u : Finset (ProdToken A B)} {x : a} :
  x in fstFinset A B u <-> exists p in u, p.val.2 = B.bot /\ p.val.1 = x := by
  unfold fstFinset
  rw [mem_foldr_fst]
  constructor
* rintro <p, hp, hx>
  have hp' := Finset.mem_filter.mp hp
  exact <p, hp'.1, hp'.2, hx>
* rintro <p, hp, hbot, hx>
  exact <p, Finset.mem_filter.mpr <hp, hbot>, hx>

theorem mem_sndFinset {u : Finset (ProdToken A B)} {y : b} :
  y in sndFinset A B u <-> exists p in u, p.val.1 = A.bot /\ p.val.2 = y := by
  unfold sndFinset
  rw [mem_foldr_snd]
  constructor
* rintro <p, hp, hy>
  have hp' := Finset.mem_filter.mp hp
  exact <p, hp'.1, hp'.2, hy>
* rintro <p, hp, hbot, hy>
  exact <p, Finset.mem_filter.mpr <hp, hbot>, hy>

theorem fstFinset_empty : fstFinset A B ({ } : Finset (ProdToken A B)) = { } := by
  ext x
  constructor
* intro hx
  rcases (mem_fstFinset A B).1 hx with <_, hp, _, _>
  exact False.elim (Finset.notMem_empty _ hp)
* intro hx
  exact False.elim (Finset.notMem_empty x hx)

theorem sndFinset_empty : sndFinset A B ({ } : Finset (ProdToken A B)) = { } := by
  ext y
  constructor
* intro hy
  rcases (mem_sndFinset A B).1 hy with <_, hp, _, _>
  exact False.elim (Finset.notMem_empty _ hp)
* intro hy
  exact False.elim (Finset.notMem_empty y hy)

theorem fstFinset_mono {u v : Finset (ProdToken A B)} (h : v subseq u) :
  fstFinset A B v subseq fstFinset A B u := by
  intro x hx
  rcases (mem_fstFinset A B).1 hx with <p, hp, hbot, hx'>
  exact (mem_fstFinset A B).2 <p, h hp, hbot, hx'>

theorem sndFinset_mono {u v : Finset (ProdToken A B)} (h : v subseq u) :
  sndFinset A B v subseq sndFinset A B u := by

```

```

intro y hy
rcases (mem_sndFinset A B).1 hy with <p, hp, hbot, hy'>
exact (mem_sndFinset A B).2 <p, h hp, hbot, hy'>

theorem fstFinset_insert_left (u : Finset (ProdToken A B)) (p : ProdToken A B)
  (hp : p.val.2 = B.bot) :
  fstFinset A B (insert p u) = insert p.val.1 (fstFinset A B u) := by
  ext x
  constructor
  * intro hx
    rcases (mem_fstFinset A B).1 hx with <q, hq, hbot, hx'>
    rcases Finset.mem_insert.mp hq with rfl | hq
    * exact Finset.mem_insert.mpr (Or.inl hx'.symm)
    * exact Finset.mem_insert.mpr (Or.inr ((mem_fstFinset A B).2 <q, hq, hbot, hx'>))
  * intro hx
    rcases Finset.mem_insert.mp hx with hx' | hx'
    * exact (mem_fstFinset A B).2 <p, Finset.mem_insert_self p u, hp, hx'.symm>
    * rcases (mem_fstFinset A B).1 hx' with <q, hq, hbot, hxq>
      exact (mem_fstFinset A B).2 <q, Finset.mem_insert_of_mem hq, hbot, hxq>

theorem fstFinset_insert_not_left (u : Finset (ProdToken A B)) (p : ProdToken A B)
  (hp : p.val.2 != B.bot) :
  fstFinset A B (insert p u) = fstFinset A B u := by
  ext x
  constructor
  * intro hx
    rcases (mem_fstFinset A B).1 hx with <q, hq, hbot, hx'>
    rcases Finset.mem_insert.mp hq with rfl | hq
    * exact False.elim (hp hbot)
    * exact (mem_fstFinset A B).2 <q, hq, hbot, hx'>
  * intro hx
    rcases (mem_fstFinset A B).1 hx with <q, hq, hbot, hx'>
    exact (mem_fstFinset A B).2 <q, Finset.mem_insert_of_mem hq, hbot, hx'>

theorem sndFinset_insert_right (u : Finset (ProdToken A B)) (p : ProdToken A B)
  (hp : p.val.1 = A.bot) :
  sndFinset A B (insert p u) = insert p.val.2 (sndFinset A B u) := by
  ext y
  constructor
  * intro hy
    rcases (mem_sndFinset A B).1 hy with <q, hq, hbot, hy'>
    rcases Finset.mem_insert.mp hq with rfl | hq
    * exact Finset.mem_insert.mpr (Or.inl hy'.symm)
    * exact Finset.mem_insert.mpr (Or.inr ((mem_sndFinset A B).2 <q, hq, hbot, hy'>))
  * intro hy
    rcases Finset.mem_insert.mp hy with hy' | hy'
    * exact (mem_sndFinset A B).2 <p, Finset.mem_insert_self p u, hp, hy'.symm>
    * rcases (mem_sndFinset A B).1 hy' with <q, hq, hbot, hyq>
      exact (mem_sndFinset A B).2 <q, Finset.mem_insert_of_mem hq, hbot, hyq>

theorem sndFinset_insert_not_right (u : Finset (ProdToken A B)) (p : ProdToken A B)
  (hp : p.val.1 != A.bot) :
  sndFinset A B (insert p u) = sndFinset A B u := by
  ext y
  constructor
  * intro hy
    rcases (mem_sndFinset A B).1 hy with <q, hq, hbot, hy'>
    rcases Finset.mem_insert.mp hq with rfl | hq
    * exact False.elim (hp hbot)
    * exact (mem_sndFinset A B).2 <q, hq, hbot, hy'>
  * intro hy
    rcases (mem_sndFinset A B).1 hy with <q, hq, hbot, hy'>
    exact (mem_sndFinset A B).2 <q, Finset.mem_insert_of_mem hq, hbot, hy'>

```



```

theorem fstFinset_singleton_left (p : ProdToken A B) (hp : p.val.2 = B.bot) :
  fstFinset A B {p} = {p.val.1} := by
  rw [show ({p} : Finset _) = insert p {} from rfl, fstFinset_insert_left A B {} p hp,
    fstFinset_empty]
  rfl

theorem sndFinset_singleton_right (p : ProdToken A B) (hp : p.val.1 = A.bot) :
  sndFinset A B {p} = {p.val.2} := by
  rw [show ({p} : Finset _) = insert p {} from rfl, sndFinset_insert_right A B {} p hp,
    sndFinset_empty]
  rfl

theorem fstFinset_funion (u v : Finset (ProdToken A B)) :
  fstFinset A B (u U ' v) = fstFinset A B u U ' fstFinset A B v := by
  ext x
  constructor
  * intro hx
    rcases (mem_fstFinset A B).1 hx with <p, hp, hbot, hx'>
    rcases mem_funion.mpr hp with hp | hp
    * exact mem_funion.mpr (Or.inl ((mem_fstFinset A B).2 <p, hp, hbot, hx'>))
    * exact mem_funion.mpr (Or.inr ((mem_fstFinset A B).2 <p, hp, hbot, hx'>))
  * intro hx
    rcases mem_funion.mpr hx with hx | hx
    * rcases (mem_fstFinset A B).1 hx with <p, hp, hbot, hx'>
      exact (mem_fstFinset A B).2 <p, mem_funion.mpr (Or.inl hp), hbot, hx'>
    * rcases (mem_fstFinset A B).1 hx with <p, hp, hbot, hx'>
      exact (mem_fstFinset A B).2 <p, mem_funion.mpr (Or.inr hp), hbot, hx'>

theorem sndFinset_funion (u v : Finset (ProdToken A B)) :
  sndFinset A B (u U ' v) = sndFinset A B u U ' sndFinset A B v := by
  ext y
  constructor
  * intro hy
    rcases (mem_sndFinset A B).1 hy with <p, hp, hbot, hy'>
    rcases mem_funion.mpr hp with hp | hp
    * exact mem_funion.mpr (Or.inl ((mem_sndFinset A B).2 <p, hp, hbot, hy'>))
    * exact mem_funion.mpr (Or.inr ((mem_sndFinset A B).2 <p, hp, hbot, hy'>))
  * intro hy
    rcases mem_funion.mpr hy with hy | hy
    * rcases (mem_sndFinset A B).1 hy with <p, hp, hbot, hy'>
      exact (mem_sndFinset A B).2 <p, mem_funion.mpr (Or.inl hp), hbot, hy'>
    * rcases (mem_sndFinset A B).1 hy with <p, hp, hbot, hy'>
      exact (mem_sndFinset A B).2 <p, mem_funion.mpr (Or.inr hp), hbot, hy'>

/-- Consistency for the product system (Scott 6.1(iii)). -/
def ProdCon (u : Finset (ProdToken A B)) : Prop :=
  fstFinset A B u in A.Con /\ sndFinset A B u in B.Con

/-- Entailment for the product system (Scott 6.1(iv'), (iv')).
Includes `ProdCon` so both projections stay available for `ent_con`. -/
def ProdEnt (u : Finset (ProdToken A B)) (p : ProdToken A B) : Prop :=
  ProdCon A B u /\
  (p.val.2 = B.bot -> A.Ent (fstFinset A B u) p.val.1) /\
  (p.val.1 = A.bot -> B.Ent (sndFinset A B u) p.val.2)

/-- **Definition 6.1.** The product information system `A x B`. -/
def productSystem : InfoSys (ProdToken A B) where
  bot := prodBot A B
  Con := {u | ProdCon A B u}
  Ent := ProdEnt A B
  con_subset := by
    intro u v hu hv

```

```

exact <A.con_subset hu.1 (fstFinset_mono A B hv),
      B.con_subset hu.2 (sndFinset_mono A B hv)>
con_sing := by
  intro p
  refine <?, ?_>
  * rcases p.property with h1 | h2
  * by_cases h2 : p.val.2 = B.bot
    * have hpeq : p = prodBot A B := Subtype.ext (Prod.ext h1 h2)
      subst hpeq
      rw [fstFinset_singleton_left A B (prodBot A B) rfl]
      exact A.con_sing A.bot
    * have hfst : fstFinset A B {p} = {} := by
      ext x
      constructor
      * intro hx
        rcases (mem_fstFinset A B).1 hx with <q, hq, hbot', _>
        have hq' : q = p := Finset.mem_singleton.mp hq
        subst hq'
        exact False.elim (h2 hbot')
      * intro hx
        exact False.elim (Finset.notMem_empty x hx)
      rw [hfst]
      exact A.con_empty
  * rw [fstFinset_singleton_left A B p h2]
    exact A.con_sing p.val.1
  * rcases p.property with h1 | h2
  * rw [sndFinset_singleton_right A B p h1]
    exact B.con_sing p.val.2
  * by_cases h1 : p.val.1 = A.bot
    * have hpeq : p = prodBot A B := Subtype.ext (Prod.ext h1 h2)
      subst hpeq
      rw [sndFinset_singleton_right A B (prodBot A B) rfl]
      exact B.con_sing B.bot
    * have hsnd : sndFinset A B {p} = {} := by
      ext y
      constructor
      * intro hy
        rcases (mem_sndFinset A B).1 hy with <q, hq, hbot', _>
        have hq' : q = p := Finset.mem_singleton.mp hq
        subst hq'
        exact False.elim (h1 hbot')
      * intro hy
        exact False.elim (Finset.notMem_empty y hy)
      rw [hsnd]
      exact B.con_empty
ent_con := by
  intro u p <<hfst, hsnd>, hL, hR>
  refine <?, ?_>
  * by_cases hp2 : p.val.2 = B.bot
    * rw [fstFinset_insert_left A B u p hp2]
      exact A.ent_con (hL hp2)
    * rw [fstFinset_insert_not_left A B u p hp2]
      exact hfst
  * by_cases hp1 : p.val.1 = A.bot
    * rw [sndFinset_insert_right A B u p hp1]
      exact B.ent_con (hR hp1)
    * rw [sndFinset_insert_not_right A B u p hp1]
      exact hsnd
ent_bot := by
  intro u hu
  exact <hu, fun _ => A.ent_bot hu.1, fun _ => B.ent_bot hu.2>
ent_refl := by
  intro u p hu hp

```

```

    refine <hu, ?_, ?_>
    * intro hbot
      exact A.ent_refl hu.1 ((mem_fstFinset A B).2 <p, hp, hbot, rfl>)
    * intro hbot
      exact B.ent_refl hu.2 ((mem_sndFinset A B).2 <p, hp, hbot, rfl>)
ent_trans := by
  intro u v c hv hu hEnts hEntc
  refine <hv, ?_, ?_>
  * intro hbot
    have hfst : forall y in fstFinset A B u, A.Ent (fstFinset A B v) y := by
      intro y hy
      rcases (mem_fstFinset A B).1 hy with <q, hq, hq2, rfl>
      exact (hEnts q hq).2.1 hq2
    exact A.ent_trans hv.1 hu.1 hfst (hEntc.2.1 hbot)
  * intro hbot
    have hsnd : forall y in sndFinset A B u, B.Ent (sndFinset A B v) y := by
      intro y hy
      rcases (mem_sndFinset A B).1 hy with <q, hq, hq1, rfl>
      exact (hEnts q hq).2.2 hq1
    exact B.ent_trans hv.2 hu.2 hsnd (hEntc.2.2 hbot)

end InfoSys

end Scott1982

```

A.26 Scott1982/Proposition62.lean

434 lines.

Lean 4 source (lines 1–400)

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Product
import Scott1982.Proposition53
import Scott1982.Proposition55

/-!
# Proposition 6.2 -- product projections and pairing

**Scott 1982, Proposition 6.2.** Approximable `fst`, `snd`, and unique pairing
`<f, g>` with `fst comp <f, g> = f` and `snd comp <f, g> = g`.
-/

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys

variable {a b ? : Type*} [DecidableEq a] [DecidableEq b] [DecidableEq ?]
variable (A : InfoSys a) (B : InfoSys b)

/-- Left-tagged product token `(X, Delta_B)` -/
def mkLeft (x : a) : ProdToken A B :=
  <(x, B.bot), Or.inr rfl>

/-- Right-tagged product token `(Delta_A, Y)` -/

```

```

def mkRight (y : b) : ProdToken A B :=
  <(A.bot, y), Or.inl rfl>

private instance instLeftCommutativeMkLeftInsert :
  LeftCommutative fun x : a =>
    (insert (mkLeft A B x) : Finset (ProdToken A B) -> Finset (ProdToken A B)) :=
    <fun x y s => insert_comm' (mkLeft A B x) (mkLeft A B y) s>

private instance instLeftCommutativeMkRightInsert :
  LeftCommutative fun y : b =>
    (insert (mkRight A B y) : Finset (ProdToken A B) -> Finset (ProdToken A B)) :=
    <fun x y s => insert_comm' (mkRight A B x) (mkRight A B y) s>

/-- Embed a finite set of `A`-tokens as left product tokens. -/
def liftLeft (v : Finset a) : Finset (ProdToken A B) :=
  Multiset.foldr (fun x : a => insert (mkLeft A B x)) ({ } : Finset (ProdToken A B)) v.1

/-- Embed a finite set of `B`-tokens as right product tokens. -/
def liftRight (w : Finset b) : Finset (ProdToken A B) :=
  Multiset.foldr (fun y : b => insert (mkRight A B y)) ({ } : Finset (ProdToken A B)) w.1

private theorem mem_foldr_liftLeft (s : Multiset a) (p : ProdToken A B) :
  p in Multiset.foldr (fun x : a => insert (mkLeft A B x)) ({ } : Finset (ProdToken A B)) s
  <->
  exists x in s, p = mkLeft A B x := by
  refine Multiset.induction_on s ?_ ?_
  * constructor
  * intro hp
    exact False.elim (Finset.notMem_empty p hp)
  * rintro <_, hx, _>
    exact False.elim (by cases hx)
  * intro x t ih
    simp only [Multiset.foldr_cons, Finset.mem_insert, ih, Multiset.mem_cons]
    constructor
  * rintro (hp | <y, hy, hp>)
    * exact <x, Or.inl rfl, hp>
    * exact <y, Or.inr hy, hp>
  * rintro <y, hy, hp>
    rcases hy with rfl | hy
    * exact Or.inl hp
    * exact Or.inr <y, hy, hp>

private theorem mem_foldr_liftRight (s : Multiset b) (p : ProdToken A B) :
  p in Multiset.foldr (fun y : b => insert (mkRight A B y)) ({ } : Finset (ProdToken A B)) s
  <->
  exists y in s, p = mkRight A B y := by
  refine Multiset.induction_on s ?_ ?_
  * constructor
  * intro hp
    exact False.elim (Finset.notMem_empty p hp)
  * rintro <_, hy, _>
    exact False.elim (by cases hy)
  * intro y t ih
    simp only [Multiset.foldr_cons, Finset.mem_insert, ih, Multiset.mem_cons]
    constructor
  * rintro (hp | <z, hz, hp>)
    * exact <y, Or.inl rfl, hp>
    * exact <z, Or.inr hz, hp>
  * rintro <z, hz, hp>
    rcases hz with rfl | hz
    * exact Or.inl hp
    * exact Or.inr <z, hz, hp>

theorem mem_liftLeft {v : Finset a} {p : ProdToken A B} :

```

```

    p in liftLeft A B v <-> exists x in v, p = mkLeft A B x := by
  unfold liftLeft
  exact mem_foldr_liftLeft A B v.1 p

theorem mem_liftRight {w : Finset b} {p : ProdToken A B} :
  p in liftRight A B w <-> exists y in w, p = mkRight A B y := by
  unfold liftRight
  exact mem_foldr_liftRight A B w.1 p

theorem fstFinset_liftLeft (v : Finset a) : fstFinset A B (liftLeft A B v) = v := by
  ext x
  constructor
  * intro hx
    rcases (mem_fstFinset A B).1 hx with <p, hp, hbot, hx'>
    rcases (mem_liftLeft A B).1 hp with <y, hy, hp'>
    subst hp'
    exact (show y = x from hx').symm |> hy
  * intro hx
    exact (mem_fstFinset A B).2
      <mkLeft A B x, (mem_liftLeft A B).2 <x, hx, rfl>, rfl, rfl>

theorem sndFinset_liftRight (w : Finset b) : sndFinset A B (liftRight A B w) = w := by
  ext y
  constructor
  * intro hy
    rcases (mem_sndFinset A B).1 hy with <p, hp, hbot, hy'>
    rcases (mem_liftRight A B).1 hp with <z, hz, hp'>
    subst hp'
    exact (show z = y from hy').symm |> hz
  * intro hy
    exact (mem_sndFinset A B).2
      <mkRight A B y, (mem_liftRight A B).2 <y, hy, rfl>, rfl, rfl>

theorem sndFinset_liftLeft (v : Finset a) :
  sndFinset A B (liftLeft A B v) subseteq ({B.bot} : Finset b) := by
  intro y hy
  rcases (mem_sndFinset A B).1 hy with <p, hp, hbot, hy'>
  rcases (mem_liftLeft A B).1 hp with <x, _, hp'>
  subst hp'
  -- mkLeft x has first component x; filter requires x = A.bot, and second is B.bot
  have : y = B.bot := hy'.symm
  exact Finset.mem_singleton.mpr this

theorem fstFinset_liftRight (w : Finset b) :
  fstFinset A B (liftRight A B w) subseteq ({A.bot} : Finset a) := by
  intro x hx
  rcases (mem_fstFinset A B).1 hx with <p, hp, hbot, hx'>
  rcases (mem_liftRight A B).1 hp with <y, _, hp'>
  subst hp'
  have : x = A.bot := hx'.symm
  exact Finset.mem_singleton.mpr this

theorem ProdCon_liftLeft {v : Finset a} (hv : v in A.Con) :
  ProdCon A B (liftLeft A B v) := by
  refine <?, ?_>
  * rw [fstFinset_liftLeft]
  exact hv
  * exact B.con_subset (B.con_sing B.bot) (sndFinset_liftLeft A B v)

theorem ProdCon_liftRight {w : Finset b} (hw : w in B.Con) :
  ProdCon A B (liftRight A B w) := by
  refine <?, ?_>
  * exact A.con_subset (A.con_sing A.bot) (fstFinset_liftRight A B w)

```

```

* rw [sndFinset_liftRight]
  exact hw

theorem ProdCon_empty : ProdCon A B ({ } : Finset (ProdToken A B)) :=
  <by rw [fstFinset_empty]; exact A.con_empty,
  by rw [sndFinset_empty]; exact B.con_empty>

/-- Product entailment of a set projects to component entailment on `fst`. -/
theorem entSet_fst_of_prodEntSet {u v : Finset (ProdToken A B)}
  (h : (productSystem A B).EntSet u v) :
  A.EntSet (fstFinset A B u) (fstFinset A B v) := by
  intro x hx
  rcases (mem_fstFinset A B).1 hx with <p, hp, hbot, rfl>
  exact (h p hp).2.1 hbot

/-- Product entailment of a set projects to component entailment on `snd`. -/
theorem entSet_snd_of_prodEntSet {u v : Finset (ProdToken A B)}
  (h : (productSystem A B).EntSet u v) :
  B.EntSet (sndFinset A B u) (sndFinset A B v) := by
  intro y hy
  rcases (mem_sndFinset A B).1 hy with <p, hp, hbot, rfl>
  exact (h p hp).2.2 hbot

namespace ApproximableMap

variable {C : InfoSys ?}

/-- **Proposition 6.2(1).** Approximable first projection. -/
def fstMap : ApproximableMap (productSystem A B) A where
  rel u v := u in (productSystem A B).Con /\ v in A.Con /\ A.EntSet (fstFinset A B u) v
  rel_dom h := h.1
  rel_cod h := h.2.1
  empty_rel := <ProdCon_empty A B, A.con_empty, by
    rw [fstFinset_empty]; exact A.entSet_empty {}>
  union_right := by
    rintro u v v' <hu, hv, huv> <_, hv', huv'>
    have hEnt : A.EntSet (fstFinset A B u) (v U ' v') :=
      proposition_2_3_vi (sys := A) huv huv'
    have hBig : fstFinset A B u U ' (v U ' v') in A.Con :=
      proposition_2_3_ii (sys := A) hu.1 hEnt
    have hCon : v U ' v' in A.Con :=
      A.con_subset hBig (subset_funion_right _ _)
    exact <hu, hCon, hEnt>
  mono := by
    rintro u u' v v' <hu, hv, huv> hEntu hEntv hu' hv'
    refine <hu', hv', ?_>
    exact proposition_2_3_iv A hu'.1 hv
      (proposition_2_3_iv A hu'.1 hu.1 (entSet_fst_of_prodEntSet A B hEntu) huv) hEntv

/-- **Proposition 6.2(2).** Approximable second projection. -/
def sndMap : ApproximableMap (productSystem A B) B where
  rel u w := u in (productSystem A B).Con /\ w in B.Con /\ B.EntSet (sndFinset A B u) w
  rel_dom h := h.1
  rel_cod h := h.2.1
  empty_rel := <ProdCon_empty A B, B.con_empty, by
    rw [sndFinset_empty]; exact B.entSet_empty {}>
  union_right := by
    rintro u w w' <hu, hw, huw> <_, hw', huw'>
    have hEnt : B.EntSet (sndFinset A B u) (w U ' w') :=
      proposition_2_3_vi (sys := B) huw huw'
    have hBig : sndFinset A B u U ' (w U ' w') in B.Con :=
      proposition_2_3_ii (sys := B) hu.2 hEnt
    have hCon : w U ' w' in B.Con :=

```

```

      B.con_subset hBig (subset_funion_right _ _)
    exact <hu, hCon, hEnt>
  mono := by
    rintro u u' w w' <hu, hw, huw> hEntu hEntw hu' hw'
    refine <hu', hw', ?_>
    exact proposition_2_3_iv B hu'.2 hw
      (proposition_2_3_iv B hu'.2 hu.2 (entSet_snd_of_prodEntSet A B hEntu) huw) hEntw

/-- Helper: `g` relates `s` to any subset of `{Delta_B}` once it relates to `{}`. -/
private theorem rel_bot_subset {D : InfoSys b} {s : Finset ?} {w : Finset b}
  (g : ApproximableMap C D) (hs : s in C.Con)
  (hg : g.rel s ({ } : Finset b)) (hw : w subseq ({D.bot} : Finset b)) :
  g.rel s w := by
  have hwCon : w in D.Con := D.con_subset (D.con_sing D.bot) hw
  have hEnt : D.EntSet ({ } : Finset b) w := by
    intro y hy
    have : y = D.bot := Finset.mem_singleton.mp (hw hy)
    subst this
    exact D.ent_bot D.con_empty
  exact g.mono hg (proposition_2_3_iii C hs) hEnt hs hwCon

/-- Helper: `f` relates `s` to any subset of `{Delta_A}` once it relates to `{}`. -/
private theorem rel_bot_subset_left {D : InfoSys a} {s : Finset ?} {v : Finset a}
  (f : ApproximableMap C D) (hs : s in C.Con)
  (hf : f.rel s ({ } : Finset a)) (hv : v subseq ({D.bot} : Finset a)) :
  f.rel s v := by
  have hvCon : v in D.Con := D.con_subset (D.con_sing D.bot) hv
  have hEnt : D.EntSet ({ } : Finset a) v := by
    intro x hx
    have : x = D.bot := Finset.mem_singleton.mp (hv hx)
    subst this
    exact D.ent_bot D.con_empty
  exact f.mono hf (proposition_2_3_iii C hs) hEnt hs hvCon

/-- **Proposition 6.2(3).** Pairing of approximable maps. -/
def pairMap (f : ApproximableMap C A) (g : ApproximableMap C B) :
  ApproximableMap C (productSystem A B) where
  rel s u := f.rel s (fstFinset A B u) /\ g.rel s (sndFinset A B u)
  rel_dom h := f.rel_dom h.1
  rel_cod h := <f.rel_cod h.1, g.rel_cod h.2>
  empty_rel := by
    rw [fstFinset_empty, sndFinset_empty]
    exact <f.empty_rel, g.empty_rel>
  union_right := by
    rintro s u u' <hf, hg> <hf', hg'>
    rw [fstFinset_funion A B u u', sndFinset_funion A B u u']
    exact <f.union_right hf hf', g.union_right hg hg'>
  mono := by
    rintro s s' u u' <hf, hg> hEnts hEntu hs' hu'
    refine <?_, ?_>
    * exact f.mono hf hEnts (entSet_fst_of_prodEntSet A B hEntu) hs' hu'.1
    * exact g.mono hg hEnts (entSet_snd_of_prodEntSet A B hEntu) hs' hu'.2

/-- Product elements are determined by their projections (Scott's uniqueness lemma). -/
theorem element_eq_of_fst_snd (z z' : (productSystem A B).Element)
  (hfst : (fstMap A B).toElement z = (fstMap A B).toElement z')
  (hsnd : (sndMap A B).toElement z = (sndMap A B).toElement z') :
  z = z' := by
  apply le_antisymm
  * intro p hp
    have hpz : ({p} : Finset (ProdToken A B)) : Set _ subseq z.carrier := by
      intro q hq
      have : q = p := Finset.mem_singleton.mp (Finset.mem_coe.1 hq)

```

```

    subst this
    exact hp
have hCon : ProdCon A B {p} := z.consistent {p} hpz
rcases p.property with h1 | h2
* -- right-tagged (or bottom): use `snd`
  have hy : p.val.2 in ((sndMap A B).toElement z).carrier := by
    refine <{p}, hpz, ?_>
    refine <hCon, B.con_sing p.val.2, ?_>
    intro y hy
    have : y = p.val.2 := Finset.mem_singleton.mp hy
    subst this
    exact B.ent_refl hCon.2 ((mem_sndFinset A B).2 <p, Finset.mem_singleton_self p, h1, rfl
>)
  have hy' : p.val.2 in ((sndMap A B).toElement z').carrier := by
    rw [ <- hsnd]
    exact hy
  rcases hy' with <u, hu, <huCon, _, hEnt>>
  have hEntp : (productSystem A B).Ent u p := by
    refine <huCon, ?_, ?_>
    * intro _
      simpa [h1] using A.ent_bot huCon.1
    * intro _
      exact hEnt p.val.2 (Finset.mem_singleton_self _)
  exact z'.closed u p hu hEntp
* -- left-tagged: use `fst`
  have hx : p.val.1 in ((fstMap A B).toElement z).carrier := by
    refine <{p}, hpz, ?_>
    refine <hCon, A.con_sing p.val.1, ?_>
    intro x hx
    have : x = p.val.1 := Finset.mem_singleton.mp hx
    subst this
    exact A.ent_refl hCon.1 ((mem_fstFinset A B).2 <p, Finset.mem_singleton_self p, h2, rfl
>)
  have hx' : p.val.1 in ((fstMap A B).toElement z').carrier := by
    rw [ <- hfst]
    exact hx
  rcases hx' with <u, hu, <huCon, _, hEnt>>
  have hEntp : (productSystem A B).Ent u p := by
    refine <huCon, ?_, ?_>
    * intro _
      exact hEnt p.val.1 (Finset.mem_singleton_self _)
    * intro _
      simpa [h2] using B.ent_bot huCon.2
  exact z'.closed u p hu hEntp
* intro p hp
  have hpz : ({p} : Finset (ProdToken A B)) : Set _ subseteq z'.carrier := by
    intro q hq
    have : q = p := Finset.mem_singleton.mp (Finset.mem_coe.1 hq)
    subst this
    exact hp
have hCon : ProdCon A B {p} := z'.consistent {p} hpz
rcases p.property with h1 | h2
* have hy : p.val.2 in ((sndMap A B).toElement z').carrier := by
  refine <{p}, hpz, ?_>
  refine <hCon, B.con_sing p.val.2, ?_>
  intro y hy
  have : y = p.val.2 := Finset.mem_singleton.mp hy
  subst this
  exact B.ent_refl hCon.2 ((mem_sndFinset A B).2 <p, Finset.mem_singleton_self p, h1, rfl
>)
  have hy' : p.val.2 in ((sndMap A B).toElement z).carrier := by
    rw [hsnd]
    exact hy

```



```

rcases hy' with <u, hu, <huCon, _, hEnt>>
have hEntp : (productSystem A B).Ent u p := by
  refine <huCon, ?, ?_>
  * intro _
    simpa [h1] using A.ent_bot huCon.1
  * intro _
    exact hEnt p.val.2 (Finset.mem_singleton_self _)
exact z.closed u p hu hEntp
* have hx : p.val.1 in ((fstMap A B).toElement z').carrier := by
  refine <{p}, hpz, ?_>
  refine <hCon, A.con_sing p.val.1, ?_>
  intro x hx
  have : x = p.val.1 := Finset.mem_singleton.mp hx
  subst this
  exact A.ent_refl hCon.1 ((mem_fstFinset A B).2 <p, Finset.mem_singleton_self p, h2, rfl
>)
  have hx' : p.val.1 in ((fstMap A B).toElement z).carrier := by
    rw [hfst]
    exact hx
rcases hx' with <u, hu, <huCon, _, hEnt>>
have hEntp : (productSystem A B).Ent u p := by
  refine <huCon, ?, ?_>
  * intro _
    exact hEnt p.val.1 (Finset.mem_singleton_self _)
  * intro _
    simpa [h2] using B.ent_bot huCon.2
exact z.closed u p hu hEntp

/-- `fst comp <f, g> = f`. -/
theorem comp_fstMap_pairMap (f : ApproximableMap C A) (g : ApproximableMap C B) :
  comp (fstMap A B) (pairMap A B f g) = f := by
  refine ApproximableMap.ext fun s v => ?_
  constructor
  * rintro <u, <hf, hg>, <hu, hv, hEnt>>
    exact f.mono hf (proposition_2_3_iii C (f.rel_dom hf)) hEnt (f.rel_dom hf) hv
  * intro hf
    refine <liftLeft A B v, ?, ?_>
  * refine <?, ?_>
    * rw [fstFinset_liftLeft]
      exact hf
    * have hs : s in C.Con := f.rel_dom hf
      have hg0 : g.rel s ({ : Finset b) :=
        g.mono g.empty_rel (C.entSet_empty s) (B.entSet_empty { }) hs B.con_empty
      exact rel_bot_subset g hs hg0 (sndFinset_liftLeft A B v)
  * refine <ProdCon_liftLeft A B (f.rel_cod hf), f.rel_cod hf, ?_>
    rw [fstFinset_liftLeft]
    exact proposition_2_3_iii A (f.rel_cod hf)

/-- `snd comp <f, g> = g`. -/
theorem comp_sndMap_pairMap (f : ApproximableMap C A) (g : ApproximableMap C B) :
  comp (sndMap A B) (pairMap A B f g) = g := by
  refine ApproximableMap.ext fun s w => ?_
  constructor
  * rintro <u, <hf, hg>, <hu, hw, hEnt>>
    exact g.mono hg (proposition_2_3_iii C (g.rel_dom hg)) hEnt (g.rel_dom hg) hw
  * intro hg
    refine <liftRight A B w, ?, ?_>
  * refine <?, ?_>
    * have hs : s in C.Con := g.rel_dom hg

```

Lean 4 source (lines 401–434)

```

have hf0 : f.rel s ({ : Finset a) :=
  f.mono f.empty_rel (C.entSet_empty s) (A.entSet_empty { }) hs A.con_empty

```

```

    exact rel_bot_subset_left f hs hf0 (fstFinset_liftRight A B w)
  * rw [sndFinset_liftRight]
    exact hg
  * refine <ProdCon_liftRight A B (g.rel_cod hg), g.rel_cod hg, ?_>
    rw [sndFinset_liftRight]
    exact proposition_2_3_iii B (g.rel_cod hg)

/-- Uniqueness of pairing (Scott Prop 6.2). -/
theorem pairMap_unique (f : ApproximableMap C A) (g : ApproximableMap C B)
  (h : ApproximableMap C (productSystem A B))
  (hfst : comp (fstMap A B) h = f) (hsnd : comp (sndMap A B) h = g) :
  h = pairMap A B f g :=
(ext_iff_toElement h (pairMap A B f g)).2 fun x =>
  element_eq_of_fst_snd A B (h.toElement x) ((pairMap A B f g).toElement x)
  (by
    have h1 : (fstMap A B).toElement (h.toElement x) = f.toElement x := by
      rw [ <- comp_toElement, hfst]
    have h2 : (fstMap A B).toElement ((pairMap A B f g).toElement x) = f.toElement x := by
      rw [ <- comp_toElement, comp_fstMap_pairMap]
    exact h1.trans h2.symm)
  (by
    have h1 : (sndMap A B).toElement (h.toElement x) = g.toElement x := by
      rw [ <- comp_toElement, hsnd]
    have h2 : (sndMap A B).toElement ((pairMap A B f g).toElement x) = g.toElement x := by
      rw [ <- comp_toElement, comp_sndMap_pairMap]
    exact h1.trans h2.symm)

end ApproximableMap

end InfoSys

end Scott1982

```

A.27 Scott1982/Sum.lean

516 lines.

Lean 4 source (lines 1–400)

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Constructive
import Scott1982.Proposition23

/-!
# Separated sum of information systems -- Definition 6.3

**Scott 1982, Definition 6.3.** Tokens of `A + B` are left copies `(X, Delta)`, right
copies `(Delta, Y)`, and a fresh bottom `(Delta, Delta)`. Consistency and entailment are
disjunctive: a consistent set lives entirely on the left or entirely on the right
(or is empty / `{False}`).

We encode tokens as an inductive type so that `(Delta_A, Delta)`, `(Delta, Delta_B)`, and `(
Delta, Delta)`
remain pairwise distinct, matching Scott's remark after Def 6.3.
-/

namespace Scott1982

```

```

open Scott1982.Constructive

namespace InfoSys

set_option linter.unusedSectionVars false

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

/-- Token type of the separated sum `A + B` (Scott 6.3(i)-(ii)).
`left X` is `(X, Delta)`, `right Y` is `(Delta, Y)`, and `bot` is `(Delta, Delta)`. -/
inductive SumToken (a b : Type*) where
  | left : a -> SumToken a b
  | right : b -> SumToken a b
  | bot : SumToken a b

instance : DecidableEq (SumToken a b)
  | .left a, .left b =>
    if h : a = b then isTrue (h |> rfl)
    else isFalse fun h' => h (SumToken.left.inj h')
  | .right a, .right b =>
    if h : a = b then isTrue (h |> rfl)
    else isFalse fun h' => h (SumToken.right.inj h')
  | .bot, .bot => isTrue rfl
  | .left _, .right _ => isFalse fun h => nomatch h
  | .left _, .bot => isFalse fun h => nomatch h
  | .right _, .left _ => isFalse fun h => nomatch h
  | .right _, .bot => isFalse fun h => nomatch h
  | .bot, .left _ => isFalse fun h => nomatch h
  | .bot, .right _ => isFalse fun h => nomatch h

variable (A : InfoSys a) (B : InfoSys b)

/-- Sum bottom `Delta_{A+B} = (Delta, Delta)`. -/
def sumBot : SumToken a b := .bot

private def lftInsert : SumToken a b -> Finset a -> Finset a
  | .left x => insert x
  | .right _ => id
  | .bot => id

private def rhtInsert : SumToken a b -> Finset b -> Finset b
  | .right y => insert y
  | .left _ => id
  | .bot => id

private instance instLeftCommutativeLftInsert :
  LeftCommutative (lftInsert : SumToken a b -> Finset a -> Finset a) :=
  <fun p q s => by
    cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl>

private instance instLeftCommutativeRhtInsert :
  LeftCommutative (rhtInsert : SumToken a b -> Finset b -> Finset b) :=
  <fun p q s => by
    cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl>

/-- Left projection `lft u` (Scott 6.3). -/
def lftFinset (u : Finset (SumToken a b)) : Finset a :=
  Multiset.foldr lftInsert ({} : Finset a) u.1

/-- Right projection `rht u` (Scott 6.3). -/
def rhtFinset (u : Finset (SumToken a b)) : Finset b :=
  Multiset.foldr rhtInsert ({} : Finset b) u.1

```

```

private theorem mem_foldr_lft (s : Multiset (SumToken a b)) (x : a) :
  x in Multiset.foldr lftInsert ({ } : Finset a) s <-> exists p in s, p = .left x := by
  refine Multiset.induction_on s ?_ ?_
* constructor
  * intro hx
    exact False.elim (Finset.notMem_empty x hx)
  * rintro <_, hp, _>
    exact False.elim (by cases hp)
* intro p t ih
  cases p with
  | left a =>
    simp only [Multiset.foldr_cons, lftInsert, Finset.mem_insert, ih, Multiset.mem_cons]
    constructor
    * rintro (hx | <q, hq, hq'>)
      * exact <.left a, Or.inl rfl, congrArg SumToken.left hx.symm>
      * exact <q, Or.inr hq, hq'>
    * rintro <q, hq, hq'>
      rcases hq with rfl | hq
      * injection hq' with hx
        exact Or.inl hx.symm
      * exact Or.inr <q, hq, hq'>
  | right b =>
    simp only [Multiset.foldr_cons, lftInsert, id_eq, ih, Multiset.mem_cons]
    constructor
    * rintro <q, hq, hq'>
      exact <q, Or.inr hq, hq'>
    * rintro <q, hq, hq'>
      rcases hq with rfl | hq
      * exact False.elim (nomatch hq')
      * exact <q, hq, hq'>
  | bot =>
    simp only [Multiset.foldr_cons, lftInsert, id_eq, ih, Multiset.mem_cons]
    constructor
    * rintro <q, hq, hq'>
      exact <q, Or.inr hq, hq'>
    * rintro <q, hq, hq'>
      rcases hq with rfl | hq
      * exact False.elim (nomatch hq')
      * exact <q, hq, hq'>

private theorem mem_foldr_rht (s : Multiset (SumToken a b)) (y : b) :
  y in Multiset.foldr rhtInsert ({ } : Finset b) s <-> exists p in s, p = .right y :=
  by
  refine Multiset.induction_on s ?_ ?_
* constructor
  * intro hy
    exact False.elim (Finset.notMem_empty y hy)
  * rintro <_, hp, _>
    exact False.elim (by cases hp)
* intro p t ih
  cases p with
  | right b =>
    simp only [Multiset.foldr_cons, rhtInsert, Finset.mem_insert, ih, Multiset.mem_cons]
    constructor
    * rintro (hy | <q, hq, hq'>)
      * exact <.right b, Or.inl rfl, congrArg SumToken.right hy.symm>
      * exact <q, Or.inr hq, hq'>
    * rintro <q, hq, hq'>
      rcases hq with rfl | hq
      * injection hq' with hy
        exact Or.inl hy.symm
      * exact Or.inr <q, hq, hq'>

```

```

| left a =>
  simp only [Multiset.foldr_cons, rhtInsert, id_eq, ih, Multiset.mem_cons]
  constructor
  * rintro <q, hq, hq'>
    exact <q, Or.inr hq, hq'>
  * rintro <q, hq, hq'>
    rcases hq with rfl | hq
    * exact False.elim (nomatch hq')
    * exact <q, hq, hq'>
| bot =>
  simp only [Multiset.foldr_cons, rhtInsert, id_eq, ih, Multiset.mem_cons]
  constructor
  * rintro <q, hq, hq'>
    exact <q, Or.inr hq, hq'>
  * rintro <q, hq, hq'>
    rcases hq with rfl | hq
    * exact False.elim (nomatch hq')
    * exact <q, hq, hq'>

theorem mem_lftFinset {u : Finset (SumToken a b)} {x : a} :
  x in lftFinset u <-> SumToken.left x in u := by
  unfold lftFinset
  rw [mem_foldr_lft]
  constructor
  * rintro <p, hp, hp'>
    rwa [ <- hp' ]
  * intro hx
    exact <.left x, hx, rfl>

theorem mem_rhtFinset {u : Finset (SumToken a b)} {y : b} :
  y in rhtFinset u <-> SumToken.right y in u := by
  unfold rhtFinset
  rw [mem_foldr_rht]
  constructor
  * rintro <p, hp, hp'>
    rwa [ <- hp' ]
  * intro hy
    exact <.right y, hy, rfl>

theorem lftFinset_empty : lftFinset ({ } : Finset (SumToken a b)) = { } := by
  ext x
  constructor
  * intro hx
    exact False.elim (Finset.notMem_empty _ ((mem_lftFinset).1 hx))
  * intro hx
    exact False.elim (Finset.notMem_empty x hx)

theorem rhtFinset_empty : rhtFinset ({ } : Finset (SumToken a b)) = { } := by
  ext y
  constructor
  * intro hy
    exact False.elim (Finset.notMem_empty _ ((mem_rhtFinset).1 hy))
  * intro hy
    exact False.elim (Finset.notMem_empty y hy)

theorem lftFinset_mono {u v : Finset (SumToken a b)} (h : v subseq u) :
  lftFinset v subseq lftFinset u := by
  intro x hx
  exact (mem_lftFinset).2 (h ((mem_lftFinset).1 hx))

theorem rhtFinset_mono {u v : Finset (SumToken a b)} (h : v subseq u) :
  rhtFinset v subseq rhtFinset u := by
  intro y hy

```

```

exact (mem_rhtFinset).2 (h ((mem_rhtFinset).1 hy))

theorem lftFinset_insert_left (u : Finset (SumToken a b)) (x : a) :
  lftFinset (insert (.left x) u) = insert x (lftFinset u) := by
  ext y
  constructor
  * intro hy
    have : SumToken.left y in insert (.left x) u := (mem_lftFinset).1 hy
    rcases Finset.mem_insert.mp this with h | h
    * injection h with hy'
      exact Finset.mem_insert.mpr (Or.inl hy')
    * exact Finset.mem_insert.mpr (Or.inr ((mem_lftFinset).2 h))
  * intro hy
    rcases Finset.mem_insert.mp hy with hy' | hy'
    * exact (mem_lftFinset).2 (Finset.mem_insert.mpr (Or.inl (congrArg SumToken.left hy')))
    * exact (mem_lftFinset).2 (Finset.mem_insert.mpr (Or.inr ((mem_lftFinset).1 hy')))

theorem lftFinset_insert_right (u : Finset (SumToken a b)) (y : b) :
  lftFinset (insert (.right y) u) = lftFinset u := by
  ext x
  constructor
  * intro hx
    have : SumToken.left x in insert (.right y) u := (mem_lftFinset).1 hx
    rcases Finset.mem_insert.mp this with h | h
    * exact False.elim (nomatch h)
    * exact (mem_lftFinset).2 h
  * intro hx
    exact (mem_lftFinset).2 (Finset.mem_insert.mpr (Or.inr ((mem_lftFinset).1 hx)))

theorem lftFinset_insert_bot (u : Finset (SumToken a b)) :
  lftFinset (insert (.bot : SumToken a b) u) = lftFinset u := by
  ext x
  constructor
  * intro hx
    have : SumToken.left x in insert .bot u := (mem_lftFinset).1 hx
    rcases Finset.mem_insert.mp this with h | h
    * exact False.elim (nomatch h)
    * exact (mem_lftFinset).2 h
  * intro hx
    exact (mem_lftFinset).2 (Finset.mem_insert.mpr (Or.inr ((mem_lftFinset).1 hx)))

theorem rhtFinset_insert_right (u : Finset (SumToken a b)) (y : b) :
  rhtFinset (insert (.right y) u) = insert y (rhtFinset u) := by
  ext z
  constructor
  * intro hz
    have : SumToken.right z in insert (.right y) u := (mem_rhtFinset).1 hz
    rcases Finset.mem_insert.mp this with h | h
    * injection h with hz'
      exact Finset.mem_insert.mpr (Or.inl hz')
    * exact Finset.mem_insert.mpr (Or.inr ((mem_rhtFinset).2 h))
  * intro hz
    rcases Finset.mem_insert.mp hz with hz' | hz'
    * exact (mem_rhtFinset).2 (Finset.mem_insert.mpr (Or.inl (congrArg SumToken.right hz')))
    * exact (mem_rhtFinset).2 (Finset.mem_insert.mpr (Or.inr ((mem_rhtFinset).1 hz')))

theorem rhtFinset_insert_left (u : Finset (SumToken a b)) (x : a) :
  rhtFinset (insert (.left x) u) = rhtFinset u := by
  ext y
  constructor
  * intro hy
    have : SumToken.right y in insert (.left x) u := (mem_rhtFinset).1 hy
    rcases Finset.mem_insert.mp this with h | h

```

```

    * exact False.elim (nomatch h)
    * exact (mem_rhtFinset).2 h
  * intro hy
    exact (mem_rhtFinset).2 (Finset.mem_insert.mpr (Or.inr ((mem_rhtFinset).1 hy)))

theorem rhtFinset_insert_bot (u : Finset (SumToken a b)) :
  rhtFinset (insert (.bot : SumToken a b) u) = rhtFinset u := by
  ext y
  constructor
  * intro hy
    have : SumToken.right y in insert .bot u := (mem_rhtFinset).1 hy
    rcases Finset.mem_insert.mp this with h | h
    * exact False.elim (nomatch h)
    * exact (mem_rhtFinset).2 h
  * intro hy
    exact (mem_rhtFinset).2 (Finset.mem_insert.mpr (Or.inr ((mem_rhtFinset).1 hy)))

theorem lftFinset_singleton_left (x : a) :
  lftFinset ({.left x} : Finset (SumToken a b)) = {x} := by
  ext y
  simp only [mem_lftFinset, Finset.mem_singleton]
  exact <SumToken.left.inj, fun h => h |> rfl>

theorem rhtFinset_singleton_right (y : b) :
  rhtFinset ({.right y} : Finset (SumToken a b)) = {y} := by
  ext z
  simp only [mem_rhtFinset, Finset.mem_singleton]
  exact <SumToken.right.inj, fun h => h |> rfl>

theorem lftFinset_singleton_bot :
  lftFinset ({.bot} : Finset (SumToken a b)) = {} := by
  ext x
  constructor
  * intro hx
    have : SumToken.left x in ({.bot} : Finset _) := (mem_lftFinset).1 hx
    exact False.elim (nomatch Finset.mem_singleton.mp this)
  * intro hx
    exact False.elim (Finset.notMem_empty x hx)

theorem rhtFinset_singleton_bot :
  rhtFinset ({.bot} : Finset (SumToken a b)) = {} := by
  ext y
  constructor
  * intro hy
    have : SumToken.right y in ({.bot} : Finset _) := (mem_rhtFinset).1 hy
    exact False.elim (nomatch Finset.mem_singleton.mp this)
  * intro hy
    exact False.elim (Finset.notMem_empty y hy)

theorem lftFinset_funion (u v : Finset (SumToken a b)) :
  lftFinset (u U ' v) = lftFinset u U ' lftFinset v := by
  ext x
  constructor
  * intro hx
    have : SumToken.left x in u U ' v := (mem_lftFinset).1 hx
    rcases mem_funion.mp this with h | h
    * exact mem_funion.mpr (Or.inl ((mem_lftFinset).2 h))
    * exact mem_funion.mpr (Or.inr ((mem_lftFinset).2 h))
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact (mem_lftFinset).2 (mem_funion.mpr (Or.inl ((mem_lftFinset).1 h)))
    * exact (mem_lftFinset).2 (mem_funion.mpr (Or.inr ((mem_lftFinset).1 h)))

```

```

theorem rhtFinset_funion (u v : Finset (SumToken a b)) :
  rhtFinset (u U ' v) = rhtFinset u U ' rhtFinset v := by
  ext y
  constructor
  * intro hy
    have : SumToken.right y in u U ' v := (mem_rhtFinset).1 hy
    rcases mem_funion.mp this with h | h
    * exact mem_funion.mpr (Or.inl ((mem_rhtFinset).2 h))
    * exact mem_funion.mpr (Or.inr ((mem_rhtFinset).2 h))
  * intro hy
    rcases mem_funion.mp hy with h | h
    * exact (mem_rhtFinset).2 (mem_funion.mpr (Or.inl ((mem_rhtFinset).1 h)))
    * exact (mem_rhtFinset).2 (mem_funion.mpr (Or.inr ((mem_rhtFinset).1 h)))

/-- Consistency for the separated sum (Scott 6.3(iii)). -/
def SumCon (u : Finset (SumToken a b)) : Prop :=
  (lftFinset u in A.Con /\ rhtFinset u = {}) \
  (lftFinset u = {} /\ rhtFinset u in B.Con)

/-- Entailment for the separated sum (Scott 6.3(iv')-(iv'')).
Includes `SumCon` so `ent_con` can update projections. -/
def SumEnt (u : Finset (SumToken a b)) (p : SumToken a b) : Prop :=
  SumCon A B u /\
  match p with
  | .bot => True
  | .left x => lftFinset u != {} /\ A.Ent (lftFinset u) x
  | .right y => rhtFinset u != {} /\ B.Ent (rhtFinset u) y

theorem SumCon_empty : SumCon A B ({} : Finset (SumToken a b)) :=
  Or.inl <by rw [lftFinset_empty]; exact A.con_empty, by rw [rhtFinset_empty]>

theorem SumCon_rht_empty_of_lft_nonempty {u : Finset (SumToken a b)}
  (hu : SumCon A B u) (hne : lftFinset u != {}) : rhtFinset u = {} := by
  rcases hu with <_, hr> | <hl, _>
  * exact hr
  * exact False.elim (hne hl)

theorem SumCon_lft_empty_of_rht_nonempty {u : Finset (SumToken a b)}
  (hu : SumCon A B u) (hne : rhtFinset u != {}) : lftFinset u = {} := by
  rcases hu with <_, hr> | <hl, _>
  * exact False.elim (hne hr)
  * exact hl

theorem SumCon_lft_con_of_lft_nonempty {u : Finset (SumToken a b)}
  (hu : SumCon A B u) (hne : lftFinset u != {}) : lftFinset u in A.Con := by
  rcases hu with <hl, _> | <hl, _>
  * exact hl
  * exact False.elim (hne hl)

theorem SumCon_rht_con_of_rht_nonempty {u : Finset (SumToken a b)}
  (hu : SumCon A B u) (hne : rhtFinset u != {}) : rhtFinset u in B.Con := by
  rcases hu with <_, hr> | <_, hr>
  * exact False.elim (hne hr)
  * exact hr

/-- **Definition 6.3.** The separated sum information system `A + B`. -/
def sumSystem : InfoSys (SumToken a b) where
  bot := sumBot
  Con := {u | SumCon A B u}
  Ent := SumEnt A B
  con_subset := by
    intro u v hu hv
    rcases hu with <hl, hr> | <hl, hr>

```


Lean 4 source (lines 401–516)

```

* exact Or.inl <A.con_subset hl (lftFinset_mono hv),
  Finset.Subset.antisymm (by
    intro y hy
    have : y in rhtFinset u := rhtFinset_mono hv hy
    rw [hr] at this
    exact False.elim (Finset.notMem_empty y this))
    (Finset.empty_subset _)>
* exact Or.inr <Finset.Subset.antisymm (by
  intro x hx
  have : x in lftFinset u := lftFinset_mono hv hx
  rw [hl] at this
  exact False.elim (Finset.notMem_empty x this))
  (Finset.empty_subset _),
  B.con_subset hr (rhtFinset_mono hv)>
con_sing := by
  intro p
  cases p with
  | left x =>
    exact Or.inl <by rw [lftFinset_singleton_left]; exact A.con_sing x,
    by
      ext y
      constructor
      * intro hy
        have : SumToken.right y in ({.left x} : Finset _) := (mem_rhtFinset).1 hy
        exact False.elim (nomatch Finset.mem_singleton.mp this)
      * intro hy
        exact False.elim (Finset.notMem_empty y hy)>
  | right y =>
    exact Or.inr <by
      ext x
      constructor
      * intro hx
        have : SumToken.left x in ({.right y} : Finset _) := (mem_lftFinset).1 hx
        exact False.elim (nomatch Finset.mem_singleton.mp this)
      * intro hx
        exact False.elim (Finset.notMem_empty x hx),
      by rw [rhtFinset_singleton_right]; exact B.con_sing y>
  | bot =>
    exact Or.inl <by rw [lftFinset_singleton_bot]; exact A.con_empty,
    rhtFinset_singleton_bot>
ent_con := by
  intro u p <hu, hEnt>
  cases p with
  | bot =>
    rcases hu with <hl, hr> | <hl, hr>
    * exact Or.inl <by rw [lftFinset_insert_bot]; exact hl,
      by rw [rhtFinset_insert_bot]; exact hr>
    * exact Or.inr <by rw [lftFinset_insert_bot]; exact hl,
      by rw [rhtFinset_insert_bot]; exact hr>
  | left x =>
    rcases hEnt with <hne, hA>
    have hr : rhtFinset u = {} := SumCon_rht_empty_of_lft_nonempty A B hu hne
    exact Or.inl <by
      rw [lftFinset_insert_left]
      exact A.ent_con hA,
      by rw [rhtFinset_insert_left]; exact hr>
  | right y =>
    rcases hEnt with <hne, hB>
    have hl : lftFinset u = {} := SumCon_lft_empty_of_rht_nonempty A B hu hne
    exact Or.inr <by rw [lftFinset_insert_right]; exact hl,
      by
        rw [rhtFinset_insert_right]

```

```

        exact B.ent_con hB>
ent_bot := by
  intro u hu
  exact <hu, trivial>
ent_refl := by
  intro u p hu hp
  refine <hu, ?_>
  cases p with
  | bot => exact trivial
  | left x =>
    have hx : x in lftFinset u := (mem_lftFinset).2 hp
    have hne : lftFinset u != {} := Finset.ne_empty_of_mem hx
    exact <hne, A.ent_refl (SumCon_lft_con_of_lft_nonempty A B hu hne) hx>
  | right y =>
    have hy : y in rhtFinset u := (mem_rhtFinset).2 hp
    have hne : rhtFinset u != {} := Finset.ne_empty_of_mem hy
    exact <hne, B.ent_refl (SumCon_rht_con_of_rht_nonempty A B hu hne) hy>
ent_trans := by
  intro u v c hv hu hEnts hEntc
  refine <hv, ?_>
  cases c with
  | bot => exact trivial
  | left x =>
    rcases hEntc with <_, <hne, hA>>
    have hlft : forall y in lftFinset u, A.Ent (lftFinset v) y := by
      intro y hy
      have hp : SumToken.left y in u := (mem_lftFinset).1 hy
      have hEy : SumEnt A B v (.left y) := hEnts _ hp
      exact hEy.2.2
    have hne' : lftFinset v != {} := by
      obtain <y, hy> := Finset.nonempty_of_ne_empty hne
      have hp : SumToken.left y in u := (mem_lftFinset).1 hy
      exact (hEnts _ hp).2.1
    refine <hne', ?_>
    exact A.ent_trans (SumCon_lft_con_of_lft_nonempty A B hv hne')
      (SumCon_lft_con_of_lft_nonempty A B hu hne) hlft hA
  | right y =>
    rcases hEntc with <_, <hne, hB>>
    have hrht : forall z in rhtFinset u, B.Ent (rhtFinset v) z := by
      intro z hz
      have hp : SumToken.right z in u := (mem_rhtFinset).1 hz
      have hEz : SumEnt A B v (.right z) := hEnts _ hp
      exact hEz.2.2
    have hne' : rhtFinset v != {} := by
      obtain <z, hz> := Finset.nonempty_of_ne_empty hne
      have hp : SumToken.right z in u := (mem_rhtFinset).1 hz
      exact (hEnts _ hp).2.1
    refine <hne', ?_>
    exact B.ent_trans (SumCon_rht_con_of_rht_nonempty A B hv hne')
      (SumCon_rht_con_of_rht_nonempty A B hu hne) hrht hB

end InfoSys

end Scott1982

```

A.28 Scott1982/Proposition64.lean

837 lines.

Lean 4 source (lines 1–400)

/-

Copyright (c) 2026 Lars Warren Ericson. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Lars Warren Ericson.

Github: <https://github.com/catskillsresearch/scott1982>

-/

```
import Scott1982.Sum
```

```
import Scott1982.Factoid33
```

```
import Scott1982.Proposition53
```

```
import Scott1982.Proposition55
```

/-!

Proposition 6.4 -- sum injections and copairing

****Scott 1982, Proposition 6.4.**** Approximable ``inl``, ``inr``, and unique copairing

``[f, g]`` with ``[f, g] comp inl = f``, ``[f, g] comp inr = g``, and ``[f, g](False) = False``.

-/

```
namespace Scott1982
```

```
open Scott1982.Constructive
```

```
namespace InfoSys
```

```
variable {a b ? : Type*} [DecidableEq a] [DecidableEq b] [DecidableEq ?]
```

```
variable (A : InfoSys a) (B : InfoSys b)
```

```
set_option linter.unusedSectionVars false
```

```
private instance instLeftCommutativeLiftSumLeft :
```

```
LeftCommutative fun x : a =>
```

```
(insert (SumToken.left (b := b) x) : Finset (SumToken a b) -> Finset _) :=
```

```
<fun _ _ s => insert_comm' _ _ s>
```

```
private instance instLeftCommutativeLiftSumRight :
```

```
LeftCommutative fun y : b =>
```

```
(insert (SumToken.right (a := a) y) : Finset (SumToken a b) -> Finset _) :=
```

```
<fun _ _ s => insert_comm' _ _ s>
```

/-- Embed ``A``-tokens as left sum tokens. -/

```
def liftSumLeft (v : Finset a) : Finset (SumToken a b) :=
```

```
Multiset.foldr (fun x : a => insert (SumToken.left (b := b) x))
```

```
({} : Finset (SumToken a b)) v.1
```

/-- Embed ``B``-tokens as right sum tokens. -/

```
def liftSumRight (w : Finset b) : Finset (SumToken a b) :=
```

```
Multiset.foldr (fun y : b => insert (SumToken.right (a := a) y))
```

```
({} : Finset (SumToken a b)) w.1
```

```
private theorem mem_foldr_liftSumLeft (s : Multiset a) (p : SumToken a b) :
```

```
p in Multiset.foldr (fun x : a => insert (SumToken.left (b := b) x))
```

```
({} : Finset (SumToken a b)) s <->
```

```
exists x in s, p = .left x := by
```

```
refine Multiset.induction_on s ?_ ?_
```

```
* constructor
```

```
* intro hp
```

```
exact False.elim (Finset.notMem_empty p hp)
```

```
* rintro <_, hx, _>
```

```
exact False.elim (by cases hx)
```

```
* intro x t ih
```

```
simp only [Multiset.foldr_cons, Finset.mem_insert, ih, Multiset.mem_cons]
```

```
constructor
```

```
* rintro (hp | <y, hy, hp>)
```

```
* exact <x, Or.inl rfl, hp>
```

```

    * exact <y, Or.inr hy, hp>
  * rintro <y, hy, hp>
    rcases hy with rfl | hy
    * exact Or.inl hp
    * exact Or.inr <y, hy, hp>

private theorem mem_foldr_liftSumRight (s : Multiset b) (p : SumToken a b) :
  p in Multiset.foldr (fun y : b => insert (SumToken.right (a := a) y))
    ({ } : Finset (SumToken a b)) s <->
    exists y in s, p = .right y := by
  refine Multiset.induction_on s ?_ ?_
  * constructor
    * intro hp
      exact False.elim (Finset.notMem_empty p hp)
    * rintro <_, hy, _>
      exact False.elim (by cases hy)
  * intro y t ih
    simp only [Multiset.foldr_cons, Finset.mem_insert, ih, Multiset.mem_cons]
    constructor
    * rintro (hp | <z, hz, hp>)
      * exact <y, Or.inl rfl, hp>
      * exact <z, Or.inr hz, hp>
    * rintro <z, hz, hp>
      rcases hz with rfl | hz
      * exact Or.inl hp
      * exact Or.inr <z, hz, hp>

theorem mem_liftSumLeft {v : Finset a} {p : SumToken a b} :
  p in liftSumLeft (b := b) v <-> exists x in v, p = .left x := by
  simpa [liftSumLeft] using mem_foldr_liftSumLeft (b := b) v.1 p

theorem mem_liftSumRight {w : Finset b} {p : SumToken a b} :
  p in liftSumRight (a := a) w <-> exists y in w, p = .right y := by
  simpa [liftSumRight] using mem_foldr_liftSumRight (a := a) w.1 p

theorem lftFinset_liftSumLeft (v : Finset a) :
  lftFinset (liftSumLeft (b := b) v) = v := by
  ext x
  simp only [mem_lftFinset, mem_liftSumLeft]
  exact <fun <y, hy, h> => by injection h with h'; exact h' |> hy,
    fun hx => <x, hx, rfl>>

theorem rhtFinset_liftSumLeft (v : Finset a) :
  rhtFinset (liftSumLeft (b := b) v) = { } := by
  ext y
  constructor
  * intro hy
    rcases (mem_liftSumLeft (b := b)).1 ((mem_rhtFinset).1 hy) with <_, _, h>
    exact False.elim (nomatch h)
  * intro hy
    exact False.elim (Finset.notMem_empty y hy)

theorem rhtFinset_liftSumRight (w : Finset b) :
  rhtFinset (liftSumRight (a := a) w) = w := by
  ext y
  simp only [mem_rhtFinset, mem_liftSumRight]
  exact <fun <z, hz, h> => by injection h with h'; exact h' |> hz,
    fun hy => <y, hy, rfl>>

theorem lftFinset_liftSumRight (w : Finset b) :
  lftFinset (liftSumRight (a := a) w) = { } := by
  ext x
  constructor

```

```

* intro hx
  rcases (mem_liftSumRight (a := a)).1 ((mem_lftFinset).1 hx) with <_, _, h>
  exact False.elim (nomatch h)
* intro hx
  exact False.elim (Finset.notMem_empty x hx)

theorem SumCon_liftSumLeft {v : Finset a} (hv : v in A.Con) :
  SumCon A B (liftSumLeft (b := b) v) :=
  Or.inl <by rw [lftFinset_liftSumLeft]; exact hv, rhtFinset_liftSumLeft (b := b) v>

theorem SumCon_liftSumRight {w : Finset b} (hw : w in B.Con) :
  SumCon A B (liftSumRight (a := a) w) :=
  Or.inr <lftFinset_liftSumRight (a := a) w, by rw [rhtFinset_liftSumRight]; exact hw>

theorem rht_eq_empty_of_sumEntSet {u u' : Finset (SumToken a b)}
  (hr : rhtFinset u = {}) (hEnt : (sumSystem A B).EntSet u u') :
  rhtFinset u' = {} := by
  ext y
  constructor
  * intro hy
    exact False.elim ((hEnt _ ((mem_rhtFinset).1 hy)).2.1 hr)
  * intro hy
    exact False.elim (Finset.notMem_empty y hy)

theorem lft_eq_empty_of_sumEntSet {u u' : Finset (SumToken a b)}
  (hl : lftFinset u = {}) (hEnt : (sumSystem A B).EntSet u u') :
  lftFinset u' = {} := by
  ext x
  constructor
  * intro hx
    exact False.elim ((hEnt _ ((mem_lftFinset).1 hx)).2.1 hl)
  * intro hx
    exact False.elim (Finset.notMem_empty x hx)

theorem entSet_lft_of_sumEntSet {u u' : Finset (SumToken a b)}
  (hEnt : (sumSystem A B).EntSet u u') :
  A.EntSet (lftFinset u) (lftFinset u') := fun _ hx =>
  (hEnt _ ((mem_lftFinset).1 hx)).2.2

theorem entSet_rht_of_sumEntSet {u u' : Finset (SumToken a b)}
  (hEnt : (sumSystem A B).EntSet u u') :
  B.EntSet (rhtFinset u) (rhtFinset u') := fun _ hy =>
  (hEnt _ ((mem_rhtFinset).1 hy)).2.2

theorem lft_mem_Con_of_SumCon_rht_empty {u : Finset (SumToken a b)}
  (hu : SumCon A B u) (_hr : rhtFinset u = {}) : lftFinset u in A.Con := by
  rcases hu with <hl, _> | <hl, _>
  * exact hl
  * rw [hl]; exact A.con_empty

theorem rht_mem_Con_of_SumCon_lft_empty {u : Finset (SumToken a b)}
  (hu : SumCon A B u) (_hl : lftFinset u = {}) : rhtFinset u in B.Con := by
  rcases hu with <_, hr> | <_, hr>
  * rw [hr]; exact B.con_empty
  * exact hr

namespace ApproximableMap

variable {C : InfoSys ?}

/-- **Proposition 6.4(1).** Left injection. -/
def inlMap : ApproximableMap A (sumSystem A B) where
  rel v u := v in A.Con /\ SumCon A B u /\ rhtFinset u = {} /\ A.EntSet v (lftFinset u)

```

```

rel_dom h := h.1
rel_cod h := h.2.1
empty_rel := <A.con_empty, SumCon_empty A B, by rw [rhtFinset_empty], A.entSet_empty {}>
union_right := by
  rintro v u u' <hv, hu, hr, hEnt> <_, hu', hr', hEnt'>
  refine <hv, ?_, ?_, ?_>
  * have hl : lftFinset (u U ' u') in A.Con := by
    rw [lftFinset_funion]
    exact A.con_subset
    (proposition_2_3_ii (sys := A) hv (proposition_2_3_vi (sys := A) hEnt hEnt'))
    (subset_funion_right _ _)
    exact Or.inl <hl, by rw [rhtFinset_funion, hr, hr']; rfl>
  * rw [rhtFinset_funion, hr, hr']; rfl
  * rw [lftFinset_funion]; exact proposition_2_3_vi (sys := A) hEnt hEnt'
mono := by
  rintro v v' u u' <hv, hu, hr, hEnt> hEntv hEntu hv' hu'
  refine <hv', hu', rht_eq_empty_of_sumEntSet A B hr hEntu, ?_>
  exact proposition_2_3_iv (sys := A) hv'
  (lft_mem_Con_of_SumCon_rht_empty A B hu hr)
  (proposition_2_3_iv (sys := A) hv' hv hEntv hEnt)
  (entSet_lft_of_sumEntSet A B hEntu)

/-- **Proposition 6.4(2).** Right injection. -/
def inrMap : ApproximableMap B (sumSystem A B) where
  rel w u := w in B.Con /\ SumCon A B u /\ lftFinset u = {} /\ B.entSet w (rhtFinset u)
  rel_dom h := h.1
  rel_cod h := h.2.1
  empty_rel := <B.con_empty, SumCon_empty A B, by rw [lftFinset_empty], B.entSet_empty {}>
  union_right := by
    rintro w u u' <hw, hu, hl, hEnt> <_, hu', hl', hEnt'>
    refine <hw, ?_, ?_, ?_>
    * have hrC : rhtFinset (u U ' u') in B.Con := by
      rw [rhtFinset_funion]
      exact B.con_subset
      (proposition_2_3_ii (sys := B) hw (proposition_2_3_vi (sys := B) hEnt hEnt'))
      (subset_funion_right _ _)
      exact Or.inr <by rw [lftFinset_funion, hl, hl']; rfl, hrC>
    * rw [lftFinset_funion, hl, hl']; rfl
    * rw [rhtFinset_funion]; exact proposition_2_3_vi (sys := B) hEnt hEnt'
  mono := by
    rintro w w' u u' <hw, hu, hl, hEnt> hEntw hEntu hw' hu'
    refine <hw', hu', lft_eq_empty_of_sumEntSet A B hl hEntu, ?_>
    exact proposition_2_3_iv (sys := B) hw'
    (rht_mem_Con_of_SumCon_lft_empty A B hu hl)
    (proposition_2_3_iv (sys := B) hw' hw hEntw hEnt)
    (entSet_rht_of_sumEntSet A B hEntu)

private theorem copair_union_disj (f : ApproximableMap A C) (g : ApproximableMap B C)
  {u : Finset (SumToken a b)} {s s' : Finset ?}
  (hu : SumCon A B u) (hs : s in C.Con) (hs' : s' in C.Con)
  (h : C.entSet {} s \\/ (lftFinset u != {} /\ f.rel (lftFinset u) s) \\/
    (rhtFinset u != {} /\ g.rel (rhtFinset u) s))
  (h' : C.entSet {} s' \\/ (lftFinset u != {} /\ f.rel (lftFinset u) s') \\/
    (rhtFinset u != {} /\ g.rel (rhtFinset u) s')) :
  C.entSet {} (s U ' s') \\/
    (lftFinset u != {} /\ f.rel (lftFinset u) (s U ' s')) \\/
    (rhtFinset u != {} /\ g.rel (rhtFinset u) (s U ' s')) := by
  rcases h with h | <hne, hf> | <hne, hg> <;> rcases h' with h' | <hne', hf'> | <hne', hg'>
  * exact Or.inl (proposition_2_3_vi (sys := C) h h')
  * have hf0 : f.rel (lftFinset u) s :=
    f.mono f.empty_rel (A.entSet_empty _) h
    (SumCon_lft_con_of_lft_nonempty A B hu hne') hs
    exact Or.inl (Or.inl <hne', f.union_right hf0 hf'>)

```

```

* have hg0 : g.rel (rhtFinset u) s :=
  g.mono g.empty_rel (B.entSet_empty _) h
  (SumCon_rht_con_of_rht_nonempty A B hu hne') hs
exact Or.inr (Or.inr <hne', g.union_right hg0 hg'>)
* have hf0 : f.rel (lftFinset u) s' :=
  f.mono f.empty_rel (A.entSet_empty _) h'
  (SumCon_lft_con_of_lft_nonempty A B hu hne) hs'
exact Or.inr (Or.inl <hne, f.union_right hf hf0>)
* exact Or.inr (Or.inl <hne, f.union_right hf hf'>)
* exact False.elim (hne' (SumCon_rht_empty_of_lft_nonempty A B hu hne))
* have hg0 : g.rel (rhtFinset u) s' :=
  g.mono g.empty_rel (B.entSet_empty _) h'
  (SumCon_rht_con_of_rht_nonempty A B hu hne) hs'
exact Or.inr (Or.inr <hne, g.union_right hg hg0>)
* exact False.elim (hne' (SumCon_lft_empty_of_rht_nonempty A B hu hne))
* exact Or.inr (Or.inr <hne, g.union_right hg hg'>)

/-- **Proposition 6.4(3).** Copairing of approximable maps. -/
def copairMap (f : ApproximableMap A C) (g : ApproximableMap B C) :
  ApproximableMap (sumSystem A B) C where
rel u s :=
  SumCon A B u /\ s in C.Con /\
    (C.EntSet {} s \ /
      (lftFinset u != {} /\ f.rel (lftFinset u) s) \ /
      (rhtFinset u != {} /\ g.rel (rhtFinset u) s))
rel_dom h := h.1
rel_cod h := h.2.1
empty_rel := <SumCon_empty A B, C.con_empty, Or.inl (C.entSet_empty {})>
union_right := by
  rintro u s s' <hu, hs, h> <_, hs', h'>
  have hU := copair_union_disj A B f g hu hs hs' h h'
  refine <hu, ?, hU>
  rcases hU with hE | <_, hf> | <_, hg>
  * exact C.con_subset (proposition_2_3_ii (sys := C) C.con_empty hE)
    (subset_funion_right _ _)
  * exact f.rel_cod hf
  * exact g.rel_cod hg
mono := by
  rintro u u' s s' <hu, hs, h> hEntu hEnts hu' hs'
  refine <hu', hs', ?_>
  rcases h with h | <hne, hf> | <hne, hg>
  * exact Or.inl (proposition_2_3_iv (sys := C) C.con_empty hs h hEnts)
  * have hne' : lftFinset u' != {} := by
    obtain <x, hx> := Finset.nonempty_of_ne_empty hne
    exact (hEntu _ ((mem_lftFinset).1 hx)).2.1
    exact Or.inr (Or.inl <hne',
      f.mono hf (entSet_lft_of_sumEntSet A B hEntu) hEnts
      (SumCon_lft_con_of_lft_nonempty A B hu' hne') hs'>)
  * have hne' : rhtFinset u' != {} := by
    obtain <y, hy> := Finset.nonempty_of_ne_empty hne
    exact (hEntu _ ((mem_rhtFinset).1 hy)).2.1
    exact Or.inr (Or.inr <hne',
      g.mono hg (entSet_rht_of_sumEntSet A B hEntu) hEnts
      (SumCon_rht_con_of_rht_nonempty A B hu' hne') hs'>)

theorem comp_copairMap_inlMap (f : ApproximableMap A C) (g : ApproximableMap B C) :
  comp (copairMap A B f g) (inlMap A B) = f := by
  refine ApproximableMap.ext fun v s => ?_
  constructor
  * rintro <u, <hv, hu, hr, hEnt>, <_, hs, hcop>>
    rcases hcop with hE | <_, hf> | <hne, _>
    * exact f.mono f.empty_rel (A.entSet_empty v) hE hv hs
    * exact f.mono hf hEnt (proposition_2_3_iii C hs) hv hs

```

```

* exact False.elim (hne hr)
* intro hf
-- Always lift `insert Delta v` so the left branch of copair is nonempty (avoids
-- classical `by_cases` on `v = {}`).
let vBot := insert A.bot v
have hvBot : vBot in A.Con := A.ent_con (A.ent_bot (f.rel_dom hf))
have hEntBot : A.EntSet vBot v :=
  fun _ hx => A.ent_refl hvBot (Finset.mem_insert_of_mem hx)
have hfBot : f.rel vBot s :=
  f.mono hf hEntBot (proposition_2_3_iii C (f.rel_cod hf)) hvBot (f.rel_cod hf)
refine <liftSumLeft (b := b) vBot, ?_, ?_>
* refine <f.rel_dom hf, SumCon_liftSumLeft A B hvBot,
  rhtFinset_liftSumLeft (b := b) vBot, ?_>
  rw [lftFinset_liftSumLeft]
  intro x hx
  rcases Finset.mem_insert.mp hx with hx | hx
  * subst hx; exact A.ent_bot (f.rel_dom hf)
  * exact A.ent_refl (f.rel_dom hf) hx
* refine <SumCon_liftSumLeft A B hvBot, f.rel_cod hf, Or.inr (Or.inl <?_, ?_>>>
  * rw [lftFinset_liftSumLeft]; exact Finset.insert_ne_empty A.bot v
  * rw [lftFinset_liftSumLeft]; exact hfBot

theorem comp_copairMap_inrMap (f : ApproximableMap A C) (g : ApproximableMap B C) :
  comp (copairMap A B f g) (inrMap A B) = g := by
  refine ApproximableMap.ext fun w s => ?_
  constructor
* rintro <u, <hw, hu, hl, hEnt>, <_, hs, hcop>>
  rcases hcop with hE | <hne, _> | <_, hg>
  * exact g.mono g.empty_rel (B.entSet_empty w) hE hw hs
  * exact False.elim (hne hl)
  * exact g.mono hg hEnt (proposition_2_3_iii C hs) hw hs
* intro hg
  let wBot := insert B.bot w
  have hwBot : wBot in B.Con := B.ent_con (B.ent_bot (g.rel_dom hg))
  have hEntBot : B.EntSet wBot w :=
    fun _ hy => B.ent_refl hwBot (Finset.mem_insert_of_mem hy)
  have hgBot : g.rel wBot s :=
    g.mono hg hEntBot (proposition_2_3_iii C (g.rel_cod hg)) hwBot (g.rel_cod hg)
  refine <liftSumRight (a := a) wBot, ?_, ?_>
* refine <g.rel_dom hg, SumCon_liftSumRight A B hwBot,
  lftFinset_liftSumRight (a := a) wBot, ?_>
  rw [rhtFinset_liftSumRight]
  intro y hy
  rcases Finset.mem_insert.mp hy with hy | hy
  * subst hy; exact B.ent_bot (g.rel_dom hg)
  * exact B.ent_refl (g.rel_dom hg) hy
* refine <SumCon_liftSumRight A B hwBot, g.rel_cod hg, Or.inr (Or.inr <?_, ?_>>>
  * rw [rhtFinset_liftSumRight]; exact Finset.insert_ne_empty B.bot w
  * rw [rhtFinset_liftSumRight]; exact hgBot

theorem copairMap_botElement (f : ApproximableMap A C) (g : ApproximableMap B C) :
  (copairMap A B f g).toElement (sumSystem A B).botElement = C.botElement := by
  apply le_antisymm
* intro Y <u, hu, <huCon, hs, hcop>>
  have onlyBot : forall p in u, p = SumToken.bot (a := a) (b := b) := by
    intro p hp
    have hEnt : (sumSystem A B).Ent {sumBot} p := hu (Finset.mem_coe.2 hp)
    cases p with
    | bot => rfl
    | left x =>
      exact False.elim (by
        have hne := hEnt.2.1
        have : lftFinset ({sumBot} : Finset (SumToken a b)) = {} := lftFinset_singleton_bot

```



```

    exact hne this)
| right y =>
  exact False.elim (by
    have hne := hEnt.2.1
    have : rhtFinset ({sumBot} : Finset (SumToken a b)) = {} := rhtFinset_singleton_bot
    exact hne this)
have hl : lftFinset u = {} := by
  ext x; constructor
  * intro hx; exact False.elim (nomatch onlyBot _ ((mem_lftFinset).1 hx))
  * intro hx; exact False.elim (Finset.notMem_empty x hx)
have hr : rhtFinset u = {} := by
  ext y; constructor
  * intro hy; exact False.elim (nomatch onlyBot _ ((mem_rhtFinset).1 hy))
  * intro hy; exact False.elim (Finset.notMem_empty y hy)
rcases hcop with hE | <hne, _> | <hne, _>
* exact C.ent_trans (C.con_sing C.bot) C.con_empty
  (fun _ hz => False.elim (Finset.notMem_empty _ hz))
  (hE Y (Finset.mem_singleton_self _))
* exact False.elim (hne hl)
* exact False.elim (hne hr)

```

Lean 4 source (lines 401–800)

```

* intro Y hY
refine <{}, fun _ hp => False.elim (Finset.notMem_empty _ (Finset.mem_coe.1 hp)), ?_>
refine <SumCon_empty A B, C.con_sing Y, Or.inl ?_>
intro Z hZ
have : Z = Y := Finset.mem_singleton.mp hZ
subst this
exact C.ent_trans C.con_empty (C.con_sing C.bot)
  (fun _ hz => by
    have : _ = C.bot := Finset.mem_singleton.mp hz
    subst this
    exact C.ent_bot C.con_empty)
hY

/-- Carrier of the left copy extracted from a sum element. -/
def sumLftCarrier (z : (sumSystem A B).Element) : Set a :=
  {x | SumToken.left x in z.carrier}

/-- Carrier of the right copy extracted from a sum element. -/
def sumRhtCarrier (z : (sumSystem A B).Element) : Set b :=
  {y | SumToken.right y in z.carrier}

/-- Left copy as an `A`-element, given some left token in `z`. -/
def sumElementLft (z : (sumSystem A B).Element)
  (x0 : a) (hx0 : SumToken.left x0 in z.carrier) : A.Element where
  carrier := sumLftCarrier A B z
  consistent := by
    intro Y hY
    have hsub : ((liftSumLeft (b := b) Y) : Set _) subseq z.carrier := by
      intro p hp
      rcases (mem_liftSumLeft (b := b)).1 (Finset.mem_coe.1 hp) with <x, hx, rfl>
      exact hY (Finset.mem_coe.2 hx)
    have h := lft_mem_Con_of_SumCon_rht_empty A B (z.consistent _ hsub)
    (rhtFinset_liftSumLeft (b := b) Y)
    rwa [lftFinset_liftSumLeft] at h
closed := by
  intro Y a hY hEnt
  if hempty : Y = {} then
    subst hempty
    have hEnt' : A.Ent {x0} a :=
      A.ent_trans (A.con_sing x0) A.con_empty
      (fun _ h => False.elim (Finset.notMem_empty _ h)) hEnt

```

```

    have hsub : (({SumToken.left x0} : Finset (SumToken a b)) : Set _) subseq z.carrier
:= by
  intro p hp
  have : p = .left x0 := Finset.mem_singleton.mp (Finset.mem_coe.1 hp)
  subst this; exact hx0
  have hSum : (sumSystem A B).Ent {SumToken.left x0} (.left a) := by
    refine <z.consistent _ hsub, <Finset.singleton_ne_empty _, ?_>>
    rw [lftFinset_singleton_left]; exact hEnt'
  exact z.closed _ _ hsub hSum
else
  have hsub : ((liftSumLeft (b := b) Y) : Set _) subseq z.carrier := by
    intro p hp
    rcases (mem_liftSumLeft (b := b)).1 (Finset.mem_coe.1 hp) with <x, hx, rfl>
    exact hY (Finset.mem_coe.2 hx)
  have hSum : (sumSystem A B).Ent (liftSumLeft (b := b) Y) (.left a) := by
    refine <z.consistent _ hsub, <?_, ?_>>
    * rw [lftFinset_liftSumLeft]; exact hempty
    * rw [lftFinset_liftSumLeft]; exact hEnt
  exact z.closed _ _ hsub hSum

/-- Right copy as a `B`-element, given some right token in `z`. -/
def sumElementRht (z : (sumSystem A B).Element)
  (y0 : b) (hy0 : SumToken.right y0 in z.carrier) : B.Element where
  carrier := sumRhtCarrier A B z
  consistent := by
    intro W hW
    have hsub : ((liftSumRight (a := a) W) : Set _) subseq z.carrier := by
      intro p hp
      rcases (mem_liftSumRight (a := a)).1 (Finset.mem_coe.1 hp) with <y, hy, rfl>
      exact hW (Finset.mem_coe.2 hy)
    have h := rht_mem_Con_of_SumCon_lft_empty A B (z.consistent _ hsub)
    (lftFinset_liftSumRight (a := a) W)
    rwa [rhtFinset_liftSumRight] at h
  closed := by
    intro W b hW hEnt
    if hempty : W = {} then
      subst hempty
      have hEnt' : B.Ent {y0} b :=
        B.ent_trans (B.con_sing y0) B.con_empty
        (fun _ h => False.elim (Finset.notMem_empty _ h)) hEnt
      have hsub : (({SumToken.right y0} : Finset (SumToken a b)) : Set _) subseq z.carrier
:= by
      intro p hp
      have : p = .right y0 := Finset.mem_singleton.mp (Finset.mem_coe.1 hp)
      subst this; exact hy0
      have hSum : (sumSystem A B).Ent {SumToken.right y0} (.right b) := by
        refine <z.consistent _ hsub, <Finset.singleton_ne_empty _, ?_>>
        rw [rhtFinset_singleton_right]; exact hEnt'
      exact z.closed _ _ hsub hSum
    else
      have hsub : ((liftSumRight (a := a) W) : Set _) subseq z.carrier := by
        intro p hp
        rcases (mem_liftSumRight (a := a)).1 (Finset.mem_coe.1 hp) with <y, hy, rfl>
        exact hW (Finset.mem_coe.2 hy)
      have hSum : (sumSystem A B).Ent (liftSumRight (a := a) W) (.right b) := by
        refine <z.consistent _ hsub, <?_, ?_>>
        * rw [rhtFinset_liftSumRight]; exact hempty
        * rw [rhtFinset_liftSumRight]; exact hEnt
      exact z.closed _ _ hsub hSum

theorem not_mem_right_of_mem_left (z : (sumSystem A B).Element)
  {x : a} {y : b} (hx : SumToken.left x in z.carrier)
  (hy : SumToken.right y in z.carrier) : False := by

```

```

let u : Finset (SumToken a b) := insert (.left x) {SumToken.right y}
have hsub : (u : Set _) subseq z.carrier := by
  intro p hp
  rcases Finset.mem_insert.mp (Finset.mem_coe.1 hp) with h | h
  * subst h; exact hx
  * have : p = .right y := Finset.mem_singleton.mp h
    subst this; exact hy
have hCon := z.consistent u hsub
have hlne : lftFinset u != {} := by
  intro h
  have : x in lftFinset u := (mem_lftFinset).2 (Finset.mem_insert_self _ _)
  rw [h] at this
  exact Finset.notMem_empty _ this
have hrne : rhtFinset u != {} := by
  intro h
  have : y in rhtFinset u :=
    (mem_rhtFinset).2 (Finset.mem_insert_of_mem (Finset.mem_singleton_self _))
  rw [h] at this
  exact Finset.notMem_empty _ this
rcases hCon with <_, hr> | <hl, _>
* exact hrne hr
* exact hlne hl

theorem inlMap_toElement_sumElementLft (z : (sumSystem A B).Element)
  (x0 : a) (hx0 : SumToken.left x0 in z.carrier) :
  (inlMap A B).toElement (sumElementLft A B z x0 hx0) = z := by
  apply le_antisymm
  * intro p <v, hv, <hvCon, huCon, hr, hEnt>>
    cases p with
    | bot => exact factoid_3_2 (sumSystem A B) z
    | left x =>
      have hEntx : A.Ent v x := by
        have : lftFinset ({SumToken.left x} : Finset (SumToken a b)) = {x} :=
          lftFinset_singleton_left x
        simpa [this] using hEnt x (Finset.mem_singleton_self x)
        exact (sumElementLft A B z x0 hx0).closed v x hv hEntx
      | right y =>
        have : rhtFinset ({SumToken.right y} : Finset (SumToken a b)) = {y} :=
          rhtFinset_singleton_right y
        exact False.elim (Finset.singleton_ne_empty y (this |> hr))
  * intro p hp
    cases p with
    | bot =>
      refine <{}, fun _ h => False.elim (Finset.notMem_empty _ (Finset.mem_coe.1 h)), ?_>
      exact <A.con_empty, (sumSystem A B).con_sing _, by rw [rhtFinset_singleton_bot],
        A.entSet_empty _>
    | left x =>
      refine <{x}, ?_, ?_>
      * intro a ha
        have : a = x := Finset.mem_singleton.mp (Finset.mem_coe.1 ha)
        subst this; exact hp
      * refine <A.con_sing x, (sumSystem A B).con_sing _, ?_, ?_>
      * ext y; constructor
      * intro hy
        have : SumToken.right y in ({SumToken.left x} : Finset _) := (mem_rhtFinset).1 hy
        exact False.elim (nomatch Finset.mem_singleton.mp this)
      * intro hy; exact False.elim (Finset.notMem_empty y hy)
      * rw [lftFinset_singleton_left]
        exact proposition_2_3_iii A (A.con_sing x)
    | right y =>
      exact False.elim (not_mem_right_of_mem_left A B z hx0 hp)

theorem inrMap_toElement_sumElementRht (z : (sumSystem A B).Element)

```

```

(y0 : b) (hy0 : SumToken.right y0 in z.carrier) :
  (inrMap A B).toElement (sumElementRht A B z y0 hy0) = z := by
apply le_antisymm
* intro p <w, hw, <hwCon, huCon, hl, hEnt>>
cases p with
| bot => exact factoid_3_2 (sumSystem A B) z
| right y =>
  have hEnty : B.Ent w y := by
    have : rhtFinset ({SumToken.right y} : Finset (SumToken a b)) = {y} :=
      rhtFinset_singleton_right y
    simpa [this] using hEnt y (Finset.mem_singleton_self y)
  exact (sumElementRht A B z y0 hy0).closed w y hw hEnty
| left x =>
  have : lftFinset ({SumToken.left x} : Finset (SumToken a b)) = {x} :=
    lftFinset_singleton_left x
  exact False.elim (Finset.singleton_ne_empty x (this |> hl))
* intro p hp
cases p with
| bot =>
  refine <{}, fun _ h => False.elim (Finset.notMem_empty _ (Finset.mem_coe.1 h)), ?_>
  exact <B.con_empty, (sumSystem A B).con_sing _, by rw [lftFinset_singleton_bot],
    B.entSet_empty _>
| right y =>
  refine <{y}, ?_, ?_>
  * intro b hb
  have : b = y := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
  subst this; exact hp
  * refine <B.con_sing y, (sumSystem A B).con_sing _, ?_, ?_>
  * ext x; constructor
  * intro hx
  have : SumToken.left x in ({SumToken.right y} : Finset _) := (mem_lftFinset).1 hx
  exact False.elim (nomatch Finset.mem_singleton.mp this)
  * intro hx; exact False.elim (Finset.notMem_empty x hx)
  * rw [rhtFinset_singleton_right]
  exact proposition_2_3_iii B (B.con_sing y)
| left x =>
  exact False.elim (not_mem_right_of_mem_left A B z hp hy0)

theorem eq_botElement_of_no_injections (z : (sumSystem A B).Element)
  (hL : forall x : a, SumToken.left x notin z.carrier)
  (hR : forall y : b, SumToken.right y notin z.carrier) :
  z = (sumSystem A B).botElement := by
apply le_antisymm
* intro p hp
cases p with
| bot =>
  change SumToken.bot in (sumSystem A B).botElement.carrier
  exact (sumSystem A B).ent_refl ((sumSystem A B).con_sing _) (Finset.mem_singleton_self _)
| left x => exact False.elim (hL x hp)
| right y => exact False.elim (hR y hp)
* exact botElement_le (sumSystem A B) z

/-- Choice-free emptiness dichotomy for `lftFinset` (avoids `Finset.decidableEq`). -/
theorem lftFinset_eq_empty_or_mem (u : Finset (SumToken a b)) :
  lftFinset u = {} \ / exists x, x in lftFinset u := by
induction u using Finset.induction_on with
| empty =>
  exact Or.inl lftFinset_empty
| insert p s _hp ih =>
  cases p with
  | left x =>
    exact Or.inr <x, by rw [lftFinset_insert_left]; exact Finset.mem_insert_self _ _>
  | right y =>

```

```

    rw [lftFinset_insert_right]; exact ih
| bot =>
    rw [lftFinset_insert_bot]; exact ih

/-- Choice-free emptiness dichotomy for `rhtFinset`. -/
theorem rhtFinset_eq_empty_or_mem (u : Finset (SumToken a b)) :
  rhtFinset u = {} \ / exists y, y in rhtFinset u := by
  induction u using Finset.induction_on with
| empty =>
  exact Or.inl rhtFinset_empty
| insert p s _hp ih =>
  cases p with
| right y =>
  exact Or.inr <y, by rw [rhtFinset_insert_right]; exact Finset.mem_insert_self _ _>
| left x =>
  rw [rhtFinset_insert_left]; exact ih
| bot =>
  rw [rhtFinset_insert_bot]; exact ih

theorem entSet_liftLft_of_left_only {u : Finset (SumToken a b)}
  (hu : SumCon A B u) (hr : rhtFinset u = {}) :
  (sumSystem A B).EntSet (liftSumLeft (b := b) (lftFinset u)) u := by
  intro p hp
  cases p with
| bot =>
  exact <SumCon_liftSumLeft A B (lft_mem_Con_of_SumCon_rht_empty A B hu hr), trivial>
| left x =>
  have hx : x in lftFinset u := (mem_lftFinset).2 hp
  refine <SumCon_liftSumLeft A B (lft_mem_Con_of_SumCon_rht_empty A B hu hr), <?_, ?_>>
  * rw [lftFinset_liftSumLeft]; exact Finset.ne_empty_of_mem hx
  * rw [lftFinset_liftSumLeft]
  exact A.ent_refl (lft_mem_Con_of_SumCon_rht_empty A B hu hr) hx
| right y =>
  exact False.elim (by
    have : y in rhtFinset u := (mem_rhtFinset).2 hp
    rw [hr] at this
    exact Finset.notMem_empty _ this)

theorem entSet_liftRht_of_right_only {u : Finset (SumToken a b)}
  (hu : SumCon A B u) (hl : lftFinset u = {}) :
  (sumSystem A B).EntSet (liftSumRight (a := a) (rhtFinset u)) u := by
  intro p hp
  cases p with
| bot =>
  exact <SumCon_liftSumRight A B (rht_mem_Con_of_SumCon_lft_empty A B hu hl), trivial>
| right y =>
  have hy : y in rhtFinset u := (mem_rhtFinset).2 hp
  refine <SumCon_liftSumRight A B (rht_mem_Con_of_SumCon_lft_empty A B hu hl), <?_, ?_>>
  * rw [rhtFinset_liftSumRight]; exact Finset.ne_empty_of_mem hy
  * rw [rhtFinset_liftSumRight]
  exact B.ent_refl (rht_mem_Con_of_SumCon_lft_empty A B hu hl) hy
| left x =>
  exact False.elim (by
    have : x in lftFinset u := (mem_lftFinset).2 hp
    rw [hl] at this
    exact Finset.notMem_empty _ this)

theorem subset_botElement_of_projections_empty {u : Finset (SumToken a b)}
  (hl : lftFinset u = {}) (hr : rhtFinset u = {}) :
  (u : Set (SumToken a b)) subseq (sumSystem A B).botElement.carrier := by
  intro p hp
  have hp' : p in u := Finset.mem_coe.1 hp
  cases p with

```

```

| bot =>
  exact (sumSystem A B).ent_refl ((sumSystem A B).con_sing _) (Finset.mem_singleton_self _)
| left x =>
  exact False.elim (by
    have : x in lftFinset u := (mem_lftFinset).2 hp'
    rw [hl] at this; exact Finset.notMem_empty _ this)
| right y =>
  exact False.elim (by
    have : y in rhtFinset u := (mem_rhtFinset).2 hp'
    rw [hr] at this; exact Finset.notMem_empty _ this)

theorem entSet_empty_of_rel_bot_like (h : ApproximableMap (sumSystem A B) C)
  (hbot : h.toElement (sumSystem A B).botElement = C.botElement)
  {u : Finset (SumToken a b)} {s : Finset ?}
  (hu : SumCon A B u) (hs : s in C.Con) (hl : lftFinset u = {}) (hr : rhtFinset u = {})
  (hh : h.rel u s) : C.EntSet {} s := by
  have hsub := subset_botElement_of_projections_empty A B hl hr
  have hcl : (sumSystem A B).closure u hu <= (sumSystem A B).botElement := by
    intro p hp
    exact (sumSystem A B).botElement.closed u p hsub hp
  have hle : C.closure s hs <= C.botElement := by
    have := (h.rel_iff_closure_le hu hs).1 hh
    have := le_trans this (toElement_mono h hcl)
    rwa [hbot] at this
  intro Y hY
  have hY' : Y in (C.closure s hs).carrier := C.subset_closure hs hY
  have hbotY : Y in C.botElement.carrier := hle hY'
  exact C.ent_trans C.con_empty (C.con_sing C.bot)
  (fun _ hz => by
    have : _ = C.bot := Finset.mem_singleton.mp hz
    subst this; exact C.ent_bot C.con_empty)
  hbotY

theorem entSet_u_of_inl_witness {u u' : Finset (SumToken a b)} {v : Finset a}
  (hlne : lftFinset u != {}) (hinl : (inlMap A B).rel v u')
  (huv : A.EntSet (lftFinset u) v) (hu : SumCon A B u)
  (hr : rhtFinset u = {}) :
  (sumSystem A B).EntSet u u' := by
  intro p hp
  have <hv, hu', hr', hEnt> := hinl
  cases p with
  | bot => exact <hu, trivial>
  | left x =>
    have hx : x in lftFinset u' := (mem_lftFinset).2 hp
    have hAx : A.Ent v x := hEnt x hx
    have hAx' : A.Ent (lftFinset u) x :=
      A.ent_trans (lft_mem_Con_of_SumCon_rht_empty A B hu hr) hv huv hAx
    exact <hu, <hlne, hAx'>>
  | right y =>
    exact False.elim (by
      have : y in rhtFinset u' := (mem_rhtFinset).2 hp
      rw [hr'] at this; exact Finset.notMem_empty _ this)

theorem entSet_u_of_inr_witness {u u' : Finset (SumToken a b)} {w : Finset b}
  (hrne : rhtFinset u != {}) (hinr : (inrMap A B).rel w u')
  (huw : B.EntSet (rhtFinset u) w) (hu : SumCon A B u)
  (hl : lftFinset u = {}) :
  (sumSystem A B).EntSet u u' := by
  intro p hp
  have <hw, hu', hl', hEnt> := hinr
  cases p with
  | bot => exact <hu, trivial>
  | right y =>

```

```

have hy : y in rhtFinset u' := (mem_rhtFinset).2 hp
have hBy : B.Ent w y := hEnt y hy
have hBy' : B.Ent (rhtFinset u) y :=
  B.ent_trans (rht_mem_Con_of_SumCon_lft_empty A B hu hl) hw huw hBy
exact <hu, <hrne, hBy'>>
| left x =>
  exact False.elim (by
    have : x in lftFinset u' := (mem_lftFinset).2 hp
    rw [hl'] at this; exact Finset.notMem_empty _ this)

/-- Uniqueness of copairing (Scott Prop 6.4). -/
theorem copairMap_unique (f : ApproximableMap A C) (g : ApproximableMap B C)
  (h : ApproximableMap (sumSystem A B) C)
  (hinl : comp h (inlMap A B) = f) (hinr : comp h (inrMap A B) = g)
  (hbot : h.toElement (sumSystem A B).botElement = C.botElement) :
  h = copairMap A B f g := by
  refine ApproximableMap.ext fun u s => ?_
  constructor
  * intro hh
  have hu : SumCon A B u := h.rel_dom hh
  have hs : s in C.Con := h.rel_cod hh
  refine <hu, hs, ?_>
  rcases lftFinset_eq_empty_or_mem u with hl | <x0, hx0>
  * rcases rhtFinset_eq_empty_or_mem u with hr | <y0, hy0>
  * exact Or.inl (entSet_empty_of_rel_bot_like A B h hbot hu hs hl hr hh)
  * have hl' : lftFinset u = {} := hl
  have hg : g.rel (rhtFinset u) s := by
    have hinr' : (comp h (inrMap A B)).rel (rhtFinset u) s := by
      refine <liftSumRight (a := a) (rhtFinset u), ?_, ?_>
      * exact <rht_mem_Con_of_SumCon_lft_empty A B hu hl',
        SumCon_liftSumRight A B (rht_mem_Con_of_SumCon_lft_empty A B hu hl'),
        lftFinset_liftSumRight (a := a) _, by
          rw [rhtFinset_liftSumRight]
          exact proposition_2_3_iii B
          (rht_mem_Con_of_SumCon_lft_empty A B hu hl')>
      * exact h.mono hh (entSet_liftRht_of_right_only A B hu hl')
      (proposition_2_3_iii C hs)
      (SumCon_liftSumRight A B (rht_mem_Con_of_SumCon_lft_empty A B hu hl')) hs
    simp [hinr] using hinr'
  exact Or.inr (Or.inr <Finset.ne_empty_of_mem hy0, hg>)
  * have hne : lftFinset u != {} := Finset.ne_empty_of_mem hx0
  have hr : rhtFinset u = {} := SumCon_rht_empty_of_lft_nonempty A B hu hne
  have hf : f.rel (lftFinset u) s := by
    have hinl' : (comp h (inlMap A B)).rel (lftFinset u) s := by
      refine <liftSumLeft (b := b) (lftFinset u), ?_, ?_>
      * exact <lft_mem_Con_of_SumCon_rht_empty A B hu hr,

```

Lean 4 source (lines 801–837)

```

SumCon_liftSumLeft A B (lft_mem_Con_of_SumCon_rht_empty A B hu hr),
rhtFinset_liftSumLeft (b := b) _, by
  rw [lftFinset_liftSumLeft]
  exact proposition_2_3_iii A
  (lft_mem_Con_of_SumCon_rht_empty A B hu hr)>
  * exact h.mono hh (entSet_liftLft_of_left_only A B hu hr)
  (proposition_2_3_iii C hs)
  (SumCon_liftSumLeft A B (lft_mem_Con_of_SumCon_rht_empty A B hu hr)) hs
  simp [hinl] using hinl'
  exact Or.inr (Or.inl <hne, hf>)
* intro hc
rcases hc with <hu, hs, hcop>
rcases hcop with hE | <hne, hf> | <hne, hg>
* have hb : h.rel {} s :=
  h.mono h.empty_rel ((sumSystem A B).entSet_empty _) hE

```

```

      (sumSystem A B).con_empty hs
    exact h.mono hb ((sumSystem A B).entSet_empty _) (proposition_2_3_iii C hs) hu hs
  * have : (comp h (inlMap A B)).rel (lftFinset u) s := by simpa [hinl] using hf
    rcases this with <u', hinl', hh'>
    have hr : rhtFinset u = {} := SumCon_rht_empty_of_lft_nonempty A B hu hne
    have hEnt : (sumSystem A B).EntSet u u' :=
      entSet_u_of_inl_witness A B hne hinl'
      (proposition_2_3_iii A (lft_mem_Con_of_SumCon_rht_empty A B hu hr)) hu hr
    exact h.mono hh' hEnt (proposition_2_3_iii C hs) hu hs
  * have : (comp h (inrMap A B)).rel (rhtFinset u) s := by simpa [hinr] using hg
    rcases this with <u', hinr', hh'>
    have hl : lftFinset u = {} := SumCon_lft_empty_of_rht_nonempty A B hu hne
    have hEnt : (sumSystem A B).EntSet u u' :=
      entSet_u_of_inr_witness A B hne hinr'
      (proposition_2_3_iii B (rht_mem_Con_of_SumCon_lft_empty A B hu hl)) hu hl
    exact h.mono hh' hEnt (proposition_2_3_iii C hs) hu hs

```

end ApproximableMap

end InfoSys

end Scott1982

A.29 Scott1982/Factoid65.lean

82 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Proposition56
import Scott1982.Factoid33

/#!
# Factoid 6.5 -- the unit domain

**Scott 1982 (end of S6, after Prop 6.2).** The trivial product of no factors is the
*unit* information system `1` with `D_1 = {Delta_1}`. It has a unique element `False_1`.
Every approximable map `1 -> A` is constant, and there is a unique approximable map
`A -> 1`, namely `const(False_1)`.
-/

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys

/-- **Factoid 6.5.** The unit information system on a singleton token type. -/
def unitSystem : InfoSys PUnit where
  bot := <>
  Con := Set.univ
  Ent := fun _ _ => True
  con_subset := by
    intro u v _ _
    exact trivial
  con_sing := by

```



```

    intro _
    exact trivial
ent_con := by
  intro u a _
  exact trivial
ent_bot := by
  intro u _
  exact trivial
ent_refl := by
  intro u a _ _
  exact trivial
ent_trans := by
  intro u v c _ _ _ _
  exact trivial

/-- The unique element of `|1|` is `False`. -/
theorem unitElement_eq_bot (x : unitSystem.Element) : x = unitSystem.botElement := by
  apply le_antisymm
  * intro a ha
  cases a
  exact trivial
  * exact botElement_le unitSystem x

namespace ApproximableMap

variable {a : Type*} [DecidableEq a] {A : InfoSys a}

/-- Every approximable map out of `1` is constant (Scott Factoid 6.5). -/
theorem approxMap_from_unit_eq_const (f : ApproximableMap unitSystem A) :
  f = constMap (A := unitSystem) (f.toElement unitSystem.botElement) :=
  constMap_unique (f.toElement unitSystem.botElement) f fun x => by
    rw [unitElement_eq_bot x]

/-- There is a unique approximable map into `1`, namely `const(False_1)`. -/
theorem approxMap_to_unit_eq_const (f : ApproximableMap A unitSystem) :
  f = constMap (A := A) unitSystem.botElement :=
  constMap_unique unitSystem.botElement f fun x => unitElement_eq_bot (f.toElement x)

/-- Elements of `|A|` are in bijection with approximable maps `1 -> A` via `const`. -/
theorem toElement_constMap_bot (b : A.Element) :
  (constMap (A := unitSystem) b).toElement unitSystem.botElement = b :=
  constMap_toElement b unitSystem.botElement

end ApproximableMap

end InfoSys

end Scott1982

```

A.30 Scott1982/FunctionSpace.lean

549 lines.

Lean 4 source (lines 1–400)

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Constructive

```

```

import Scott1982.Proposition23

/-!
# Function space -- Definition 7.1

**Scott 1982, Definition 7.1.** Tokens of  $A \rightarrow B$  are pairs  $(u, v)$  of consistent
sets. Consistency of a finite set  $w$  of such pairs asks that whenever a selection of
inputs is jointly consistent, the corresponding outputs are jointly consistent.
Entailment is the least approximable map generated by  $w$ :
 $w \Vdash (u', v')$  iff the union of those  $v_i$  whose  $u_i$  are entailed by  $u'$  entails  $v'$ .

Constructively we package the right-hand side of (iv) as an existential witness: some
 $s \subseteq w$  of pairs whose inputs are entailed by  $u'$  and whose output-union entails  $v'$ 
(avoiding an undecidable filter on EntSet).
-/

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys

set_option linter.unusedSectionVars false

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

/-- Token type of the function space  $A \rightarrow B$  (Scott 7.1(i)). -/
def FunToken (A : InfoSys a) (B : InfoSys b) : Type _ :=
  {p : Finset a  $\times$  Finset b // p.1 in A.Con  $\wedge$  p.2 in B.Con}

instance (A : InfoSys a) (B : InfoSys b) : DecidableEq (FunToken A B) :=
  fun p q =>
    match decidableEq_finset p.val.1 q.val.1, decidableEq_finset p.val.2 q.val.2 with
    | isTrue h1, isTrue h2 => isTrue (Subtype.ext (Prod.ext h1 h2))
    | isFalse n1, _ =>
      isFalse fun h => n1 (congrArg (fun r : FunToken A B => r.val.1) h)
    | isTrue _, isFalse n2 =>
      isFalse fun h => n2 (congrArg (fun r : FunToken A B => r.val.2) h)

variable (A : InfoSys a) (B : InfoSys b)

/-- The empty pair is a function-space token. -/
theorem funBot_property : ({ } : Finset a) in A.Con  $\wedge$  ({ } : Finset b) in B.Con :=
  <A.con_empty, B.con_empty>

/-- Function-space bottom  $\Delta_{A \rightarrow B} = ({ }, { })$  (Scott 7.1(ii)). -/
def funBot : FunToken A B :=
  <({ }, { }), funBot_property A B>

theorem funion_left_comm (a b c : Finset a) :
  a  $\cup$  (b  $\cup$  c) = b  $\cup$  (a  $\cup$  c) := by
  ext x
  constructor
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inl h)))
    * rcases mem_funion.mp h with h | h
      * exact mem_funion.mpr (Or.inl h)
      * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inr h)))
  * intro hx
    rcases mem_funion.mp hx with h | h

```

```

    * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inl h)))
    * rcases mem_funion.mp h with h | h
    * exact mem_funion.mpr (Or.inl h)
    * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inr h)))

theorem funion_left_comm' (a b c : Finset b) :
  a U ' (b U ' c) = b U ' (a U ' c) := by
  ext x
  constructor
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inl h)))
    * rcases mem_funion.mp h with h | h
    * exact mem_funion.mpr (Or.inl h)
    * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inr h)))
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inl h)))
    * rcases mem_funion.mp h with h | h
    * exact mem_funion.mpr (Or.inl h)
    * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inr h)))

private theorem funion_assoc (a b c : Finset a) :
  (a U ' b) U ' c = a U ' (b U ' c) := by
  ext x
  constructor
  * intro hx
    rcases mem_funion.mp hx with h | h
    * rcases mem_funion.mp h with h | h
    * exact mem_funion.mpr (Or.inl h)
    * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inl h)))
    * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inr h)))
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact mem_funion.mpr (Or.inl (mem_funion.mpr (Or.inl h)))
    * rcases mem_funion.mp h with h | h
    * exact mem_funion.mpr (Or.inl (mem_funion.mpr (Or.inr h)))
    * exact mem_funion.mpr (Or.inr h)

private theorem funion_comm (a b : Finset a) : a U ' b = b U ' a := by
  ext x
  constructor
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact mem_funion.mpr (Or.inr h)
    * exact mem_funion.mpr (Or.inl h)
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact mem_funion.mpr (Or.inr h)
    * exact mem_funion.mpr (Or.inl h)

private theorem funion_assoc' (a b c : Finset b) :
  (a U ' b) U ' c = a U ' (b U ' c) := by
  ext x
  constructor
  * intro hx
    rcases mem_funion.mp hx with h | h
    * rcases mem_funion.mp h with h | h
    * exact mem_funion.mpr (Or.inl h)
    * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inl h)))
    * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inr h)))
  * intro hx
    rcases mem_funion.mp hx with h | h

```

```

    * exact mem_funion.mpr (Or.inl (mem_funion.mpr (Or.inl h)))
    * rcases mem_funion.mp h with h | h
    * exact mem_funion.mpr (Or.inl (mem_funion.mpr (Or.inr h)))
    * exact mem_funion.mpr (Or.inr h)

private theorem funion_comm' (a b : Finset b) : a ∪ b = b ∪ a := by
  ext x
  constructor
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact mem_funion.mpr (Or.inr h)
    * exact mem_funion.mpr (Or.inl h)
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact mem_funion.mpr (Or.inr h)
    * exact mem_funion.mpr (Or.inl h)

instance instLeftCommutativeFunInput :
  LeftCommutative fun p : FunToken A B => (funion p.val.1 : Finset a -> Finset a) :=
  <fun p q s => funion_left_comm p.val.1 q.val.1 s>

instance instLeftCommutativeFunOutput :
  LeftCommutative fun p : FunToken A B => (funion p.val.2 : Finset b -> Finset b) :=
  <fun p q s => funion_left_comm' p.val.2 q.val.2 s>

/-- Union of input components of a finite set of function-space tokens. -/
def funInputUnion (s : Finset (FunToken A B)) : Finset a :=
  Multiset.foldr (fun p : FunToken A B => funion p.val.1) ({} : Finset a) s.1

/-- Union of output components of a finite set of function-space tokens. -/
def funOutputUnion (s : Finset (FunToken A B)) : Finset b :=
  Multiset.foldr (fun p : FunToken A B => funion p.val.2) ({} : Finset b) s.1

private theorem mem_foldr_funInput (t : Multiset (FunToken A B)) (x : a) :
  x in Multiset.foldr (fun p : FunToken A B => funion p.val.1) ({} : Finset a) t <=>
    exists p in t, x in p.val.1 := by
  refine Multiset.induction_on t ?_ ?_
  * constructor
    * intro hx
      exact False.elim (Finset.notMem_empty x hx)
    * rintro <_, hp, _>
      exact False.elim (by cases hp)
  * intro p t ih
    simp only [Multiset.foldr_cons, mem_funion, ih, Multiset.mem_cons]
    constructor
    * rintro (hx | <q, hq, hx>)
      * exact <p, Or.inl rfl, hx>
      * exact <q, Or.inr hq, hx>
    * rintro <q, hq, hx>
      rcases hq with rfl | hq
      * exact Or.inl hx
      * exact Or.inr <q, hq, hx>

private theorem mem_foldr_funOutput (t : Multiset (FunToken A B)) (y : b) :
  y in Multiset.foldr (fun p : FunToken A B => funion p.val.2) ({} : Finset b) t <=>
    exists p in t, y in p.val.2 := by
  refine Multiset.induction_on t ?_ ?_
  * constructor
    * intro hy
      exact False.elim (Finset.notMem_empty y hy)
    * rintro <_, hp, _>
      exact False.elim (by cases hp)
  * intro p t ih

```

```

simp only [Multiset.foldr_cons, mem_funion, ih, Multiset.mem_cons]
constructor
* rintro (hy | <q, hq, hy>)
  * exact <p, Or.inl rfl, hy>
  * exact <q, Or.inr hq, hy>
* rintro <q, hq, hy>
  rcases hq with rfl | hq
  * exact Or.inl hy
  * exact Or.inr <q, hq, hy>

theorem mem_funInputUnion {s : Finset (FunToken A B)} {x : a} :
  x in funInputUnion A B s <-> exists p in s, x in p.val.1 := by
  simpa [funInputUnion] using mem_foldr_funInput A B s.1 x

theorem mem_funOutputUnion {s : Finset (FunToken A B)} {y : b} :
  y in funOutputUnion A B s <-> exists p in s, y in p.val.2 := by
  simpa [funOutputUnion] using mem_foldr_funOutput A B s.1 y

theorem funInputUnion_empty : funInputUnion A B ({ } : Finset (FunToken A B)) = { } :=
  rfl

theorem funOutputUnion_empty : funOutputUnion A B ({ } : Finset (FunToken A B)) = { } :=
  rfl

theorem funInputUnion_singleton (p : FunToken A B) :
  funInputUnion A B {p} = p.val.1 := by
  ext x
  constructor
  * intro hx
    rcases (mem_funInputUnion A B).1 hx with <q, hq, hx'>
    have : q = p := Finset.mem_singleton.mp hq
    subst this
    exact hx'
  * intro hx
    exact (mem_funInputUnion A B).2 <p, Finset.mem_singleton_self p, hx>

theorem funOutputUnion_singleton (p : FunToken A B) :
  funOutputUnion A B {p} = p.val.2 := by
  ext y
  constructor
  * intro hy
    rcases (mem_funOutputUnion A B).1 hy with <q, hq, hy'>
    have : q = p := Finset.mem_singleton.mp hq
    subst this
    exact hy'
  * intro hy
    exact (mem_funOutputUnion A B).2 <p, Finset.mem_singleton_self p, hy>

theorem funInputUnion_funion (s t : Finset (FunToken A B)) :
  funInputUnion A B (s U ' t) = funInputUnion A B s U ' funInputUnion A B t := by
  ext x
  constructor
  * intro hx
    rcases (mem_funInputUnion A B).1 hx with <p, hp, hx'>
    rcases mem_funion.mpr hp with hp | hp
    * exact mem_funion.mpr (Or.inl ((mem_funInputUnion A B).2 <p, hp, hx'>))
    * exact mem_funion.mpr (Or.inr ((mem_funInputUnion A B).2 <p, hp, hx'>))
  * intro hx
    rcases mem_funion.mp hx with hx | hx
    * rcases (mem_funInputUnion A B).1 hx with <p, hp, hx'>
      exact (mem_funInputUnion A B).2 <p, mem_funion.mpr (Or.inl hp), hx'>
    * rcases (mem_funInputUnion A B).1 hx with <p, hp, hx'>
      exact (mem_funInputUnion A B).2 <p, mem_funion.mpr (Or.inr hp), hx'>

```

```

theorem funOutputUnion_funion (s t : Finset (FunToken A B)) :
  funOutputUnion A B (s U ' t) = funOutputUnion A B s U ' funOutputUnion A B t := by
  ext y
  constructor
  * intro hy
    rcases (mem_funOutputUnion A B).1 hy with <p, hp, hy'>
    rcases mem_funion.mpr hp with hp | hp
    * exact mem_funion.mpr (Or.inl ((mem_funOutputUnion A B).2 <p, hp, hy'>))
    * exact mem_funion.mpr (Or.inr ((mem_funOutputUnion A B).2 <p, hp, hy'>))
  * intro hy
    rcases mem_funion.mpr hy with hy | hy
    * rcases (mem_funOutputUnion A B).1 hy with <p, hp, hy'>
      exact (mem_funOutputUnion A B).2 <p, mem_funion.mpr (Or.inl hp), hy'>
    * rcases (mem_funOutputUnion A B).1 hy with <p, hp, hy'>
      exact (mem_funOutputUnion A B).2 <p, mem_funion.mpr (Or.inr hp), hy'>

theorem funInputUnion_insert (s : Finset (FunToken A B)) (p : FunToken A B) :
  funInputUnion A B (insert p s) = p.val.1 U ' funInputUnion A B s := by
  have h : insert p s = ({p} : Finset (FunToken A B)) U ' s := by
    ext q
    constructor
    * intro hq
      rcases Finset.mem_insert.mpr hq with hqp | hq
      * exact mem_funion.mpr (Or.inl (hqp |> Finset.mem_singleton_self p))
      * exact mem_funion.mpr (Or.inr hq)
    * intro hq
      rcases mem_funion.mpr hq with hq | hq
      * exact Finset.mem_insert.mpr (Or.inl (Finset.mem_singleton.mpr hq))
      * exact Finset.mem_insert_of_mem hq
  rw [h, funInputUnion_funion, funInputUnion_singleton]

theorem funOutputUnion_insert (s : Finset (FunToken A B)) (p : FunToken A B) :
  funOutputUnion A B (insert p s) = p.val.2 U ' funOutputUnion A B s := by
  have h : insert p s = ({p} : Finset (FunToken A B)) U ' s := by
    ext q
    constructor
    * intro hq
      rcases Finset.mem_insert.mpr hq with hqp | hq
      * exact mem_funion.mpr (Or.inl (hqp |> Finset.mem_singleton_self p))
      * exact mem_funion.mpr (Or.inr hq)
    * intro hq
      rcases mem_funion.mpr hq with hq | hq
      * exact Finset.mem_insert.mpr (Or.inl (Finset.mem_singleton.mpr hq))
      * exact Finset.mem_insert_of_mem hq
  rw [h, funOutputUnion_funion, funOutputUnion_singleton]

/-- Consistency for the function space (Scott 7.1(iii)). -/
def FunCon (w : Finset (FunToken A B)) : Prop :=
  forall s subseteq w, funInputUnion A B s in A.Con -> funOutputUnion A B s in B.Con

/-- Entailment for the function space (Scott 7.1(iv), constructive witness form).
Includes `FunCon` so `ent_con` has the ambient consistency hypothesis. -/
def FunEnt (w : Finset (FunToken A B)) (p : FunToken A B) : Prop :=
  FunCon A B w /\
  exists s subseteq w, (forall q in s, A.EntSet p.val.1 q.val.1) /\
  B.EntSet (funOutputUnion A B s) p.val.2

theorem entSet_inputUnion_of_ent {u' : Finset a} {s : Finset (FunToken A B)}
  (h : forall q in s, A.EntSet u' q.val.1) :
  A.EntSet u' (funInputUnion A B s) := by
  intro x hx
  rcases (mem_funInputUnion A B).1 hx with <q, hq, hx'>

```

```

exact h q hq x hx'

private theorem funInputUnion_con_of_ent {u' : Finset a} {s : Finset (FunToken A B)}
  (hu' : u' in A.Con) (h : forall q in s, A.EntSet u' q.val.1) :
  funInputUnion A B s in A.Con :=
  A.con_subset (proposition_2_3_ii A hu' (entSet_inputUnion_of_ent A B h))
  (subset_funion_right _ _)

private theorem insert_filter_ne {p : FunToken A B} {t : Finset (FunToken A B)}
  (hp : p in t) : insert p (t.filter (* != p)) = t := by
  ext q
  constructor
  * intro hq
    rcases Finset.mem_insert.mp hq with rfl | hq
    * exact hp
    * exact (Finset.mem_filter.mp hq).1
  * intro hq
    if h : q = p then
      exact Finset.mem_insert.mpr (Or.inl h)
    else
      exact Finset.mem_insert.mpr (Or.inr (Finset.mem_filter.mpr <hq, h>))

private theorem filter_ne_subset_of_subset_insert {p : FunToken A B}
  {t w : Finset (FunToken A B)} (ht : t subseq insert p w) :
  t.filter (* != p) subseq w := by
  intro q hq
  have <hqt, hne> := Finset.mem_filter.mp hq
  have : q in insert p w := ht hqt
  rcases Finset.mem_insert.mp this with rfl | hqW
  * exact False.elim (hne rfl)
  * exact hqW

/-- Combine entailment witnesses along a finite set of tokens (for `ent_trans`). -/
private theorem exists_combined_witness {u' : Finset a} {s : Finset (FunToken A B)}
  {v : Finset (FunToken A B)}
  (hv : FunCon A B v) (hu' : u' in A.Con)
  (hEntIn : forall q in s, A.EntSet u' q.val.1)
  (hWit : forall q in s, exists s_q subseq v, (forall r in s_q, A.EntSet q.val.1 r
    .val.1) /\
    B.EntSet (funOutputUnion A B s_q) q.val.2) :
  exists S subseq v, (forall r in S, A.EntSet u' r.val.1) /\
  B.EntSet (funOutputUnion A B S) (funOutputUnion A B s) := by
  induction s using Finset.induction_on with
  | empty =>
    refine <{}, Finset.empty_subset _, ?, ?>
    * intro _ hq
      exact False.elim (Finset.notMem_empty _ hq)
    * intro y hy
      exact False.elim (Finset.notMem_empty y hy)
  | insert q s' hqnot ih =>
    have hEntIn' : forall r in s', A.EntSet u' r.val.1 := fun r hr =>
      hEntIn r (Finset.mem_insert_of_mem hr)
    have hWit' : forall r in s', exists s_r subseq v, (forall t in s_r, A.EntSet r.
      val.1 t.val.1) /\
      B.EntSet (funOutputUnion A B s_r) r.val.2 := fun r hr =>
      hWit r (Finset.mem_insert_of_mem hr)
    obtain <S', hS'sub, hS'in, hS'out> := ih hEntIn' hWit'
    obtain <Sq, hSqsub, hSqin, hSqout> := hWit q (Finset.mem_insert_self q s')
    refine <S' U ' Sq, funion_subset_iff.mpr <hS'sub, hSqsub>, ?, ?>
    * intro r hr
      rcases mem_funion.mp hr with hr | hr
      * exact hS'in r hr
      * have hEnt_q : A.EntSet u' q.val.1 := hEntIn q (Finset.mem_insert_self q s')

```

```

    have hEnt_r : A.EntSet q.val.1 r.val.1 := hSqin r hr
    exact proposition_2_3_iv A hu' q.property.1 hEnt_q hEnt_r
* rw [funOutputUnion_funion, funOutputUnion_insert]
    have hinS' : funInputUnion A B S' in A.Con :=
      funInputUnion_con_of_ent A B hu' hS'in
    have houtS' : funOutputUnion A B S' in B.Con := hv S' hS'sub hinS'
    have hinSq : funInputUnion A B Sq in A.Con :=
      funInputUnion_con_of_ent A B q.property.1 hSqin
    have houtSq : funOutputUnion A B Sq in B.Con := hv Sq hSqsub hinSq
    have hinUnion : funInputUnion A B (S' U ' Sq) in A.Con := by
      rw [funInputUnion_funion]
    have hEnt : A.EntSet u' (funInputUnion A B S' U ' funInputUnion A B Sq) :=
      proposition_2_3_vi A
      (entSet_inputUnion_of_ent A B hS'in)
      (proposition_2_3_iv A hu' q.property.1
        (hEntIn q (Finset.mem_insert_self q s')))

```

Lean 4 source (lines 401–549)

```

    (entSet_inputUnion_of_ent A B hSqin))
    exact A.con_subset (proposition_2_3_ii A hu' hEnt) (subset_funion_right _ _)
    have houtUnion : funOutputUnion A B S' U ' funOutputUnion A B Sq in B.Con := by
      have : funOutputUnion A B (S' U ' Sq) in B.Con := hv _ (funion_subset_iff.mpr <hS'
sub, hSqsub>) hinUnion
      rwa [funOutputUnion_funion] at this
    have hEnt1 : B.EntSet (funOutputUnion A B S' U ' funOutputUnion A B Sq)
      (funOutputUnion A B s') :=
      proposition_2_3_v B houtS' houtUnion (subset_funion_left _ _) hS'out
      (Finset.Subset.refl _)
    have hEnt2 : B.EntSet (funOutputUnion A B S' U ' funOutputUnion A B Sq) q.val.2 :=
      proposition_2_3_v B houtSq houtUnion (subset_funion_right _ _) hSqout
      (Finset.Subset.refl _)
    exact proposition_2_3_vi B hEnt2 hEnt1

/-- Downward closure of function-space consistency. -/
theorem functionSystem_con_subset {w w' : Finset (FunToken A B)}
  (hw : FunCon A B w) (hw' : w' subseq w) : FunCon A B w' :=
  fun s hs hin => hw s (fun _ hq => hw' (hs hq)) hin

/-- Singletons are consistent in the function space. -/
theorem functionSystem_con_sing (p : FunToken A B) : FunCon A B {p} := by
  intro s hs _hin
  have hsub : funOutputUnion A B s subseq p.val.2 := by
    intro y hy
    rcases (mem_funOutputUnion A B).1 hy with <q, hq, hy'>
    have hq' : q = p := Finset.mem_singleton.mp (hs hq)
    cases hq'
    exact hy'
  exact B.con_subset p.property.2 hsub

/-- Adding an entailed function-space token preserves consistency. -/
theorem functionSystem_ent_con {w : Finset (FunToken A B)} {p : FunToken A B}
  (hEnt : FunEnt A B w p) : FunCon A B (insert p w) := by
  obtain <hw, s, hs, hEntIn, hEntOut> := hEnt
  intro t ht hin
  if hp : p in t then
    let t' := t.filter (* != p)
    have ht_eq : insert p t' = t := insert_filter_ne A B hp
    have ht_sub : t' subseq w := filter_ne_subset_of_subset_insert A B ht
    have hin' : p.val.1 U ' funInputUnion A B t' in A.Con := by
      rwa [← ht_eq, funInputUnion_insert] at hin
    have hsub : s U ' t' subseq w := funion_subset_iff.mpr <hs, ht_sub>
    have hEnt_in_s : A.EntSet p.val.1 (funInputUnion A B s) :=
      entSet_inputUnion_of_ent A B hEntIn

```



```

have hEnt_big : A.EntSet (p.val.1 U ' funInputUnion A B t') (funInputUnion A B s) :=
  proposition_2_3_v A p.property.1 hin' (subset_funion_left _ _) hEnt_in_s
  (Finset.Subset.refl _)
have hbig : (p.val.1 U ' funInputUnion A B t') U ' funInputUnion A B s in A.Con :=
  proposition_2_3_ii A hin' hEnt_big
have hin_st' : funInputUnion A B s U ' funInputUnion A B t' in A.Con := by
  have heq :
    (p.val.1 U ' funInputUnion A B t') U ' funInputUnion A B s =
      p.val.1 U ' (funInputUnion A B s U ' funInputUnion A B t') := by
      rw [funion_assoc, funion_comm (funInputUnion A B t')]
  have hbig' : p.val.1 U ' (funInputUnion A B s U ' funInputUnion A B t') in A.Con :=
  by
    rwa [heq] at hbig
    exact A.con_subset hbig' (subset_funion_right _ _)
have hin_union : funInputUnion A B (s U ' t') in A.Con := by
  rwa [funInputUnion_funion]
have hout_union : funOutputUnion A B s U ' funOutputUnion A B t' in B.Con := by
  have : funOutputUnion A B (s U ' t') in B.Con := hw _ hsub hin_union
  rwa [funOutputUnion_funion] at this
have hout_s : funOutputUnion A B s in B.Con :=
  B.con_subset hout_union (subset_funion_left _ _)
have hEnt_out' : B.EntSet (funOutputUnion A B s U ' funOutputUnion A B t') p.val.2 :=
  proposition_2_3_v B hout_s hout_union (subset_funion_left _ _) hEntOut
  (Finset.Subset.refl _)
have hfinal :
  (funOutputUnion A B s U ' funOutputUnion A B t') U ' p.val.2 in B.Con :=
  proposition_2_3_ii B hout_union hEnt_out'
have hout_t : p.val.2 U ' funOutputUnion A B t' in B.Con := by
  have heq :
    (funOutputUnion A B s U ' funOutputUnion A B t') U ' p.val.2 =
      funOutputUnion A B s U ' (p.val.2 U ' funOutputUnion A B t') := by
      rw [funion_assoc', funion_comm' (funOutputUnion A B t')]
  have hfinal' :
    funOutputUnion A B s U ' (p.val.2 U ' funOutputUnion A B t') in B.Con := by
    rwa [heq] at hfinal
    exact B.con_subset hfinal' (subset_funion_right _ _)
have : funOutputUnion A B t in B.Con := by
  rwa [← ht_eq, funOutputUnion_insert]
exact this
else
  have ht' : t subseq w := by
    intro q hq
    have : q in insert p w := ht hq
    rcases Finset.mem_insert.mp this with rfl | hqW
    * exact False.elim (hp hq)
    * exact hqW
  exact hw t ht' hin

/-- The function-space bottom is entailed by every consistent set. -/
theorem functionSystem_ent_bot {w : Finset (FunToken A B)} (hw : FunCon A B w) :
  FunEnt A B w (funBot A B) :=
  <hw, {}, Finset.empty_subset _,
  fun q hq => False.elim (Finset.notMem_empty q hq),
  fun y hy => False.elim (Finset.notMem_empty y hy)>

/-- Function-space entailment is reflexive on members. -/
theorem functionSystem_ent_refl {w : Finset (FunToken A B)} {p : FunToken A B}
  (hw : FunCon A B w) (hp : p in w) : FunEnt A B w p :=
  <hw, {p},
  fun q hq => by
    cases Finset.mem_singleton.mp hq
    exact hp,
  fun q hq => by

```

```

    cases Finset.mem_singleton.mp hq
    exact proposition_2_3_iii A p.property.1,
  by
    rw [funOutputUnion_singleton]
    exact proposition_2_3_iii B p.property.2>

/-- Function-space entailment is transitive. -/
theorem functionSystem_ent_trans {u v : Finset (FunToken A B)} {c : FunToken A B}
  (hv : FunCon A B v) (hu : FunCon A B u)
  (hEnts : forall y in u, FunEnt A B v y) (hEnt : FunEnt A B u c) :
  FunEnt A B v c := by
  obtain <_, s, hs, hEntIn, hEntOut> := hEnt
  refine <hv, ?_>
  have hWit : forall q in s, exists s_q subseq v, (forall r in s_q, A.EntSet q.val.1
    r.val.1) /\
    B.EntSet (funOutputUnion A B s_q) q.val.2 := by
    intro q hq
    exact (hEnts q (hs hq)).2
  obtain <S, hSsub, hSin, hSout> :=
    exists_combined_witness A B hv c.property.1 hEntIn hWit
  refine <S, hSsub, hSin, ?_>
  have hin_s : funInputUnion A B s in A.Con :=
    funInputUnion_con_of_ent A B c.property.1 hEntIn
  have hout_s : funOutputUnion A B s in B.Con := hu s hs hin_s
  have hin_S : funInputUnion A B S in A.Con :=
    funInputUnion_con_of_ent A B c.property.1 hSin
  have hout_S : funOutputUnion A B S in B.Con := hv S hSsub hin_S
  exact proposition_2_3_iv B hout_S hout_s hSout hEntOut

/-- **Definition 7.1** The function-space information system `A -> B`. -/
def functionSystem : InfoSys (FunToken A B) where
  bot := funBot A B
  Con := {w | FunCon A B w}
  Ent := FunEnt A B
  con_subset := functionSystem_con_subset A B
  con_sing := functionSystem_con_sing A B
  ent_con := functionSystem_ent_con A B
  ent_bot := functionSystem_ent_bot A B
  ent_refl := functionSystem_ent_refl A B
  ent_trans := functionSystem_ent_trans A B

end InfoSys

end Scott1982

```

A.31 Scott1982/Theorem72.lean

1020 lines.

Lean 4 source (lines 1–400)

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.FunctionSpace
import Scott1982.Proposition53
import Scott1982.Proposition55
import Scott1982.Proposition62
import Scott1982.Factoid32

```

```

/-!
# Theorem 7.2 -- function space elements, apply, and curry

**Scott 1982, Theorem 7.2.** Approximable maps  $A \rightarrow B$  are exactly the elements of
 $|A \rightarrow B|$ . There is approximable  $\text{apply} : (B \rightarrow C) \times B \rightarrow C$  with  $\text{apply}(g,y) = g(y)$ ,
and for each  $h : A \times B \rightarrow C$  a unique  $\text{curry } h : A \rightarrow (B \rightarrow C)$  satisfying
 $h = \text{apply} \text{ comp } \langle (\text{curry } h) \text{ comp } \text{fst}, \text{snd} \rangle$ .

( $A \rightarrow B$  as an  $\text{InfoSys}$  is already Def 7.1 /  $\text{functionSystem}$ .)
-/

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys

set_option linter.unusedSectionVars false

universe u v w

variable {a : Type u} {b : Type v} {? : Type w}
[DecidableEq a] [DecidableEq b] [DecidableEq ?]
variable (A : InfoSys a) (B : InfoSys b) (C : InfoSys ?)

/-- Consistency of a packaged function-space token. -/
theorem mkFunToken_property (u : Finset a) (v : Finset b) (hu : u in A.Con) (hv : v in B.
  Con) :
  u in A.Con /\ v in B.Con :=
  <hu, hv>

/-- Package a consistent pair as a function-space token. -/
def mkFunToken (u : Finset a) (v : Finset b) (hu : u in A.Con) (hv : v in B.Con) :
  FunToken A B :=
  <(u, v), mkFunToken_property A B u v hu hv>

theorem mkFunToken_eq (p : FunToken A B) :
  mkFunToken A B p.val.1 p.val.2 p.property.1 p.property.2 = p :=
  Subtype.ext rfl

private theorem funion_self (u : Finset a) : u U ' u = u := by
  ext x
  constructor
  * intro hx
    rcases mem_funion.mp hx with h | h <;> exact h
  * intro hx
    exact mem_funion.mpr (Or.inl hx)

private theorem funion_comm_b (u v : Finset b) : u U ' v = v U ' u := by
  ext x
  constructor
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact mem_funion.mpr (Or.inr h)
    * exact mem_funion.mpr (Or.inl h)
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact mem_funion.mpr (Or.inr h)
    * exact mem_funion.mpr (Or.inl h)

theorem rel_input_output_union (f : ApproximableMap A B)
  (s : Finset (FunToken A B))

```

```

(hrel : forall q in s, f.rel q.val.1 q.val.2)
(hin : funInputUnion A B s in A.Con) :
f.rel (funInputUnion A B s) (funOutputUnion A B s) := by
induction s using Finset.induction_on with
| empty =>
  rw [funInputUnion_empty, funOutputUnion_empty]
  exact f.empty_rel
| insert q s' _hq ih =>
  have hrel' : forall r in s', f.rel r.val.1 r.val.2 := fun r hr =>
    hrel r (Finset.mem_insert_of_mem hr)
  have hin' : funInputUnion A B s' in A.Con :=
    A.con_subset hin (by
      rw [funInputUnion_insert]
      exact subset_funion_right _ _)
  have ih' := ih hrel' hin'
  have hqrel : f.rel q.val.1 q.val.2 := hrel q (Finset.mem_insert_self q s')
  have hinU : q.val.1 U ' funInputUnion A B s' in A.Con := by
    rwa [funInputUnion_insert] at hin
  have hEnt_q : A.EntSet (q.val.1 U ' funInputUnion A B s') q.val.1 :=
    fun x hx => A.ent_refl hinU (subset_funion_left _ _ hx)
  have hEnt_s : A.EntSet (q.val.1 U ' funInputUnion A B s') (funInputUnion A B s') :=
    fun x hx => A.ent_refl hinU (subset_funion_right _ _ hx)
  have h1 : f.rel (q.val.1 U ' funInputUnion A B s') q.val.2 :=
    f.mono hqrel hEnt_q (proposition_2_3_iii B q.property.2) hinU q.property.2
  have h2 : f.rel (q.val.1 U ' funInputUnion A B s') (funOutputUnion A B s') :=
    f.mono ih' hEnt_s (proposition_2_3_iii B (f.rel_cod ih')) hinU (f.rel_cod ih')
  rw [funInputUnion_insert, funOutputUnion_insert]
  exact f.union_right h1 h2

open ApproximableMap

theorem funCon_of_approxMap (f : ApproximableMap A B) (w : Finset (FunToken A B))
(hw : forall p in w, f.rel p.val.1 p.val.2) : FunCon A B w :=
fun s hs hin =>
  f.rel_cod (rel_input_output_union A B f s (fun q hq => hw q (hs hq)) hin)

/-- Consistency of the token-set of an approximable map. -/
theorem approxMap_toElement_consistent (f : ApproximableMap A B)
(Y : Finset (FunToken A B))
(hY : (Y : Set (FunToken A B)) subseq {p : FunToken A B | f.rel p.val.1 p.val.2}) :
Y in (functionSystem A B).Con :=
funCon_of_approxMap A B f Y fun _ hp => hY (Finset.mem_coe.2 hp)

/-- Deductive closure of the token-set of an approximable map. -/
theorem approxMap_toElement_closed (f : ApproximableMap A B)
(Y : Finset (FunToken A B)) (p : FunToken A B)
(hY : (Y : Set (FunToken A B)) subseq {p : FunToken A B | f.rel p.val.1 p.val.2})
(hEnt : (functionSystem A B).Ent Y p) :
p in {q : FunToken A B | f.rel q.val.1 q.val.2} := by
obtain <_hCon, s, hs, hEntIn, hEntOut> := hEnt
have hrel : forall q in s, f.rel q.val.1 q.val.2 := fun q hq =>
  hY (Finset.mem_coe.2 (hs hq))
have hin_s : funInputUnion A B s in A.Con :=
  A.con_subset
    (proposition_2_3_ii A p.property.1 (entSet_inputUnion_of_ent A B hEntIn))
    (subset_funion_right _ _)
have hIO := rel_input_output_union A B f s hrel hin_s
have hU : f.rel p.val.1 (funOutputUnion A B s) :=
  f.mono hIO (entSet_inputUnion_of_ent A B hEntIn)
  (proposition_2_3_iii B (f.rel_cod hIO)) p.property.1 (f.rel_cod hIO)
exact f.mono hU (proposition_2_3_iii A p.property.1) hEntOut p.property.1 p.property.2

/-- **Theorem 7.2.** Approximable map as an element of `|A -> B|`. -/

```

```

def approxMap_toElement (f : ApproximableMap A B) : (functionSystem A B).Element where
  carrier := {p : FunToken A B | f.rel p.val.1 p.val.2}
  consistent := approxMap_toElement_consistent A B f
  closed := approxMap_toElement_closed A B f

theorem mem_approxMap_toElement (f : ApproximableMap A B) {p : FunToken A B} :
  p in (approxMap_toElement A B f).carrier <=> f.rel p.val.1 p.val.2 :=
  Iff.rfl

/-- Domain of the relation recovered from a function-space element. -/
theorem element_toApproxMap_rel_dom (x : (functionSystem A B).Element) :
  forall {u : Finset a} {v : Finset b},
    (exists (hu : u in A.Con) (hv : v in B.Con), mkFunToken A B u v hu hv in x.carrier)
  ->
    u in A.Con :=
  fun <hu, _, _> => hu

/-- Codomain of the relation recovered from a function-space element. -/
theorem element_toApproxMap_rel_cod (x : (functionSystem A B).Element) :
  forall {u : Finset a} {v : Finset b},
    (exists (hu : u in A.Con) (hv : v in B.Con), mkFunToken A B u v hu hv in x.carrier)
  ->
    v in B.Con :=
  fun <_, hv, _> => hv

/-- The empty pair is related. -/
theorem element_toApproxMap_empty_rel (x : (functionSystem A B).Element) :
  exists (hu : ({ } : Finset a) in A.Con) (hv : ({ } : Finset b) in B.Con),
    mkFunToken A B { } { } hu hv in x.carrier := by
  refine <A.con_empty, B.con_empty, ?_>
  change funBot A B in x.carrier
  exact factoid_3_2 (functionSystem A B) x

/-- Output-union of two related pairs remains related. -/
theorem element_toApproxMap_union_right (x : (functionSystem A B).Element) :
  forall {u : Finset a} {v v' : Finset b},
    (exists (hu : u in A.Con) (hv : v in B.Con), mkFunToken A B u v hu hv in x.carrier)
  ->
    (exists (hu : u in A.Con) (hv : v' in B.Con), mkFunToken A B u v' hu hv in x.
      carrier) ->
      exists (hu : u in A.Con) (hv : v U' v' in B.Con),
        mkFunToken A B u (v U' v') hu hv in x.carrier := by
  rintro u v v' <hu, hv, hp> <hu', hv', hq>
  have hvU : v U' v' in B.Con := by
  have hCon : FunCon A B
    (insert (mkFunToken A B u v' hu' hv') {mkFunToken A B u v hu hv}) := by
  apply x.consistent
  intro r hr
  rcases Finset.mem_insert.mp (Finset.mem_coe.1 hr) with rfl | hr
  * exact hq
  * have : r = mkFunToken A B u v hu hv := Finset.mem_singleton.mp hr
    subst this
    exact hp
  have hin : funInputUnion A B
    (insert (mkFunToken A B u v' hu' hv') {mkFunToken A B u v hu hv}) in A.Con := by
  rw [funInputUnion_insert, funInputUnion_singleton]
  change u U' u in A.Con
  rw [funion_self]
  exact hu
  have hout := hCon _ (Finset.Subset.refl _) hin
  rw [funOutputUnion_insert, funOutputUnion_singleton, funion_comm_b] at hout
  exact hout
  refine <hu, hvU, ?_>

```

```

let p := mkFunToken A B u v hu hv
let q := mkFunToken A B u v' hu' hv'
let r := mkFunToken A B u (v U ' v') hu hvU
let w : Finset (FunToken A B) := insert q {p}
have hwsb : (w : Set _) subseq x.carrier := by
  intro t ht
  rcases Finset.mem_insert.mp (Finset.mem_coe.1 ht) with rfl | ht
  * exact hq
  * have : t = p := Finset.mem_singleton.mp ht
    subst this
    exact hp
have hEnt : FunEnt A B w r := by
  refine <x.consistent w hwsb, w, Finset.Subset.refl _, ?, ?_>
  * intro t ht
  rcases Finset.mem_insert.mp ht with rfl | ht
  * exact proposition_2_3_iii A hu'
  * have : t = p := Finset.mem_singleton.mp ht
    subst this
    exact proposition_2_3_iii A hu
  * have houtEq : funOutputUnion A B w = v' U ' v := by
    simp only [w, q, p, funOutputUnion_insert, funOutputUnion_singleton, mkFunToken]
    rw [houtEq, funion_comm_b]
    exact proposition_2_3_iii B hvU
exact x.closed w r hwsb hEnt

/-- The recovered relation is monotone for entailment. -/
theorem element_toApproxMap_mono (x : (functionSystem A B).Element) :
  forall {u u' : Finset a} {v v' : Finset b},
    (exists (hu : u in A.Con) (hv : v in B.Con), mkFunToken A B u v hu hv in x.carrier)
  ) ->
  A.EntSet u' u -> B.EntSet v v' -> u' in A.Con -> v' in B.Con ->
  exists (hu : u' in A.Con) (hv : v' in B.Con),
    mkFunToken A B u' v' hu hv in x.carrier := by
  rintro u u' v v' <hu, hv, hp> hEntu hEntv hu' hv'
  refine <hu', hv', ?_>
  let p := mkFunToken A B u v hu hv
  let r := mkFunToken A B u' v' hu' hv'
  have hwsb : ({p} : Finset (FunToken A B)) : Set _ subseq x.carrier := by
    intro t ht
    have : t = p := Finset.mem_singleton.mp (Finset.mem_coe.1 ht)
    subst this
    exact hp
  have hEnt : FunEnt A B {p} r := by
    refine <x.consistent {p} hwsb, {p}, Finset.Subset.refl _, ?, ?_>
    * intro t ht
      have : t = p := Finset.mem_singleton.mp ht
      subst this
      exact hEntu
    * rw [funOutputUnion_singleton]
      exact hEntv
  exact x.closed {p} r hwsb hEnt

/-- **Theorem 7.2.** Element of  $|A \rightarrow B|$  as an approximable map. -/
def element_toApproxMap (x : (functionSystem A B).Element) : ApproximableMap A B where
  rel u v := exists (hu : u in A.Con) (hv : v in B.Con), mkFunToken A B u v hu hv in x.
  carrier
  rel_dom := element_toApproxMap_rel_dom A B x
  rel_cod := element_toApproxMap_rel_cod A B x
  empty_rel := element_toApproxMap_empty_rel A B x
  union_right := element_toApproxMap_union_right A B x
  mono := element_toApproxMap_mono A B x

theorem element_toApproxMap_approxMap_toElement (f : ApproximableMap A B) :

```

```

    element_toApproxMap A B (approxMap_toElement A B f) = f := by
refine ApproximableMap.ext fun u v => ?_
constructor
* rintro <hu, hv, hp>
  simp [mem_approxMap_toElement, mkFunToken] using hp
* intro hrel
  exact <f.rel_dom hrel, f.rel_cod hrel, by
    simp [mem_approxMap_toElement, mkFunToken] using hrel>

theorem approxMap_toElement_element_toApproxMap (x : (functionSystem A B).Element) :
  approxMap_toElement A B (element_toApproxMap A B x) = x := by
  apply le_antisymm
* intro p hp
  change (element_toApproxMap A B x).rel p.val.1 p.val.2 at hp
  obtain <_, _, hmem> := hp
  simp [mkFunToken_eq A B p] using hmem
* intro p hp
  change (element_toApproxMap A B x).rel p.val.1 p.val.2
  refine <p.property.1, p.property.2, ?_>
  simp [mkFunToken_eq A B p] using hp

/-- **Theorem 7.2.** Approximable `apply : (B -> C) x B -> C`. -/
def applyMap : ApproximableMap (productSystem (functionSystem B C) B) C where
  rel s t :=
    exists (hs : ProdCon (functionSystem B C) B s) (ht : t in C.Con),
      FunEnt B C (fstFinset (functionSystem B C) B s)
        (mkFunToken B C (sndFinset (functionSystem B C) B s) t hs.2 ht)
  rel_dom := fun <hs, _, _> => hs
  rel_cod := fun <_, ht, _> => ht
  empty_rel := by
    have hs : ProdCon (functionSystem B C) B {} :=
      <by rw [fstFinset_empty]; exact (functionSystem B C).con_empty,
        by rw [sndFinset_empty]; exact B.con_empty>
    refine <hs, C.con_empty, ?_>
    have hfst : fstFinset (functionSystem B C) B ({} : Finset _) = {} := fstFinset_empty _
    have hsnd : sndFinset (functionSystem B C) B ({} : Finset _) = {} := sndFinset_empty _
    simp only [hfst, hsnd]
    exact (functionSystem B C).ent_bot (functionSystem B C).con_empty
  union_right := by
    rintro s t t' <hs, ht, hEntt> <_, ht', hEntt'>
    obtain <hw, s1, hs1, hIn1, hOut1> := hEntt
    obtain <_, s2, hs2, hIn2, hOut2> := hEntt'
    have hsub : s1 U ' s2 subseteq fstFinset (functionSystem B C) B s :=
      fununion_subset_iff.mpr <hs1, hs2>
    have hin : funInputUnion B C (s1 U ' s2) in B.Con := by
      rw [funInputUnion_funion]
      have hEntU : B.EntSet (sndFinset (functionSystem B C) B s)
        (funInputUnion B C s1 U ' funInputUnion B C s2) :=
        proposition_2_3_vi B
        (entSet_inputUnion_of_ent B C hIn1)
        (entSet_inputUnion_of_ent B C hIn2)
      exact B.con_subset (proposition_2_3_ii B hs.2 hEntU) (subset_funion_right _ _)
    have hout : funOutputUnion B C s1 U ' funOutputUnion B C s2 in C.Con := by
      have := hw _ hsub hin
      rwa [funOutputUnion_funion] at this
    have htt' : t U ' t' in C.Con := by
      have hEntt2 : C.EntSet (funOutputUnion B C s1 U ' funOutputUnion B C s2) (t U ' t') :=
        proposition_2_3_vi C
        (proposition_2_3_v C (C.con_subset hout (subset_funion_left _ _)) hout
          (subset_funion_left _ _) hOut1 (Finset.Subset.refl _))
        (proposition_2_3_v C (C.con_subset hout (subset_funion_right _ _)) hout
          (subset_funion_right _ _) hOut2 (Finset.Subset.refl _))
      exact C.con_subset (proposition_2_3_ii C hout hEntt2) (subset_funion_right _ _)

```

```

refine <hs, htt', hw, s1 U ' s2, hsub, ?_, ?_>
* intro q hq
rcases mem_funion.mp hq with hq | hq
* exact hIn1 q hq
* exact hIn2 q hq
* rw [funOutputUnion_funion]
exact proposition_2_3_vi C
  (proposition_2_3_v C (C.con_subset hout (subset_funion_left _ _)) hout
   (subset_funion_left _ _) hOut1 (Finset.Subset.refl _))
  (proposition_2_3_v C (C.con_subset hout (subset_funion_right _ _)) hout
   (subset_funion_right _ _) hOut2 (Finset.Subset.refl _))
mono := by
  rintro s s' t t' <hs, ht, hEnt> hEnts hEntt hs' ht'
  refine <hs', ht', ?_>
  have hfst := entSet_fst_of_prodEntSet (functionSystem B C) B hEnts
  have hsnd := entSet_snd_of_prodEntSet (functionSystem B C) B hEnts
  have htok :
    (functionSystem B C).Ent (fstFinset (functionSystem B C) B s)
    (mkFunToken B C (sndFinset (functionSystem B C) B s) t hs.2 ht) := hEnt
  have hmid :
    (functionSystem B C).Ent (fstFinset (functionSystem B C) B s')
    (mkFunToken B C (sndFinset (functionSystem B C) B s) t hs.2 ht) :=
    (functionSystem B C).ent_trans hs'.1 hs.1 (fun y hy => hfst y hy) htok
  obtain <hw', s1, hs1, hIn1, hOut1> := hmid
  have hIn1' : forall q in s1, B.EntSet (sndFinset (functionSystem B C) B s') q.val.1 :=
    fun q hq => proposition_2_3_iv B hs'.2 hs.2 hsnd (hIn1 q hq)
  have hin : funInputUnion B C s1 in B.Con :=
    B.con_subset
    (proposition_2_3_ii B hs'.2 (entSet_inputUnion_of_ent B C hIn1'))
    (subset_funion_right _ _)
  have hout : funOutputUnion B C s1 in C.Con := hw' s1 hs1 hin
  refine <hw', s1, hs1, hIn1', ?_>
  exact proposition_2_3_iv C hout ht hOut1 hEntt

/-! ## Curry helpers -/

/-- Product token set encoding a pair of consistent sets `(u, v)`. -/
def liftPair (u : Finset a) (v : Finset b) : Finset (ProdToken A B) :=
  liftLeft A B u U ' liftRight A B v

theorem liftLeft_funion (u u' : Finset a) :
  liftLeft A B (u U ' u') = liftLeft A B u U ' liftLeft A B u' := by
  ext p
  constructor
  * intro hp
  rcases (mem_liftLeft A B).1 hp with <x, hx, rfl>
  rcases mem_funion.mp hx with hx | hx
  * exact mem_funion.mpr (Or.inl ((mem_liftLeft A B).2 <x, hx, rfl>))
  * exact mem_funion.mpr (Or.inr ((mem_liftLeft A B).2 <x, hx, rfl>))
  * intro hp
  rcases mem_funion.mp hp with hp | hp
  * rcases (mem_liftLeft A B).1 hp with <x, hx, rfl>
  exact (mem_liftLeft A B).2 <x, mem_funion.mpr (Or.inl hx), rfl>
  * rcases (mem_liftLeft A B).1 hp with <x, hx, rfl>
  exact (mem_liftLeft A B).2 <x, mem_funion.mpr (Or.inr hx), rfl>

theorem liftRight_funion (v v' : Finset b) :
  liftRight A B (v U ' v') = liftRight A B v U ' liftRight A B v' := by
  ext p
  constructor
  * intro hp
  rcases (mem_liftRight A B).1 hp with <y, hy, rfl>
  rcases mem_funion.mp hy with hy | hy

```



```

    * exact mem_funion.mpr (Or.inl ((mem_liftRight A B).2 <y, hy, rfl>))
    * exact mem_funion.mpr (Or.inr ((mem_liftRight A B).2 <y, hy, rfl>))
  * intro hp
  rcases mem_funion.mp hp with hp | hp
  * rcases (mem_liftRight A B).1 hp with <y, hy, rfl>
    exact (mem_liftRight A B).2 <y, mem_funion.mpr (Or.inl hy), rfl>
  * rcases (mem_liftRight A B).1 hp with <y, hy, rfl>
    exact (mem_liftRight A B).2 <y, mem_funion.mpr (Or.inr hy), rfl>

theorem ProdCon_liftPair {u : Finset a} {v : Finset b}
  (hu : u in A.Con) (hv : v in B.Con) : ProdCon A B (liftPair A B u v) := by
  constructor
  * unfold liftPair
  rw [fstFinset_funion, fstFinset_liftLeft]
  have hEnt : A.EntSet u (fstFinset A B (liftRight A B v)) := fun x hx => by
    have : x = A.bot := Finset.mem_singleton.mp (fstFinset_liftRight A B v hx)
    subst this

```

Lean 4 source (lines 401–800)

```

    exact A.ent_bot hu
    exact proposition_2_3_ii A hu hEnt
  * unfold liftPair
  rw [sndFinset_funion, sndFinset_liftRight]
  have hEnt : B.EntSet v (sndFinset A B (liftLeft A B u)) := fun y hy => by
    have : y = B.bot := Finset.mem_singleton.mp (sndFinset_liftLeft A B u hy)
    subst this
    exact B.ent_bot hv
  have h := proposition_2_3_ii B hv hEnt
  rwa [funion_comm_b] at h

theorem fstFinset_liftPair (u : Finset a) (v : Finset b) :
  fstFinset A B (liftPair A B u v) =
    u U ' fstFinset A B (liftRight A B v) := by
  unfold liftPair
  rw [fstFinset_funion, fstFinset_liftLeft]

theorem sndFinset_liftPair (u : Finset a) (v : Finset b) :
  sndFinset A B (liftPair A B u v) =
    sndFinset A B (liftLeft A B u) U ' v := by
  unfold liftPair
  rw [sndFinset_funion, sndFinset_liftRight]

/-- Product entailment between lifted pairs from component entailments. -/
theorem entSet_liftPair {u u' : Finset a} {v v' : Finset b}
  (hu' : u' in A.Con) (hv' : v' in B.Con)
  (hEntu : A.EntSet u' u) (hEntv : B.EntSet v' v) :
  (productSystem A B).EntSet (liftPair A B u' v') (liftPair A B u v) := by
  intro p hp
  have hCon : ProdCon A B (liftPair A B u' v') := ProdCon_liftPair A B hu' hv'
  unfold liftPair at hp
  rcases mem_funion.mp hp with hp | hp
  * rcases (mem_liftLeft A B).1 hp with <x, hx, rfl>
    refine <hCon, ?_, ?_>
  * intro _
    have hsub : u' subseteq fstFinset A B (liftPair A B u' v') := by
      intro z hz
      rw [fstFinset_liftPair]
      exact subset_funion_left _ _ hz
    exact A.ent_trans hCon.1 hu'
      (fun z hz => A.ent_refl hCon.1 (hsub hz)) (hEntu x hx)
  * intro _
    exact B.ent_bot hCon.2
  * rcases (mem_liftRight A B).1 hp with <y, hy, rfl>

```

```

refine <hCon, ?_, ?_>
* intro _
  exact A.ent_bot hCon.1
* intro _
  have hsub : v' ⊆ sndFinset A B (liftPair A B u' v') := by
    intro z hz
    rw [sndFinset_liftPair]
    exact subset_funion_right _ _ hz
  exact B.ent_trans hCon.2 hv'
    (fun z hz => B.ent_refl hCon.2 (hsub hz)) (hEntv y hy)

/-- Combine pointwise `h`-relations along a finite set of function tokens. -/
theorem rel_of_funTokens (h : ApproximableMap (productSystem A B) C)
  (u : Finset a) (hu : u in A.Con) (t : Finset (FunToken B C))
  (hps : forall p in t, h.rel (liftPair A B u p.val.1) p.val.2)
  (hin : funInputUnion B C t in B.Con) :
  h.rel (liftPair A B u (funInputUnion B C t)) (funOutputUnion B C t) := by
  induction t using Finset.induction_on with
  | empty =>
    rw [funInputUnion_empty, funOutputUnion_empty]
    have hdom : liftPair A B u {} in (productSystem A B).Con :=
      ProdCon_liftPair A B hu B.con_empty
    exact h.mono h.empty_rel
      ((productSystem A B).entSet_empty _)
      (C.entSet_empty {}) hdom C.con_empty
  | insert p t' hpnot ih =>
    have hps' : forall q in t', h.rel (liftPair A B u q.val.1) q.val.2 :=
      fun q hq => hps q (Finset.mem_insert_of_mem hq)
    have hin' : funInputUnion B C t' in B.Con :=
      B.con_subset hin (by
        rw [funInputUnion_insert]
        exact subset_funion_right _ _)
    have ih' := ih hps' hin'
    have hp : h.rel (liftPair A B u p.val.1) p.val.2 :=
      hps p (Finset.mem_insert_self p t')
    have hinU : p.val.1 U ' funInputUnion B C t' in B.Con := by
      rwa [funInputUnion_insert] at hin
    have hEnt_p : B.EntSet (p.val.1 U ' funInputUnion B C t') p.val.1 :=
      fun y hy => B.ent_refl hinU (subset_funion_left _ _ hy)
    have hEnt_t : B.EntSet (p.val.1 U ' funInputUnion B C t') (funInputUnion B C t') :=
      fun y hy => B.ent_refl hinU (subset_funion_right _ _ hy)
    have hdomU : liftPair A B u (p.val.1 U ' funInputUnion B C t') in
      (productSystem A B).Con :=
      ProdCon_liftPair A B hu hinU
    have h1 : h.rel (liftPair A B u (p.val.1 U ' funInputUnion B C t')) p.val.2 :=
      h.mono hp
      (entSet_liftPair A B hu hinU (proposition_2_3_iii A hu) hEnt_p)
      (proposition_2_3_iii C (h.rel_cod hp)) hdomU (h.rel_cod hp)
    have h2 : h.rel (liftPair A B u (p.val.1 U ' funInputUnion B C t'))
      (funOutputUnion B C t') :=
      h.mono ih'
      (entSet_liftPair A B hu hinU (proposition_2_3_iii A hu) hEnt_t)
      (proposition_2_3_iii C (h.rel_cod ih')) hdomU (h.rel_cod ih')
    rw [funInputUnion_insert, funOutputUnion_insert]
    exact h.union_right h1 h2

theorem funCon_of_curry_pointwise (h : ApproximableMap (productSystem A B) C)
  {u : Finset a} (hu : u in A.Con) {s : Finset (FunToken B C)}
  (hps : forall p in s, h.rel (liftPair A B u p.val.1) p.val.2) :
  FunCon B C s :=
  fun t ht hin =>
    h.rel_cod (rel_of_funTokens A B C h u hu t (fun p hp => hps p (ht hp)) hin)

```

```

/-- **Theorem 7.2.** Approximable `curry h : A -> (B -> C)` -/
def curryMap (h : ApproximableMap (productSystem A B) C) :
  ApproximableMap A (functionSystem B C) where
  rel u s :=
    u in A.Con /\ forall p in s, h.rel (liftPair A B u p.val.1) p.val.2
  rel_dom hrel := hrel.1
  rel_cod hrel := funCon_of_curry_pointwise A B C h hrel.1 hrel.2
  empty_rel := <A.con_empty, fun _ hp => False.elim (Finset.notMem_empty _ hp)>
  union_right := by
    rintro u s' <hu, hps> <_, hps'>
    refine <hu, ?_>
    intro p hp
    rcases mem_funion.mp hp with hp | hp
    * exact hps p hp
    * exact hps' p hp
  mono := by
    rintro u u' s s' <hu, hps> hEntu hEnts hu' hs'
    refine <hu', ?_>
    intro p hp
    -- FunEnt s p from EntSet
    have hEntp : FunEnt B C s p := hEnts p hp
    obtain <_, t, ht, hIn, hOut> := hEntp
    have hps_t : forall q in t, h.rel (liftPair A B u q.val.1) q.val.2 :=
      fun q hq => hps q (ht hq)
    have hin : funInputUnion B C t in B.Con :=
      B.con_subset
      (proposition_2_3_ii B p.property.1 (entSet_inputUnion_of_ent B C hIn))
      (subset_funion_right _ _)
    have hmid : h.rel (liftPair A B u (funInputUnion B C t)) (funOutputUnion B C t) :=
      rel_of_funTokens A B C h u hu t hps_t hin
    have hmid' : h.rel (liftPair A B u' p.val.1) (funOutputUnion B C t) :=
      h.mono hmid
      (entSet_liftPair A B hu' p.property.1 hEntu
       (entSet_inputUnion_of_ent B C hIn))
      (proposition_2_3_iii C (h.rel_cod hmid))
      (ProdCon_liftPair A B hu' p.property.1) (h.rel_cod hmid)
    exact h.mono hmid'
      (proposition_2_3_iii (productSystem A B) (ProdCon_liftPair A B hu' p.property.1))
      hOut (ProdCon_liftPair A B hu' p.property.1) p.property.2

/-- Product element `` assembled from the two projections. -/
def pairElements (x : A.Element) (y : B.Element) : (productSystem A B).Element where
  carrier := {p : ProdToken A B |
    (p.val.2 = B.bot -> p.val.1 in x.carrier) /\ (p.val.1 = A.bot -> p.val.2 in y.
    carrier)}
  consistent := by
    intro Y hY
    refine <?_, ?_>
    * have hsub : (fstFinset A B Y) subseq x.carrier := by
      intro a ha
      rcases (mem_fstFinset A B).1 ha with <p, hp, hbot, rfl>
      exact (hY (Finset.mem_coe.2 hp)).1 hbot
      exact x.consistent _ hsub
    * have hsub : (sndFinset A B Y) subseq y.carrier := by
      intro b hb
      rcases (mem_sndFinset A B).1 hb with <p, hp, hbot, rfl>
      exact (hY (Finset.mem_coe.2 hp)).2 hbot
      exact y.consistent _ hsub
  closed := by
    intro Y p hY hEnt
    obtain <hCon, hL, hR> := hEnt
    refine <?_, ?_>
    * intro hbot

```

```

have hEntA : A.Ent (fstFinset A B Y) p.val.1 := hL hbot
have hsub : (fstFinset A B Y) subseq x.carrier := by
  intro a ha
  rcases (mem_fstFinset A B).1 ha with <q, hq, hbot', rfl>
  exact (hY (Finset.mem_coe.2 hq)).1 hbot'
exact x.closed _ _ hsub hEntA
* intro hbot
have hEntB : B.Ent (sndFinset A B Y) p.val.2 := hR hbot
have hsub : (sndFinset A B Y) subseq y.carrier := by
  intro b hb
  rcases (mem_sndFinset A B).1 hb with <q, hq, hbot', rfl>
  exact (hY (Finset.mem_coe.2 hq)).2 hbot'
exact y.closed _ _ hsub hEntB

theorem fstMap_pairElements (x : A.Element) (y : B.Element) :
  (fstMap A B).toElement (pairElements A B x y) = x := by
  apply le_antisymm
* intro a <s, hs, <hCon, _, hEnt>>
  have hsub : (fstFinset A B s) subseq x.carrier := by
    intro b hb
    rcases (mem_fstFinset A B).1 hb with <p, hp, hbot, rfl>
    exact (hs (Finset.mem_coe.2 hp)).1 hbot
  exact x.closed _ a hsub (hEnt a (Finset.mem_singleton_self _))
* intro a ha
  refine <liftLeft A B {a}, ?_, ?_>
* intro p hp
  rcases (mem_liftLeft A B).1 (Finset.mem_coe.1 hp) with <b, hb, rfl>
  have : b = a := Finset.mem_singleton.mp hb
  subst this
  exact <fun _ => ha, fun _ => factoid_3_2 B y>
* refine <ProdCon_liftLeft A B (A.con_sing a), A.con_sing a, ?_>
  rw [fstFinset_liftLeft]
  exact proposition_2_3_iii A (A.con_sing a)

theorem sndMap_pairElements (x : A.Element) (y : B.Element) :
  (sndMap A B).toElement (pairElements A B x y) = y := by
  apply le_antisymm
* intro b <s, hs, <hCon, _, hEnt>>
  have hsub : (sndFinset A B s) subseq y.carrier := by
    intro c hc
    rcases (mem_sndFinset A B).1 hc with <p, hp, hbot, rfl>
    exact (hs (Finset.mem_coe.2 hp)).2 hbot
  exact y.closed _ b hsub (hEnt b (Finset.mem_singleton_self _))
* intro b hb
  refine <liftRight A B {b}, ?_, ?_>
* intro p hp
  rcases (mem_liftRight A B).1 (Finset.mem_coe.1 hp) with <c, hc, rfl>
  have : c = b := Finset.mem_singleton.mp hc
  subst this
  exact <fun _ => factoid_3_2 A x, fun _ => hb>
* refine <ProdCon_liftRight A B (B.con_sing b), B.con_sing b, ?_>
  rw [sndFinset_liftRight]
  exact proposition_2_3_iii B (B.con_sing b)

/-- Tokens in `fstFinset s` for `s subseq <x,y>` lie in `x`. -/
theorem mem_fst_of_subset_pairElements {x : A.Element} {y : B.Element}
  {s : Finset (ProdToken A B)}
  (hs : s subseq (pairElements A B x y).carrier) {a : a}
  (ha : a in fstFinset A B s) : a in x.carrier := by
  have hCon : ProdCon A B s := (pairElements A B x y).consistent s hs
  have : a in ((fstMap A B).toElement (pairElements A B x y)).carrier := by
    refine <s, hs, <hCon, A.con_sing a, ?_>>
    intro b hb

```

```

    have : b = a := Finset.mem_singleton.mp hb
    subst this
    exact A.ent_refl hCon.1 ha
    simpa [fstMap_pairElements] using this

/-- Tokens in `sndFinset s` for `s` subseteq <x,y>` lie in `y`. -/
theorem mem_snd_of_subset_pairElements {x : A.Element} {y : B.Element}
  {s : Finset (ProdToken A B)}
  (hs : s subseteq (pairElements A B x y).carrier) {b : b}
  (hb : b in sndFinset A B s) : b in y.carrier := by
  have hCon : ProdCon A B s := (pairElements A B x y).consistent s hs
  have : b in ((sndMap A B).toElement (pairElements A B x y)).carrier := by
    refine <s, hs, <hCon, B.con_sing b, ?_>>
    intro c hc
    have : c = b := Finset.mem_singleton.mp hc
    subst this
    exact B.ent_refl hCon.2 hb
  simpa [sndMap_pairElements] using this

theorem subset_pairElements_liftLeft {x : A.Element} {y : B.Element}
  {u : Finset a} (hu : u subseteq x.carrier) :
  (liftLeft A B u) subseteq (pairElements A B x y).carrier := by
  intro p hp
  rcases (mem_liftLeft A B).1 (Finset.mem_coe.1 hp) with <a, ha, rfl>
  have hax : a in x.carrier := hu (Finset.mem_coe.2 ha)
  have : a in ((fstMap A B).toElement (pairElements A B x y)).carrier := by
    simpa [fstMap_pairElements] using hax
  rcases this with <s, hs, <hCon, _, hEnt>>
  exact (pairElements A B x y).closed s (mkLeft A B a) hs <hCon,
    fun _ => hEnt a (Finset.mem_singleton_self _),
    fun _ => B.ent_bot hCon.2>

theorem subset_pairElements_liftRight {x : A.Element} {y : B.Element}
  {v : Finset b} (hv : v subseteq y.carrier) :
  (liftRight A B v) subseteq (pairElements A B x y).carrier := by
  intro p hp
  rcases (mem_liftRight A B).1 (Finset.mem_coe.1 hp) with <b, hb, rfl>
  have hby : b in y.carrier := hv (Finset.mem_coe.2 hb)
  have : b in ((sndMap A B).toElement (pairElements A B x y)).carrier := by
    simpa [sndMap_pairElements] using hby
  rcases this with <s, hs, <hCon, _, hEnt>>
  exact (pairElements A B x y).closed s (mkRight A B b) hs <hCon,
    fun _ => A.ent_bot hCon.1,
    fun _ => hEnt b (Finset.mem_singleton_self _)>

theorem subset_pairElements_liftPair {x : A.Element} {y : B.Element}
  {u : Finset a} {v : Finset b} (hu : u subseteq x.carrier) (hv : v subseteq y.carrier) :
  (liftPair A B u v) subseteq (pairElements A B x y).carrier := by
  intro p hp
  have hp' : p in liftLeft A B u U' liftRight A B v := Finset.mem_coe.1 hp
  rcases mem_funion.mp hp' with hp | hp
  * exact subset_pairElements_liftLeft A B hu (Finset.mem_coe.2 hp)
  * exact subset_pairElements_liftRight A B hv (Finset.mem_coe.2 hp)

/-- **Theorem 7.2.** `apply(g, y) = g(y)`. -/
theorem applyMap_toElement (g : (functionSystem B C).Element) (y : B.Element) :
  (applyMap (B := B) (C := C)).toElement
    (pairElements (functionSystem B C) B g y) =
    (element_toApproxMap B C g).toElement y := by
  apply le_antisymm
  * intro Z <s, hs, <hCon, hZcon, hEnt>>
    obtain <hw, t, ht, hIn, hOut> := hEnt
    have ht_sub : t subseteq g.carrier := by

```

```

    intro p hp
    exact mem_fst_of_subset_pairElements (functionSystem B C) B hs (ht (Finset.mem_coe.1 hp))
have hsnd_sub : (sndFinset (functionSystem B C) B s) subseq y.carrier := by
  intro b hb
  exact mem_snd_of_subset_pairElements (functionSystem B C) B hs hb
have hin : funInputUnion B C t in B.Con :=
  B.con_subset
  (proposition_2_3_ii B hCon.2 (entSet_inputUnion_of_ent B C hIn))
  (subset_funion_right _ _)
have hin_sub : (funInputUnion B C t) subseq y.carrier := by
  intro b hb
  rcases (mem_funInputUnion B C).1 (Finset.mem_coe.1 hb) with <q, hq, hb'>
  exact y.closed _ b hsnd_sub (hIn q hq b hb')
have hEntZ : FunEnt B C t
  (mkFunToken B C (funInputUnion B C t) {Z} hin (C.con_sing Z)) := by
  refine <fun s' hs' hin' => hw s' (fun q hq => ht (hs' hq)) hin', t, Finset.Subset.refl _,
  ?_, hOut>
  intro q hq y' hy'
  exact B.ent_refl hin ((mem_funInputUnion B C).2 <q, hq, hy'>)
have hmem : mkFunToken B C (funInputUnion B C t) {Z} hin (C.con_sing Z) in g.carrier :=
  g.closed t _ ht_sub hEntZ
exact <funInputUnion B C t, hin_sub, <hin, C.con_sing Z, hmem>>
* intro Z <v, hv, <hvCon, hZcon, hmem>>
let p : FunToken B C := mkFunToken B C v {Z} hvCon hZcon
have hp : p in g.carrier := hmem
let s := liftPair (functionSystem B C) B {p} v
have hsCon : ProdCon (functionSystem B C) B s :=
  ProdCon_liftPair (functionSystem B C) B ((functionSystem B C).con_sing p) hvCon
have hs_sub : s subseq (pairElements (functionSystem B C) B g y).carrier :=
  subset_pairElements_liftPair (functionSystem B C) B
  (by
    intro q hq
    have : q = p := Finset.mem_singleton.mp (Finset.mem_coe.1 hq)
    subst this
    exact hp)
  hv
refine <s, hs_sub, <hsCon, C.con_sing Z, ?_>>
refine <?_, {p}, ?_, ?_, ?_>
* -- FunCon of fst s from g.consistent
  intro t ht hin
  have ht_sub : t subseq g.carrier := by
    intro q hq
    exact mem_fst_of_subset_pairElements (functionSystem B C) B hs_sub (ht hq)
  exact funCon_of_approxMap B C (element_toApproxMap B C g) t
  (fun q hq => <q.property.1, q.property.2, by
    simpa [mkFunToken_eq] using ht_sub (Finset.mem_coe.2 hq)>)) t (Finset.Subset.refl _)
hin
* -- {p} subseq fst s
  intro q hq
  have : q = p := Finset.mem_singleton.mp hq
  subst this
  rw [show s = liftLeft (functionSystem B C) B {p} U ' liftRight (functionSystem B C) B v
from rfl]
  rw [fstFinset_funion, fstFinset_liftLeft]
  exact subset_funion_left _ _ (Finset.mem_singleton_self _)
* -- snd s |- v = p.input
  intro q hq
  have : q = p := Finset.mem_singleton.mp hq
  subst this
  intro b hb
  have hb' : b in v := hb
  -- snd s = snd(liftLeft) U ' v entails v
  have : b in sndFinset (functionSystem B C) B s := by

```

```

    rw [sndFinset_liftPair]
    exact subset_funion_right _ _ hb'
    exact B.ent_refl hsCon.2 this
* -- output {Z} |- {Z}
    rw [funOutputUnion_singleton]
    exact proposition_2_3_iii C (C.con_sing Z)

/-- The uncurrying of `k : A -> (B -> C)` -/
def uncurryMap (k : ApproximableMap A (functionSystem B C)) :
  ApproximableMap (productSystem A B) C :=
  comp (applyMap (B := B) (C := C))
    (pairMap (functionSystem B C) B (comp k (fstMap A B)) (sndMap A B))

/-- **Theorem 7.2.** `h = apply comp <(curry h) comp fst, snd>`. -/
theorem uncurry_curryMap (h : ApproximableMap (productSystem A B) C) :
  uncurryMap A B C (curryMap A B C h) = h := by
  refine ApproximableMap.ext fun s w => ?_
  constructor
* rintro <m, <hk, hsnd>, <hCon, hw, hEnt>>
  -- hk : (curry h comp fst).rel s (fst m) i.e. exists v, fst.rel s v /\ curry.rel v (fst m)
  obtain <v, hfst, <hv, hcurry>> := hk
  obtain <_, t, ht, hIn, hOut> := hEnt
  -- Each q in t is in fst m, and curry says h.rel (liftPair v q.1) q.2
  have hps : forall q in t, h.rel (liftPair A B v q.val.1) q.val.2 :=
    fun q hq => hcurry q (ht hq)
  have hin : funInputUnion B C t in B.Con :=
    B.con_subset
      (proposition_2_3_ii B hCon.2 (entSet_inputUnion_of_ent B C hIn))
      (subset_funion_right _ _)
  have hmid : h.rel (liftPair A B v (funInputUnion B C t)) (funOutputUnion B C t) :=
    rel_of_funTokens A B C h v hv t hps hin
  -- fst.rel s v means EntSet (fst s) v; snd.rel s (snd m) means EntSet (snd s) (snd m)
  -- and snd m |- funInputUnion t
  have hEntIn : B.EntSet (sndFinset A B s) (funInputUnion B C t) :=
    proposition_2_3_iv B (hsnd.1.2) hin
      (proposition_2_3_iv B (hsnd.1.2) hCon.2 hsnd.2.2
        (entSet_inputUnion_of_ent B C hIn))
      (proposition_2_3_iii B hin)
  -- Actually hsnd : sndMap.rel s (snd m) = <ProdCon s, snd m in Con, EntSet (snd s) (snd m)>
  -- Simplify:
  have hs : s in (productSystem A B).Con := hfst.1
  have hEntv : A.EntSet (fstFinset A B s) v := hfst.2.2
  have hEntsnd : B.EntSet (sndFinset A B s) (sndFinset (functionSystem B C) B m) := hsnd.2.2
  have hEntIn' : B.EntSet (sndFinset A B s) (funInputUnion B C t) :=
    proposition_2_3_iv B hs.2 hin

```

Lean 4 source (lines 801–1020)

```

    (proposition_2_3_iv B hs.2 hCon.2 hEntsnd (entSet_inputUnion_of_ent B C hIn))
    (proposition_2_3_iii B hin)
  have hdom : liftPair A B (fstFinset A B s) (sndFinset A B s) in
    (productSystem A B).Con :=
    ProdCon_liftPair A B hs.1 hs.2
  -- Relate liftPair (fst s) (snd s) to liftPair v (inputUnion) via entSet_liftPair
  have hmid' : h.rel (liftPair A B (fstFinset A B s) (sndFinset A B s))
    (funOutputUnion B C t) :=
    h.mono hmid (entSet_liftPair A B hs.1 hs.2 hEntv hEntIn')
      (proposition_2_3_iii C (h.rel_cod hmid)) hdom (h.rel_cod hmid)
  -- Need: liftPair (fst s) (snd s) is "entailed by" s as product sets, or s relates via mono
  -- from a subset relation
  -- Key: s as product tokens "contains" the information of liftPair (fst s) (snd s)
  -- Use: EntSet s (liftPair (fst s) (snd s)) then mono of h from... wait h relates liftPair

```

```

to output, need s h w.
-- Show (productSystem).EntSet s (liftPair (fst s) (snd s))
have hEntLift : (productSystem A B).EntSet s (liftPair A B (fstFinset A B s) (sndFinset A B
s)) := by
  intro p hp
  unfold liftPair at hp
  rcases mem_funion.mp hp with hp | hp
  * rcases (mem_liftLeft A B).1 hp with <a, ha, rfl>
    refine <hs, ?_, ?_>
    * intro _
      exact A.ent_refl hs.1 ha
    * intro _
      exact B.ent_bot hs.2
  * rcases (mem_liftRight A B).1 hp with <b, hb, rfl>
    refine <hs, ?_, ?_>
    * intro _
      exact A.ent_bot hs.1
    * intro _
      exact B.ent_refl hs.2 hb
  exact h.mono hmid' hEntLift hOut hs hw
* intro hh
  have hs : s in (productSystem A B).Con := h.rel_dom hh
  have hw : w in C.Con := h.rel_cod hh
  let tok : FunToken B C := mkFunToken B C (sndFinset A B s) w hs.2 hw
  have hLiftCon : ProdCon A B (liftPair A B (fstFinset A B s) (sndFinset A B s)) :=
    ProdCon_liftPair A B hs.1 hs.2
  have hEntLift : (productSystem A B).EntSet
    (liftPair A B (fstFinset A B s) (sndFinset A B s)) s := by
    intro p hp
    rcases p.property with hR | hL
    * refine <hLiftCon, fun _ => ?_, fun _ => ?_>
      * rw [hR]; exact A.ent_bot hLiftCon.1
      * have hb : p.val.2 in sndFinset A B s :=
        (mem_sndFinset A B).2 <p, hp, hR, rfl>
        have hb' : p.val.2 in
          sndFinset A B (liftPair A B (fstFinset A B s) (sndFinset A B s)) := by
            rw [sndFinset_liftPair]
            exact subset_funion_right _ _ hb
            exact B.ent_refl hLiftCon.2 hb'
      * refine <hLiftCon, fun _ => ?_, fun _ => ?_>
      * have ha : p.val.1 in fstFinset A B s :=
        (mem_fstFinset A B).2 <p, hp, hL, rfl>
        have ha' : p.val.1 in
          fstFinset A B (liftPair A B (fstFinset A B s) (sndFinset A B s)) := by
            rw [fstFinset_liftPair]
            exact subset_funion_left _ _ ha
            exact A.ent_refl hLiftCon.1 ha'
      * rw [hL]; exact B.ent_bot hLiftCon.2
  have hrel : h.rel (liftPair A B (fstFinset A B s) (sndFinset A B s)) w :=
    h.mono hh hEntLift (proposition_2_3_iii C hw) hLiftCon hw
  have hcurry_tok : (curryMap A B C h).rel (fstFinset A B s) {tok} := by
    refine <hs.1, ?_>
    intro p hp
    have : p = tok := Finset.mem_singleton.mp hp
    subst this
    exact hrel
  let m := liftPair (functionSystem B C) B {tok} (sndFinset A B s)
  have hmCon : ProdCon (functionSystem B C) B m :=
    ProdCon_liftPair (functionSystem B C) B ((functionSystem B C).con_sing tok) hs.2
  have hfst_m : fstFinset (functionSystem B C) B m =
    {tok} U 'fstFinset (functionSystem B C) B
    (liftRight (functionSystem B C) B (sndFinset A B s)) := by
    change fstFinset _ _ (liftLeft _ _ {tok} U 'liftRight _ _ _) = _

```



```

    rw [fstFinset_funion, fstFinset_liftLeft]
have hsnd_m : sndFinset (functionSystem B C) B m =
  sndFinset (functionSystem B C) B (liftLeft (functionSystem B C) B {tok}) U '
  sndFinset A B s := by
  change sndFinset _ _ (liftLeft _ _ {tok} U ' liftRight _ _ _) = _
  rw [sndFinset_funion, sndFinset_liftRight]
have hcurry_fst : (curryMap A B C h).rel (fstFinset A B s)
  (fstFinset (functionSystem B C) B m) := by
  refine <hs.1, ?_>
  intro p hp
  have hp' : p in {tok} U ' fstFinset (functionSystem B C) B
    (liftRight (functionSystem B C) B (sndFinset A B s)) := by
    rw [hfst_m] at hp
    exact hp
  rcases mem_funion.mp hp' with hp' | hp'
* have : p = tok := Finset.mem_singleton.mp hp'
  subst this
  exact hrel
* have hpbot : p = (functionSystem B C).bot :=
  Finset.mem_singleton.mp
  (fstFinset_liftRight (functionSystem B C) B (sndFinset A B s) hp')
  subst hpbot
  change h.rel (liftPair A B (fstFinset A B s) (funBot B C).val.1) (funBot B C).val.2
  simp only [funBot]
  exact h.mono h.empty_rel ((productSystem A B).entSet_empty _)
  (C.entSet_empty {}) (ProdCon_liftPair A B hs.1 B.con_empty) C.con_empty
refine <m, <?_, ?_>, ?_>
* -- (curry comp fst).rel s (fst m)
  exact <fstFinset A B s, <hs, hs.1, proposition_2_3_iii A hs.1>, hcurry_fst>
* -- snd.rel s (snd m)
  refine <hs, hmCon.2, ?_>
  intro b hb
  have hb' : b in
    sndFinset (functionSystem B C) B (liftLeft (functionSystem B C) B {tok}) U '
    sndFinset A B s := by
    rwa [hsnd_m] at hb
  rcases mem_funion.mp hb' with hb' | hb'
* have : b = B.bot := Finset.mem_singleton.mp
  (sndFinset_liftLeft (functionSystem B C) B {tok} hb')
  subst this
  exact B.ent_bot hs.2
* exact B.ent_refl hs.2 hb'
* -- apply.rel m w
  refine <hmCon, hw, ?_>
  refine <(curryMap A B C h).rel_cod hcurry_fst, {tok}, ?_, ?_, ?_>
* intro q hq
  have : q = tok := Finset.mem_singleton.mp hq
  subst this
  have : tok in fstFinset (functionSystem B C) B m := by
    rw [hfst_m]
    exact subset_funion_left _ _ (Finset.mem_singleton_self _)
  exact this
* intro q hq
  have : q = tok := Finset.mem_singleton.mp hq
  subst this
  intro b hb
  have : b in sndFinset (functionSystem B C) B m := by
    rw [hsnd_m]
    exact subset_funion_right _ _ hb
  exact B.ent_refl hmCon.2 this
* rw [funOutputUnion_singleton]
  exact proposition_2_3_iii C hw

```

```

/-- **Theorem 7.2.** `curry` is the unique map satisfying the universal equation. -/
theorem curryMap_unique (h : ApproximableMap (productSystem A B) C)
  (k : ApproximableMap A (functionSystem B C))
  (huniv : uncurryMap A B C k = h) :
  k = curryMap A B C h := by
have hk : uncurryMap A B C k = uncurryMap A B C (curryMap A B C h) := by
  rw [huniv, uncurry_curryMap]
refine (ext_iff_toElement k (curryMap A B C h)).2 fun x => ?_
have happly : forall y : B.Element,
  (element_toApproxMap B C (k.toElement x)).toElement y =
    (element_toApproxMap B C ((curryMap A B C h).toElement x)).toElement y := by
intro y
have hpair_k :
  (pairMap (functionSystem B C) B (comp k (fstMap A B)) (sndMap A B)).toElement
    (pairElements A B x y) =
    pairElements (functionSystem B C) B (k.toElement x) y := by
apply element_eq_of_fst_snd (functionSystem B C) B
* have h1 : (fstMap (functionSystem B C) B).toElement
  ((pairMap (functionSystem B C) B (comp k (fstMap A B)) (sndMap A B)).toElement
    (pairElements A B x y)) = k.toElement x := by
  rw [ <- comp_toElement, comp_fstMap_pairMap, comp_toElement, fstMap_pairElements]
  exact h1.trans (fstMap_pairElements (functionSystem B C) B (k.toElement x) y).symm
* have h1 : (sndMap (functionSystem B C) B).toElement
  ((pairMap (functionSystem B C) B (comp k (fstMap A B)) (sndMap A B)).toElement
    (pairElements A B x y)) = y := by
  rw [ <- comp_toElement, comp_sndMap_pairMap, sndMap_pairElements]
  exact h1.trans (sndMap_pairElements (functionSystem B C) B (k.toElement x) y).symm
have hpair_c :
  (pairMap (functionSystem B C) B
    (comp (curryMap A B C h) (fstMap A B)) (sndMap A B)).toElement
    (pairElements A B x y) =
    pairElements (functionSystem B C) B ((curryMap A B C h).toElement x) y := by
apply element_eq_of_fst_snd (functionSystem B C) B
* have h1 : (fstMap (functionSystem B C) B).toElement
  ((pairMap (functionSystem B C) B
    (comp (curryMap A B C h) (fstMap A B)) (sndMap A B)).toElement
    (pairElements A B x y)) = (curryMap A B C h).toElement x := by
  rw [ <- comp_toElement, comp_fstMap_pairMap, comp_toElement, fstMap_pairElements]
  exact h1.trans
  (fstMap_pairElements (functionSystem B C) B ((curryMap A B C h).toElement x) y).symm
* have h1 : (sndMap (functionSystem B C) B).toElement
  ((pairMap (functionSystem B C) B
    (comp (curryMap A B C h) (fstMap A B)) (sndMap A B)).toElement
    (pairElements A B x y)) = y := by
  rw [ <- comp_toElement, comp_sndMap_pairMap, sndMap_pairElements]
  exact h1.trans
  (sndMap_pairElements (functionSystem B C) B ((curryMap A B C h).toElement x) y).symm
have hu1 : (uncurryMap A B C k).toElement (pairElements A B x y) =
  (applyMap (B := B) (C := C)).toElement
  (pairElements (functionSystem B C) B (k.toElement x) y) := by
  simp only [uncurryMap, comp_toElement, hpair_k]
have hu2 : (uncurryMap A B C (curryMap A B C h)).toElement (pairElements A B x y) =
  (applyMap (B := B) (C := C)).toElement
  (pairElements (functionSystem B C) B ((curryMap A B C h).toElement x) y) := by
  simp only [uncurryMap, comp_toElement, hpair_c]
have heq : (uncurryMap A B C k).toElement (pairElements A B x y) =
  (uncurryMap A B C (curryMap A B C h)).toElement (pairElements A B x y) := by
  rw [hk]
  rw [ <- applyMap_toElement, <- applyMap_toElement, <- hu1, <- hu2, heq]
have hx : element_toApproxMap B C (k.toElement x) =
  element_toApproxMap B C ((curryMap A B C h).toElement x) :=
  (ext_iff_toElement _ _).2 happly
have : k.toElement x = (curryMap A B C h).toElement x := by

```

```

    rw [ <- approxMap_toElement_element_toApproxMap B C (k.toElement x),
      <- approxMap_toElement_element_toApproxMap B C ((curryMap A B C h).toElement x), hx]
    rw [this]

/-- **Theorem 7.2, first sentence (Scott 1982).** Approximable maps `A -> B` are
exactly the elements of the Definition 7.1 function-space system `|A -> B|`.
The `apply` / `curry` clauses of the same numbered theorem are proved above
and are not this compared declaration. -/
theorem theorem_7_2 {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]
  (A : InfoSys a) (B : InfoSys b) :
    (forall f : ApproximableMap A B,
      element_toApproxMap A B (approxMap_toElement A B f) = f) /\
    (forall x : (functionSystem A B).Element,
      approxMap_toElement A B (element_toApproxMap A B x) = x) :=
    <element_toApproxMap_approxMap_toElement A B,
      approxMap_toElement_element_toApproxMap A B>

end InfoSys

end Scott1982

```

A.32 Scott1982/Fixpoint.lean

348 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Theorem72
import Scott1982.Factoid33

/-!
# Least fixed-point operator -- Theorem 7.3

**Scott 1982, Theorem 7.3.** Unique approximable `fix : (A -> A) -> A` such that
for every approximable `f : A -> A`:
(ii) `f(fix(f)) subseq fix(f)` (in fact equality);
(iii) if `f(x) subseq x` then `fix(f) subseq x`.

`w (fix) v` means some `w`-chain from `{}` reaches a set that entails `v`.
-/

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys

set_option linter.unusedSectionVars false

variable {a : Type*} [DecidableEq a] (A : InfoSys a)

/-- A length-`n` chain under the approximable map generated by `w`. -/
def FixChain (w : Finset (FunToken A A)) : Nat -> Finset a -> Prop
| 0, v => v = {}
| n + 1, v =>
  exists u : Finset a, FixChain w n u /\

```

```

    exists (hu : u in A.Con) (hv : v in A.Con),
    FunEnt A A w (mkFunToken A A u v hu hv)

/-- Reachability of `v` from `{}` by a finite `w`-chain. -/
def FixReach (w : Finset (FunToken A A)) (v : Finset a) : Prop :=
  exists n, FixChain A w n v

theorem fixChain_empty (w : Finset (FunToken A A)) : FixChain A w 0 {} :=
  rfl

theorem fixChain_con {w : Finset (FunToken A A)} {n : Nat} {v : Finset a}
  (h : FixChain A w n v) : v in A.Con := by
  match n with
  | 0 =>
    rw [h]
    exact A.con_empty
  | n + 1 =>
    obtain <_, _, _, hv, _> := h
    exact hv

theorem fixReach_con {w : Finset (FunToken A A)} {v : Finset a}
  (h : FixReach A w v) : v in A.Con := by
  obtain <_, hn> := h
  exact fixChain_con A hn

theorem fixReach_empty (w : Finset (FunToken A A)) : FixReach A w {} :=
  <0, fixChain_empty A w>

theorem fixChain_pad_front {w : Finset (FunToken A A)} {n : Nat} {v : Finset a}
  (hw : FunCon A A w) (h : FixChain A w n v) : FixChain A w (n + 1) v := by
  induction n generalizing v with
  | zero =>
    rw [h]
    refine <{}, rfl, A.con_empty, A.con_empty, ?_>
    exact (functionSystem A A).ent_bot hw
  | succ n ih =>
    obtain <u, hu, huC, hvC, hEnt> := h
    exact <u, ih hu, huC, hvC, hEnt>

theorem fixChain_pad {w : Finset (FunToken A A)} {n k : Nat} {v : Finset a}
  (hw : FunCon A A w) (h : FixChain A w n v) : FixChain A w (n + k) v := by
  induction k with
  | zero => exact h
  | succ k ih => exact fixChain_pad_front A hw ih

theorem funEnt_union_step {w : Finset (FunToken A A)}
  {u u' v v' : Finset a}
  (hw : FunCon A A w) (hu : u in A.Con) (hu' : u' in A.Con)
  (hv : v in A.Con) (hv' : v' in A.Con)
  (hEnt : FunEnt A A w (mkFunToken A A u v hu hv))
  (hEnt' : FunEnt A A w (mkFunToken A A u' v' hu' hv'))
  (huU : u U' u' in A.Con) :
  exists hvU : v U' v' in A.Con,
  FunEnt A A w (mkFunToken A A (u U' u') (v U' v')) huU hvU := by
  obtain <_, s1, hs1, hIn1, hOut1> := hEnt
  obtain <_, s2, hs2, hIn2, hOut2> := hEnt'
  have hsub : s1 U' s2 subseq w := funion_subset_iff.mpr <hs1, hs2>
  have hin : funInputUnion A A (s1 U' s2) in A.Con := by
    rw [funInputUnion_funion]
  have hEntU : A.EntSet (u U' u')
    (funInputUnion A A s1 U' funInputUnion A A s2) :=
    proposition_2_3_vi A
    (proposition_2_3_v A hu huU (subset_funion_left _ _))

```

```

      (entSet_inputUnion_of_ent A A hIn1) (Finset.Subset.refl _)
      (proposition_2_3_v A hu' huU (subset_funion_right _ _))
      (entSet_inputUnion_of_ent A A hIn2) (Finset.Subset.refl _)
    exact A.con_subset (proposition_2_3_ii A huU hEntU) (subset_funion_right _ _)
  have hout : funOutputUnion A A s1 U ' funOutputUnion A A s2 in A.Con := by
    have := hw _ hsub hin
    rwa [funOutputUnion_funion] at this
  have hvU : v U ' v' in A.Con := by
    have hEnt0 : A.EntSet (funOutputUnion A A s1 U ' funOutputUnion A A s2) (v U ' v') :=
      proposition_2_3_vi A
      (proposition_2_3_v A
        (A.con_subset hout (subset_funion_left _ _)) hout
        (subset_funion_left _ _) hOut1 (Finset.Subset.refl _))
      (proposition_2_3_v A
        (A.con_subset hout (subset_funion_right _ _)) hout
        (subset_funion_right _ _) hOut2 (Finset.Subset.refl _))
    exact A.con_subset (proposition_2_3_ii A hout hEnt0) (subset_funion_right _ _)
  refine <hvU, hw, s1 U ' s2, hsub, ?_, ?_>
* intro q hq
  rcases mem_funion.mp hq with hq | hq
* exact proposition_2_3_v A huU (subset_funion_left _ _) (hIn1 q hq)
  (Finset.Subset.refl _)
* exact proposition_2_3_v A hu' huU (subset_funion_right _ _) (hIn2 q hq)
  (Finset.Subset.refl _)
* rw [funOutputUnion_funion]
  exact proposition_2_3_vi A
  (proposition_2_3_v A
    (A.con_subset hout (subset_funion_left _ _)) hout
    (subset_funion_left _ _) hOut1 (Finset.Subset.refl _))
  (proposition_2_3_v A
    (A.con_subset hout (subset_funion_right _ _)) hout
    (subset_funion_right _ _) hOut2 (Finset.Subset.refl _))

theorem fixChain_union {w : Finset (FunToken A A)} {n : Nat} {v v' : Finset a}
  (hw : FunCon A A w) (h : FixChain A w n v) (h' : FixChain A w n v') :
  FixChain A w n (v U ' v') := by
  induction n generalizing v v' with
  | zero =>
    rw [h, h']
    rfl
  | succ n ih =>
    obtain <u, hu, huC, hvC, hEnt> := h
    obtain <u', hu', huC', hvC', hEnt'> := h'
    have huU : u U ' u' in A.Con := fixChain_con A (ih hu hu')
    obtain <hvU, hEntU> := funEnt_union_step A hw huC huC' hvC hvC' hEnt hEnt' huU
    exact <u U ' u', ih hu hu', huU, hvU, hEntU>

theorem fixReach_union {w : Finset (FunToken A A)} {v v' : Finset a}
  (hw : FunCon A A w) (h : FixReach A w v) (h' : FixReach A w v') :
  FixReach A w (v U ' v') := by
  obtain <n, hn> := h
  obtain <m, hm> := h'
  have hn' : FixChain A w (n + m) v := fixChain_pad A hw hn
  have hm' : FixChain A w (m + n) v' := fixChain_pad A hw hm
  have hm'' : FixChain A w (n + m) v' := by rwa [Nat.add_comm] at hm'
  exact <n + m, fixChain_union A hw hn' hm''>

theorem fixChain_mono_w {w w' : Finset (FunToken A A)} {n : Nat} {v : Finset a}
  (hw : FunCon A A w) (hw' : FunCon A A w') (h : FixChain A w n v)
  (hEntw : (functionSystem A A).EntSet w' w) : FixChain A w' n v := by
  induction n generalizing v with
  | zero => exact h
  | succ n ih =>

```

```

    obtain <u, hu, huC, hvC, hEnt> := h
    refine <u, ih hu, huC, hvC, ?_>
    exact (functionSystem A A).ent_trans hw' hw hEntw hEnt

theorem fixReach_mono_w {w w' : Finset (FunToken A A)} {v : Finset a}
  (hw : FunCon A A w) (hw' : FunCon A A w') (h : FixReach A w v)
  (hEntw : (functionSystem A A).EntSet w' w) : FixReach A w' v := by
  obtain <n, hn> := h
  exact <n, fixChain_mono_w A hw hw' hn hEntw>

namespace ApproximableMap

/-- **Theorem 7.3.** Approximable least fixed-point operator. -/
def fixMap : ApproximableMap (functionSystem A A) A where
  rel w v :=
    w in (functionSystem A A).Con /\ v in A.Con /\
    exists v0, FixReach A w v0 /\ A.EntSet v0 v
  rel_dom h := h.1
  rel_cod h := h.2.1
  empty_rel :=
    <(functionSystem A A).con_empty, A.con_empty, {}, fixReach_empty A _, A.entSet_empty {}>
  union_right := by
    rintro w v v' <hw, hv, v0, hv0, hEnt0> <_, hv', v0', hv0', hEnt0'>
    have hv0C := fixReach_con A hv0
    have hv0'C := fixReach_con A hv0'
    have hUreach : FixReach A w (v0 U ' v0') := fixReach_union A hw hv0 hv0'
    have hv0U := fixReach_con A hUreach
    have hEntU : A.EntSet (v0 U ' v0') (v U ' v') :=
      proposition_2_3_vi A
      (proposition_2_3_v A hv0C hv0U (subset_funion_left _ _) hEnt0 (Finset.Subset.refl _))
      (proposition_2_3_v A hv0'C hv0U (subset_funion_right _ _) hEnt0' (Finset.Subset.refl _))
    )
    refine <hw, ?_, v0 U ' v0', hUreach, hEntU>
    exact A.con_subset (proposition_2_3_ii A hv0U hEntU) (subset_funion_right _ _)
  mono := by
    rintro w w' v v' <hw, hv, v0, hv0, hEnt0> hEntw hEntv hw' hv'
    refine <hw', hv', v0, fixReach_mono_w A hw hw' hv0 hEntw, ?_>
    exact proposition_2_3_iv A (fixReach_con A hv0) hv hEnt0 hEntv

/-- Least fixed point of an approximable endomap, as an element. -/
def fixElement (f : ApproximableMap A A) : A.Element :=
  (fixMap A).toElement (approxMap_toElement A A f)

/-- Tokens of `w` subseqeq approxMap_toElement f` satisfy `f.rel`. -/
private theorem rel_of_mem_approx (f : ApproximableMap A A)
  {w : Finset (FunToken A A)}
  (hw : w subseqeq (approxMap_toElement A A f).carrier) {p : FunToken A A}
  (hp : p in w) : f.rel p.val.1 p.val.2 := by
  simpa [mem_approxMap_toElement] using hw (Finset.mem_coe.2 hp)

/-- A `w`-chain under tokens from `f` stays inside a pre-fixed point. -/
theorem fixChain_subset_pre_fixed (f : ApproximableMap A A) (x : A.Element)
  (hpre : f.toElement x <= x) {w : Finset (FunToken A A)}
  (hw : w subseqeq (approxMap_toElement A A f).carrier) {n : Nat} {u : Finset a}
  (h : FixChain A w n u) : u subseqeq x.carrier := by
  induction n generalizing u with
  | zero =>
    intro b hb
    rw [h] at hb
    exact False.elim (Finset.notMem_empty b hb)
  | succ n ih =>
    obtain <u0, hu0, hu0C, huC, hEnt> := h
    have hu0_sub := ih hu0

```

```

obtain <_, s, hs, hIn, hOut> := hEnt
have hps : forall q in s, f.rel q.val.1 q.val.2 := fun q hq =>
  rel_of_mem_approx A f hw (hs hq)
have hin : funInputUnion A A s in A.Con :=
  A.con_subset
  (proposition_2_3_ii A huOC (entSet_inputUnion_of_ent A A hIn))
  (subset_funion_right _ _)
have hrel : f.rel (funInputUnion A A s) (funOutputUnion A A s) :=
  rel_input_output_union A A f s hps hin
have hrel' : f.rel u0 (funOutputUnion A A s) :=
  f.mono hrel (entSet_inputUnion_of_ent A A hIn)
  (proposition_2_3_iii A (f.rel_cod hrel)) huOC (f.rel_cod hrel)
have hrel'' : f.rel u0 u :=
  f.mono hrel' (proposition_2_3_iii A huOC) hOut huOC huC
intro b hb
have hsing : f.rel u0 {b} :=
  f.mono hrel'' (proposition_2_3_iii A huOC)
  (fun z hz => by
    rw [Finset.mem_singleton] at hz; subst hz
    exact A.ent_refl huC hb)
  huOC (A.con_sing b)
exact hpre <u0, hu0_sub, hsing>

/-- **Theorem 7.3(iii).** `fix(f)` is below every pre-fixed point. -/
theorem fixElement_le_of_pre_fixed (f : ApproximableMap A A) (x : A.Element)
  (hpre : f.toElement x <= x) : fixElement A f <= x := by
  intro a <w, hw, <hwCon, haCon, v0, hv0, hEnt>>
  obtain <n, hn> := hv0
  have hsub := fixChain_subset_pre_fixed A f x hpre hw hn
  have hEnta : A.Ent v0 a := hEnt a (Finset.mem_singleton_self _)
  exact x.closed v0 a hsub hEnta

/-- Extend a chain by one `f`-step using a token from `approxMap_toElement f`. -/
theorem fixChain_step_of_rel (f : ApproximableMap A A)
  {w : Finset (FunToken A A)} {n : Nat} {u v : Finset a}
  (hw : w ⊆ (approxMap_toElement A A f).carrier)
  (hu : FixChain A w n u) (hrel : f.rel u v) :
  exists w' : Finset (FunToken A A),
    w' ⊆ (approxMap_toElement A A f).carrier /\
      FunCon A A w' /\ FixChain A w' (n + 1) v := by
  let p := mkFunToken A A u v (f.rel_dom hrel) (f.rel_cod hrel)
  have hp : p in (approxMap_toElement A A f).carrier := by
    simpa [p, mem_approxMap_toElement, mkFunToken] using hrel
  let w' := insert p w
  have hw' : w' ⊆ (approxMap_toElement A A f).carrier := by
    intro q hq
    rcases Finset.mem_insert.mp (Finset.mem_coe.1 hq) with rfl | hq
    * exact hp
    * exact hw (Finset.mem_coe.2 hq)
  have hw'Con : FunCon A A w' :=
    (approxMap_toElement A A f).consistent w' hw'
  have hwCon : FunCon A A w :=
    (approxMap_toElement A A f).consistent w hw
  have hu' : FixChain A w' n u :=
    fixChain_mono_w A hwCon hw'Con hu
  (fun q hq => (functionSystem A A).ent_refl hw'Con (Finset.mem_insert_of_mem hq))
  refine <w', hw', hw'Con, u, hu', f.rel_dom hrel, f.rel_cod hrel, ?_>
  refine <hw'Con, {p}, ?_, ?_, ?_>
  * intro q hq
    cases Finset.mem_singleton.mp hq
    exact Finset.mem_insert_self _ _
  * intro q hq
    cases Finset.mem_singleton.mp hq

```

```

    exact proposition_2_3_iii A (f.rel_dom hrel)
* rw [funOutputUnion_singleton]
    exact proposition_2_3_iii A (f.rel_cod hrel)

/-- **Theorem 7.3(ii).** `f(fix(f)) subseteq fix(f)`. -/
theorem toElement_fixElement_le (f : ApproximableMap A A) :
  f.toElement (fixElement A f) <= fixElement A f := by
  intro a <u, hu, hrel>
  obtain <w, hw, <hwCon, _, v0, hv0, hEnt>> :=
    exists_rel_of_subset_image (fixMap A) (approxMap_toElement A A f) u hu
  -- v0 entails u, and f.rel u {a}; mono to get f.rel v0 {a} after adjusting
  have hv0C := fixReach_con A hv0
  have huC : u in A.Con := f.rel_dom hrel
  have hrel' : f.rel v0 {a} :=
    f.mono hrel hEnt
    (proposition_2_3_iii A (f.rel_cod hrel)) hv0C (f.rel_cod hrel)
  obtain <n, hn> := hv0
  obtain <w', hw', hw'Con, hchain> := fixChain_step_of_rel A f hw hn hrel'
  refine <w', hw', <hw'Con, A.con_sing a, {a}, <n + 1, hchain>, ?_>>
  exact proposition_2_3_iii A (A.con_sing a)

/-- **Theorem 7.3.** `fix(f)` is a fixed point. -/
theorem fixElement_fixed (f : ApproximableMap A A) :
  f.toElement (fixElement A f) = fixElement A f := by
  apply le_antisymm
  * exact toElement_fixElement_le A f
  * have hpre : f.toElement (f.toElement (fixElement A f)) <=
      f.toElement (fixElement A f) :=
      toElement_mono f (toElement_fixElement_le A f)
    exact fixElement_le_of_pre_fixed A f (f.toElement (fixElement A f)) hpre

/-- **Theorem 7.3.** Uniqueness of the least fixed-point operator on elements. -/
theorem fixElement_unique (f : ApproximableMap A A) (phi : A.Element)
  (hfix : f.toElement phi <= phi)
  (hleast : forall x : A.Element, f.toElement x <= x -> phi <= x) :
  phi = fixElement A f := by
  apply le_antisymm
  * exact hleast _ (toElement_fixElement_le A f)
  * exact fixElement_le_of_pre_fixed A f phi hfix

/-- Uniqueness of `fix` as an approximable operator (Scott Thm 7.3). -/
theorem fixMap_unique (phi : ApproximableMap (functionSystem A A) A)
  (hpre : forall f : ApproximableMap A A,
    f.toElement (phi.toElement (approxMap_toElement A A f)) <=
      phi.toElement (approxMap_toElement A A f))
  (hleast : forall f : ApproximableMap A A, forall x : A.Element,
    f.toElement x <= x -> phi.toElement (approxMap_toElement A A f) <= x) :
  phi = fixMap A :=
  (ext_iff_toElement phi (fixMap A)).2 fun g => by
    let hf := element_toApproxMap A A g
    have hg : approxMap_toElement A A hf = g :=
      approxMap_toElement_element_toApproxMap A A g
    rw [← hg]
    exact fixElement_unique A hf (phi.toElement (approxMap_toElement A A hf))
      (hpre hf) (hleast hf)

end ApproximableMap

end InfoSys

end Scott1982

```


A.33 Scott1982/Proposition74.lean

171 lines.

Lean 4 source

```
/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Fixpoint
import Scott1982.Proposition54
import Scott1982.Proposition55
import Scott1982.Factoid33
import Scott1982.Factoid35

/-!
# Proposition 7.4 -- Plotkin's equational characterization of `fix`

**Scott 1982, Proposition 7.4.** The least fixed-point operator is uniquely
determined by:
(i) `fix_A : (A -> A) -> A` for every system `A`;
(ii) `fix(f) = f(fix(f))`;
(iii) `h(fix_A(f)) = fix_B(g)` whenever `h comp f = g comp h` and `h(False_A) = False_B`.
-/

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys
namespace ApproximableMap

variable {a b : Type*} [DecidableEq a] [DecidableEq b]
variable (A : InfoSys a) (B : InfoSys b)

/-- **Proposition 7.4(ii).** -/
theorem proposition_7_4_ii (f : ApproximableMap A A) :
  f.toElement (fixElement A f) = fixElement A f :=
  fixElement_fixed A f

theorem toElement_comp_fix_le (f : ApproximableMap A A) (g : ApproximableMap B B)
  (h : ApproximableMap A B)
  (hcomm : comp h f = comp g h) :
  g.toElement (h.toElement (fixElement A f)) <= h.toElement (fixElement A f) := by
  have h1 : g.toElement (h.toElement (fixElement A f)) =
    h.toElement (f.toElement (fixElement A f)) := by
    rw [ <- comp_toElement, <- hcomm, comp_toElement ]
  rw [h1, proposition_7_4_ii A f]

theorem fixElement_le_toElement_fix (f : ApproximableMap A A) (g : ApproximableMap B B)
  (h : ApproximableMap A B)
  (hcomm : comp h f = comp g h) :
  fixElement B g <= h.toElement (fixElement A f) :=
  fixElement_le_of_pre_fixed B g (h.toElement (fixElement A f))
  (toElement_comp_fix_le A B f g h hcomm)

private theorem closure_empty_eq_bot :
  A.closure ({ } : Finset a) A.con_empty = A.botElement := by
  apply le_antisymm
  * intro a ha
```

```

    exact A.ent_trans (A.con_sing A.bot) A.con_empty (A.entSet_empty _) ha
* intro a ha
    exact A.ent_trans A.con_empty (A.con_sing A.bot)
      (fun x hx => by
        have : x = A.bot := Finset.mem_singleton.mp hx
        subst this
        exact A.ent_bot A.con_empty)
    ha

theorem fixChain_toElement_le_fix (f : ApproximableMap A A) (g : ApproximableMap B B)
  (h : ApproximableMap A B)
  (hcomm : comp h f = comp g h)
  (hstrict : h.toElement A.botElement = B.botElement)
  {w : Finset (FunToken A A)}
  (hw : w ⊆ (approxMap_toElement A A f).carrier) {n : Nat} {u : Finset a}
  (hchain : FixChain A w n u) :
  h.toElement (A.closure u (fixChain_con A hchain)) ≤ fixElement B g := by
induction n generalizing u with
| zero =>
  have hu : u = {} := hchain
  subst hu
  simp [closure_empty_eq_bot A, hstrict] using
    (botElement_le B (fixElement B g))
| succ n ih =>
  obtain <u0, hu0, hu0C, huC, hEnt> := hchain
  have ih' := ih hu0
  obtain <_, s, hs, hIn, hOut> := hEnt
  have hps : forall q in s, f.rel q.val.1 q.val.2 := fun q hq => by
    simp [mem_approxMap_toElement] using hw (Finset.mem_coe.2 (hs hq))
  have hin : funInputUnion A A s in A.Con :=
    A.con_subset
      (proposition_2_3_ii A hu0C (entSet_inputUnion_of_ent A A hIn))
      (subset_funion_right _ _)
  have hrel : f.rel (funInputUnion A A s) (funOutputUnion A A s) :=
    rel_input_output_union A A f s hps hin
  have hrel' : f.rel u0 (funOutputUnion A A s) :=
    f.mono hrel (entSet_inputUnion_of_ent A A hIn)
      (proposition_2_3_iii A (f.rel_cod hrel)) hu0C (f.rel_cod hrel)
  have hrel'' : f.rel u0 u :=
    f.mono hrel' (proposition_2_3_iii A hu0C) hOut hu0C huC
  have huf : A.closure u huC ≤ f.toElement (A.closure u0 hu0C) := by
    intro a ha
    have hsub : u ⊆ (f.toElement (A.closure u0 hu0C)).carrier := by
      intro b hb
      have hsing : f.rel u0 {b} :=
        f.mono hrel'' (proposition_2_3_iii A hu0C)
          (fun z hz => by
            rw [Finset.mem_singleton] at hz; subst hz
            exact A.ent_refl huC hb)
        hu0C (A.con_sing b)
      refine <u0, fun z hz => A.subset_closure hu0C hz, hsing>
      exact (f.toElement (A.closure u0 hu0C)).closed u a hsub ha
    have hhuf : h.toElement (A.closure u huC) ≤
      h.toElement (f.toElement (A.closure u0 hu0C)) :=
      toElement_mono h huf
    have hcomm' : h.toElement (f.toElement (A.closure u0 hu0C)) =
      g.toElement (h.toElement (A.closure u0 hu0C)) := by
      rw [← comp_toElement, hcomm, comp_toElement]
    have hg : g.toElement (h.toElement (A.closure u0 hu0C)) ≤
      g.toElement (fixElement B g) :=
      toElement_mono g ih'
    have hfixg : g.toElement (fixElement B g) = fixElement B g :=
      proposition_7_4_ii B g

```

```

    calc
      h.toElement (A.closure u huC) <= h.toElement (f.toElement (A.closure u0 hu0C)) := hhuf
      _ = g.toElement (h.toElement (A.closure u0 hu0C)) := hcomm'
      _ <= g.toElement (fixElement B g) := hgle
      _ = fixElement B g := hfixg

theorem toElement_fix_le_fixElement (f : ApproximableMap A A) (g : ApproximableMap B B)
  (h : ApproximableMap A B)
  (hcomm : comp h f = comp g h)
  (hstrict : h.toElement A.botElement = B.botElement) :
  h.toElement (fixElement A f) <= fixElement B g := by
  intro Z <u, hu, hrel>
  obtain <w, hw, <_, _, v0, hv0, hEnt>> :=
    exists_rel_of_subset_image (fixMap A) (approxMap_toElement A A f) u hu
  obtain <n, hn> := hv0
  have hcl := fixChain_toElement_le_fix A B f g h hcomm hstrict hw hn
  have hv0C := fixChain_con A hn
  have hrel' : h.rel v0 {Z} :=
    h.mono hrel hEnt (proposition_2_3_iii B (h.rel_cod hrel)) hv0C (h.rel_cod hrel)
  have hZ : Z in (h.toElement (A.closure v0 hv0C)).carrier :=
    <v0, A.subset_closure hv0C, hrel'>
  exact hcl hZ

/-- **Proposition 7.4(iii).** Naturality of `fix` for strict commuting maps. -/
theorem proposition_7_4_iii (f : ApproximableMap A A) (g : ApproximableMap B B)
  (h : ApproximableMap A B)
  (hcomm : comp h f = comp g h)
  (hstrict : h.toElement A.botElement = B.botElement) :
  h.toElement (fixElement A f) = fixElement B g := by
  apply le_antisymm
  * exact toElement_fix_le_fixElement A B f g h hcomm hstrict
  * exact fixElement_le_toElement_fix A B f g h hcomm

/-- **Proposition 7.4.** Uniqueness via the Theorem 7.3 least-fixed-point property
(Plotkin's (ii) is the fixed-point equation; leastness is 7.3(iii)). -/
theorem proposition_7_4_unique (f : ApproximableMap A A) (phi : A.Element)
  (hii : f.toElement phi = phi)
  (hleat : forall x : A.Element, f.toElement x <= x -> phi <= x) :
  phi = fixElement A f :=
  fixElement_unique A f phi (le_of_eq hii) hleat

/-- Identity is strict, so 7.4(iii) specializes to a tautology on `fix` itself. -/
theorem idMap_botElement : (idMap A).toElement A.botElement = A.botElement :=
  idMap_toElement A A.botElement

end ApproximableMap

end InfoSys

end Scott1982

```

A.34 Scott1982/Factoid75.lean

800 lines.

Lean 4 source (lines 1–400)

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

```

```

import Mathlib.Data.Multiset.MapFold
import Scott1982.FunctionSpace
import Scott1982.Theorem72
import Scott1982.Factoid25
import Scott1982.Factoid33
import Scott1982.Factoid35

/--!
# Factoid 7.5 -- strict mappings, `strictify`, BOOL, and the conditional

**Scott 1982 (after Prop 7.4).** Strict approximable maps; Scott?s `strict` /
`strictify` operator; InfoSys `A -> ? B`; flat BOOL; `A x A ~= (BOOL -> ? A)`.
-/

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys

set_option linter.unusedSectionVars false

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

/-- `{}` |- u`. -/
def EmptyEntails (A : InfoSys a) (u : Finset a) : Prop :=
  A.entSet ({}` : Finset a) u

theorem emptyEntails_empty (A : InfoSys a) : EmptyEntails A ({}` : Finset a) :=
  A.entSet_empty {}

theorem emptyEntails_mem_botElement (A : InfoSys a) {X : a}
  (h : A.ent ({}` : Finset a) X) : X in A.botElement.carrier :=
  A.ent_trans (A.con_sing A.bot) A.con_empty (A.entSet_empty _) h

theorem mem_botElement_emptyEntails (A : InfoSys a) {X : a}
  (h : X in A.botElement.carrier) : A.ent ({}` : Finset a) X :=
  A.ent_trans A.con_empty (A.con_sing A.bot)
  (fun x hx => by
    have : x = A.bot := Finset.mem_singleton.mp hx
    subst this
    exact A.ent_bot A.con_empty)
  h

theorem emptyEntails_funion {? : Type*} [DecidableEq ?] (sys : InfoSys ?)
  {u v : Finset ?} :
  EmptyEntails sys (u U ' v) <-> EmptyEntails sys u /\ EmptyEntails sys v := by
  constructor
  * intro h
    exact <fun x hx => h x (subset_funion_left _ _ hx),
      fun x hx => h x (subset_funion_right _ _ hx)>
  * intro <hu, hv> x hx
    rcases mem_funion.mp hx with hx | hx
  * exact hu x hx
  * exact hv x hx

/-- `f` is strict when `f(False) = False`. -/
def IsStrict (A : InfoSys a) (B : InfoSys b) (f : ApproximableMap A B) : Prop :=
  f.toElement A.botElement = B.botElement

variable (A : InfoSys a) (B : InfoSys b)

```

namespace ApproximableMap

/-- **Factoid 7.5.** Largest strict approximable map contained in `f` (Scott?s `strict`). -/
def strictify (f : ApproximableMap A B) : ApproximableMap A B where

```

rel u v :=
  u in A.Con /\ v in B.Con /\
    (EmptyEntails B v /\ (f.rel u v /\ not EmptyEntails A u))
rel_dom h := h.1
rel_cod h := h.2.1
empty_rel := <A.con_empty, B.con_empty, Or.inl (emptyEntails_empty B)>
union_right := by
  rintro u v v' <hu, hv, hvcase> <_, hv', hv'case>
  have hvU : v U ' v' in B.Con := by
    rcases hvcase with hE | <hf, _>
    * rcases hv'case with hE' | <hf', _>
    * exact B.con_subset
      (proposition_2_3_ii B B.con_empty (proposition_2_3_vi B hE hE'))
      (subset_funion_right _ _)
    * exact f.rel_cod (f.union_right
      (f.mono f.empty_rel (A.entSet_empty u) hE hu hv) hf')
    * rcases hv'case with hE' | <hf', _>
    * exact f.rel_cod (f.union_right hf
      (f.mono f.empty_rel (A.entSet_empty u) hE' hu hv'))
    * exact f.rel_cod (f.union_right hf hf')
  refine <hu, hvU, ?_>
  rcases hvcase with hE | <hf, hn>
  * rcases hv'case with hE' | <hf', hn'>
  * exact Or.inl (proposition_2_3_vi B hE hE')
  * exact Or.inr <f.union_right
    (f.mono f.empty_rel (A.entSet_empty u) hE hu hv) hf', hn'>
  * rcases hv'case with hE' | <hf', hn'>
  * exact Or.inr <f.union_right hf
    (f.mono f.empty_rel (A.entSet_empty u) hE' hu hv'), hn'>
  * exact Or.inr <f.union_right hf hf', hn>
mono := by
  rintro u u' v v' <hu, hv, hcase> hEntu hEntv hu' hv'
  refine <hu', hv', ?_>
  rcases hcase with hE | <hf, hn>
  * exact Or.inl (fun y hy => B.ent_trans B.con_empty hv hE (hEntv y hy))
  * refine Or.inr <f.mono hf hEntu hEntv hu' hv', fun hE' => hn ?_>
    exact fun x hx => A.ent_trans A.con_empty hu' hE' (hEntu x hx)

```

theorem IsStrict_strictify (f : ApproximableMap A B) :

```

  IsStrict A B (strictify A B f) := by
  apply le_antisymm
  * intro Z <u, hu, <_, _, hcase>>
  have huE : EmptyEntails A u := fun x hx =>
    mem_botElement_emptyEntails A (hu (Finset.mem_coe.2 hx))
  rcases hcase with hE | <_, hn>
  * exact emptyEntails_mem_botElement B (hE Z (Finset.mem_singleton_self _))
  * exact False.elim (hn huE)
  * exact botElement_le B _

```

theorem strictify_toElement_le (f : ApproximableMap A B) (x : A.Element) :

```

  (strictify A B f).toElement x <= f.toElement x := by
  intro Z <u, hu, <_, _, hcase>>
  rcases hcase with hE | <hf, _>
  * exact (botElement_le B (f.toElement x))
    (emptyEntails_mem_botElement B (hE Z (Finset.mem_singleton_self _)))
  * exact <u, hu, hf>

```

end ApproximableMap

```

/-! ## BOOL -/

inductive BoolToken
| bot
| tru
| fls
deriving DecidableEq

/-- Points of the flat BOOL domain: undefined, true, or false. -/
inductive BoolPoint
| undef
| isTrue
| isFalse

def boolSat : BoolPoint -> BoolToken -> Prop
| .undef, .bot => True
| .undef, _ => False
| .isTrue, .bot => True
| .isTrue, .tru => True
| .isTrue, .fls => False
| .isFalse, .bot => True
| .isFalse, .fls => True
| .isFalse, .tru => False

theorem boolSat_bot (p : BoolPoint) : boolSat p .bot := by
cases p <;> trivial

theorem boolSat_sing (t : BoolToken) : exists p, boolSat p t := by
cases t with
| bot => exact <.undef, trivial>
| tru => exact <.isTrue, trivial>
| fls => exact <.isFalse, trivial>

/-- **Factoid 7.5.** Flat BOOL information system. -/
def boolSystem : InfoSys BoolToken :=
Factoid25.ofSatisfaction boolSat .bot boolSat_bot boolSat_sing

/-! ## Strict function space `A -> ? B` -/

/-- Token of `A -> ? B`: a function-space token that itself respects empty input. -/
def IsStrictToken (p : FunToken A B) : Prop :=
EmptyEntails A p.val.1 -> EmptyEntails B p.val.2

def StrictFunToken : Type _ :=
{p : FunToken A B // IsStrictToken A B p}

instance : DecidableEq (StrictFunToken A B) :=
fun p q =>
match (inferInstance : DecidableEq (FunToken A B)) p.val q.val with
| isTrue h => isTrue (Subtype.ext h)
| isFalse n => isFalse fun h => n (congrArg Subtype.val h)

/-- Forget strict tokens to ordinary function-space tokens. -/
def toFunFinset (w : Finset (StrictFunToken A B)) : Finset (FunToken A B) :=
<Multiset.map (Subtype.val : StrictFunToken A B -> FunToken A B) w.1,
(Multiset.nodup_map_iff_of_injective Subtype.val_injective).2 w.nodup>

theorem mem_toFunFinset {w : Finset (StrictFunToken A B)} {p : FunToken A B} :
p in toFunFinset A B w <-> exists q in w, q.val = p := by
change (p in Multiset.map (Subtype.val : StrictFunToken A B -> FunToken A B) w.1) <-> _
constructor
* intro hp
rcases Multiset.mem_map.mp hp with <q, hq, heq>

```

```

    refine <q, hq, heq>
* intro <q, hq, heq>
    exact Multiset.mem_map.mpr <q, hq, heq>

theorem toFunFinset_subset {s w : Finset (StrictFunToken A B)} (h : s subseq w) :
  toFunFinset A B s subseq toFunFinset A B w := by
  intro p hp
  rcases (mem_toFunFinset A B).1 hp with <q, hq, rfl>
  exact (mem_toFunFinset A B).2 <q, h hq, rfl>

theorem toFunFinset_insert (w : Finset (StrictFunToken A B)) (p : StrictFunToken A B) :
  toFunFinset A B (insert p w) = insert p.val (toFunFinset A B w) := by
  ext q
  constructor
* intro hq
  rcases (mem_toFunFinset A B).1 hq with <r, hr, rfl>
  rcases Finset.mem_insert.mp hr with rfl | hr
  * exact Finset.mem_insert_self _ _
  * exact Finset.mem_insert_of_mem ((mem_toFunFinset A B).2 <r, hr, rfl>)
* intro hq
  rcases Finset.mem_insert.mp hq with rfl | hq
  * exact (mem_toFunFinset A B).2 <p, Finset.mem_insert_self _ _, rfl>
  * rcases (mem_toFunFinset A B).1 hq with <r, hr, rfl>
    exact (mem_toFunFinset A B).2 <r, Finset.mem_insert_of_mem hr, rfl>

theorem toFunFinset_singleton (p : StrictFunToken A B) :
  toFunFinset A B ({p} : Finset (StrictFunToken A B)) = {p.val} := by
  ext q
  constructor
* intro hq
  rcases (mem_toFunFinset A B).1 hq with <r, hr, rfl>
  exact Finset.mem_singleton.mpr (congrArg Subtype.val (Finset.mem_singleton.mp hr))
* intro hq
  have : q = p.val := Finset.mem_singleton.mp hq
  subst this
  exact (mem_toFunFinset A B).2 <p, Finset.mem_singleton_self p, rfl>

theorem toFunFinset_empty :
  toFunFinset A B ({ } : Finset (StrictFunToken A B)) = ({ } : Finset (FunToken A B)) := by
  ext p
  constructor
* intro hp
  rcases (mem_toFunFinset A B).1 hp with <_, hq, _>
  exact False.elim (Finset.notMem_empty _ hq)
* intro hp
  exact False.elim (Finset.notMem_empty _ hp)

/-- Consistency for `A -> ? B` (Scott 7.5: FunCon plus empty-input clause). -/
def StrictFunCon (w : Finset (StrictFunToken A B)) : Prop :=
  FunCon A B (toFunFinset A B w) /\
  (forall s subseq w,
    EmptyEntails A (funInputUnion A B (toFunFinset A B s)) ->
    EmptyEntails B (funOutputUnion A B (toFunFinset A B s)))

/-- Entailment for `A -> ? B`. -/
def StrictFunEnt (w : Finset (StrictFunToken A B)) (p : StrictFunToken A B) : Prop :=
  StrictFunCon A B w /\ FunEnt A B (toFunFinset A B w) p.val

def strictBot : StrictFunToken A B :=
  <funBot A B, fun _ => emptyEntails_empty B>

private theorem insert_filter_ne_strict {p : StrictFunToken A B}
  {t : Finset (StrictFunToken A B)} (hp : p in t) :

```

```

    insert p (t.filter (* != p)) = t := by
  ext q
  constructor
  * intro hq
    rcases Finset.mem_insert.mp hq with rfl | hq
    * exact hp
    * exact (Finset.mem_filter.mp hq).1
  * intro hq
    if h : q = p then
      exact Finset.mem_insert.mpr (Or.inl h)
    else
      exact Finset.mem_insert.mpr (Or.inr (Finset.mem_filter.mpr <hq, h>))

private theorem filter_ne_subset_of_subset_insert_strict {p : StrictFunToken A B}
  {t w : Finset (StrictFunToken A B)} (ht : t subteq insert p w) :
  t.filter (* != p) subteq w := by
  intro q hq
  have <hqt, hne> := Finset.mem_filter.mp hq
  have : q in insert p w := ht hqt
  rcases Finset.mem_insert.mp this with rfl | hqW
  * exact False.elim (hne rfl)
  * exact hqW

/-- **Factoid 7.5.** The strict function-space information system `A -> ? B`. -/
def strictFunctionSystem : InfoSys (StrictFunToken A B) where
  bot := strictBot A B
  Con := {w | StrictFunCon A B w}
  Ent := StrictFunEnt A B
  con_subset := by
    intro w w' <hwFun, hwE> hw'
    refine <fun s hs hin =>
      hwFun s (fun q hq => (toFunFinset_subset A B hw') (hs hq)) hin, ?_>
    intro s hs hE
    exact hwE s (fun q hq => hw' (hs hq)) hE
  con_sing := by
    intro p
    refine <?_, ?_>
    * intro s hs _hin
      have hs' : s subteq {p.val} := by
        intro q hq
        have : q in toFunFinset A B ({p} : Finset (StrictFunToken A B)) := hs hq
        rwa [toFunFinset_singleton] at this
      have hsub : funOutputUnion A B s subteq p.val.val.2 := by
        intro y hy
        rcases (mem_funOutputUnion A B).1 hy with <q, hq, hy'>
        have : q = p.val := Finset.mem_singleton.mp (hs' hq)
        cases this
        exact hy'
      exact B.con_subset p.val.property.2 hsub
    * intro s hs hE y hy
      rcases (mem_funOutputUnion A B).1 hy with <q, hq, hy'>
      rcases (mem_toFunFinset A B).1 hq with <r, hr, rfl>
      have hr' : r = p := Finset.mem_singleton.mp (hs hr)
      cases hr'
      have hEp : EmptyEntails A p.val.val.1 := by
        intro x hx
        exact hE x ((mem_funInputUnion A B).2
          <p.val, (mem_toFunFinset A B).2 <p, hr, rfl>, hx>)
        exact p.property hEp y hy'
  ent_con := by
    intro w p <hwFun, hwE>, hFunEnt
    have hFunInsert : FunCon A B (insert p.val (toFunFinset A B w)) :=
      (functionSystem A B).ent_con hFunEnt

```



```

have hFunInsert' : FunCon A B (toFunFinset A B (insert p w)) :=
  (toFunFinset_insert A B w p).symm |> hFunInsert
refine <hFunInsert', ?_>
intro t ht hE
if hp : p in t then
  let t' := t.filter (* != p)
  have ht_eq : insert p t' = t := insert_filter_ne_strict A B hp
  have ht'sub : t' subseteq w := filter_ne_subset_of_subset_insert_strict A B ht
  have hE' : EmptyEntails A (p.val.val.1 U ' funInputUnion A B (toFunFinset A B t')) := by
    rwa [ <- ht_eq, toFunFinset_insert, funInputUnion_insert ] at hE
  have <hEp, hEt'> := (emptyEntails_funion A).1 hE'
  have hOutp : EmptyEntails B p.val.val.2 := p.property hEp
  have hOutt' : EmptyEntails B (funOutputUnion A B (toFunFinset A B t')) :=
    hwE t' ht'sub hEt'
  have : EmptyEntails B (p.val.val.2 U ' funOutputUnion A B (toFunFinset A B t')) :=
    (emptyEntails_funion B).2 <hOutp, hOutt'>
  rwa [ <- ht_eq, toFunFinset_insert, funOutputUnion_insert ]
else
  have ht' : t subseteq w := by
    intro q hq
    have : q in insert p w := ht hq
    rcases Finset.mem_insert.mp this with rfl | hqW
    * exact False.elim (hp hq)
    * exact hqW
  exact hwE t ht' hE
ent_bot := by
  intro w hw
  refine <hw, (functionSystem A B).ent_bot hw.1>
ent_refl := by
  intro w p hw hp
  have hpFun : p.val in toFunFinset A B w :=
    (mem_toFunFinset A B).2 <p, hp, rfl>
  exact <hw, (functionSystem A B).ent_refl hw.1 hpFun>
ent_trans := by
  intro u v c hv hu hEnts <_, hFunEnt>
  have hFunEnts : forall y in toFunFinset A B u,
    FunEnt A B (toFunFinset A B v) y := by
    intro y hy
    rcases (mem_toFunFinset A B).1 hy with <q, hq, rfl>
    exact (hEnts q hq).2
  have hFun : FunEnt A B (toFunFinset A B v) c.val :=
    (functionSystem A B).ent_trans hv.1 hu.1 hFunEnts hFunEnt
  exact <hv, hFun>

/#!/ ## Strict maps as elements; retract via `strictify` -/

theorem IsStrictToken_of_IsStrict (f : ApproximableMap A B) (hf : IsStrict A B f)
  {p : FunToken A B} (hrel : f.rel p.val.1 p.val.2) : IsStrictToken A B p := by
  intro hE
  have hsub : p.val.1 subseteq A.botElement.carrier := fun x hx =>
    emptyEntails_mem_botElement A (hE x (Finset.mem_coe.1 hx))
  have hout : p.val.2 subseteq (f.toElement A.botElement).carrier := by
    intro y hy
    refine <p.val.1, hsub, ?_>
    have hy' : y in p.val.2 := Finset.mem_coe.1 hy
    exact f.mono hrel (proposition_2_3_iii A p.property.1)
    (fun z hz => by
      have : z = y := Finset.mem_singleton.mp hz
      subst this
      exact B.ent_refl p.property.2 hy')
    p.property.1 (B.con_sing y)
  rw [hf] at hout
  exact fun y hy => mem_botElement_emptyEntails B (hout (Finset.mem_coe.2 hy))

```

```
open ApproximableMap
```

```
/-- Package a strict approximable map as an element of  $\lambda A \rightarrow ? B$ . -/
def approxMap_toStrictElement (f : ApproximableMap A B) (hf : IsStrict A B f) :
  (strictFunctionSystem A B).Element where
  carrier := {p : StrictFunToken A B | f.rel p.val.val.1 p.val.val.2}
  consistent := by
    intro Y hY
    refine <?_, ?_>
    * exact funCon_of_approxMap A B f (toFunFinset A B Y) fun q hq => by
      rcases (mem_toFunFinset A B).1 hq with <p, hp, rfl>
      exact hY (Finset.mem_coe.2 hp)
    * intro s hs hE
      have hrel : forall q in toFunFinset A B s, f.rel q.val.1 q.val.2 := by
        intro q hq
```

Lean 4 source (lines 401–800)

```
      rcases (mem_toFunFinset A B).1 hq with <p, hp, rfl>
      exact hY (Finset.mem_coe.2 (hs hp))
  have hin : funInputUnion A B (toFunFinset A B s) in A.Con := by
    have hsub : (funInputUnion A B (toFunFinset A B s)) subseq A.botElement.carrier :=
      fun x hx => emptyEntails_mem_botElement A (hE x (Finset.mem_coe.1 hx))
    exact A.botElement.consistent _ hsub
  have hIO := rel_input_output_union A B f (toFunFinset A B s) hrel hin
  exact IsStrictToken_of_IsStrict A B f hf
    (p := mkFunToken A B (funInputUnion A B (toFunFinset A B s))
      (funOutputUnion A B (toFunFinset A B s)) hin (f.rel_cod hIO))
    hIO hE
closed := by
  intro Y p hY hEnt
  obtain <_, hFunEnt> := hEnt
  obtain <_, s, hs, hEntIn, hEntOut> := hFunEnt
  have hrel : forall q in s, f.rel q.val.1 q.val.2 := by
    intro q hq
    have hq' : q in toFunFinset A B Y := hs hq
    rcases (mem_toFunFinset A B).1 hq' with <r, hr, rfl>
    exact hY (Finset.mem_coe.2 hr)
  have hin_s : funInputUnion A B s in A.Con :=
    A.con_subset
    (proposition_2_3_ii A p.val.property.1 (entSet_inputUnion_of_ent A B hEntIn))
    (subset_funion_right _ _)
  have hIO := rel_input_output_union A B f s hrel hin_s
  have hU : f.rel p.val.val.1 (funOutputUnion A B s) :=
    f.mono hIO (entSet_inputUnion_of_ent A B hEntIn)
    (proposition_2_3_iii B (f.rel_cod hIO)) p.val.property.1 (f.rel_cod hIO)
  exact f.mono hU (proposition_2_3_iii A p.val.property.1) hEntOut
    p.val.property.1 p.val.property.2

/-- Scott's retract map  $\lambda A \rightarrow B \mid \rightarrow \lambda A \rightarrow ? B$  induced by `strictify`. -/
def strictifyElement (x : (functionSystem A B).Element) :
  (strictFunctionSystem A B).Element :=
  approxMap_toStrictElement A B (strictify A B (element_toApproxMap A B x))
  (IsStrict_strictify A B _)

/-! ## Conditional:  $\lambda A \times A \rightarrow (BOOL \rightarrow ? A)$  -/

def boolTrue : boolSystem.Element :=
  boolSystem.closure {BoolToken.tru} (boolSystem.con_sing _)

def boolFalse : boolSystem.Element :=
  boolSystem.closure {BoolToken.fls} (boolSystem.con_sing _)
```

```

theorem not_ent_empty_tru : not boolSystem.Ent {} BoolToken.tru := by
  intro <_, hAll>
  exact hAll .undef (fun _ h => absurd h (Finset.notMem_empty _))

theorem not_ent_empty_fls : not boolSystem.Ent {} BoolToken.fls := by
  intro <_, hAll>
  exact hAll .undef (fun _ h => absurd h (Finset.notMem_empty _))

theorem boolCon_trichotomy {u : Finset BoolToken} (hu : u in boolSystem.Con) :
  EmptyEntails boolSystem u \ / boolSystem.Ent u .tru \ / boolSystem.Ent u .fls := by
  obtain <p, hp> := hu
  cases p with
  | undef =>
    refine Or.inl fun t ht => ?_
    have hsat := hp t ht
    cases t with
    | bot =>
      exact <<.undef, fun _ h => False.elim (Finset.notMem_empty _ h)>,
        fun _ _ => boolSat_bot _>
    | tru => exact False.elim hsat
    | fls => exact False.elim hsat
  | isTrue =>
    if htru : BoolToken.tru in u then
      exact Or.inr (Or.inl <<.isTrue, hp>, fun q hq => hq _ htru>)
    else
      refine Or.inl fun t ht => ?_
      have hsat := hp t ht
      cases t with
      | bot =>
        exact <<.undef, fun _ h => False.elim (Finset.notMem_empty _ h)>,
          fun _ _ => boolSat_bot _>
      | tru => exact False.elim (htru ht)
      | fls => exact False.elim hsat
  | isFalse =>
    if hfls : BoolToken.fls in u then
      exact Or.inr (Or.inr <<.isFalse, hp>, fun q hq => hq _ hfls>)
    else
      refine Or.inl fun t ht => ?_
      have hsat := hp t ht
      cases t with
      | bot =>
        exact <<.undef, fun _ h => False.elim (Finset.notMem_empty _ h)>,
          fun _ _ => boolSat_bot _>
      | tru => exact False.elim hsat
      | fls => exact False.elim (hfls ht)

theorem subset_of_entSet_mem {x : A.Element} {v v' : Finset a}
  (hv : v subseq x.carrier) (hEnt : A.EntSet v v') : v' subseq x.carrier := by
  intro y hy
  exact x.closed v y hv (hEnt y (Finset.mem_coe.1 hy))

def condMap (x y : A.Element) : ApproximableMap boolSystem A where
  rel u v :=
    u in boolSystem.Con /\ v in A.Con /\
    (EmptyEntails boolSystem u -> EmptyEntails A v) /\
    (boolSystem.Ent u .tru -> (v : Set a) subseq x.carrier) /\
    (boolSystem.Ent u .fls -> (v : Set a) subseq y.carrier)
  rel_dom h := h.1
  rel_cod h := h.2.1
  empty_rel := by
    refine <boolSystem.con_empty, A.con_empty, fun _ => emptyEntails_empty A, ?_, ?_>
    * exact fun h => False.elim (not_ent_empty_tru h)
    * exact fun h => False.elim (not_ent_empty_fls h)

```

```

union_right := by
  rintro u v v' <hu, hv, hE, ht, hf> <_, hv', hE', ht', hf'>
  have hvU : v U ' v' in A.Con := by
    rcases boolCon_trichotomy hu with hEmp | hTru | hFls
    * have hsub : (v U ' v') subseq A.botElement.carrier := by
      intro z hz
      rcases mem_funion.mp (Finset.mem_coe.1 hz) with hz | hz
      * exact emptyEntails_mem_botElement A (hE hEmp z hz)
      * exact emptyEntails_mem_botElement A (hE' hEmp z hz)
    exact A.botElement.consistent _ hsub
  * have hsub : (v U ' v') subseq x.carrier := by
    intro z hz
    rcases mem_funion.mp (Finset.mem_coe.1 hz) with hz | hz
    * exact ht hTru (Finset.mem_coe.2 hz)
    * exact ht' hTru (Finset.mem_coe.2 hz)
  exact x.consistent _ hsub
  * have hsub : (v U ' v') subseq y.carrier := by
    intro z hz
    rcases mem_funion.mp (Finset.mem_coe.1 hz) with hz | hz
    * exact hf hFls (Finset.mem_coe.2 hz)
    * exact hf' hFls (Finset.mem_coe.2 hz)
  exact y.consistent _ hsub
refine <hu, hvU, fun hEmp => (emptyEntails_funion A).2 <hE hEmp, hE' hEmp>, ?_, ?_>
* intro hTru z hz
  rcases mem_funion.mp (Finset.mem_coe.1 hz) with hz | hz
  * exact ht hTru (Finset.mem_coe.2 hz)
  * exact ht' hTru (Finset.mem_coe.2 hz)
* intro hFls z hz
  rcases mem_funion.mp (Finset.mem_coe.1 hz) with hz | hz
  * exact hf hFls (Finset.mem_coe.2 hz)
  * exact hf' hFls (Finset.mem_coe.2 hz)
mono := by
  rintro u u' v v' <hu, hv, hE, ht, hf> hEntu hEntv hu' hv'
  refine <hu', hv', ?_, ?_, ?_>
  * intro hEmp
    have hEmpu : EmptyEntails boolSystem u := fun t ht' =>
      boolSystem.ent_trans boolSystem.con_empty hu' hEmp (hEntu t ht')
    exact fun z hz => A.ent_trans A.con_empty hv (hE hEmpu) (hEntv z hz)
  * intro hTru
    rcases boolCon_trichotomy hu with hEmp | hTruU | hFlsU
    * have hEv' : EmptyEntails A v' := fun z hz =>
      A.ent_trans A.con_empty hv (hE hEmp) (hEntv z hz)
    exact fun z hz =>
      (botElement_le A x) (emptyEntails_mem_botElement A (hEv' z (Finset.mem_coe.1 hz)))
  * exact subset_of_entSet_mem A (ht hTruU) hEntv
  * exfalso
    have hFls' : boolSystem.Ent u' .fls :=
      boolSystem.ent_trans hu' hu (fun t ht' => hEntu t ht') hFlsU
    obtain <<p, hp>, hAllT> := hTru
    obtain <_, hAllF> := hFls'
    have htru : boolSat p .tru := hAllT p hp
    have hfls : boolSat p .fls := hAllF p hp
    cases p <|> simp only [boolSat] at htru hfls
  * intro hFls
    rcases boolCon_trichotomy hu with hEmp | hTruU | hFlsU
    * have hEv' : EmptyEntails A v' := fun z hz =>
      A.ent_trans A.con_empty hv (hE hEmp) (hEntv z hz)
    exact fun z hz =>
      (botElement_le A y) (emptyEntails_mem_botElement A (hEv' z (Finset.mem_coe.1 hz)))
  * exfalso
    have hTru' : boolSystem.Ent u' .tru :=
      boolSystem.ent_trans hu' hu (fun t ht' => hEntu t ht') hTruU
    obtain <<p, hp>, hAllF> := hFls

```

```

    obtain <_, hAllT> := hTru'
    have hfls : boolSat p .fls := hAllF p hp
    have htru : boolSat p .tru := hAllT p hp
    cases p <;> simp only [boolSat] at htru hfls
    * exact subset_of_entSet_mem A (hf hflsU) hEntv

theorem IsStrict_condMap (x y : A.Element) : IsStrict boolSystem A (condMap A x y) := by
  apply le_antisymm
  * intro Z <u, hu, <_, _, hE, _, _>>
    have hEmp : EmptyEntails boolSystem u := fun t ht =>
      mem_botElement_emptyEntails boolSystem (hu (Finset.mem_coe.2 ht))
    exact emptyEntails_mem_botElement A (hE hEmp Z (Finset.mem_singleton_self _))
  * exact botElement_le A _

theorem not_ent_tru_fls : not boolSystem.Ent {BoolToken.tru} BoolToken.fls := by
  intro <<p, hp>, hAll>
  have htru : boolSat p .tru := hp .tru (Finset.mem_singleton_self _)
  have hfls : boolSat p .fls := hAll p hp
  cases p <;> simp only [boolSat] at htru hfls

theorem not_ent_fls_tru : not boolSystem.Ent {BoolToken.fls} BoolToken.tru := by
  intro <<p, hp>, hAll>
  have hfls : boolSat p .fls := hp .fls (Finset.mem_singleton_self _)
  have htru : boolSat p .tru := hAll p hp
  cases p <;> simp only [boolSat] at htru hfls

theorem not_emptyEntails_singleton_tru :
  not EmptyEntails boolSystem ({BoolToken.tru} : Finset BoolToken) :=
  fun h => not_ent_empty_tru (h .tru (Finset.mem_singleton_self _))

theorem not_emptyEntails_singleton_fls :
  not EmptyEntails boolSystem ({BoolToken.fls} : Finset BoolToken) :=
  fun h => not_ent_empty_fls (h .fls (Finset.mem_singleton_self _))

theorem condMap_toElement_boolTrue (x y : A.Element) :
  (condMap A x y).toElement boolTrue = x := by
  apply le_antisymm
  * intro Z <u, hu, <huC, _, hE, ht, _>>
    rcases boolCon_trichotomy huC with hEmp | hTru | hFls
    * exact (botElement_le A x)
      (emptyEntails_mem_botElement A (hE hEmp Z (Finset.mem_singleton_self _)))
    * exact ht hTru (Finset.mem_coe.2 (Finset.mem_singleton_self _))
    * exfalso
      obtain <<_, _>, hAll> := hFls
      have : boolSat .isTrue .fls := hAll .isTrue fun t ht' =>
        (hu (Finset.mem_coe.2 ht')).2 .isTrue fun s hs => by
          have : s = .tru := Finset.mem_singleton.mp hs
          subst this; trivial
      simp only [boolSat] at this
  * intro Z hZ
    refine <{BoolToken.tru}, ?_, boolSystem.con_sing _, A.con_sing _, ?_, ?_, ?_>
  * intro t ht
    have : t = .tru := Finset.mem_singleton.mp (Finset.mem_coe.1 ht)
    subst this
    exact (boolSystem.subset_closure (boolSystem.con_sing _))
      (Finset.mem_coe.2 (Finset.mem_singleton_self _))
  * exact fun hEmp => False.elim (not_emptyEntails_singleton_tru hEmp)
  * intro _ z hz
    have : z = Z := Finset.mem_singleton.mp (Finset.mem_coe.1 hz)
    subst this; exact hZ
  * exact fun hFls => False.elim (not_ent_tru_fls hFls)

theorem condMap_toElement_boolFalse (x y : A.Element) :

```

```

    (condMap A x y).toElement boolFalse = y := by
  apply le_antisymm
* intro Z <u, hu, <huC, _, hE, _, hf>>
  rcases boolCon_trichotomy huC with hEmp | hTru | hFls
* exact (botElement_le A y)
  (emptyEntails_mem_botElement A (hE hEmp Z (Finset.mem_singleton_self _)))
* ex falso
  obtain <<_, _>, hAll> := hTru
  have : boolSat .isFalse .tru := hAll .isFalse fun t ht' =>
    (hu (Finset.mem_coe.2 ht')).2 .isFalse fun s hs => by
    have : s = .fls := Finset.mem_singleton.mp hs
    subst this; trivial
  simp only [boolSat] at this
* exact hf hFls (Finset.mem_coe.2 (Finset.mem_singleton_self _))
* intro Z hZ
  refine <{BoolToken.fls}, ?_, boolSystem.con_sing _, A.con_sing _, ?_, ?_, ?_>
* intro t ht
  have : t = .fls := Finset.mem_singleton.mp (Finset.mem_coe.1 ht)
  subst this
  exact (boolSystem.subset_closure (boolSystem.con_sing _))
    (Finset.mem_coe.2 (Finset.mem_singleton_self _))
* exact fun hEmp => False.elim (not_emptyEntails_singleton_fls hEmp)
* exact fun hTru => False.elim (not_ent_fls_tru hTru)
* intro _ z hz
  have : z = Z := Finset.mem_singleton.mp (Finset.mem_coe.1 hz)
  subst this; exact hZ

/-- `cond` as an element of `|BOOL -> ? A|`. -/
def condElement (z : (productSystem A A).Element) :
  (strictFunctionSystem boolSystem A).Element :=
  approxMap_toStrictElement boolSystem A
  (condMap A ((fstMap A A).toElement z) ((sndMap A A).toElement z))
  (IsStrict_condMap A _ _)

/-- One direction of the conditional isomorphism on pairs. -/
theorem condMap_pair_roundtrip (x y : A.Element) :
  (condMap A x y).toElement boolTrue = x /\
  (condMap A x y).toElement boolFalse = y :=
  <condMap_toElement_boolTrue A x y, condMap_toElement_boolFalse A x y>

theorem entSet_singleton_tru_of_ent_tru {u : Finset BoolToken}
  (_hu : u in boolSystem.Con) (h : boolSystem.Ent u .tru) :
  boolSystem.EntSet ({BoolToken.tru} : Finset BoolToken) u := by
  intro t ht
  obtain <<p, hp>, hAll> := h
  have htru : boolSat p .tru := hAll p hp
  cases p with
  | undef => simp only [boolSat] at htru
  | isFalse => simp only [boolSat] at htru
  | isTrue =>
    have hsat : boolSat .isTrue t := hp t ht
    cases t with
    | bot =>
      exact <<.isTrue, fun s hs => by
        have : s = .tru := Finset.mem_singleton.mp hs
        subst this; trivial>, fun _ _ => boolSat_bot _>
    | tru =>
      exact <<.isTrue, fun s hs => by
        have : s = .tru := Finset.mem_singleton.mp hs
        subst this; trivial>, fun q hq => hq .tru (Finset.mem_singleton_self _)>
    | fls => exact False.elim hsat

theorem entSet_singleton_fls_of_ent_fls {u : Finset BoolToken}

```

```

    (_hu : u in boolSystem.Con) (h : boolSystem.Ent u .fls) :
    boolSystem.EntSet ({BoolToken.fls} : Finset BoolToken) u := by
intro t ht
obtain <<p, hp>, hAll> := h
have hfls : boolSat p .fls := hAll p hp
cases p with
| undef => simp only [boolSat] at hfls
| isTrue => simp only [boolSat] at hfls
| isFalse =>
  have hsat : boolSat .isFalse t := hp t ht
  cases t with
  | bot =>
    exact <<.isFalse, fun s hs => by
      have : s = .fls := Finset.mem_singleton.mp hs
      subst this; trivial>, fun _ _ => boolSat_bot _>
  | fls =>
    exact <<.isFalse, fun s hs => by
      have : s = .fls := Finset.mem_singleton.mp hs
      subst this; trivial>, fun q hq => hq .fls (Finset.mem_singleton_self _)>
  | tru => exact False.elim hsat

/-- Every strict BOOL-map is `cond` of its values at true and false. -/
theorem condMap_unique (f : ApproximableMap boolSystem A) (hf : IsStrict boolSystem A f) :
  f = condMap A (f.toElement boolTrue) (f.toElement boolFalse) := by
  refine ApproximableMap.ext fun u v => ?_
  constructor
  * intro hrel
    refine <f.rel_dom hrel, f.rel_cod hrel, ?_, ?_, ?_>
  * intro hEmp
    have hsub : u subseq boolSystem.botElement.carrier := fun t ht =>
      emptyEntails_mem_botElement boolSystem (hEmp t (Finset.mem_coe.1 ht))
    have hout : v subseq (f.toElement boolSystem.botElement).carrier := by
      intro y hy
      refine <u, hsub, f.mono hrel (proposition_2_3_iii boolSystem (f.rel_dom hrel))
        (fun z hz => by
          have : z = y := Finset.mem_singleton.mp hz
          subst this
          exact A.ent_refl (f.rel_cod hrel) (Finset.mem_coe.1 hy))
        (f.rel_dom hrel) (A.con_sing y)>
    rw [hf] at hout
    exact fun y hy => mem_botElement_emptyEntails A (hout (Finset.mem_coe.2 hy))
  * intro hTru y hy
    have hEntU : boolSystem.EntSet {BoolToken.tru} u :=
      entSet_singleton_tru_of_ent_tru (f.rel_dom hrel) hTru
    refine <{BoolToken.tru}, ?_, ?_>
  * exact boolSystem.subset_closure (boolSystem.con_sing _)
  * exact f.mono hrel hEntU
    (fun z hz => by
      have : z = y := Finset.mem_singleton.mp hz
      subst this
      exact A.ent_refl (f.rel_cod hrel) (Finset.mem_coe.1 hy))
    (boolSystem.con_sing _) (A.con_sing y)
  * intro hFls y hy
    have hEntU : boolSystem.EntSet {BoolToken.fls} u :=
      entSet_singleton_fls_of_ent_fls (f.rel_dom hrel) hFls
    refine <{BoolToken.fls}, ?_, ?_>
  * exact boolSystem.subset_closure (boolSystem.con_sing _)
  * exact f.mono hrel hEntU
    (fun z hz => by
      have : z = y := Finset.mem_singleton.mp hz
      subst this
      exact A.ent_refl (f.rel_cod hrel) (Finset.mem_coe.1 hy))
    (boolSystem.con_sing _) (A.con_sing y)

```

```

* intro <hu, hv, hE, ht, hfl>
rcases boolCon_trichotomy hu with hEmp | hTru | hFls
* have hrel0 : f.rel u {} :=
  f.mono f.empty_rel (boolSystem.entSet_empty u) (A.entSet_empty {}) hu A.con_empty
exact f.mono hrel0 (proposition_2_3_iii boolSystem hu) (hE hEmp) hu hv
* have hsub : v ⊆ f.toElement boolTrue.carrier := ht hTru
obtain <w, hw, hrelw> := exists_rel_of_subset_image f boolTrue v hsub
have hEntW : boolSystem.EntSet {BoolToken.tru} w := fun t ht' =>
  hw (Finset.mem_coe.2 ht')
have hrelTru : f.rel {BoolToken.tru} v :=
  f.mono hrelw hEntW (proposition_2_3_iii A (f.rel_cod hrelw))
  (boolSystem.con_sing _) (f.rel_cod hrelw)
have hEntU : boolSystem.EntSet u {BoolToken.tru} := fun t ht' => by
  have : t = .tru := Finset.mem_singleton.mp ht'
  subst this; exact hTru
exact f.mono hrelTru hEntU (proposition_2_3_iii A hv) hu hv
* have hsub : v ⊆ f.toElement boolFalse.carrier := hfl hFls
obtain <w, hw, hrelw> := exists_rel_of_subset_image f boolFalse v hsub
have hEntW : boolSystem.EntSet {BoolToken.fls} w := fun t ht' =>
  hw (Finset.mem_coe.2 ht')
have hrelFls : f.rel {BoolToken.fls} v :=
  f.mono hrelw hEntW (proposition_2_3_iii A (f.rel_cod hrelw))
  (boolSystem.con_sing _) (f.rel_cod hrelw)
have hEntU : boolSystem.EntSet u {BoolToken.fls} := fun t ht' => by
  have : t = .fls := Finset.mem_singleton.mp ht'
  subst this; exact hFls
exact f.mono hrelFls hEntU (proposition_2_3_iii A hv) hu hv

/-- Domain isomorphism data: `A x A ~ (BOOL -> ? A)` via `cond`. -/
theorem conditional_iso (x y : A.Element) :
  (condMap A x y).toElement boolTrue = x /\
  (condMap A x y).toElement boolFalse = y /\
  forall (f : ApproximableMap boolSystem A), IsStrict boolSystem A f ->
    f = condMap A (f.toElement boolTrue) (f.toElement boolFalse) :=
  <condMap_toElement_boolTrue A x y, condMap_toElement_boolFalse A x y,
  fun f hf => condMap_unique A f hf>

end InfoSys

end Scott1982

```

A.35 Scott1982/Factoid76.lean

361 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Theorem72
import Scott1982.Proposition56

/-!
# Factoid 7.6 -- combinators as approximable operators

**Scott 1982 (*Higher-type functions and the combinators*)** Once function spaces
exist as domains, the familiar combinators become approximable operators:

```



```

* `const : B -> (A -> B)` with `const(b) = constMap b`
* `pair : (C -> A) x (C -> B) -> (C -> A x B)` with `pair(f, g) = <f, g>`
* `comp : (B -> C) x (A -> B) -> (A -> C)` with `comp(g, f) = g comp f`

Each is obtained by currying a composite of `apply`, `fst`/`snd`, and `pair`
(already approximable), so approximability is immediate.
-/

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys

open ApproximableMap

set_option linter.unusedSectionVars false

variable {a b ? : Type*} [DecidableEq a] [DecidableEq b] [DecidableEq ?]
variable (A : InfoSys a) (B : InfoSys b) (C : InfoSys ?)

/#! ## Helpers: pairing with projections -/

theorem pairMap_toElement_pairElements
  {X : InfoSys a} {Y : InfoSys b} {Z : InfoSys ?}
  (f : ApproximableMap Z X) (g : ApproximableMap Z Y) (z : Z.Element) :
  (pairMap X Y f g).toElement z =
    pairElements X Y (f.toElement z) (g.toElement z) := by
  refine element_eq_of_fst_snd X Y _ _ ?_ ?_
  * rw [ <- comp_toElement, comp_fstMap_pairMap, fstMap_pairElements]
  * rw [ <- comp_toElement, comp_sndMap_pairMap, sndMap_pairElements]

theorem pairMap_comp_fst_snd_toElement
  {P : InfoSys a} {X : InfoSys b} {Y : InfoSys ?}
  (k : ApproximableMap P (functionSystem Y X))
  (p : P.Element) (y : Y.Element) :
  (pairMap (functionSystem Y X) Y (comp k (fstMap P Y)) (sndMap P Y)).toElement
    (pairElements P Y p y) =
    pairElements (functionSystem Y X) Y (k.toElement p) y := by
  refine element_eq_of_fst_snd (functionSystem Y X) Y _ _ ?_ ?_
  * rw [ <- comp_toElement, comp_fstMap_pairMap, comp_toElement, fstMap_pairElements,
    fstMap_pairElements]
  * rw [ <- comp_toElement, comp_sndMap_pairMap, sndMap_pairElements, sndMap_pairElements]

/#! ## `const : B -> (A -> B)` -/

/-- **Factoid 7.6.** Operator `const : B -> (A -> B)`, as `curry fst`. -/
def constOp : ApproximableMap B (functionSystem A B) :=
  curryMap B A B (fstMap B A)

theorem constOp_uncurry : uncurryMap B A B (constOp A B) = fstMap B A := by
  simp only [constOp, uncurry_curryMap]

theorem constOp_toElement (b : B.Element) :
  (constOp A B).toElement b =
    approxMap_toElement A B (constMap (A := A) b) := by
  have huniq :
    element_toApproxMap A B ((constOp A B).toElement b) = constMap (A := A) b := by
  refine constMap_unique (A := A) b _ fun x => ?_
  have hpair := pairMap_comp_fst_snd_toElement (constOp A B) b x
  calc
    (element_toApproxMap A B ((constOp A B).toElement b)).toElement x
      = (uncurryMap B A B (constOp A B)).toElement (pairElements B A b x) := by

```

```

      simp only [uncurryMap, comp_toElement]
      rw [hpair, applyMap_toElement]
    _ = (fstMap B A).toElement (pairElements B A b x) := by rw [constOp_uncurry]
    _ = b := fstMap_pairElements B A b x
  rw [ <- huniq, approxMap_toElement_element_toApproxMap]

/--! ## `pair : (C -> A) x (C -> B) -> (C -> A x B)` -/

/-- Uncurried pairing: `(C -> A) x (C -> B) x C -> A x B`. -/
def pairOpUncurry : ApproximableMap
  (productSystem (productSystem (functionSystem C A) (functionSystem C B)) C)
  (productSystem A B) :=
  let P := productSystem (functionSystem C A) (functionSystem C B)
  pairMap A B
    (comp (applyMap (B := C) (C := A))
      (pairMap (functionSystem C A) C
        (comp (fstMap (functionSystem C A) (functionSystem C B)) (fstMap P C))
        (sndMap P C)))
    (comp (applyMap (B := C) (C := B))
      (pairMap (functionSystem C B) C
        (comp (sndMap (functionSystem C A) (functionSystem C B)) (fstMap P C))
        (sndMap P C)))

/-- **Factoid 7.6.** Operator `pair : (C -> A) x (C -> B) -> (C -> A x B)`. -/
def pairOp : ApproximableMap
  (productSystem (functionSystem C A) (functionSystem C B))
  (functionSystem C (productSystem A B)) :=
  curryMap (productSystem (functionSystem C A) (functionSystem C B)) C (productSystem A B)
  (pairOpUncurry A B C)

theorem pairOp_uncurry :
  uncurryMap (productSystem (functionSystem C A) (functionSystem C B)) C (productSystem A B)
  (pairOp A B C) =
  pairOpUncurry A B C := by
  simp only [pairOp, uncurryCurryMap]

theorem pairOpUncurry_toElement (f : ApproximableMap C A) (g : ApproximableMap C B)
  (x : C.Element) :
  (pairOpUncurry A B C).toElement
    (pairElements (productSystem (functionSystem C A) (functionSystem C B)) C
      (pairElements (functionSystem C A) (functionSystem C B)
        (approxMap_toElement C A f) (approxMap_toElement C B g))
      x) =
  (pairMap A B f g).toElement x := by
  let P := productSystem (functionSystem C A) (functionSystem C B)
  let fg := pairElements (functionSystem C A) (functionSystem C B)
    (approxMap_toElement C A f) (approxMap_toElement C B g)
  have hfst_arg :
    (pairMap (functionSystem C A) C
      (comp (fstMap (functionSystem C A) (functionSystem C B)) (fstMap P C))
      (sndMap P C)).toElement
      (pairElements P C fg x) =
    pairElements (functionSystem C A) C (approxMap_toElement C A f) x := by
  refine element_eq_of_fst_snd (functionSystem C A) C _ _ ?_ ?_
  * have hL :
    (fstMap (functionSystem C A) C).toElement
      ((pairMap (functionSystem C A) C
        (comp (fstMap (functionSystem C A) (functionSystem C B)) (fstMap P C))
        (sndMap P C)).toElement
        (pairElements P C fg x)) =
    approxMap_toElement C A f := by
  rw [ <- comp_toElement, comp_fstMap_pairMap, comp_toElement, fstMap_pairElements,
    fstMap_pairElements]

```

```

    exact hL.trans (fstMap_pairElements (functionSystem C A) C _ _).symm
* have hL :
  (sndMap (functionSystem C A) C).toElement
    ((pairMap (functionSystem C A) C
      (comp (fstMap (functionSystem C A) (functionSystem C B)) (fstMap P C))
      (sndMap P C)).toElement
      (pairElements P C fg x)) =
    x := by
  rw [ <- comp_toElement, comp_sndMap_pairMap, sndMap_pairElements]
  exact hL.trans (sndMap_pairElements (functionSystem C A) C _ _).symm
have hsnd_arg :
  (pairMap (functionSystem C B) C
    (comp (sndMap (functionSystem C A) (functionSystem C B)) (fstMap P C))
    (sndMap P C)).toElement
    (pairElements P C fg x) =
  pairElements (functionSystem C B) C (approxMap_toElement C B g) x := by
refine element_eq_of_fst_snd (functionSystem C B) C _ _ ?_ ?_
* have hL :
  (fstMap (functionSystem C B) C).toElement
    ((pairMap (functionSystem C B) C
      (comp (sndMap (functionSystem C A) (functionSystem C B)) (fstMap P C))
      (sndMap P C)).toElement
      (pairElements P C fg x)) =
  approxMap_toElement C B g := by
  rw [ <- comp_toElement, comp_fstMap_pairMap, comp_toElement, fstMap_pairElements,
    sndMap_pairElements]
  exact hL.trans (fstMap_pairElements (functionSystem C B) C _ _).symm
* have hL :
  (sndMap (functionSystem C B) C).toElement
    ((pairMap (functionSystem C B) C
      (comp (sndMap (functionSystem C A) (functionSystem C B)) (fstMap P C))
      (sndMap P C)).toElement
      (pairElements P C fg x)) =
  x := by
  rw [ <- comp_toElement, comp_sndMap_pairMap, sndMap_pairElements]
  exact hL.trans (sndMap_pairElements (functionSystem C B) C _ _).symm
refine element_eq_of_fst_snd A B _ _ ?_ ?_
* have L :
  (fstMap A B).toElement
    ((pairOpUncurry A B C).toElement (pairElements P C fg x)) =
  f.toElement x := by
  simp only [pairOpUncurry]
  rw [ <- comp_toElement, comp_fstMap_pairMap, comp_toElement, hfst_arg,
    applyMap_toElement, element_toApproxMap_approxMap_toElement]
  have R :
    (fstMap A B).toElement ((pairMap A B f g).toElement x) = f.toElement x := by
    rw [ <- comp_toElement, comp_fstMap_pairMap]
  exact L.trans R.symm
* have L :
  (sndMap A B).toElement
    ((pairOpUncurry A B C).toElement (pairElements P C fg x)) =
  g.toElement x := by
  simp only [pairOpUncurry]
  rw [ <- comp_toElement, comp_sndMap_pairMap, comp_toElement, hsnd_arg,
    applyMap_toElement, element_toApproxMap_approxMap_toElement]
  have R :
    (sndMap A B).toElement ((pairMap A B f g).toElement x) = g.toElement x := by
    rw [ <- comp_toElement, comp_sndMap_pairMap]
  exact L.trans R.symm

theorem pairOp_toElement (f : ApproximableMap C A) (g : ApproximableMap C B) :
  (pairOp A B C).toElement
    (pairElements (functionSystem C A) (functionSystem C B)

```

```

      (approxMap_toElement C A f) (approxMap_toElement C B g)) =
      approxMap_toElement C (productSystem A B) (pairMap A B f g) := by
let P := productSystem (functionSystem C A) (functionSystem C B)
let fg := pairElements (functionSystem C A) (functionSystem C B)
      (approxMap_toElement C A f) (approxMap_toElement C B g)
have huniq :
  element_toApproxMap C (productSystem A B) ((pairOp A B C).toElement fg) =
    pairMap A B f g := by
  refine (ext_iff_toElement _ _).2 fun x => ?_
  have hpair :=
    pairMap_comp_fst_snd_toElement (X := productSystem A B) (pairOp A B C) fg x
  calc
    (element_toApproxMap C (productSystem A B) ((pairOp A B C).toElement fg)).toElement x
    = (uncurryMap P C (productSystem A B) (pairOp A B C)).toElement
      (pairElements P C fg x) := by
      simp only [uncurryMap, comp_toElement]
      rw [hpair, applyMap_toElement]
    _ = (pairOpUncurry A B C).toElement (pairElements P C fg x) := by
      rw [pairOp_uncurry]
    _ = (pairMap A B f g).toElement x := pairOpUncurry_toElement A B C f g x
  rw [ <- huniq, approxMap_toElement_element_toApproxMap]

/--! ## `comp : (B -> C) x (A -> B) -> (A -> C)` -/

/-- Uncurried composition: `(B -> C) x (A -> B) x A -> C`. -/
def compOpUncurry : ApproximableMap
  (productSystem (productSystem (functionSystem B C) (functionSystem A B)) A) C :=
  let Q := productSystem (functionSystem B C) (functionSystem A B)
  comp (applyMap (B := B) (C := C))
    (pairMap (functionSystem B C) B
      (comp (fstMap (functionSystem B C) (functionSystem A B)) (fstMap Q A))
      (comp (applyMap (B := A) (C := B))
        (pairMap (functionSystem A B) A
          (comp (sndMap (functionSystem B C) (functionSystem A B)) (fstMap Q A))
          (sndMap Q A))))

/-- **Factoid 7.6.** Operator `comp : (B -> C) x (A -> B) -> (A -> C)`. -/
def compOp : ApproximableMap
  (productSystem (functionSystem B C) (functionSystem A B))
  (functionSystem A C) :=
  curryMap (productSystem (functionSystem B C) (functionSystem A B)) A C
  (compOpUncurry A B C)

theorem compOp_uncurry :
  uncurryMap (productSystem (functionSystem B C) (functionSystem A B)) A C (compOp A B C) =
  compOpUncurry A B C := by
  simp only [compOp, uncurry_curryMap]

theorem compOpUncurry_toElement (g : ApproximableMap B C) (f : ApproximableMap A B)
  (x : A.Element) :
  (compOpUncurry A B C).toElement
    (pairElements (productSystem (functionSystem B C) (functionSystem A B)) A
      (pairElements (functionSystem B C) (functionSystem A B)
        (approxMap_toElement B C g) (approxMap_toElement A B f))
      x) =
  (comp g f).toElement x := by
let Q := productSystem (functionSystem B C) (functionSystem A B)
let gf := pairElements (functionSystem B C) (functionSystem A B)
  (approxMap_toElement B C g) (approxMap_toElement A B f)
have hinner :
  (pairMap (functionSystem A B) A
    (comp (sndMap (functionSystem B C) (functionSystem A B)) (fstMap Q A))
    (sndMap Q A)).toElement

```

```

    (pairElements Q A gf x) =
    pairElements (functionSystem A B) A (approxMap_toElement A B f) x := by
  refine element_eq_of_fst_snd (functionSystem A B) A _ _ ?_ ?_
  * have hL :
    (fstMap (functionSystem A B) A).toElement
      ((pairMap (functionSystem A B) A
        (comp (sndMap (functionSystem B C) (functionSystem A B)) (fstMap Q A))
        (sndMap Q A)).toElement
        (pairElements Q A gf x)) =
    approxMap_toElement A B f := by
  rw [ <- comp_toElement, comp_fstMap_pairMap, comp_toElement, fstMap_pairElements,
    sndMap_pairElements]
  exact hL.trans (fstMap_pairElements (functionSystem A B) A _ _).symm
  * have hL :
    (sndMap (functionSystem A B) A).toElement
      ((pairMap (functionSystem A B) A
        (comp (sndMap (functionSystem B C) (functionSystem A B)) (fstMap Q A))
        (sndMap Q A)).toElement
        (pairElements Q A gf x)) =
    x := by
  rw [ <- comp_toElement, comp_sndMap_pairMap, sndMap_pairElements]
  exact hL.trans (sndMap_pairElements (functionSystem A B) A _ _).symm
have hf :
  (comp (applyMap (B := A) (C := B))
    (pairMap (functionSystem A B) A
      (comp (sndMap (functionSystem B C) (functionSystem A B)) (fstMap Q A))
      (sndMap Q A))).toElement
    (pairElements Q A gf x) =
  f.toElement x := by
  rw [comp_toElement, hinner, applyMap_toElement, element_toApproxMap_approxMap_toElement]
have houter :
  (pairMap (functionSystem B C) B
    (comp (fstMap (functionSystem B C) (functionSystem A B)) (fstMap Q A))
    (comp (applyMap (B := A) (C := B))
      (pairMap (functionSystem A B) A
        (comp (sndMap (functionSystem B C) (functionSystem A B)) (fstMap Q A))
        (sndMap Q A)))).toElement
    (pairElements Q A gf x) =
  pairElements (functionSystem B C) B (approxMap_toElement B C g) (f.toElement x) := by
  refine element_eq_of_fst_snd (functionSystem B C) B _ _ ?_ ?_
  * have hL :
    (fstMap (functionSystem B C) B).toElement
      ((pairMap (functionSystem B C) B
        (comp (fstMap (functionSystem B C) (functionSystem A B)) (fstMap Q A))
        (comp (applyMap (B := A) (C := B))
          (pairMap (functionSystem A B) A
            (comp (sndMap (functionSystem B C) (functionSystem A B)) (fstMap Q A))
            (sndMap Q A)))).toElement
          (pairElements Q A gf x)) =
    approxMap_toElement B C g := by
  rw [ <- comp_toElement, comp_fstMap_pairMap, comp_toElement, fstMap_pairElements,
    fstMap_pairElements]
  exact hL.trans (fstMap_pairElements (functionSystem B C) B _ _).symm
  * have hL :
    (sndMap (functionSystem B C) B).toElement
      ((pairMap (functionSystem B C) B
        (comp (fstMap (functionSystem B C) (functionSystem A B)) (fstMap Q A))
        (comp (applyMap (B := A) (C := B))
          (pairMap (functionSystem A B) A
            (comp (sndMap (functionSystem B C) (functionSystem A B)) (fstMap Q A))
            (sndMap Q A)))).toElement
          (pairElements Q A gf x)) =
    f.toElement x := by

```

```

    rw [ <- comp_toElement, comp_sndMap_pairMap, hf]
    exact hL.trans (sndMap_pairElements (functionSystem B C) B _ _).symm
    simp only [compOpUncurry, comp_toElement]
    rw [houter, applyMap_toElement, element_toApproxMap_approxMap_toElement]

theorem compOp_toElement (g : ApproximableMap B C) (f : ApproximableMap A B) :
  (compOp A B C).toElement
    (pairElements (functionSystem B C) (functionSystem A B)
      (approxMap_toElement B C g) (approxMap_toElement A B f)) =
    approxMap_toElement A C (comp g f) := by
  let Q := productSystem (functionSystem B C) (functionSystem A B)
  let gf := pairElements (functionSystem B C) (functionSystem A B)
    (approxMap_toElement B C g) (approxMap_toElement A B f)
  have huniq :
    element_toApproxMap A C ((compOp A B C).toElement gf) = comp g f := by
    refine (ext_iff_toElement _ _).2 fun x => ?_
    have hpair := pairMap_comp_fst_snd_toElement (X := C) (compOp A B C) gf x
    calc
      (element_toApproxMap A C ((compOp A B C).toElement gf)).toElement x
      = (uncurryMap Q A C (compOp A B C)).toElement (pairElements Q A gf x) := by
        simp only [uncurryMap, comp_toElement]
        rw [hpair, applyMap_toElement]
      _ = (compOpUncurry A B C).toElement (pairElements Q A gf x) := by
        rw [compOp_uncurry]
      _ = (comp g f).toElement x := compOpUncurry_toElement A B C g f x
  rw [ <- huniq, approxMap_toElement_element_toApproxMap]

end InfoSys

end Scott1982

```

A.36 Scott1982/Factoid77.lean

373 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Theorem72
import Scott1982.Factoid65
import Mathlib.CategoryTheory.Monoidal.Cartesian.Basic
import Mathlib.CategoryTheory.Monoidal.Closed.Basic
import Mathlib.CategoryTheory.Adjunction.Basic

/-!
# Factoid 7.7 -- cartesian closed structure (stretch goal)

**Scott 1982 (*Categories again*).** Props 6.2 and Theorem 7.2 (with unit `1` from
Factoid 6.5) show that information systems and approximable maps form a cartesian
closed category. This file packages that as Mathlib `Category`,
`CartesianMonoidalCategory`, and `MonoidalClosed`.

**Constructivity note.** The Scott-side data (products, curry/uncurry) is constructive.
Wiring it through Mathlib's `CategoryTheory` (chosen finite products / adjunction
constructors) introduces `Classical.choice`. This is an artifact of Lean's category
library, not of Scott's argument; the constructive core of the formalization is

```

```

unaffected.
-/

universe u

namespace Scott1982

open Scott1982.Constructive
open CategoryTheory
open CategoryTheory.Limits
open CategoryTheory.MonoidalCategory

namespace InfoSys

/#!/ ## Bundled objects -/

/-- Bundled information system (object of the Scott category). -/
structure InfoSysObj : Type (u + 1) where
  Token : Type u
  [decEq : DecidableEq Token]
  sys : InfoSys Token

attribute [instance] InfoSysObj.decEq

namespace InfoSysObj

def prod (A B : InfoSysObj) : InfoSysObj where
  Token := ProdToken A.sys B.sys
  sys := productSystem A.sys B.sys

def exp (A B : InfoSysObj) : InfoSysObj where
  Token := FunToken A.sys B.sys
  sys := functionSystem A.sys B.sys

def unit : InfoSysObj where
  Token := PUnit
  sys := unitSystem

end InfoSysObj

/#!/ ## Diagrammatic composition (`f.acomp g` = apply `f` then `g` = `f ? g`) -/

namespace ApproximableMap

def acomp {a b ? : Type*} [DecidableEq a] [DecidableEq b] [DecidableEq ?]
  {A : InfoSys a} {B : InfoSys b} {C : InfoSys ?}
  (f : ApproximableMap A B) (g : ApproximableMap B C) : ApproximableMap A C :=
  ApproximableMap.comp g f

theorem acomp_toElement {a b ? : Type*} [DecidableEq a] [DecidableEq b] [DecidableEq ?]
  {A : InfoSys a} {B : InfoSys b} {C : InfoSys ?}
  (f : ApproximableMap A B) (g : ApproximableMap B C) (x : A.Element) :
  (accomp f g).toElement x = g.toElement (f.toElement x) :=
  comp_toElement g f x

theorem id_acomp {a b : Type*} [DecidableEq a] [DecidableEq b]
  {A : InfoSys a} {B : InfoSys b} (f : ApproximableMap A B) :
  acomp (idMap A) f = f :=
  (ext_iff_toElement _ _).2 fun x => by rw [accomp_toElement, idMap_toElement]

theorem acomp_id {a b : Type*} [DecidableEq a] [DecidableEq b]
  {A : InfoSys a} {B : InfoSys b} (f : ApproximableMap A B) :
  acomp f (idMap B) = f :=

```

```

(ext_iff_toElement _ _).2 fun x => by rw [acompl_toElement, idMap_toElement]

theorem acompl_assoc {a b ? ? : Type*}
  [DecidableEq a] [DecidableEq b] [DecidableEq ?] [DecidableEq ?]
  {A : InfoSys a} {B : InfoSys b} {C : InfoSys ?} {D : InfoSys ?}
  (f : ApproximableMap A B) (g : ApproximableMap B C) (h : ApproximableMap C D) :
  acompl (acompl f g) h = acompl f (acompl g h) :=
  (ext_iff_toElement _ _).2 fun x => by simp only [acompl_toElement]

def swapMap {a b : Type*} [DecidableEq a] [DecidableEq b]
  (A : InfoSys a) (B : InfoSys b) :
  ApproximableMap (productSystem A B) (productSystem B A) :=
  pairMap B A (sndMap A B) (fstMap A B)

theorem acompl_swap_fst {a b : Type*} [DecidableEq a] [DecidableEq b]
  (A : InfoSys a) (B : InfoSys b) :
  acompl (swapMap A B) (fstMap B A) = sndMap A B := by
  simp only [acompl, swapMap, compl_fstMap_pairMap]

theorem acompl_swap_snd {a b : Type*} [DecidableEq a] [DecidableEq b]
  (A : InfoSys a) (B : InfoSys b) :
  acompl (swapMap A B) (sndMap B A) = fstMap A B := by
  simp only [acompl, swapMap, compl_sndMap_pairMap]

theorem swapMap_acompl_swapMap {a b : Type*} [DecidableEq a] [DecidableEq b]
  (A : InfoSys a) (B : InfoSys b) :
  acompl (swapMap A B) (swapMap B A) = idMap (productSystem A B) := by
  have hfst : acompl (acompl (swapMap A B) (swapMap B A)) (fstMap A B) = fstMap A B := by
    calc
      acompl (acompl (swapMap A B) (swapMap B A)) (fstMap A B)
        = acompl (swapMap A B) (acompl (swapMap B A) (fstMap A B)) := acompl_assoc _ _ _
      _ = acompl (swapMap A B) (sndMap B A) := by rw [acompl_swap_fst]
      _ = fstMap A B := acompl_swap_snd A B
  have hsnd : acompl (acompl (swapMap A B) (swapMap B A)) (sndMap A B) = sndMap A B := by
    calc
      acompl (acompl (swapMap A B) (swapMap B A)) (sndMap A B)
        = acompl (swapMap A B) (acompl (swapMap B A) (sndMap A B)) := acompl_assoc _ _ _
      _ = acompl (swapMap A B) (fstMap B A) := by rw [acompl_swap_snd]
      _ = sndMap A B := acompl_swap_fst A B
  exact (pairMap_unique A B (fstMap A B) (sndMap A B) _ hfst hsnd).trans
    (pairMap_unique A B (fstMap A B) (sndMap A B) (idMap _))
    (id_acompl _ (id_acompl _)).symm

theorem curry_uncurry {a b ? : Type*} [DecidableEq a] [DecidableEq b] [DecidableEq ?]
  (A : InfoSys a) (B : InfoSys b) (C : InfoSys ?)
  (k : ApproximableMap A (functionSystem B C)) :
  curryMap A B C (uncurryMap A B C k) = k :=
  Eq.symm (curryMap_unique A B C (uncurryMap A B C k) k rfl)

theorem swap_fst_toElement {a b : Type*} [DecidableEq a] [DecidableEq b]
  (A : InfoSys a) (B : InfoSys b) (p : (productSystem A B).Element) :
  (fstMap B A).toElement ((swapMap A B).toElement p) = (sndMap A B).toElement p := by
  rw [ <- acompl_toElement, acompl_swap_fst]

theorem swap_snd_toElement {a b : Type*} [DecidableEq a] [DecidableEq b]
  (A : InfoSys a) (B : InfoSys b) (p : (productSystem A B).Element) :
  (sndMap B A).toElement ((swapMap A B).toElement p) = (fstMap A B).toElement p := by
  rw [ <- acompl_toElement, acompl_swap_snd]

end ApproximableMap

open ApproximableMap

```



```

/-! ## Category -/

instance : Category.{u, u + 1} InfoSysObj.{u} where
  Hom A B := ApproximableMap A.sys B.sys
  id A := idMap A.sys
  comp f g := acomp f g
  id_comp := id_acomp
  comp_id := acomp_id
  assoc := acomp_assoc

/-! ## Finite products -/

def terminalLimitCone : LimitCone (Functor.empty.{0} InfoSysObj.{u}) where
  cone := asEmptyCone InfoSysObj.unit
  isLimit := IsTerminal.ofUniqueHom
    (fun (X : InfoSysObj.{u}) => constMap (A := X.sys) unitSystem.botElement)
  fun (_X : InfoSysObj.{u}) m => approxMap_to_unit_eq_const m

def productLimitCone (X Y : InfoSysObj.{u}) : LimitCone (pair X Y) where
  cone := BinaryFan.mk (P := InfoSysObj.prod X Y)
    (fstMap X.sys Y.sys) (sndMap X.sys Y.sys)
  isLimit := by
    refine BinaryFan.IsLimit.mk _
      (fun {T : InfoSysObj.{u}} (f : T --> X) (g : T --> Y) => pairMap X.sys Y.sys f g)
      ?_ ?_ ?_
    * intro T f g
      exact @comp_fstMap_pairMap X.Token Y.Token T.Token _ _ _
        X.sys Y.sys T.sys f g
    * intro T f g
      exact @comp_sndMap_pairMap X.Token Y.Token T.Token _ _ _
        X.sys Y.sys T.sys f g
    * intro T f g m hf hg
      exact @pairMap_unique X.Token Y.Token T.Token _ _ _
        X.sys Y.sys T.sys f g m hf hg

noncomputable instance : CartesianMonoidalCategory.{u, u + 1} InfoSysObj.{u} :=
  CartesianMonoidalCategory.ofChosenFiniteProducts terminalLimitCone productLimitCone

theorem tensorObj_eq_prod (A Y : InfoSysObj.{u}) : A ? Y = InfoSysObj.prod A Y := rfl

/-! ## Braided curry (Mathlib convention) -/

/-- `Hom(A ? Y, Z) -> Hom(Y, A -> Z)` via Scott curry after swap. -/
noncomputable def curryRight (A Y Z : InfoSysObj.{u}) (h : A ? Y --> Z) :
  Y --> InfoSysObj.exp A Z :=
  curryMap Y.sys A.sys Z.sys (acomap (swapMap Y.sys A.sys) h)

/-- Inverse of `curryRight`. -/
noncomputable def uncurryRight (A Y Z : InfoSysObj.{u}) (k : Y --> InfoSysObj.exp A Z) :
  A ? Y --> Z :=
  acomp (swapMap A.sys Y.sys) (uncurryMap Y.sys A.sys Z.sys k)

theorem uncurryRight_curryRight (A Y Z : InfoSysObj.{u}) (h : A ? Y --> Z) :
  uncurryRight A Y Z (curryRight A Y Z h) = h := by
  simp only [uncurryRight, curryRight]
  have huc := uncurry_curryMap Y.sys A.sys Z.sys (acomap (swapMap Y.sys A.sys) h)
  calc
    acomp (swapMap A.sys Y.sys)
      (uncurryMap Y.sys A.sys Z.sys
        (curryMap Y.sys A.sys Z.sys (acomap (swapMap Y.sys A.sys) h)))
      = acomp (swapMap A.sys Y.sys) (acomap (swapMap Y.sys A.sys) h) := by rw [huc]
    _ = acomp (acomap (swapMap A.sys Y.sys) (swapMap Y.sys A.sys)) h :=
      (acomap_assoc _ _ _).symm

```

```

_ = acomp (idMap (productSystem A.sys Y.sys)) h := by rw [swapMap_acomp_swapMap]
_ = h := id_acomp _

theorem curryRight_uncurryRight (A Y Z : InfoSysObj.{u}) (k : Y --> InfoSysObj.exp A Z) :
  curryRight A Y Z (uncurryRight A Y Z k) = k := by
  simp only [curryRight, uncurryRight]
  have hred :
    acomp (swapMap Y.sys A.sys)
      (acom (swapMap A.sys Y.sys) (uncurryMap Y.sys A.sys Z.sys k)) =
      uncurryMap Y.sys A.sys Z.sys k := by
  calc
    acomp (swapMap Y.sys A.sys)
      (acom (swapMap A.sys Y.sys) (uncurryMap Y.sys A.sys Z.sys k))
      = acomp (acom (swapMap Y.sys A.sys) (swapMap A.sys Y.sys))
        (uncurryMap Y.sys A.sys Z.sys k) := (acom_assoc _ _ _).symm
    _ = acomp (idMap (productSystem Y.sys A.sys))
      (uncurryMap Y.sys A.sys Z.sys k) := by rw [swapMap_acomp_swapMap]
    _ = uncurryMap Y.sys A.sys Z.sys k := id_acomp _
  refine Eq.trans (congrArg (curryMap Y.sys A.sys Z.sys) hred)
    (curry_uncurry Y.sys A.sys Z.sys k)

/-- Hom-set equivalence for the exponential (Scott curry after braiding). -/
noncomputable def tensorExpEquiv (A : InfoSysObj.{u}) (Y Z : InfoSysObj.{u}) :
  (A ? Y --> Z) ? (Y --> InfoSysObj.exp A Z) where
  toFun := curryRight A Y Z
  invFun := uncurryRight A Y Z
  left_inv := uncurryRight_curryRight A Y Z
  right_inv := curryRight_uncurryRight A Y Z

/-- `A ? f` is the pairing `` -/
theorem tensorLeft_map_eq (A : InfoSysObj.{u}) {Y Y' : InfoSysObj.{u}} (f : Y --> Y') :
  (tensorLeft A).map f =
    pairMap A.sys Y'.sys (fstMap A.sys Y.sys) (acom (sndMap A.sys Y.sys) f) := by
  refine pairMap_unique A.sys Y'.sys
    (fstMap A.sys Y.sys) (acom (sndMap A.sys Y.sys) f)
    ((tensorLeft A).map f) ?_ ?_
  * change ((tensorLeft A).map f) ? fstMap A.sys Y'.sys = fstMap A.sys Y.sys
  exact CartesianMonoidalCategory.whiskerLeft_fst (X := A) (Y := Y) (Z := Y') f
  * change ((tensorLeft A).map f) ? sndMap A.sys Y'.sys = sndMap A.sys Y.sys ? f
  exact CartesianMonoidalCategory.whiskerLeft_snd (X := A) (Y := Y) (Z := Y') f

/-! ## Left naturality of uncurry / MonoidalClosed -/

private theorem lift_snd_toElement (A : InfoSysObj.{u}) {Y Y' : InfoSysObj.{u}}
  (f : Y' --> Y) (p : (A ? Y').sys.Element) :
  (sndMap A.sys Y.sys).toElement
    ((pairMap A.sys Y.sys (fstMap A.sys Y'.sys)
      (acom (sndMap A.sys Y'.sys) f)).toElement p) =
  f.toElement ((sndMap A.sys Y'.sys).toElement p) := by
  have e :
    acomp (pairMap A.sys Y.sys (fstMap A.sys Y'.sys) (acom (sndMap A.sys Y'.sys) f))
      (sndMap A.sys Y.sys) =
    acomp (sndMap A.sys Y'.sys) f := by
  simp only [acom, comp_sndMap_pairMap]
  simpa [acom_toElement] using congrArg (fun phi => phi.toElement p) e

private theorem lift_fst_toElement (A : InfoSysObj.{u}) {Y Y' : InfoSysObj.{u}}
  (f : Y' --> Y) (p : (A ? Y').sys.Element) :
  (fstMap A.sys Y.sys).toElement
    ((pairMap A.sys Y.sys (fstMap A.sys Y'.sys)
      (acom (sndMap A.sys Y'.sys) f)).toElement p) =
  (fstMap A.sys Y'.sys).toElement p := by
  have e :

```

```

    acomp (pairMap A.sys Y.sys (fstMap A.sys Y'.sys) (acom (sndMap A.sys Y'.sys) f))
      (fstMap A.sys Y.sys) =
    fstMap A.sys Y'.sys := by
    simp only [acom, comp_fstMap_pairMap]
    simpa [acom_toElement] using congrArg (fun phi => phi.toElement p) e

private theorem pair_after_swap_toElement (A Y Z : InfoSysObj.{u})
  (phi : Y --> InfoSysObj.exp A Z) (p : (A ? Y).sys.Element) :
  (pairMap (functionSystem A.sys Z.sys) A.sys
    (ApproximableMap.comp phi (fstMap Y.sys A.sys))
    (sndMap Y.sys A.sys)).toElement ((swapMap A.sys Y.sys).toElement p) =
  pairElements (functionSystem A.sys Z.sys) A.sys
    (phi.toElement ((fstMap Y.sys A.sys).toElement ((swapMap A.sys Y.sys).toElement p)))
    ((sndMap Y.sys A.sys).toElement ((swapMap A.sys Y.sys).toElement p)) := by
  refine element_eq_of_fst_snd _ _ _ ? _ ? _
  * rw [← comp_toElement, comp_fstMap_pairMap, comp_toElement]
    exact (fstMap_pairElements _ _ _).symm
  * rw [← comp_toElement, comp_sndMap_pairMap]
    exact (sndMap_pairElements _ _ _).symm

theorem uncurryRight_toElement (A Y Z : InfoSysObj.{u})
  (phi : Y --> InfoSysObj.exp A Z) (q : (A ? Y).sys.Element) :
  (uncurryRight A Y Z phi).toElement q =
  (element_toApproxMap A.sys Z.sys
    (phi.toElement ((fstMap Y.sys A.sys).toElement
      ((swapMap A.sys Y.sys).toElement q))))).toElement
    ((sndMap Y.sys A.sys).toElement ((swapMap A.sys Y.sys).toElement q)) := by
  simp only [uncurryRight, acomp_toElement, uncurryMap]
  change (ApproximableMap.comp (applyMap (B := A.sys) (C := Z.sys))
    (pairMap _ _ (ApproximableMap.comp phi (fstMap _ _)) (sndMap _ _))).toElement
    ((swapMap A.sys Y.sys).toElement q) = _
  rw [comp_toElement, pair_after_swap_toElement A Y Z phi q, applyMap_toElement]

theorem uncurryRight_comp_left (A : InfoSysObj.{u}) {Y Y' Z : InfoSysObj.{u}}
  (f : Y' --> Y) (g : Y --> InfoSysObj.exp A Z) :
  uncurryRight A Y' Z (f ? g) = (tensorLeft A).map f ? uncurryRight A Y Z g := by
  rw [tensorLeft_map_eq]
  refine (ext_iff_toElement _ _).2 fun p => ?_
  set lift :=
    pairMap A.sys Y.sys (fstMap A.sys Y'.sys) (acom (sndMap A.sys Y'.sys) f)
  have hgoal :
    (uncurryRight A Y' Z (f ? g)).toElement p =
    (uncurryRight A Y Z g).toElement (lift.toElement p) := by
  rw [uncurryRight_toElement A Y' Z (f ? g) p,
    uncurryRight_toElement A Y Z g (lift.toElement p)]
  rw [swap_fst_toElement, swap_snd_toElement, swap_fst_toElement, swap_snd_toElement,
    lift_snd_toElement A f p, lift_fst_toElement A f p]
  simp only [CategoryStruct.comp, acomp_toElement]
  change _ = (acom lift (uncurryRight A Y Z g)).toElement p
  exact hgoal.trans (acom_toElement lift (uncurryRight A Y Z g) p).symm

theorem tensorExpEquiv_natural (A : InfoSysObj.{u}) {Y' Y Z : InfoSysObj.{u}}
  (f : Y' --> Y) (g : A ? Y --> Z) :
  tensorExpEquiv A Y' Z ((tensorLeft A).map f ? g) =
  f ? tensorExpEquiv A Y Z g := by
  apply (tensorExpEquiv A Y' Z).symm.injective
  change uncurryRight A Y' Z
    (tensorExpEquiv A Y' Z ((tensorLeft A).map f ? g)) =
    uncurryRight A Y' Z (f ? tensorExpEquiv A Y Z g)
  simp only [tensorExpEquiv, Equiv.coe_fn_mk]
  have h := uncurryRight_comp_left A f (curryRight A Y Z g)
  rw [uncurryRight_curryRight] at h
  rw [uncurryRight_curryRight, h]

```

```

rfl

/-- Right adjoint to `tensorLeft A` from the curry equivalence. -/
noncomputable def expFunctor (A : InfoSysObj.{u}) : InfoSysObj.{u} ? InfoSysObj.{u} :=
  Adjunction.rightAdjointOfEquiv (F := tensorLeft A) (tensorExpEquiv A)
  fun _Y' _Y _Z f g => by
    simpa using tensorExpEquiv_natural A f g

noncomputable def tensorExpAdjunction (A : InfoSysObj.{u}) :
  tensorLeft A dashv expFunctor A :=
  Adjunction.adjunctionOfEquivRight (F := tensorLeft A) (tensorExpEquiv A)
  fun _Y' _Y _Z f g => by
    simpa using tensorExpEquiv_natural A f g

theorem expFunctor_obj (A Z : InfoSysObj.{u}) :
  (expFunctor A).obj Z = InfoSysObj.exp A Z :=
  rfl

noncomputable instance (A : InfoSysObj.{u}) : Closed A where
  rightAdj := expFunctor A
  adj := tensorExpAdjunction A

noncomputable instance : MonoidalClosed InfoSysObj.{u} where
  closed _ := inferInstance

/-- The Mathlib exponential is Scott's function space on objects. -/
theorem ihom_obj_eq_exp (A B : InfoSysObj.{u}) :
  (ihom A).obj B = InfoSysObj.exp A B :=
  rfl

end InfoSys

end Scott1982

```

A.37 Scott1982/Factoid81.lean

672 lines.

Lean 4 source (lines 1–400)

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Product
import Scott1982.Sum

/-!
# Factoid 8.1 -- tree / S-expression domain `T` ~ A + (T x T)`

**Scott 1982, S8.** Given `A`, the domain `T` of unlabelled binary trees with atoms
from `A` is defined so tokens, consistency, and entailment match the separated sum
of `A` with the product `T x T` (Scott clauses (1)–(12)). The domain equation then
holds by this matching: we package it as an `InfoSys` on an inductive token type
together with the unfolding map into `A + (T x T)`.
-/

namespace Scott1982

open Scott1982.Constructive

```

```

namespace InfoSys

set_option linter.unusedSectionVars false

variable {a : Type*} [DecidableEq a]

/--! ## Tokens (Scott S8 (1)-(3)) -/

/-- Tokens of the tree system: sum tagging of atoms and product left/right copies. -/
inductive TreeToken (a : Type*) where
  | bot : TreeToken a
  | atom : a -> TreeToken a
  | pairL : TreeToken a -> TreeToken a
  | pairR : TreeToken a -> TreeToken a
  deriving DecidableEq

/-- Tree bottom `Delta_T`. -/
def treeBot : TreeToken a := .bot

/--! ## Projections -/

private def atomInsert : TreeToken a -> Finset a -> Finset a
  | .atom x => insert x
  | .bot | .pairL _ | .pairR _ => id

private def pairLInsert : TreeToken a -> Finset (TreeToken a) -> Finset (TreeToken a)
  | .pairL t => insert t
  | .bot | .atom _ | .pairR _ => id

private def pairRInsert : TreeToken a -> Finset (TreeToken a) -> Finset (TreeToken a)
  | .pairR t => insert t
  | .bot | .atom _ | .pairL _ => id

private instance : LeftCommutative (atomInsert : TreeToken a -> Finset a -> Finset a) :=
  <fun p q s => by cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl>

private instance :
  LeftCommutative (pairLInsert : TreeToken a -> Finset (TreeToken a) -> Finset (TreeToken
    a)) :=
  <fun p q s => by cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl>

private instance :
  LeftCommutative (pairRInsert : TreeToken a -> Finset (TreeToken a) -> Finset (TreeToken
    a)) :=
  <fun p q s => by cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl>

def atomFinset (u : Finset (TreeToken a)) : Finset a :=
  Multiset.foldr atomInsert ({} : Finset a) u.1

def pairLFinset (u : Finset (TreeToken a)) : Finset (TreeToken a) :=
  Multiset.foldr pairLInsert ({} : Finset (TreeToken a)) u.1

def pairRFinset (u : Finset (TreeToken a)) : Finset (TreeToken a) :=
  Multiset.foldr pairRInsert ({} : Finset (TreeToken a)) u.1

private theorem mem_foldr_atom (s : Multiset (TreeToken a)) (x : a) :
  x in Multiset.foldr atomInsert ({} : Finset a) s <-> exists p in s, p = .atom x :=
  by
  refine Multiset.induction_on s ?_ ?_
  * constructor
  * intro hx; exact False.elim (Finset.notMem_empty x hx)
  * rintro <_, hp, >; exact False.elim (by cases hp)

```

```

* intro p t ih
cases p with
| atom a =>
  simp only [Multiset.foldr_cons, atomInsert, Finset.mem_insert, ih, Multiset.mem_cons]
  constructor
  * rintro (hx | <q, hq, hq'>)
    * exact <.atom a, Or.inl rfl, congrArg TreeToken.atom hx.symm>
    * exact <q, Or.inr hq, hq'>
  * rintro <q, hq, hq'>
    rcases hq with rfl | hq
    * injection hq' with hx; exact Or.inl hx.symm
    * exact Or.inr <q, hq, hq'>
| bot | pairL _ | pairR _ =>
  simp only [Multiset.foldr_cons, atomInsert, id_eq, ih, Multiset.mem_cons]
  constructor
  * rintro <q, hq, hq'>; exact <q, Or.inr hq, hq'>
  * rintro <q, hq, hq'>
    rcases hq with rfl | hq
    * exact False.elim (nomatch hq')
    * exact <q, hq, hq'>

private theorem mem_foldr_pairL (s : Multiset (TreeToken a)) (t : TreeToken a) :
  t in Multiset.foldr pairLInsert ({} : Finset (TreeToken a)) s <->
    exists p in s, p = .pairL t := by
  refine Multiset.induction_on s ?_ ?_
* constructor
  * intro ht; exact False.elim (Finset.notMem_empty t ht)
  * rintro <_, hp, _>; exact False.elim (by cases hp)
* intro p rest ih
cases p with
| pairL a =>
  simp only [Multiset.foldr_cons, pairLInsert, Finset.mem_insert, ih, Multiset.mem_cons]
  constructor
  * rintro (ht | <q, hq, hq'>)
    * exact <.pairL a, Or.inl rfl, congrArg TreeToken.pairL ht.symm>
    * exact <q, Or.inr hq, hq'>
  * rintro <q, hq, hq'>
    rcases hq with rfl | hq
    * injection hq' with ht; exact Or.inl ht.symm
    * exact Or.inr <q, hq, hq'>
| bot | atom _ | pairR _ =>
  simp only [Multiset.foldr_cons, pairLInsert, id_eq, ih, Multiset.mem_cons]
  constructor
  * rintro <q, hq, hq'>; exact <q, Or.inr hq, hq'>
  * rintro <q, hq, hq'>
    rcases hq with rfl | hq
    * exact False.elim (nomatch hq')
    * exact <q, hq, hq'>

private theorem mem_foldr_pairR (s : Multiset (TreeToken a)) (t : TreeToken a) :
  t in Multiset.foldr pairRInsert ({} : Finset (TreeToken a)) s <->
    exists p in s, p = .pairR t := by
  refine Multiset.induction_on s ?_ ?_
* constructor
  * intro ht; exact False.elim (Finset.notMem_empty t ht)
  * rintro <_, hp, _>; exact False.elim (by cases hp)
* intro p rest ih
cases p with
| pairR a =>
  simp only [Multiset.foldr_cons, pairRInsert, Finset.mem_insert, ih, Multiset.mem_cons]
  constructor
  * rintro (ht | <q, hq, hq'>)
    * exact <.pairR a, Or.inl rfl, congrArg TreeToken.pairR ht.symm>

```

```

    * exact <q, Or.inr hq, hq'>
  * rintro <q, hq, hq'>
    rcases hq with rfl | hq
    * injection hq' with ht; exact Or.inl ht.symm
    * exact Or.inr <q, hq, hq'>
| bot | atom _ | pairL _ =>
  simp only [Multiset.foldr_cons, pairRInsert, id_eq, ih, Multiset.mem_cons]
  constructor
  * rintro <q, hq, hq'>; exact <q, Or.inr hq, hq'>
  * rintro <q, hq, hq'>
    rcases hq with rfl | hq
    * exact False.elim (nomatch hq')
    * exact <q, hq, hq'>

theorem mem_atomFinset {u : Finset (TreeToken a)} {x : a} :
  x in atomFinset u <-> .atom x in u := by
  constructor
  * intro hx
    rcases (mem_foldr_atom u.1 x).1 hx with <p, hp, hp'>
    subst hp'; exact Finset.mem_def.2 hp
  * intro hx
    exact (mem_foldr_atom u.1 x).2 <.atom x, Finset.mem_def.1 hx, rfl>

theorem mem_pairLFinset {u : Finset (TreeToken a)} {t : TreeToken a} :
  t in pairLFinset u <-> .pairL t in u := by
  constructor
  * intro ht
    rcases (mem_foldr_pairL u.1 t).1 ht with <p, hp, hp'>
    subst hp'; exact Finset.mem_def.2 hp
  * intro ht
    exact (mem_foldr_pairL u.1 t).2 <.pairL t, Finset.mem_def.1 ht, rfl>

theorem mem_pairRFinset {u : Finset (TreeToken a)} {t : TreeToken a} :
  t in pairRFinset u <-> .pairR t in u := by
  constructor
  * intro ht
    rcases (mem_foldr_pairR u.1 t).1 ht with <p, hp, hp'>
    subst hp'; exact Finset.mem_def.2 hp
  * intro ht
    exact (mem_foldr_pairR u.1 t).2 <.pairR t, Finset.mem_def.1 ht, rfl>

theorem atomFinset_empty : atomFinset ({} : Finset (TreeToken a)) = {} := rfl
theorem pairLFinset_empty : pairLFinset ({} : Finset (TreeToken a)) = {} := rfl
theorem pairRFinset_empty : pairRFinset ({} : Finset (TreeToken a)) = {} := rfl

theorem atomFinset_mono {u v : Finset (TreeToken a)} (h : v ⊆ u) :
  atomFinset v ⊆ atomFinset u := fun _ hx => mem_atomFinset.2 (h (mem_atomFinset.1 hx))

theorem pairLFinset_mono {u v : Finset (TreeToken a)} (h : v ⊆ u) :
  pairLFinset v ⊆ pairLFinset u := fun _ ht => mem_pairLFinset.2 (h (mem_pairLFinset.1 ht))

theorem pairRFinset_mono {u v : Finset (TreeToken a)} (h : v ⊆ u) :
  pairRFinset v ⊆ pairRFinset u := fun _ ht => mem_pairRFinset.2 (h (mem_pairRFinset.1 ht))

theorem atomFinset_insert_atom (u : Finset (TreeToken a)) (x : a) :
  atomFinset (insert (.atom x) u) = insert x (atomFinset u) := by
  ext y; simp only [Finset.mem_insert, mem_atomFinset]
  constructor
  * rintro (h | h)
    * exact Or.inl (TreeToken.atom.inj h)

```

```

    * exact Or.inr h
* rintro (rfl | h)
    * exact Or.inl rfl
    * exact Or.inr h

theorem atomFinset_insert_bot (u : Finset (TreeToken a)) :
  atomFinset (insert (.bot : TreeToken a) u) = atomFinset u := by
  ext x; simp only [mem_atomFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

theorem atomFinset_insert_pairL (u : Finset (TreeToken a)) (t : TreeToken a) :
  atomFinset (insert (.pairL t) u) = atomFinset u := by
  ext x; simp only [mem_atomFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

theorem atomFinset_insert_pairR (u : Finset (TreeToken a)) (t : TreeToken a) :
  atomFinset (insert (.pairR t) u) = atomFinset u := by
  ext x; simp only [mem_atomFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

theorem pairLFinset_insert_pairL (u : Finset (TreeToken a)) (t : TreeToken a) :
  pairLFinset (insert (.pairL t) u) = insert t (pairLFinset u) := by
  ext s; simp only [Finset.mem_insert, mem_pairLFinset]
  constructor
  * rintro (h | h)
    * exact Or.inl (TreeToken.pairL.inj h)
    * exact Or.inr h
  * rintro (rfl | h)
    * exact Or.inl rfl
    * exact Or.inr h

theorem pairLFinset_insert_bot (u : Finset (TreeToken a)) :
  pairLFinset (insert (.bot : TreeToken a) u) = pairLFinset u := by
  ext t; simp only [mem_pairLFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

theorem pairLFinset_insert_atom (u : Finset (TreeToken a)) (x : a) :
  pairLFinset (insert (.atom x) u) = pairLFinset u := by
  ext t; simp only [mem_pairLFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

theorem pairLFinset_insert_pairR (u : Finset (TreeToken a)) (t : TreeToken a) :
  pairLFinset (insert (.pairR t) u) = pairLFinset u := by
  ext s; simp only [mem_pairLFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

theorem pairRFinset_insert_pairR (u : Finset (TreeToken a)) (t : TreeToken a) :
  pairRFinset (insert (.pairR t) u) = insert t (pairRFinset u) := by
  ext s; simp only [Finset.mem_insert, mem_pairRFinset]
  constructor
  * rintro (h | h)

```



```

    * exact Or.inl (TreeToken.pairR.inj h)
    * exact Or.inr h
  * rintro (rfl | h)
    * exact Or.inl rfl
    * exact Or.inr h

theorem pairRFinset_insert_bot (u : Finset (TreeToken a)) :
  pairRFinset (insert (.bot : TreeToken a) u) = pairRFinset u := by
  ext t; simp only [mem_pairRFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

theorem pairRFinset_insert_atom (u : Finset (TreeToken a)) (x : a) :
  pairRFinset (insert (.atom x) u) = pairRFinset u := by
  ext t; simp only [mem_pairRFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

theorem pairRFinset_insert_pairL (u : Finset (TreeToken a)) (t : TreeToken a) :
  pairRFinset (insert (.pairL t) u) = pairRFinset u := by
  ext s; simp only [mem_pairRFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

theorem atomFinset_singleton_atom (x : a) :
  atomFinset ({.atom x} : Finset (TreeToken a)) = {x} := by
  ext y; simp only [mem_atomFinset, Finset.mem_singleton]
  exact <fun h => TreeToken.atom.inj h, fun h => congrArg TreeToken.atom h>

theorem pairLFinset_singleton_pairL (t : TreeToken a) :
  pairLFinset ({.pairL t} : Finset (TreeToken a)) = {t} := by
  ext s; simp only [mem_pairLFinset, Finset.mem_singleton]
  exact <fun h => TreeToken.pairL.inj h, fun h => congrArg TreeToken.pairL h>

theorem pairRFinset_singleton_pairR (t : TreeToken a) :
  pairRFinset ({.pairR t} : Finset (TreeToken a)) = {t} := by
  ext s; simp only [mem_pairRFinset, Finset.mem_singleton]
  exact <fun h => TreeToken.pairR.inj h, fun h => congrArg TreeToken.pairR h>

theorem atomFinset_singleton_bot :
  atomFinset ({.bot} : Finset (TreeToken a)) = {} := by
  ext x
  constructor
  * intro hx
    exact False.elim (nomatch mem_atomFinset.1 hx)
  * intro hx
    exact False.elim (Finset.notMem_empty x hx)

theorem pairLFinset_singleton_bot :
  pairLFinset ({.bot} : Finset (TreeToken a)) = {} := by
  ext t
  constructor
  * intro ht
    exact False.elim (nomatch mem_pairLFinset.1 ht)
  * intro ht
    exact False.elim (Finset.notMem_empty t ht)

theorem pairRFinset_singleton_bot :
  pairRFinset ({.bot} : Finset (TreeToken a)) = {} := by
  ext t

```

```

constructor
* intro ht
  exact False.elim (nomatch mem_pairRFinset.1 ht)
* intro ht
  exact False.elim (Finset.notMem_empty t ht)

theorem pairLFinset_singleton_atom (x : a) :
  pairLFinset ({.atom x} : Finset (TreeToken a)) = {} := by
  ext t
  constructor
  * intro ht
    exact False.elim (nomatch mem_pairLFinset.1 ht)
  * intro ht
    exact False.elim (Finset.notMem_empty t ht)

theorem pairRFinset_singleton_atom (x : a) :
  pairRFinset ({.atom x} : Finset (TreeToken a)) = {} := by
  ext t
  constructor
  * intro ht
    exact False.elim (nomatch mem_pairRFinset.1 ht)
  * intro ht
    exact False.elim (Finset.notMem_empty t ht)

theorem atomFinset_singleton_pairL (t : TreeToken a) :
  atomFinset ({.pairL t} : Finset (TreeToken a)) = {} := by
  ext x
  constructor
  * intro hx
    exact False.elim (nomatch mem_atomFinset.1 hx)
  * intro hx
    exact False.elim (Finset.notMem_empty x hx)

theorem atomFinset_singleton_pairR (t : TreeToken a) :
  atomFinset ({.pairR t} : Finset (TreeToken a)) = {} := by
  ext x
  constructor
  * intro hx
    exact False.elim (nomatch mem_atomFinset.1 hx)
  * intro hx
    exact False.elim (Finset.notMem_empty x hx)

theorem pairRFinset_singleton_pairL (t : TreeToken a) :
  pairRFinset ({.pairL t} : Finset (TreeToken a)) = {} := by
  ext s
  constructor
  * intro hs
    exact False.elim (nomatch mem_pairRFinset.1 hs)
  * intro hs
    exact False.elim (Finset.notMem_empty s hs)

theorem pairLFinset_singleton_pairR (t : TreeToken a) :
  pairLFinset ({.pairR t} : Finset (TreeToken a)) = {} := by
  ext s
  constructor
  * intro hs
    exact False.elim (nomatch mem_pairLFinset.1 hs)
  * intro hs
    exact False.elim (Finset.notMem_empty s hs)

/-! ## Consistency and entailment -/

variable (A : InfoSys a)

```

```

/-- Consistency for trees (Scott S8 (4)-(7)). -/
inductive TreeCon : Finset (TreeToken a) -> Prop where
  | left (u : Finset (TreeToken a))
    (hA : atomFinset u in A.Con)

Lean 4 source (lines 401–672)

    (hL : pairLFinset u = {}) (hR : pairRFinset u = {}) : TreeCon u
  | right (u : Finset (TreeToken a))
    (hA : atomFinset u = {})
    (hL : TreeCon (pairLFinset u)) (hR : TreeCon (pairRFinset u)) : TreeCon u

theorem TreeCon_empty : TreeCon A ({ } : Finset (TreeToken a)) :=
  TreeCon.left _ (by rw [atomFinset_empty]; exact A.con_empty)
  pairLFinset_empty pairRFinset_empty

/-- Payload of tree entailment (Scott S8 (8)-(12)), recursive on the token. -/
def TreeEntPayload (u : Finset (TreeToken a)) : TreeToken a -> Prop
  | .bot => True
  | .atom x => atomFinset u != {} /\ A.Ent (atomFinset u) x
  | .pairL t =>
    atomFinset u = {} /\ pairLFinset u != {} /\
    (TreeCon A (pairLFinset u) /\ TreeEntPayload (pairLFinset u) t) /\
    TreeCon A (pairRFinset u)
  | .pairR t =>
    atomFinset u = {} /\ pairRFinset u != {} /\
    (TreeCon A (pairRFinset u) /\ TreeEntPayload (pairRFinset u) t) /\
    TreeCon A (pairLFinset u)

/-- Entailment for trees. -/
def TreeEnt (u : Finset (TreeToken a)) (p : TreeToken a) : Prop :=
  TreeCon A u /\ TreeEntPayload A u p

theorem TreeCon_pairs_empty_of_left {u : Finset (TreeToken a)}
  (hu : TreeCon A u) (hA : atomFinset u != {}) :
  pairLFinset u = {} /\ pairRFinset u = {} := by
  cases hu with
  | left _ _ hL hR => exact <hL, hR>
  | right _ hAt _ _ => exact False.elim (hA hAt)

theorem TreeCon_atoms_empty_of_right {u : Finset (TreeToken a)}
  (hu : TreeCon A u) (hP : pairLFinset u != {} /\ pairRFinset u != {}) :
  atomFinset u = {} := by
  cases hu with
  | left _ _ hL hR =>
    rcases hP with h | h
    * exact False.elim (h hL)
    * exact False.elim (h hR)
  | right _ hA _ _ => exact hA

theorem TreeCon_atom_con_of_atoms {u : Finset (TreeToken a)}
  (hu : TreeCon A u) (hne : atomFinset u != {}) : atomFinset u in A.Con := by
  cases hu with
  | left _ hA _ _ => exact hA
  | right _ hA _ _ => exact False.elim (hne hA)

theorem TreeCon_pairL_of_right {u : Finset (TreeToken a)}
  (hu : TreeCon A u) (_hA : atomFinset u = {}) : TreeCon A (pairLFinset u) := by
  cases hu with
  | left _ _ hL _ => rw [hL]; exact TreeCon_empty A
  | right _ _ hL _ => exact hL

theorem TreeCon_pairR_of_right {u : Finset (TreeToken a)}

```

```

    (hu : TreeCon A u) (_hA : atomFinset u = {}) : TreeCon A (pairRFinset u) := by
cases hu with
| left _ _ _ hR => rw [hR]; exact TreeCon_empty A
| right _ _ _ hR => exact hR

theorem TreeCon_singleton (p : TreeToken a) : TreeCon A {p} := by
induction p with
| bot =>
  exact TreeCon.left _
    (by rw [atomFinset_singleton_bot]; exact A.con_empty)
    pairLFinset_singleton_bot pairRFinset_singleton_bot
| atom x =>
  exact TreeCon.left _
    (by rw [atomFinset_singleton_atom]; exact A.con_sing x)
    (pairLFinset_singleton_atom x) (pairRFinset_singleton_atom x)
| pairL t ih =>
  exact TreeCon.right _
    (atomFinset_singleton_pairL t)
    (by rw [pairLFinset_singleton_pairL]; exact ih)
    (by rw [pairRFinset_singleton_pairL]; exact TreeCon_empty A)
| pairR t ih =>
  exact TreeCon.right _
    (atomFinset_singleton_pairR t)
    (by rw [pairLFinset_singleton_pairR]; exact TreeCon_empty A)
    (by rw [pairRFinset_singleton_pairR]; exact ih)

theorem TreeEntPayload_of_mem {u : Finset (TreeToken a)} {p : TreeToken a}
  (hu : TreeCon A u) (hp : p in u) : TreeEntPayload A u p := by
induction p generalizing u with
| bot => exact trivial
| atom x =>
  have hx : x in atomFinset u := mem_atomFinset.2 hp
  exact <Finset.ne_empty_of_mem hx,
    A.ent_refl (TreeCon_atom_con_of_atoms A hu (Finset.ne_empty_of_mem hx)) hx>
| pairL t ih =>
  have ht : t in pairLFinset u := mem_pairLFinset.2 hp
  have hne := Finset.ne_empty_of_mem ht
  have hAt := TreeCon_atoms_empty_of_right A hu (Or.inl hne)
  exact <hAt, hne, <TreeCon_pairL_of_right A hu hAt,
    ih (TreeCon_pairL_of_right A hu hAt) ht>, TreeCon_pairR_of_right A hu hAt>
| pairR t ih =>
  have ht : t in pairRFinset u := mem_pairRFinset.2 hp
  have hne := Finset.ne_empty_of_mem ht
  have hAt := TreeCon_atoms_empty_of_right A hu (Or.inr hne)
  exact <hAt, hne, <TreeCon_pairR_of_right A hu hAt,
    ih (TreeCon_pairR_of_right A hu hAt) ht>, TreeCon_pairL_of_right A hu hAt>

theorem TreeEnt_of_mem {u : Finset (TreeToken a)} {p : TreeToken a}
  (hu : TreeCon A u) (hp : p in u) : TreeEnt A u p :=
<hu, TreeEntPayload_of_mem A hu hp>

theorem TreeCon_insert_of_ent {u : Finset (TreeToken a)} {p : TreeToken a}
  (h : TreeEnt A u p) : TreeCon A (insert p u) := by
induction p generalizing u with
| bot =>
  rcases h with <hu, _>
  cases hu with
  | left _ hA hL hR =>
    exact TreeCon.left _
      (by rw [atomFinset_insert_bot]; exact hA)
      (by rw [pairLFinset_insert_bot]; exact hL)
      (by rw [pairRFinset_insert_bot]; exact hR)
  | right _ hA hL hR =>

```

```

    exact TreeCon.right _
      (by rw [atomFinset_insert_bot]; exact hA)
      (by rw [pairLFinset_insert_bot]; exact hL)
      (by rw [pairRFinset_insert_bot]; exact hR)
| atom x =>
  rcases h with <hu, hne, hA>
  have hLR := TreeCon_pairs_empty_of_left A hu hne
  exact TreeCon.left _
    (by rw [atomFinset_insert_atom]; exact A.ent_con hA)
    (by rw [pairLFinset_insert_atom]; exact hLR.1)
    (by rw [pairRFinset_insert_atom]; exact hLR.2)
| pairL t ih =>
  rcases h with <_, hAt, _, hEntL, hConR>
  exact TreeCon.right _
    (by rw [atomFinset_insert_pairL]; exact hAt)
    (by rw [pairLFinset_insert_pairL]; exact ih hEntL)
    (by rw [pairRFinset_insert_pairL]; exact hConR)
| pairR t ih =>
  rcases h with <_, hAt, _, hEntR, hConL>
  exact TreeCon.right _
    (by rw [atomFinset_insert_pairR]; exact hAt)
    (by rw [pairLFinset_insert_pairR]; exact hConL)
    (by rw [pairRFinset_insert_pairR]; exact ih hEntR)

theorem TreeEntPayload_trans {u v : Finset (TreeToken a)} {c : TreeToken a}
  (hv : TreeCon A v) (hu : TreeCon A u)
  (hEnts : forall y in u, TreeEnt A v y) (hPay : TreeEntPayload A u c) :
  TreeEntPayload A v c := by
  induction c generalizing u v with
  | bot => exact trivial
  | atom x =>
    rcases hPay with <hne, hA>
    have hlft : forall y in atomFinset u, A.Ent (atomFinset v) y := by
      intro y hy
      exact (hEnts _ (mem_atomFinset.1 hy)).2.2
    have hne' : atomFinset v != {} := by
      obtain <y, hy> := Finset.nonempty_of_ne_empty hne
      exact (hEnts _ (mem_atomFinset.1 hy)).2.1
    exact <hne', A.ent_trans (TreeCon_atom_con_of_atoms A hv hne')
      (TreeCon_atom_con_of_atoms A hu hne) hlft hA>
  | pairL t ih =>
    rcases hPay with <hAt, hne, <hConL, hPayL>, _>
    have hAtv : atomFinset v = {} := by
      obtain <s, hs> := Finset.nonempty_of_ne_empty hne
      exact (hEnts _ (mem_pairLFinset.1 hs)).2.1
    have hne' : pairLFinset v != {} := by
      obtain <s, hs> := Finset.nonempty_of_ne_empty hne
      exact (hEnts _ (mem_pairLFinset.1 hs)).2.2.1
    have hEntsL : forall y in pairLFinset u, TreeEnt A (pairLFinset v) y := by
      intro y hy
      have hEy := hEnts _ (mem_pairLFinset.1 hy)
      exact <hEy.2.2.2.1.1, hEy.2.2.2.1.2>
    have hConRv : TreeCon A (pairRFinset v) := by
      obtain <s, hs> := Finset.nonempty_of_ne_empty hne
      exact (hEnts _ (mem_pairLFinset.1 hs)).2.2.2.2
    refine <hAtv, hne', <TreeCon_pairL_of_right A hv hAtv, ?_>, hConRv>
    exact ih (TreeCon_pairL_of_right A hv hAtv) hConL hEntsL hPayL
  | pairR t ih =>
    rcases hPay with <hAt, hne, <hConR, hPayR>, _>
    have hAtv : atomFinset v = {} := by
      obtain <s, hs> := Finset.nonempty_of_ne_empty hne
      exact (hEnts _ (mem_pairRFinset.1 hs)).2.1
    have hne' : pairRFinset v != {} := by

```

```

    obtain <s, hs> := Finset.nonempty_of_ne_empty hne
    exact (hEnts _ (mem_pairRFinset.1 hs)).2.2.1
  have hEntsR : forall y in pairRFinset u, TreeEnt A (pairRFinset v) y := by
    intro y hy
    have hEy := hEnts _ (mem_pairRFinset.1 hy)
    exact <hEy.2.2.2.1.1, hEy.2.2.2.1.2>
  have hConLv : TreeCon A (pairLFinset v) := by
    obtain <s, hs> := Finset.nonempty_of_ne_empty hne
    exact (hEnts _ (mem_pairRFinset.1 hs)).2.2.2.2
  refine <hAtv, hne', <TreeCon_pairR_of_right A hv hAtv, ?_>, hConLv>
  exact ih (TreeCon_pairR_of_right A hv hAtv) hConR hEntsR hPayR

theorem TreeEnt_trans {u v : Finset (TreeToken a)} {c : TreeToken a}
  (hv : TreeCon A v) (hu : TreeCon A u)
  (hEnts : forall y in u, TreeEnt A v y) (hEntc : TreeEnt A u c) :
  TreeEnt A v c :=
  <hv, TreeEntPayload_trans A hv hu hEnts hEntc.2>

theorem TreeCon_subset {u : Finset (TreeToken a)} (hu : TreeCon A u) :
  forall {v}, v subteq u -> TreeCon A v := by
  induction hu with
  | left u hA hL hR =>
    intro v hv
    refine TreeCon.left v (A.con_subset hA (atomFinset_mono hv)) ?_ ?_
    * exact Finset.Subset.antisymm
      (fun t ht => by
        have : t in pairLFinset u := pairLFinset_mono hv ht
        rw [hL] at this; exact False.elim (Finset.notMem_empty t this))
      (Finset.empty_subset _)
    * exact Finset.Subset.antisymm
      (fun t ht => by
        have : t in pairRFinset u := pairRFinset_mono hv ht
        rw [hR] at this; exact False.elim (Finset.notMem_empty t this))
      (Finset.empty_subset _)
  | right u hA hL hR ihL ihR =>
    intro v hv
    refine TreeCon.right v
      (Finset.Subset.antisymm
        (fun x hx => by
          have : x in atomFinset u := atomFinset_mono hv hx
          rw [hA] at this; exact False.elim (Finset.notMem_empty x this))
        (Finset.empty_subset _))
      (ihL (pairLFinset_mono hv)) (ihR (pairRFinset_mono hv))

/-- **Factoid 8.1.** Tree information system solving `T ~ A + (T x T)` by construction. -/
def treeSystem : InfoSys (TreeToken a) where
  bot := treeBot
  Con := {u | TreeCon A u}
  Ent := TreeEnt A
  con_subset := fun hu hv => TreeCon_subset A hu hv
  con_sing := TreeCon_singleton A
  ent_con := fun h => TreeCon_insert_of_ent A h
  ent_bot := fun hu => <hu, trivial>
  ent_refl := fun hu hp => TreeEnt_of_mem A hu hp
  ent_trans := fun hv hu hEnts hEntc => TreeEnt_trans A hv hu hEnts hEntc

/-! ## Domain equation

Scott secures `T ~ A + (T x T)` by choosing tokens so that `T` is literally the
sum-of-product system. Here `TreeToken` is the inductive presentation of that
tagging (clauses (1)-(3)), and `TreeCon`/`TreeEnt` are the sum x product clauses
((4)-(12)). Unfolding compares with the official `sumSystem` / `productSystem`
carriers; `pairL bot` and `pairR bot` both land on the unique product bottom,

```

```

matching Scott?s single `(Delta,(Delta,Delta))`.
-/

/-- Unfold a tree token into the sum-of-product carrier. -/
def treeUnfold (t : TreeToken a) :
  SumToken a (ProdToken (treeSystem A) (treeSystem A)) :=
  match t with
  | .bot => .bot
  | .atom x => .left x
  | .pairL t => .right <(t, treeBot), Or.inr rfl>
  | .pairR t => .right <(treeBot, t), Or.inl rfl>

theorem treeUnfold_bot : treeUnfold A treeBot = SumToken.bot := rfl

theorem treeUnfold_atom (x : a) : treeUnfold A (.atom x) = SumToken.left x := rfl

/-- The right-hand side of the domain equation is the official sum of products. -/
def treeRhs : InfoSys (SumToken a (ProdToken (treeSystem A) (treeSystem A))) :=
  sumSystem A (productSystem (treeSystem A) (treeSystem A))

theorem treeRhs_eq_sum_product :
  treeRhs A = sumSystem A (productSystem (treeSystem A) (treeSystem A)) :=
  rfl

end InfoSys

end Scott1982

```

A.38 Scott1982/Factoid82.lean

2406 lines.

Lean 4 source (lines 1–400)

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.FunctionSpace
import Scott1982.Sum
import Mathlib.Data.Finset.Card

/-!
# Factoid 8.2 -- fun -calculus model `D` ~ A + (D -> D)`

**Scott 1982, S8.** `D` and `Con_D` are mutually recursive. Lean rejects putting the
FunCon-style clause (7) inside an inductive (negative occurrence), so the primary
path uses well-founded **`LamConDepth`** (`termination_by finsetMaxDepth`) and
inductive **`LamEntDepth`** (Scott (8)–(11)). Staged `LamConN` / early `LamToken`
remain scaffolding only.

**Status:** Def 2.1 packaged as `lambdaSystem` on `LamDToken` (depth WF); domain
equation via `lamUnfold` / `lamRhs` into `sumSystem A (functionSystem D D)`.
-/

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys

```

```

set_option linter.unusedSectionVars false

variable {a : Type*} [DecidableEq a]

/-! ## Raw tokens + decidable equality -/

inductive RawLamToken (a : Type*) where
| bot : RawLamToken a
| atom : a -> RawLamToken a
| funTok : List (RawLamToken a) -> List (RawLamToken a) -> RawLamToken a

def listToFinset {b : Type*} [DecidableEq b] : List b -> Finset b
| [] => {}
| x :: xs => insert x (listToFinset xs)

theorem mem_listToFinset {b : Type*} [DecidableEq b] {x : b} {xs : List b} :
  x in listToFinset xs <-> x in xs := by
  induction xs with
  | nil =>
    constructor
    * intro h; exact False.elim (Finset.notMem_empty x h)
    * intro h; cases h
  | cons a as ih =>
    simp only [listToFinset, Finset.mem_insert, List.mem_cons, ih]

mutual
  def rawLamBEq : RawLamToken a -> RawLamToken a -> Bool
  | .bot, .bot => true
  | .atom x, .atom y => decide (x = y)
  | .funTok u v, .funTok u' v' => rawLamListBEq u u' && rawLamListBEq v v'
  | _, _ => false
  def rawLamListBEq : List (RawLamToken a) -> List (RawLamToken a) -> Bool
  | [], [] => true
  | x :: xs, y :: ys => rawLamBEq x y && rawLamListBEq xs ys
  | _, _ => false
end

mutual
  theorem rawLamBEq_eq : forall a b : RawLamToken a, rawLamBEq a b = true <-> a = b
  | .bot, .bot => by simp [rawLamBEq]
  | .atom x, .atom y => by
    simp only [rawLamBEq, decide_eq_true_eq]
    exact <congrArg RawLamToken.atom, RawLamToken.atom.inj>
  | .funTok u v, .funTok u' v' => by
    simp only [rawLamBEq, Bool.and_eq_true]
    constructor
    * intro <hu, hv>
    rw [(rawLamListBEq_eq u u').1 hu, (rawLamListBEq_eq v v').1 hv]
    * intro h
    injection h with hu hv
    subst hu; subst hv
    exact <(rawLamListBEq_eq u u').2 rfl, (rawLamListBEq_eq v v').2 rfl>
  | .bot, .atom _ | .bot, .funTok _ _ | .atom _, .bot | .atom _, .funTok _ _
  | .funTok _ _, .bot | .funTok _ _, .atom _ => by
    constructor
    * intro h; simp [rawLamBEq] at h
    * intro h; cases h
  theorem rawLamListBEq_eq : forall u v : List (RawLamToken a), rawLamListBEq u v = true <->
    u = v
  | [], [] => by simp [rawLamListBEq]
  | x :: xs, y :: ys => by
    simp only [rawLamListBEq, Bool.and_eq_true]

```



```

    constructor
    * intro <hx, hxs>
      rw [(rawLamBEq_eq x y).1 hx, (rawLamListBEq_eq xs ys).1 hxs]
    * intro h
      injection h with hx hxs
      subst hx; subst hxs
      exact <(rawLamBEq_eq x x).2 rfl, (rawLamListBEq_eq xs xs).2 rfl>
| [], _ :: _ | _ :: _, [] => by
  constructor
  * intro h; simp [rawLamListBEq] at h
  * intro h; cases h
end

instance : DecidableEq (RawLamToken a) := fun a b =>
  if h : rawLamBEq a b = true then isTrue ((rawLamBEq_eq a b).1 h)
  else isFalse fun hab => h ((rawLamBEq_eq a b).2 hab)

/-! ## Projections -/

private def lamAtomInsert : RawLamToken a -> Finset a -> Finset a
| .atom x => insert x
| .bot | .funTok _ _ => id

private def lamFunInsert :
  RawLamToken a -> Finset (List (RawLamToken a) x List (RawLamToken a)) ->
  Finset (List (RawLamToken a) x List (RawLamToken a))
| .funTok u v => insert (u, v)
| .bot | .atom _ => id

private instance : LeftCommutative (lamAtomInsert : RawLamToken a -> Finset a -> Finset a)
:=
  <fun p q s => by cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl>

private instance : LeftCommutative
  (lamFunInsert : RawLamToken a ->
    Finset (List (RawLamToken a) x List (RawLamToken a)) ->
    Finset (List (RawLamToken a) x List (RawLamToken a))) :=
  <fun p q s => by cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl>

def lamAtomFinset (u : Finset (RawLamToken a)) : Finset a :=
  Multiset.foldr lamAtomInsert {} u.1

def lamFunFinset (u : Finset (RawLamToken a)) :
  Finset (List (RawLamToken a) x List (RawLamToken a)) :=
  Multiset.foldr lamFunInsert {} u.1

theorem mem_lamAtomFinset {u : Finset (RawLamToken a)} {x : a} :
  x in lamAtomFinset u <-> .atom x in u := by
  have aux (s : Multiset (RawLamToken a)) :
    x in Multiset.foldr lamAtomInsert ({ : Finset a) s <-> exists p in s, p = .atom x
  := by
    refine Multiset.induction_on s ?_ ?_
  * constructor
  * intro hx; exact False.elim (Finset.notMem_empty x hx)
  * rintro <_, hp, _>; exact False.elim (by cases hp)
  * intro p t ih
    cases p with
    | atom a =>
      simp only [Multiset.foldr_cons, lamAtomInsert, Finset.mem_insert, ih, Multiset.mem_cons]
  ]
  constructor
  * rintro (hx | <q, hq, hq'>)
    * exact <.atom a, Or.inl rfl, congrArg RawLamToken.atom hx.symm>

```

```

    * exact <q, Or.inr hq, hq'>
  * rintro <q, hq, hq'>
    rcases hq with rfl | hq
    * injection hq' with hx; exact Or.inl hx.symm
    * exact Or.inr <q, hq, hq'>
| bot | funTok _ _ =>
  simp only [Multiset.foldr_cons, lamAtomInsert, id_eq, ih, Multiset.mem_cons]
  constructor
  * rintro <q, hq, hq'>; exact <q, Or.inr hq, hq'>
  * rintro <q, hq, hq'>
    rcases hq with rfl | hq
    * exact False.elim (nomatch hq')
    * exact <q, hq, hq'>
constructor
* intro hx
  rcases (aux u.1).1 hx with <p, hp, hp'>
  subst hp'; exact Finset.mem_def.2 hp
* intro hx
  exact (aux u.1).2 <.atom x, Finset.mem_def.1 hx, rfl>

theorem mem_lamFunFinset {u : Finset (RawLamToken a)}
  {p : List (RawLamToken a) x List (RawLamToken a)} :
  p in lamFunFinset u <-> .funTok p.1 p.2 in u := by
  have aux (s : Multiset (RawLamToken a)) :
    p in Multiset.foldr lamFunInsert {} s <-> exists t in s, t = .funTok p.1 p.2 := by
    refine Multiset.induction_on s ?_ ?_
  * constructor
    * intro hp; exact False.elim (Finset.notMem_empty p hp)
    * rintro <_, ht, _>; exact False.elim (by cases ht)
  * intro t rest ih
    cases t with
    | funTok u v =>
      simp only [Multiset.foldr_cons, lamFunInsert, Finset.mem_insert, ih, Multiset.mem_cons]
      constructor
      * rintro (hp | <q, hq, hq'>)
        * exact <.funTok u v, Or.inl rfl, by cases hp; rfl>
        * exact <q, Or.inr hq, hq'>
      * rintro <q, hq, hq'>
        rcases hq with rfl | hq
        * injection hq' with hu hv
          exact Or.inl (Prod.ext hu.symm hv.symm)
        * exact Or.inr <q, hq, hq'>
    | bot | atom _ =>
      simp only [Multiset.foldr_cons, lamFunInsert, id_eq, ih, Multiset.mem_cons]
      constructor
      * rintro <q, hq, hq'>; exact <q, Or.inr hq, hq'>
      * rintro <q, hq, hq'>
        rcases hq with rfl | hq
        * exact False.elim (nomatch hq')
        * exact <q, hq, hq'>
  constructor
  * intro hp
    rcases (aux u.1).1 hp with <t, ht, ht'>
    subst ht'; exact Finset.mem_def.2 ht
  * intro hp
    exact (aux u.1).2 <.funTok p.1 p.2, Finset.mem_def.1 hp, rfl>

theorem lamAtomFinset_empty : lamAtomFinset ({ } : Finset (RawLamToken a)) = { } := rfl
theorem lamFunFinset_empty : lamFunFinset ({ } : Finset (RawLamToken a)) = { } := rfl

theorem lamAtomFinset_mono {u v : Finset (RawLamToken a)} (h : v subseq u) :
  lamAtomFinset v subseq lamAtomFinset u :=
  fun _ hx => mem_lamAtomFinset.2 (h (mem_lamAtomFinset.1 hx))

```

```

theorem lamFunFinset_mono {u v : Finset (RawLamToken a)} (h : v ⊆ u) :
  lamFunFinset v ⊆ lamFunFinset u :=
  fun _ hp => mem_lamFunFinset.2 (h (mem_lamFunFinset.1 hp))

theorem lamAtomFinset_insert_bot (u : Finset (RawLamToken a)) :
  lamAtomFinset (insert (.bot : RawLamToken a) u) = lamAtomFinset u := by
  ext x; simp only [mem_lamAtomFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

theorem lamAtomFinset_insert_atom (u : Finset (RawLamToken a)) (x : a) :
  lamAtomFinset (insert (.atom x) u) = insert x (lamAtomFinset u) := by
  ext y; simp only [Finset.mem_insert, mem_lamAtomFinset]
  constructor
  * rintro (h | h)
    * exact Or.inl (RawLamToken.atom.inj h)
    * exact Or.inr h
  * rintro (rfl | h)
    * exact Or.inl rfl
    * exact Or.inr h

theorem lamAtomFinset_insert_funTok (u : Finset (RawLamToken a))
  (xs ys : List (RawLamToken a)) :
  lamAtomFinset (insert (.funTok xs ys) u) = lamAtomFinset u := by
  ext x; simp only [mem_lamAtomFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

theorem lamFunFinset_insert_bot (u : Finset (RawLamToken a)) :
  lamFunFinset (insert (.bot : RawLamToken a) u) = lamFunFinset u := by
  ext p; simp only [mem_lamFunFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

theorem lamFunFinset_insert_atom (u : Finset (RawLamToken a)) (x : a) :
  lamFunFinset (insert (.atom x) u) = lamFunFinset u := by
  ext p; simp only [mem_lamFunFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

theorem lamFunFinset_insert_funTok (u : Finset (RawLamToken a))
  (xs ys : List (RawLamToken a)) :
  lamFunFinset (insert (.funTok xs ys) u) = insert (xs, ys) (lamFunFinset u) := by
  ext p; simp only [Finset.mem_insert, mem_lamFunFinset]
  constructor
  * rintro (h | h)
    * exact Or.inl (Prod.ext (RawLamToken.funTok.inj h).1 (RawLamToken.funTok.inj h).2)
    * exact Or.inr h
  * rintro (rfl | h)
    * exact Or.inl rfl
    * exact Or.inr h

private def lamInUnionInsert [DecidableEq (RawLamToken a)] :
  List (RawLamToken a) × List (RawLamToken a) → Finset (RawLamToken a) →
  Finset (RawLamToken a)
| p, acc => listToFinset p.1 ∪ acc

private def lamOutUnionInsert [DecidableEq (RawLamToken a)] :

```

```

List (RawLamToken a) x List (RawLamToken a) -> Finset (RawLamToken a) ->
  Finset (RawLamToken a)
| p, acc => listToFinset p.2 U ' acc

private instance [DecidableEq (RawLamToken a)] :
  LeftCommutative (lamInUnionInsert (a := a)) :=
<fun p q s => by
  simp only [lamInUnionInsert]
  ext x; simp only [mem_funion]
  constructor <;> * rintro (h | h | h); exacts [Or.inr (Or.inl h), Or.inl h, Or.inr (Or.inr h
  )]>

private instance [DecidableEq (RawLamToken a)] :
  LeftCommutative (lamOutUnionInsert (a := a)) :=
<fun p q s => by
  simp only [lamOutUnionInsert]
  ext x; simp only [mem_funion]
  constructor <;> * rintro (h | h | h); exacts [Or.inr (Or.inl h), Or.inl h, Or.inr (Or.inr h
  )]>

def lamInputUnion (s : Finset (List (RawLamToken a) x List (RawLamToken a))) :
  Finset (RawLamToken a) :=
  Multiset.foldr (lamInUnionInsert (a := a)) {} s.1

def lamOutputUnion (s : Finset (List (RawLamToken a) x List (RawLamToken a))) :
  Finset (RawLamToken a) :=
  Multiset.foldr (lamOutUnionInsert (a := a)) {} s.1

theorem lamInputUnion_empty :
  lamInputUnion ({ : Finset (List (RawLamToken a) x List (RawLamToken a))) = {} := by
  rfl

theorem lamOutputUnion_empty :
  lamOutputUnion ({ : Finset (List (RawLamToken a) x List (RawLamToken a))) = {} := by
  rfl

theorem mem_lamInputUnion {s : Finset (List (RawLamToken a) x List (RawLamToken a))}
  {t : RawLamToken a} :
  t in lamInputUnion s <-> exists p in s, t in listToFinset p.1 := by
  have aux (m : Multiset (List (RawLamToken a) x List (RawLamToken a))) :
    t in Multiset.foldr (lamInUnionInsert (a := a)) {} m <->
      exists p in m, t in listToFinset p.1 := by
    refine Multiset.induction_on m ?_ ?_
  * constructor
  * intro ht; exact False.elim (Finset.notMem_empty t ht)
  * rintro <_, hp, _>; exact False.elim (by cases hp)
  * intro p rest ih
  simp only [Multiset.foldr_cons, lamInUnionInsert, mem_funion, ih, Multiset.mem_cons]
  constructor
  * rintro (ht | <q, hq, ht>)
  * exact <p, Or.inl rfl, ht>
  * exact <q, Or.inr hq, ht>
  * rintro <q, hq, ht>
  rcases hq with rfl | hq
  * exact Or.inl ht
  * exact Or.inr <q, hq, ht>
  simpa [lamInputUnion] using aux s.1

theorem mem_lamOutputUnion {s : Finset (List (RawLamToken a) x List (RawLamToken a))}
  {t : RawLamToken a} :
  t in lamOutputUnion s <-> exists p in s, t in listToFinset p.2 := by
  have aux (m : Multiset (List (RawLamToken a) x List (RawLamToken a))) :
    t in Multiset.foldr (lamOutUnionInsert (a := a)) {} m <->

```

```

    exists p in m, t in listToFinset p.2 := by
  refine Multiset.induction_on m ?_ ?_
* constructor
* intro ht; exact False.elim (Finset.notMem_empty t ht)
* rintro <_, hp, _>; exact False.elim (by cases hp)
* intro p rest ih
  simp only [Multiset.foldr_cons, lamOutUnionInsert, mem_funion, ih, Multiset.mem_cons]
  constructor
  * rintro (ht | <q, hq, ht>)
    * exact <p, Or.inl rfl, ht>
    * exact <q, Or.inr hq, ht>
  * rintro <q, hq, ht>
    rcases hq with rfl | hq
    * exact Or.inl ht
    * exact Or.inr <q, hq, ht>
  simp [lamOutputUnion] using aux s.1

/-! ## Staged consistency -/

variable (A : InfoSys a)

/-- Stage-`n` consistency (Scott (4)-(7), stratified). -/
def LamConN : Nat -> Finset (RawLamToken a) -> Prop
| 0, u => lamAtomFinset u in A.Con /\ lamFunFinset u = {}
| n + 1, u =>
  LamConN n u /\
  (lamAtomFinset u = {} /\
   (forall p in lamFunFinset u,
    LamConN n (listToFinset p.1) /\ LamConN n (listToFinset p.2)) /\
   forall s subseq lamFunFinset u,
    LamConN n (lamInputUnion s) -> LamConN n (lamOutputUnion s))

def LamCon (u : Finset (RawLamToken a)) : Prop :=
  exists n, LamConN A n u

theorem LamConN_mono {n m : Nat} (hmn : n <= m) {u : Finset (RawLamToken a)}
  (hu : LamConN A n u) : LamConN A m u := by
  induction hmn with
  | refl => exact hu
  | step _ ih => exact Or.inl ih

theorem LamCon_empty : LamCon A ({.bot} : Finset (RawLamToken a)) :=
  <0, by
    change lamAtomFinset {.bot} in A.Con /\ lamFunFinset {.bot} = {}
    rw [lamAtomFinset_empty, lamFunFinset_empty]
    exact <A.con_empty, rfl>>

theorem LamCon_singleton_bot : LamCon A ({.bot} : Finset (RawLamToken a)) := by
  refine <0, ?_>
  change lamAtomFinset _ in A.Con /\ lamFunFinset _ = {}
  constructor
  * have h : lamAtomFinset ({.bot} : Finset (RawLamToken a)) = {} := by
    ext x
    constructor
    * intro hx; exact False.elim (nomatch mem_lamAtomFinset.1 hx)
    * intro hx; exact False.elim (Finset.notMem_empty x hx)
  rw [h]; exact A.con_empty
* ext p
  constructor
  * intro hp; exact False.elim (nomatch mem_lamFunFinset.1 hp)
  * intro hp; exact False.elim (Finset.notMem_empty p hp)

```

Lean 4 source (lines 401–800)

```

theorem LamCon_singleton_atom (x : a) :
  LamCon A ({.atom x} : Finset (RawLamToken a)) := by
  refine <0, ?_>
  change lamAtomFinset _ in A.Con /\ lamFunFinset _ = {}
  constructor
  * have h : lamAtomFinset ({.atom x} : Finset (RawLamToken a)) = {x} := by
    ext y
    simp only [mem_lamAtomFinset, Finset.mem_singleton]
    exact <RawLamToken.atom.inj, congrArg RawLamToken.atom>
  rw [h]; exact A.con_sing x
  * ext p
    constructor
    * intro hp; exact False.elim (nomatch mem_lamFunFinset.1 hp)
    * intro hp; exact False.elim (Finset.notMem_empty p hp)

theorem LamCon_subset {u : Finset (RawLamToken a)} (hu : LamCon A u) :
  forall {v}, v subseq u -> LamCon A v := by
  rcases hu with <n, hu>
  induction n generalizing u with
  | zero =>
    intro v hv
    refine <0, ?_>
    change lamAtomFinset v in A.Con /\ lamFunFinset v = {}
    rcases (hu : lamAtomFinset u in A.Con /\ lamFunFinset u = {}) with <hA, hF>
    exact <A.con_subset hA (lamAtomFinset_mono hv),
      Finset.Subset.antisymm
        (fun p hp => by
          have : p in lamFunFinset u := lamFunFinset_mono hv hp
          rw [hF] at this; exact False.elim (Finset.notMem_empty p this))
        (Finset.empty_subset _)>
  | succ n ih =>
    intro v hv
    rcases hu with hu | <hA, hWF, hFun>
    * exact ih hu hv
    * refine <n + 1, Or.inr <?_, ?_, ?_>>
      * exact Finset.Subset.antisymm
        (fun x hx => by
          have : x in lamAtomFinset u := lamAtomFinset_mono hv hx
          rw [hA] at this; exact False.elim (Finset.notMem_empty x this))
        (Finset.empty_subset _))
    * intro p hp; exact hWF p (lamFunFinset_mono hv hp)
    * intro s hs hIn
      exact hFun s (fun p hp => lamFunFinset_mono hv (hs hp)) hIn

/-- Well-formed tokens (Scott (1)-(3)). -/
def IsLamToken : RawLamToken a -> Prop
| .bot | .atom _ => True
| .funTok u v => LamCon A (listToFinset u) /\ LamCon A (listToFinset v)

def LamToken : Type _ := { t : RawLamToken a // IsLamToken A t }

instance : DecidableEq (LamToken A) := Subtype.instDecidableEq

def lamBot : LamToken A := <.bot, trivial>

def lamForget (t : LamToken A) : RawLamToken a := t.val

private def lamForgetInsert :
  LamToken A -> Finset (RawLamToken a) -> Finset (RawLamToken a) :=
  fun t => insert (lamForget A t)

private instance : LeftCommutative (lamForgetInsert (A := A)) :=
  <fun p q s => insert_comm' (lamForget A p) (lamForget A q) s>

```

```

def lamForgetFinset (u : Finset (LamToken A)) : Finset (RawLamToken a) :=
  Multiset.foldr (lamForgetInsert (A := A)) {} u.1

private theorem mem_foldr_lamForget (s : Multiset (LamToken A)) (t : RawLamToken a) :
  t in Multiset.foldr (lamForgetInsert (A := A)) {} s <-> exists p in s, p.val = t :=
  by
  refine Multiset.induction_on s ?_ ?_
  * constructor
  * intro ht; exact False.elim (Finset.notMem_empty t ht)
  * rintro <_, hp, _>; exact False.elim (by cases hp)
  * intro p rest ih
  simp only [Multiset.foldr_cons, lamForgetInsert, Finset.mem_insert, ih, Multiset.mem_cons]
  constructor
  * rintro (rfl | <q, hq, hq'>)
  * exact <p, Or.inl rfl, rfl>
  * exact <q, Or.inr hq, hq'>
  * rintro <q, hq, hq'>
  rcases hq with rfl | hq
  * exact Or.inl hq'.symm
  * exact Or.inr <q, hq, hq'>

theorem mem_lamForgetFinset {u : Finset (LamToken A)} {t : RawLamToken a} :
  t in lamForgetFinset A u <-> exists p in u, p.val = t := by
  simpa [lamForgetFinset] using mem_foldr_lamForget A u.1 t

theorem lamForgetFinset_empty : lamForgetFinset A ({ } : Finset (LamToken A)) = { } :=
  rfl

theorem lamForgetFinset_singleton (p : LamToken A) :
  lamForgetFinset A {p} = {p.val} := by
  ext t
  constructor
  * intro ht
  rcases (mem_lamForgetFinset A).1 ht with <q, hq, hq'>
  have : q = p := Finset.mem_singleton.mp hq
  subst this; exact Finset.mem_singleton.mpr hq'.symm
  * intro ht
  exact (mem_lamForgetFinset A).2
    <p, Finset.mem_singleton_self p, (Finset.mem_singleton.mp ht).symm>

theorem lamAtomFinset_singleton_funTok (u v : List (RawLamToken a)) :
  lamAtomFinset ({.funTok u v} : Finset (RawLamToken a)) = { } := by
  ext x
  constructor
  * intro hx; exact False.elim (nomatch mem_lamAtomFinset.1 hx)
  * intro hx; exact False.elim (Finset.notMem_empty x hx)

theorem lamFunFinset_singleton_funTok (u v : List (RawLamToken a)) :
  lamFunFinset ({.funTok u v} : Finset (RawLamToken a)) = {(u, v)} := by
  ext p
  simp only [mem_lamFunFinset, Finset.mem_singleton]
  constructor
  * intro h
  exact Prod.ext (RawLamToken.funTok.inj h).1 (RawLamToken.funTok.inj h).2
  * intro h
  cases h
  rfl

theorem lamInputUnion_singleton (p : List (RawLamToken a) x List (RawLamToken a)) :
  lamInputUnion ({p} : Finset (List (RawLamToken a) x List (RawLamToken a))) =
    listToFinset p.1 := by
  ext t

```

```

change t in listToFinset p.1 U ' {} <-> t in listToFinset p.1
simp only [mem_funion, Finset.notMem_empty, or_false]

theorem lamOutputUnion_singleton (p : List (RawLamToken a) x List (RawLamToken a)) :
  lamOutputUnion ({p} : Finset (List (RawLamToken a) x List (RawLamToken a))) =
    listToFinset p.2 := by
  ext t
  change t in listToFinset p.2 U ' {} <-> t in listToFinset p.2
  simp only [mem_funion, Finset.notMem_empty, or_false]

theorem LamConN_empty (n : Nat) : LamConN A n ({ } : Finset (RawLamToken a)) := by
  induction n with
  | zero =>
    change lamAtomFinset { } in A.Con /\ lamFunFinset { } = { }
    rw [lamAtomFinset_empty, lamFunFinset_empty]
    exact <A.con_empty, rfl>
  | succ n ih => exact Or.inl ih

theorem LamCon_singleton_funTok (u v : List (RawLamToken a))
  (hu : LamCon A (listToFinset u)) (hv : LamCon A (listToFinset v)) :
  LamCon A ({.funTok u v} : Finset (RawLamToken a)) := by
  rcases hu with <n, hu>
  rcases hv with <m, hv>
  let k := max n m
  refine <k + 1, Or.inr <?_, ?_, ?_>>
  * exact lamAtomFinset_singleton_funTok u v
  * intro p hp
    have : p = (u, v) := by
      have h := Finset.mem_singleton.mp (by
        rw [ <- lamFunFinset_singleton_funTok u v]; exact hp)
      exact h
    subst this
    exact <LamConN_mono A (Nat.le_max_left n m) hu,
      LamConN_mono A (Nat.le_max_right n m) hv>
  * intro s hs hIn
    have hFun : lamFunFinset ({.funTok u v} : Finset (RawLamToken a)) = {(u, v)} :=
      lamFunFinset_singleton_funTok u v
    rw [hFun] at hs
    if hmem : (u, v) in s then
      have hsin : s = {(u, v)} := by
        ext p
        constructor
        * intro hp
          exact Finset.mem_singleton.mpr (Finset.mem_singleton.mp (hs hp))
        * intro hp
          have : p = (u, v) := Finset.mem_singleton.mp hp
          subst this; exact hmem
      subst hsin
      rw [lamOutputUnion_singleton]
      exact LamConN_mono A (Nat.le_max_right n m) hv
    else
      have hempty : s = { } := by
        ext p
        constructor
        * intro hp
          exact False.elim (hmem (by
            have : p = (u, v) := Finset.mem_singleton.mp (hs hp)
            subst this; exact hp))
        * intro hp
          exact False.elim (Finset.notMem_empty p hp)
      subst hempty
      exact LamConN_empty A k

```



```

theorem LamCon_of_forget_singleton (p : LamToken A) :
  LamCon A (lamForgetFinset A {p}) := by
  rw [lamForgetFinset_singleton]
  match p with
  | <.bot, _> => exact LamCon_singleton_bot A
  | <.atom x, _> => exact LamCon_singleton_atom A x
  | <.funTok u v, hWF> => exact LamCon_singleton_funTok A u v hWF.1 hWF.2

/-- Bundled consistency on `LamToken` finsets. -/
def LamTokenCon (u : Finset (LamToken A)) : Prop :=
  LamCon A (lamForgetFinset A u)

theorem LamTokenCon_empty : LamTokenCon A ({ } : Finset (LamToken A)) := by
  simpa [LamTokenCon, lamForgetFinset_empty] using LamCon_empty A

theorem LamTokenCon_singleton (p : LamToken A) : LamTokenCon A {p} :=
  LamCon_of_forget_singleton A p

theorem LamTokenCon_subset {u v : Finset (LamToken A)}
  (hu : LamTokenCon A u) (hv : v ⊆ u) : LamTokenCon A v :=
  LamCon_subset A hu (by
    intro t ht
    rcases (mem_lamForgetFinset A).1 ht with <p, hp, rfl>
    exact (mem_lamForgetFinset A).2 <p, hv hp, rfl>)

/-! ## Staged entailment (Scott (8)-(11)) -- definitions only

Full `ent_refl` / `ent_con` / `ent_trans` for the `InfoSys` instance remain; the
definitions below package Scott's clauses (8)-(11) at stages matching `LamConN`.
-/

/-- Stage-`n` entailment on raw tokens (Scott (8)-(11), stratified). -/
def LamEntN : Nat -> Finset (RawLamToken a) -> RawLamToken a -> Prop
| 0, u, .bot => LamConN A 0 u
| 0, u, .atom x =>
  LamConN A 0 u /\ lamAtomFinset u != { } /\ A.Ent (lamAtomFinset u) x
| 0, _, .funTok _ _ => False
| n + 1, u, t =>
  LamEntN n u t /\
  match t with
  | .bot => LamConN A (n + 1) u
  | .atom x =>
    LamConN A (n + 1) u /\ lamFunFinset u = { } /\
    lamAtomFinset u != { } /\ A.Ent (lamAtomFinset u) x
  | .funTok u' v' =>
    LamConN A (n + 1) u /\ lamAtomFinset u = { } /\
    LamConN A n (listToFinset u') /\ LamConN A n (listToFinset v') /\
    exists s ⊆ lamFunFinset u,
    (forall p in s, forall t in listToFinset p.1, LamEntN n (listToFinset u')
      t) /\
    (forall t in listToFinset v', LamEntN n (lamOutputUnion s) t)

/-- Unstaged raw entailment. -/
def LamEnt (u : Finset (RawLamToken a)) (t : RawLamToken a) : Prop :=
  exists n, LamEntN A n u t

theorem LamEntN_mono {n m : Nat} (hmn : n <= m) {u : Finset (RawLamToken a)}
  {t : RawLamToken a} (ht : LamEntN A n u t) : LamEntN A m u t := by
  induction hmn with
  | refl => exact ht
  | step _ ih => exact Or.inl ih

theorem LamEnt_bot {u : Finset (RawLamToken a)} (hu : LamCon A u) :

```

```

    LamEnt A u .bot := by
rcases hu with <n, hu>
cases n with
| zero => exact <0, hu>
| succ n => exact <n + 1, Or.inr hu>

theorem LamEntN_con {n : Nat} {u : Finset (RawLamToken a)} {t : RawLamToken a}
  (ht : LamEntN A n u t) : LamConN A n u := by
  match n, t with
  | 0, .bot => exact ht
  | 0, .atom _ => exact ht.1
  | 0, .funTok _ _ => exact False.elim ht
  | n + 1, t =>
    rcases ht with ht | ht
    * exact Or.inl (LamEntN_con (n := n) ht)
    * match t with
      | .bot => exact ht
      | .atom _ => exact ht.1
      | .funTok _ _ => exact ht.1

theorem LamEnt_con {u : Finset (RawLamToken a)} {t : RawLamToken a}
  (ht : LamEnt A u t) : LamCon A u := by
  rcases ht with <n, ht>
  exact <n, LamEntN_con A ht>

/-! ## Inversion lemmas for staged Con -/

theorem LamConN_fun_empty {n : Nat} {u : Finset (RawLamToken a)}
  (hu : LamConN A n u) (hne : lamAtomFinset u != {}) : lamFunFinset u = {} := by
  induction n generalizing u with
  | zero => exact hu.2
  | succ n ih =>
    rcases hu with hu | <hA, _, _>
    * exact ih hu hne
    * exact False.elim (hne hA)

theorem LamConN_atom_empty {n : Nat} {u : Finset (RawLamToken a)}
  (hu : LamConN A n u) (hne : lamFunFinset u != {}) : lamAtomFinset u = {} := by
  induction n generalizing u with
  | zero => exact False.elim (hne hu.2)
  | succ n ih =>
    rcases hu with hu | <hA, _, _>
    * exact ih hu hne
    * exact hA

theorem LamConN_atom_con {n : Nat} {u : Finset (RawLamToken a)}
  (hu : LamConN A n u) (hne : lamAtomFinset u != {}) : lamAtomFinset u in A.Con := by
  induction n generalizing u with
  | zero => exact hu.1
  | succ n ih =>
    rcases hu with hu | <hA, _, _>
    * exact ih hu hne
    * exact False.elim (hne hA)

theorem LamConN_fun_wf {n : Nat} {u : Finset (RawLamToken a)}
  {p : List (RawLamToken a) x List (RawLamToken a)}
  (hu : LamConN A (n + 1) u) (hp : p in lamFunFinset u) :
  LamConN A n (listToFinset p.1) /\ LamConN A n (listToFinset p.2) := by
  have aux {m : Nat} {w : Finset (RawLamToken a)}
    (hw : LamConN A m w) (hq : p in lamFunFinset w) :
    exists k, k + 1 <= m /\
      LamConN A k (listToFinset p.1) /\ LamConN A k (listToFinset p.2) := by
    induction m generalizing w with

```

```

| zero =>
  exact False.elim (by
    have : p in ({} : Finset _) := by rw [ <- hw.2]; exact hq
    exact Finset.notMem_empty p this)
| succ m ih =>
  rcases hw with hw | <_, hWF, _>
  * rcases ih hw hq with <k, hk, hk'>
    exact <k, Nat.le_succ_of_le hk, hk'>
  * exact <m, Nat.le_refl _, hWF p hq>
rcases aux hu hp with <k, hk, h1, h2>
have hk' : k <= n := Nat.le_of_succ_le_succ hk
exact <LamConN_mono A hk' h1, LamConN_mono A hk' h2>

/-- FunCon from an explicit right-branch witness at stage `n+1`. -/
theorem LamConN_fun_FunCon_right {n : Nat} {u : Finset (RawLamToken a)}
  (_hA : lamAtomFinset u = {})
  (_hWF : forall p in lamFunFinset u,
    LamConN A n (listToFinset p.1) /\ LamConN A n (listToFinset p.2))
  (hFun : forall s subseq lamFunFinset u,
    LamConN A n (lamInputUnion s) -> LamConN A n (lamOutputUnion s))
  {s : Finset (List (RawLamToken a) x List (RawLamToken a))}
  (hs : s subseq lamFunFinset u) (hIn : LamConN A n (lamInputUnion s)) :
  LamConN A n (lamOutputUnion s) :=
  hFun s hs hIn

/-- If `u` has function tokens at stage `n`, some earlier right branch supplies FunCon. -/
theorem LamConN_exists_FunCon {n : Nat} {u : Finset (RawLamToken a)}
  (hu : LamConN A n u) (hne : lamFunFinset u != {}) :
  exists k, k + 1 <= n /\ lamAtomFinset u = {} /\
    (forall p in lamFunFinset u,
      LamConN A k (listToFinset p.1) /\ LamConN A k (listToFinset p.2)) /\
    (forall s subseq lamFunFinset u,
      LamConN A k (lamInputUnion s) -> LamConN A k (lamOutputUnion s)) := by
  induction n generalizing u with
  | zero => exact False.elim (hne hu.2)
  | succ n ih =>
    rcases hu with hu | <hA, hWF, hFun>
    * rcases ih hu hne with <k, hk, H>
      exact <k, Nat.le_succ_of_le hk, H>
    * exact <n, Nat.le_refl _, hA, hWF, hFun>

/-- Atom/bot-only sets that are consistent at stage `n` are consistent at stage `0`. -/
theorem LamConN_down_atom {n : Nat} {u : Finset (RawLamToken a)}
  (hu : LamConN A n u) (hF : lamFunFinset u = {}) : LamConN A 0 u := by
  induction n generalizing u with
  | zero => exact hu
  | succ n ih =>
    rcases hu with hu | <hA, _, _>
    * exact ih hu hF
    * exact <by rw [hA]; exact A.con_empty, hF>

/-- Apply a stage-`k` FunCon at stage `m` >= `k` when the input union has no function tokens. -/
theorem LamConN_FunCon_mono_atom {k m : Nat} (hkm : k <= m)
  {u : Finset (RawLamToken a)}
  (hFun : forall s subseq lamFunFinset u,
    LamConN A k (lamInputUnion s) -> LamConN A k (lamOutputUnion s))
  {s : Finset (List (RawLamToken a) x List (RawLamToken a))}
  (hs : s subseq lamFunFinset u)
  (hIn : LamConN A m (lamInputUnion s))
  (hF : lamFunFinset (lamInputUnion s) = {}) :
  LamConN A m (lamOutputUnion s) := by
  have h0 := LamConN_down_atom A hIn hF

```

```

have hk : LamConN A k (lamInputUnion s) := LamConN_mono A (Nat.zero_le k) h0
exact LamConN_mono A hkm (hFun s hs hk)

/-- Full FunCon at stage `n+1` (uses `exists_FunCon` + atom-input lift).
Nested function tokens in the input union remain for the general `ent_con` argument. -/
theorem LamConN_fun_FunCon_atom {n : Nat} {u : Finset (RawLamToken a)}
  (hu : LamConN A (n + 1) u)
  {s : Finset (List (RawLamToken a) x List (RawLamToken a))}
  (hs : s ⊆ lamFunFinset u) (hIn : LamConN A n (lamInputUnion s))
  (hF : lamFunFinset (lamInputUnion s) = {}) :
  LamConN A n (lamOutputUnion s) := by
if empty : s = {} then
  subst empty
  simp [lamOutputUnion_empty] using LamConN_empty A n
else
  have hne : lamFunFinset u ≠ {} := by
    obtain <p, hp> := Finset.nonempty_of_ne_empty empty
    exact Finset.ne_empty_of_mem (hs hp)
  rcases LamConN_exists_FunCon A hu hne with <k, hk, _, _, hFun>
  have hkn : k ≤ n := Nat.le_of_succ_le_succ hk
  exact LamConN_FunCon_mono_atom A hkn hFun hs hIn hF

/-- Membership entailment at a fixed stage (Scott (v) / ent_refl on raw tokens). -/
theorem LamEntN_of_mem {n : Nat} {u : Finset (RawLamToken a)} {t : RawLamToken a}
  (hu : LamConN A n u) (ht : t in u) : LamEntN A n u t := by

```

Lean 4 source (lines 801–1200)

```

induction n generalizing u t with
| zero =>
  match t with
  | .bot => exact hu
  | .atom x =>
    exact <hu, Finset.ne_empty_of_mem (mem_lamAtomFinset.2 ht),
      A.ent_refl hu.1 (mem_lamAtomFinset.2 ht)>
  | .funTok _ _ =>
    exact False.elim (by
      have hp : (_, _) in lamFunFinset u := mem_lamFunFinset.2 ht
      rw [hu.2] at hp; exact Finset.notMem_empty _ hp)
| succ n ih =>
  rcases hu with hu | hRight
  * exact Or.inl (ih hu ht)
  * have huFull : LamConN A (n + 1) u := Or.inr hRight
    match t with
    | .bot => exact Or.inr huFull
    | .atom x =>
      exact False.elim (by
        have : x in lamAtomFinset u := mem_lamAtomFinset.2 ht
        rw [hRight.1] at this; exact Finset.notMem_empty x this)
    | .funTok xs ys =>
      have hmem : (xs, ys) in lamFunFinset u := mem_lamFunFinset.2 ht
      have hWF := LamConN_fun_wf A huFull hmem
      refine Or.inr <huFull, hRight.1, hWF.1, hWF.2, <{(xs, ys)}, ?, ?, ?_>>
      * intro p hp
        have : p = (xs, ys) := Finset.mem_singleton.mp hp
        subst this
        exact hmem
      * intro p hp t ht'
        have : p = (xs, ys) := Finset.mem_singleton.mp hp
        subst this
        exact ih hWF.1 ht'
      * intro t ht'
        rw [lamOutputUnion_singleton (xs, ys)]
        exact ih hWF.2 ht'

```

```

theorem LamEnt_of_mem {u : Finset (RawLamToken a)} {t : RawLamToken a}
  (hu : LamCon A u) (ht : t in u) : LamEnt A u t := by
  rcases hu with <n, hu>
  exact <n, LamEntN_of_mem A hu ht>

/-- Bundled entailment on `LamToken`. -/
def LamTokenEnt (u : Finset (LamToken A)) (p : LamToken A) : Prop :=
  LamEnt A (lamForgetFinset A u) p.val

theorem LamTokenEnt_bot {u : Finset (LamToken A)} (hu : LamTokenCon A u) :
  LamTokenEnt A u (lamBot A) :=
  LamEnt_bot A hu

theorem LamTokenEnt_of_mem {u : Finset (LamToken A)} {p : LamToken A}
  (hu : LamTokenCon A u) (hp : p in u) : LamTokenEnt A u p :=
  LamEnt_of_mem A hu ((mem_lamForgetFinset A).2 <p, hp, rfl>)

/-! ## ent_con: inserting an entailed token preserves consistency -/

theorem lamInputUnion_insert
  (s : Finset (List (RawLamToken a) x List (RawLamToken a)))
  (p : List (RawLamToken a) x List (RawLamToken a)) :
  lamInputUnion (insert p s) = listToFinset p.1 U ' lamInputUnion s := by
ext t
constructor
* intro ht
  rcases (mem_lamInputUnion).1 ht with <q, hq, ht'>
  rcases Finset.mem_insert.mp hq with rfl | hq
  * exact mem_funion.2 (Or.inl ht')
  * exact mem_funion.2 (Or.inr ((mem_lamInputUnion).2 <q, hq, ht'>))
* intro ht
  rcases mem_funion.1 ht with ht | ht
  * exact (mem_lamInputUnion).2 <p, Finset.mem_insert_self p s, ht>
  * rcases (mem_lamInputUnion).1 ht with <q, hq, ht'>
    exact (mem_lamInputUnion).2 <q, Finset.mem_insert_of_mem hq, ht'>

theorem lamOutputUnion_insert
  (s : Finset (List (RawLamToken a) x List (RawLamToken a)))
  (p : List (RawLamToken a) x List (RawLamToken a)) :
  lamOutputUnion (insert p s) = listToFinset p.2 U ' lamOutputUnion s := by
ext t
constructor
* intro ht
  rcases (mem_lamOutputUnion).1 ht with <q, hq, ht'>
  rcases Finset.mem_insert.mp hq with rfl | hq
  * exact mem_funion.2 (Or.inl ht')
  * exact mem_funion.2 (Or.inr ((mem_lamOutputUnion).2 <q, hq, ht'>))
* intro ht
  rcases mem_funion.1 ht with ht | ht
  * exact (mem_lamOutputUnion).2 <p, Finset.mem_insert_self p s, ht>
  * rcases (mem_lamOutputUnion).1 ht with <q, hq, ht'>
    exact (mem_lamOutputUnion).2 <q, Finset.mem_insert_of_mem hq, ht'>

theorem lamInputUnion_funion
  (s t : Finset (List (RawLamToken a) x List (RawLamToken a))) :
  lamInputUnion (s U ' t) = lamInputUnion s U ' lamInputUnion t := by
ext x
constructor
* intro hx
  rcases (mem_lamInputUnion).1 hx with <p, hp, hx'>
  rcases mem_funion.1 hp with hp | hp
  * exact mem_funion.2 (Or.inl ((mem_lamInputUnion).2 <p, hp, hx'>))

```

```

    * exact mem_funion.2 (Or.inr ((mem_lamInputUnion).2 <p, hp, hx'>))
* intro hx
rcases mem_funion.1 hx with hx | hx
* rcases (mem_lamInputUnion).1 hx with <p, hp, hx'>
  exact (mem_lamInputUnion).2 <p, mem_funion.2 (Or.inl hp), hx'>
* rcases (mem_lamInputUnion).1 hx with <p, hp, hx'>
  exact (mem_lamInputUnion).2 <p, mem_funion.2 (Or.inr hp), hx'>

theorem lamOutputUnion_funion
  (s t : Finset (List (RawLamToken a) x List (RawLamToken a))) :
  lamOutputUnion (s U ' t) = lamOutputUnion s U ' lamOutputUnion t := by
ext x
constructor
* intro hx
rcases (mem_lamOutputUnion).1 hx with <p, hp, hx'>
rcases mem_funion.1 hp with hp | hp
* exact mem_funion.2 (Or.inl ((mem_lamOutputUnion).2 <p, hp, hx'>))
* exact mem_funion.2 (Or.inr ((mem_lamOutputUnion).2 <p, hp, hx'>))
* intro hx
rcases mem_funion.1 hx with hx | hx
* rcases (mem_lamOutputUnion).1 hx with <p, hp, hx'>
  exact (mem_lamOutputUnion).2 <p, mem_funion.2 (Or.inl hp), hx'>
* rcases (mem_lamOutputUnion).1 hx with <p, hp, hx'>
  exact (mem_lamOutputUnion).2 <p, mem_funion.2 (Or.inr hp), hx'>

theorem LamConN_insert_bot {n : Nat} {u : Finset (RawLamToken a)}
  (hu : LamConN A n u) : LamConN A n (insert (.bot : RawLamToken a) u) := by
induction n generalizing u with
| zero =>
  exact <by rw [lamAtomFinset_insert_bot]; exact hu.1,
    by rw [lamFunFinset_insert_bot]; exact hu.2>
| succ n ih =>
  rcases hu with hu | <hA, hWF, hFun>
  * exact Or.inl (ih hu)
  * exact Or.inr <by rw [lamAtomFinset_insert_bot]; exact hA,
    fun p hp => hWF p (by rwa [lamFunFinset_insert_bot] at hp),
    fun s hs hIn =>
      hFun s (fun p hp => by rw [← lamFunFinset_insert_bot u]; exact hs hp) hIn>

/-- Inserting an entailed bot or atom preserves stage-`n` consistency.
Function-token `ent_con` (FunCon on `insert`) remains. -/
theorem LamConN_insert_of_ent_atom {n : Nat} {u : Finset (RawLamToken a)} {x : a}
  (ht : LamEntN A n u (.atom x)) : LamConN A n (insert (.atom x) u) := by
induction n generalizing u with
| zero =>
  rcases ht with <<_, hF>, _, hEnt>
  exact <by rw [lamAtomFinset_insert_atom]; exact A.ent_con hEnt,
    by rw [lamFunFinset_insert_atom]; exact hF>
| succ n ih =>
  rcases ht with ht | <_, hF, _, hEnt>
  * exact Or.inl (ih ht)
  * exact LamConN_mono A (Nat.zero_le (n + 1))
    <by rw [lamAtomFinset_insert_atom]; exact A.ent_con hEnt,
      by rw [lamFunFinset_insert_atom]; exact hF>

theorem LamConN_insert_of_ent_bot {n : Nat} {u : Finset (RawLamToken a)}
  (ht : LamEntN A n u .bot) : LamConN A n (insert (.bot : RawLamToken a) u) :=
  LamConN_insert_bot A (LamEntN_con A ht)

theorem lamForgetFinset_insert (u : Finset (LamToken A)) (p : LamToken A) :
  lamForgetFinset A (insert p u) = insert p.val (lamForgetFinset A u) := by
ext t
constructor

```

```

* intro ht
  rcases (mem_lamForgetFinset A).1 ht with <q, hq, rfl>
  rcases Finset.mem_insert.mp hq with rfl | hq
  * exact Finset.mem_insert_self _ _
  * exact Finset.mem_insert_of_mem ((mem_lamForgetFinset A).2 <q, hq, rfl>)
* intro ht
  rcases Finset.mem_insert.mp ht with rfl | ht
  * exact (mem_lamForgetFinset A).2 <p, Finset.mem_insert_self p u, rfl>
  * rcases (mem_lamForgetFinset A).1 ht with <q, hq, rfl>
    exact (mem_lamForgetFinset A).2 <q, Finset.mem_insert_of_mem hq, rfl>

/-- Def 2.1 progress: `con_subset`, `con_sing`, `ent_bot`, `ent_refl`;
`ent_con` for bot/atom; `ent_con`/`ent_trans` on `funTok` remain. -/
theorem lambdaSystem_con_axioms :
  (forall {u v : Finset (LamToken A)}, LamTokenCon A u -> v subseteq u -> LamTokenCon
    A v) /\
  (forall p : LamToken A, LamTokenCon A {p}) /\
  (forall {u : Finset (LamToken A)}, LamTokenCon A u -> LamTokenEnt A u (lamBot A))
  /\
  (forall {u : Finset (LamToken A)} {p : LamToken A},
    LamTokenCon A u -> p in u -> LamTokenEnt A u p) :=
<fun hu hv => LamTokenCon_subset A hu hv,
  LamTokenCon_singleton A,
  fun hu => LamTokenEnt_bot A hu,
  fun hu hp => LamTokenEnt_of_mem A hu hp>

/-! ## Depth measure (for well-founded Con) -/

private theorem max_le_max_nat {a b c d : Nat} (h1 : a <= c) (h2 : b <= d) :
  max a b <= max c d :=
  max_le (le_trans h1 (le_max_left _ _)) (le_trans h2 (le_max_right _ _))

def rawDepth : RawLamToken a -> Nat
| .bot | .atom _ => 0
| .funTok u v =>
  1 + Nat.max
    (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u)
    (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 v)

theorem foldl_max_mono_init {u : List (RawLamToken a)} {i j : Nat} (hij : i <= j) :
  List.foldl (fun d t => Nat.max d (rawDepth t)) i u <=
  List.foldl (fun d t => Nat.max d (rawDepth t)) j u := by
  induction u generalizing i j with
  | nil => exact hij
  | cons a as ih => exact ih (max_le_max_nat hij le_rfl)

theorem foldl_max_ge_init (u : List (RawLamToken a)) (init : Nat) :
  init <= List.foldl (fun d t => Nat.max d (rawDepth t)) init u := by
  induction u generalizing init with
  | nil => exact le_rfl
  | cons a as ih =>
    exact Nat.le_trans (Nat.le_max_left init (rawDepth a)) (ih _)

theorem foldl_list_le_of_mem {t : RawLamToken a} {u : List (RawLamToken a)}
  (ht : t in u) :
  rawDepth t <= List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u := by
  induction u with
  | nil => cases ht
  | cons a as ih =>
    simp only [List.mem_cons] at ht
    rcases ht with rfl | ht

```

```

    * exact Nat.le_trans (Nat.le_max_right 0 (rawDepth t))
      (foldl_max_ge_init as (Nat.max 0 (rawDepth t)))
    * exact Nat.le_trans (ih ht) (foldl_max_mono_init (Nat.zero_le _))

theorem rawDepth_funTok_gt_left {u v : List (RawLamToken a)} {t : RawLamToken a}
  (ht : t in u) : rawDepth t < rawDepth (.funTok u v) := by
  have h : rawDepth t <=
    Nat.max (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u)
      (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 v) :=
    Nat.le_trans (foldl_list_le_of_mem ht) (Nat.le_max_left _ _)
  simp only [rawDepth]
  exact Nat.lt_of_le_of_lt h (Nat.lt_add_of_pos_left (Nat.succ_pos 0))

theorem rawDepth_funTok_gt_right {u v : List (RawLamToken a)} {t : RawLamToken a}
  (ht : t in v) : rawDepth t < rawDepth (.funTok u v) := by
  have h : rawDepth t <=
    Nat.max (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u)
      (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 v) :=
    Nat.le_trans (foldl_list_le_of_mem ht) (Nat.le_max_right _ _)
  simp only [rawDepth]
  exact Nat.lt_of_le_of_lt h (Nat.lt_add_of_pos_left (Nat.succ_pos 0))

private def maxDepthInsert : RawLamToken a -> Nat -> Nat :=
  fun t d => Nat.max (rawDepth t) d

private instance : LeftCommutative (maxDepthInsert (a := a)) :=
  <fun a b d => by simp only [maxDepthInsert]; exact Nat.max_left_comm _ _ _>

def finsetMaxDepth (u : Finset (RawLamToken a)) : Nat :=
  Multiset.foldr (maxDepthInsert (a := a)) 0 u.1

theorem le_finsetMaxDepth_of_mem {u : Finset (RawLamToken a)} {t : RawLamToken a}
  (ht : t in u) : rawDepth t <= finsetMaxDepth u := by
  simp only [finsetMaxDepth]
  have ht' : t in u.1 := Finset.mem_def.1 ht
  revert ht'
  refine Multiset.induction_on u.1 ?_ ?_
  * intro ht'; cases ht'
  * intro a s ih ht'
    simp only [Multiset.mem_cons] at ht'
    simp only [Multiset.foldr_cons, maxDepthInsert]
    rcases ht' with rfl | ht'
    * exact Nat.le_max_left _ _
    * exact Nat.le_trans (ih ht') (Nat.le_max_right _ _)

theorem foldr_max_le_of_forall {s : Multiset (RawLamToken a)} {b : Nat}
  (h : forall t in s, rawDepth t <= b) :
  Multiset.foldr (maxDepthInsert (a := a)) 0 s <= b := by
  revert h
  refine Multiset.induction_on s ?_ ?_
  * intro; exact Nat.zero_le _
  * intro a s ih h
    simp only [Multiset.foldr_cons, maxDepthInsert]
    exact Nat.max_le.2 <h a (Multiset.mem_cons_self _ _),
      ih fun t ht => h t (Multiset.mem_cons_of_mem ht)>

theorem foldr_max_lt_of_forall {s : Multiset (RawLamToken a)} {b : Nat}
  (hb : 0 < b) (h : forall t in s, rawDepth t < b) :
  Multiset.foldr (maxDepthInsert (a := a)) 0 s < b := by
  revert h
  refine Multiset.induction_on s ?_ ?_
  * intro; exact hb
  * intro a s ih h

```



```

simp only [Multiset.foldr_cons, maxDepthInsert]
exact Nat.max_lt.2 <h a (Multiset.mem_cons_self _ _),
  ih fun t ht => h t (Multiset.mem_cons_of_mem ht)>

theorem finsetMaxDepth_listToFinset_le (u : List (RawLamToken a)) :
  finsetMaxDepth (listToFinset u) <=
    List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u := by
simp only [finsetMaxDepth]
exact foldr_max_le_of_forall fun t ht =>
  foldl_list_le_of_mem ((mem_listToFinset).1 (Finset.mem_def.2 ht))

theorem finsetMaxDepth_list_lt_funTok (u v : List (RawLamToken a)) :
  finsetMaxDepth (listToFinset u) < rawDepth (.funTok u v) := by
have h1 := finsetMaxDepth_listToFinset_le u
have h2 : List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u <=
  Nat.max (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u)
    (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 v) :=
  Nat.le_max_left _ _
simp only [rawDepth]
exact Nat.lt_of_le_of_lt (Nat.le_trans h1 h2) (Nat.lt_add_of_pos_left (Nat.succ_pos 0))

theorem finsetMaxDepth_list_lt_funTok_out (u v : List (RawLamToken a)) :
  finsetMaxDepth (listToFinset v) < rawDepth (.funTok u v) := by
have h1 := finsetMaxDepth_listToFinset_le v
have h2 : List.foldl (fun d t => Nat.max d (rawDepth t)) 0 v <=
  Nat.max (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u)
    (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 v) :=
  Nat.le_max_right _ _
simp only [rawDepth]
exact Nat.lt_of_le_of_lt (Nat.le_trans h1 h2) (Nat.lt_add_of_pos_left (Nat.succ_pos 0))

theorem finsetMaxDepth_of_mem_funTok {u : Finset (RawLamToken a)}
  {xs ys : List (RawLamToken a)} (h : .funTok xs ys in u) :
  finsetMaxDepth (listToFinset xs) < finsetMaxDepth u /\
    finsetMaxDepth (listToFinset ys) < finsetMaxDepth u :=
  <Nat.lt_of_lt_of_le (finsetMaxDepth_list_lt_funTok xs ys) (le_finsetMaxDepth_of_mem h),
    Nat.lt_of_lt_of_le (finsetMaxDepth_list_lt_funTok_out xs ys) (le_finsetMaxDepth_of_mem h)>

theorem finsetMaxDepth_inputUnion_lt {u : Finset (RawLamToken a)}
  {s : Finset (List (RawLamToken a) x List (RawLamToken a))}
  (hs : s subseq lamFunFinset u) (hfun : lamFunFinset u != {}) :
  finsetMaxDepth (lamInputUnion s) < finsetMaxDepth u := by
obtain <p0, hp0> := Finset.nonempty_of_ne_empty hfun
have htok0 : .funTok p0.1 p0.2 in u := mem_lamFunFinset.1 hp0
have hpos : 0 < finsetMaxDepth u := by
  have hraw : 0 < rawDepth (.funTok p0.1 p0.2) := by
    simp only [rawDepth]
    exact Nat.add_pos_left (Nat.succ_pos 0) _
  exact Nat.lt_of_lt_of_le hraw (le_finsetMaxDepth_of_mem htok0)
refine foldr_max_lt_of_forall (s := (lamInputUnion s).1) hpos ?_
intro t ht
have ht' : t in lamInputUnion s := Finset.mem_def.2 ht
rcases (mem_lamInputUnion).1 ht' with <p, hp, htp>
have htok : .funTok p.1 p.2 in u := mem_lamFunFinset.1 (hs hp)
exact Nat.lt_of_lt_of_le
  (rawDepth_funTok_gt_left ((mem_listToFinset).1 htp))
  (le_finsetMaxDepth_of_mem htok)

theorem finsetMaxDepth_outputUnion_lt {u : Finset (RawLamToken a)}
  {s : Finset (List (RawLamToken a) x List (RawLamToken a))}
  (hs : s subseq lamFunFinset u) (hfun : lamFunFinset u != {}) :
  finsetMaxDepth (lamOutputUnion s) < finsetMaxDepth u := by

```

```

obtain <p0, hp0> := Finset.nonempty_of_ne_empty hfun
have htok0 : .funTok p0.1 p0.2 in u := mem_lamFunFinset.1 hp0
have hpos : 0 < finsetMaxDepth u := by
  have hraw : 0 < rawDepth (.funTok p0.1 p0.2) := by
    simp only [rawDepth]
    exact Nat.add_pos_left (Nat.succ_pos 0) _
  exact Nat.lt_of_lt_of_le hraw (le_finsetMaxDepth_of_mem htok0)
refine foldr_max_lt_of_forall (s := (lamOutputUnion s).1) hpos ?_
intro t ht
have ht' : t in lamOutputUnion s := Finset.mem_def.2 ht
rcases (mem_lamOutputUnion).1 ht' with <p, hp, htp>
have htok : .funTok p.1 p.2 in u := mem_lamFunFinset.1 (hs hp)
exact Nat.lt_of_lt_of_le
  (rawDepth_funTok_gt_right ((mem_listToFinset).1 htp))
  (le_finsetMaxDepth_of_mem htok)

/-- Depth-based consistency: FunCon in the recursive clause (Scott (7)).
Uses `decidableEq_finset` so the empty-fun branch does not pull `Classical.choice`. -/
def LamConDepth (u : Finset (RawLamToken a)) : Prop :=
  haveI : DecidableEq (Finset (List (RawLamToken a) x List (RawLamToken a))) :=
    decidableEq_finset
  if _hF : lamFunFinset u = {} then
    lamAtomFinset u in A.Con
  else
    lamAtomFinset u = {} /\
      (forall p in lamFunFinset u,
        LamConDepth (listToFinset p.1) /\ LamConDepth (listToFinset p.2)) /\
      (forall s subseqeq lamFunFinset u,
        LamConDepth (lamInputUnion s) -> LamConDepth (lamOutputUnion s))
termination_by finsetMaxDepth u
decreasing_by
  * exact (finsetMaxDepth_of_mem_funTok (mem_lamFunFinset.1 ?_?)).1
  * exact (finsetMaxDepth_of_mem_funTok (mem_lamFunFinset.1 ?_?)).2
  * exact finsetMaxDepth_inputUnion_lt ?_? ?_?
  * exact finsetMaxDepth_outputUnion_lt ?_? ?_?

theorem LamConDepth_empty : LamConDepth A ({ } : Finset (RawLamToken a)) := by
  haveI : DecidableEq (Finset (List (RawLamToken a) x List (RawLamToken a))) :=
    decidableEq_finset
  rw [LamConDepth.eq_1, dif_pos lamFunFinset_empty, lamAtomFinset_empty]
  exact A.con_empty

theorem LamConDepth_of_fun_empty {u : Finset (RawLamToken a)}
  (hF : lamFunFinset u = { }) :
  LamConDepth A u <-> lamAtomFinset u in A.Con := by
  haveI : DecidableEq (Finset (List (RawLamToken a) x List (RawLamToken a))) :=
    decidableEq_finset
  rw [LamConDepth.eq_1, dif_pos hF]

theorem LamConDepth_of_fun_nonempty {u : Finset (RawLamToken a)}
  (hF : lamFunFinset u != { }) :

```

Lean 4 source (lines 1201–1600)

```

LamConDepth A u <->
  lamAtomFinset u = { } /\
    (forall p in lamFunFinset u,
      LamConDepth A (listToFinset p.1) /\ LamConDepth A (listToFinset p.2)) /\
    (forall s subseqeq lamFunFinset u,
      LamConDepth A (lamInputUnion s) -> LamConDepth A (lamOutputUnion s)) := by
  haveI : DecidableEq (Finset (List (RawLamToken a) x List (RawLamToken a))) :=
    decidableEq_finset
  rw [LamConDepth.eq_1, dif_neg hF]

```

```

theorem LamConDepth_mono {u v : Finset (RawLamToken a)}
  (huv : u ⊆ v) (hv : LamConDepth A v) : LamConDepth A u := by
  haveI : DecidableEq (Finset (List (RawLamToken a) × List (RawLamToken a))) :=
    decidableEq_finset
  have hFunMono := lamFunFinset_mono huv
  have hAtomMono := lamAtomFinset_mono huv
  by_cases hFv : lamFunFinset v = {}
  * have hFu : lamFunFinset u = {} :=
    Finset.Subset.antisymm
      (fun p hp => by
        have := hFunMono hp
        rw [hFv] at this; exact False.elim (Finset.notMem_empty p this))
      (Finset.empty_subset _)
    exact (LamConDepth_of_fun_empty A hFu).2
      (A.con_subset ((LamConDepth_of_fun_empty A hFv).1 hv) hAtomMono)
  * have hv' := (LamConDepth_of_fun_nonempty A hFv).1 hv
    rcases hv' with <hAv, hWF, hFun>
    by_cases hFu : lamFunFinset u = {}
    * have hAu : lamAtomFinset u = {} :=
      Finset.Subset.antisymm
        (fun x hx => by
          have := hAtomMono hx
          rw [hAv] at this; exact False.elim (Finset.notMem_empty x this))
        (Finset.empty_subset _)
      exact (LamConDepth_of_fun_empty A hFu).2 (by rw [hAu]; exact A.con_empty)
    * refine (LamConDepth_of_fun_nonempty A hFu).2 <?_, ?_, ?_>
      * exact Finset.Subset.antisymm
        (fun x hx => by
          have := hAtomMono hx
          rw [hAv] at this; exact False.elim (Finset.notMem_empty x this))
        (Finset.empty_subset _)
      * intro p hp; exact hWF p (hFunMono hp)
      * intro s hs hIn
        exact hFun s (fun p hp => hFunMono (hs hp)) hIn

theorem LamConDepth_singleton_bot :
  LamConDepth A ({.bot} : Finset (RawLamToken a)) := by
  haveI : DecidableEq (Finset (List (RawLamToken a) × List (RawLamToken a))) :=
    decidableEq_finset
  have hF : lamFunFinset ({.bot} : Finset (RawLamToken a)) = {} := by
    ext p; constructor
    * intro hp; exact False.elim (nomatch mem_lamFunFinset.1 hp)
    * intro hp; exact False.elim (Finset.notMem_empty p hp)
  exact (LamConDepth_of_fun_empty A hF).2 (by
    have h : lamAtomFinset ({.bot} : Finset (RawLamToken a)) = {} := by
      ext x; constructor
      * intro hx; exact False.elim (nomatch mem_lamAtomFinset.1 hx)
      * intro hx; exact False.elim (Finset.notMem_empty x hx)
    rw [h]; exact A.con_empty)

theorem LamConDepth_singleton_atom (x : a) :
  LamConDepth A ({.atom x} : Finset (RawLamToken a)) := by
  haveI : DecidableEq (Finset (List (RawLamToken a) × List (RawLamToken a))) :=
    decidableEq_finset
  have hF : lamFunFinset ({.atom x} : Finset (RawLamToken a)) = {} := by
    ext p; constructor
    * intro hp; exact False.elim (nomatch mem_lamFunFinset.1 hp)
    * intro hp; exact False.elim (Finset.notMem_empty p hp)
  exact (LamConDepth_of_fun_empty A hF).2 (by
    have h : lamAtomFinset ({.atom x} : Finset (RawLamToken a)) = {x} := by
      ext y; simp only [mem_lamAtomFinset, Finset.mem_singleton]
      exact <RawLamToken.atom.inj, congrArg RawLamToken.atom>
    rw [h]; exact A.con_sing x)

```

```

/-! ## Depth-based entailment (Scott (8)-(11), inductive FunEnt-style) -/

/-- Depth-based entailment as an inductive Prop (Scott (8)-(11)).
Premises are positive, so FunEnt-style recursion is allowed -- unlike FunCon
inside an inductive `LamCon`. -/
inductive LamEntDepth : Finset (RawLamToken a) -> RawLamToken a -> Prop where
  | bot {u : Finset (RawLamToken a)}
    (hu : LamConDepth A u) : LamEntDepth u .bot
  | atom {u : Finset (RawLamToken a)} {x : a}
    (hu : LamConDepth A u)
    (hF : lamFunFinset u = {})
    (hne : lamAtomFinset u != {})
    (hEnt : A.Ent (lamAtomFinset u) x) : LamEntDepth u (.atom x)
  | funTok {u : Finset (RawLamToken a)} {xs ys : List (RawLamToken a)}
    {s : Finset (List (RawLamToken a) x List (RawLamToken a))}
    (hu : LamConDepth A u)
    (hA : lamAtomFinset u = {})
    (hConIn : LamConDepth A (listToFinset xs))
    (hConOut : LamConDepth A (listToFinset ys))
    (hs : s subseq lamFunFinset u)
    /- Function tokens are only entailed in the function world (cf. `SumEnt` right). -/
    (hWorld : lamFunFinset u != {})
    /- Atom/bot inputs only when `xs` is atom-world -- blocks mixed-world FunCon gaps. -/
    (hInCompat : lamFunFinset (listToFinset xs) = {} ->
      lamFunFinset (lamInputUnion s) = {})
    (hEntIn : forall p in s, forall z in listToFinset p.1, LamEntDepth (listToFinset xs
    ) z)
    (hEntOut : forall z in listToFinset ys, LamEntDepth (lamOutputUnion s) z) :
    LamEntDepth u (.funTok xs ys)

theorem LamEntDepth_con {u : Finset (RawLamToken a)} {t : RawLamToken a}
  (ht : LamEntDepth A u t) : LamConDepth A u := by
  cases ht with
  | bot hu => exact hu
  | atom hu _ _ => exact hu
  | funTok hu _ _ _ _ => exact hu

theorem LamEntDepth_bot {u : Finset (RawLamToken a)} (hu : LamConDepth A u) :
  LamEntDepth A u .bot :=
  LamEntDepth.bot hu

theorem LamEntDepth_of_mem {u : Finset (RawLamToken a)} {t : RawLamToken a}
  (hu : LamConDepth A u) (ht : t in u) : LamEntDepth A u t := by
  suffices forall M u t, Nat.max (finsetMaxDepth u) (rawDepth t) = M ->
    LamConDepth A u -> t in u -> LamEntDepth A u t from
  this _ _ rfl hu ht
intro M
induction M using Nat.strong_induction_on with
| h M ih =>
  intro u t hM hu ht
  match t with
  | .bot => exact LamEntDepth_bot A hu
  | .atom x =>
    cases decidableEq_finset (lamFunFinset u) {} with
  | isTrue hF =>
    have hx : x in lamAtomFinset u := mem_lamAtomFinset.2 ht
    exact LamEntDepth.atom hu hF (Finset.ne_empty_of_mem hx)
    (A.ent_refl ((LamConDepth_of_fun_empty A hF).1 hu) hx)
  | isFalse hF =>
    have hA := ((LamConDepth_of_fun_nonempty A hF).1 hu).1
    have hx : x in lamAtomFinset u := mem_lamAtomFinset.2 ht
    rw [hA] at hx; exact False.elim (Finset.notMem_empty x hx)

```

```

| .funTok xs ys =>
cases decidableEq_finset (lamFunFinset u) {} with
| isTrue hF =>
  have : (xs, ys) in lamFunFinset u := mem_lamFunFinset.2 ht
  rw [hF] at this; exact False.elim (Finset.notMem_empty _ this)
| isFalse hF =>
  have hu' := (LamConDepth_of_fun_nonempty A hF).1 hu
  rcases hu' with <hA, hWF, _hFun>
  have hmem : (xs, ys) in lamFunFinset u := mem_lamFunFinset.2 ht
  have <hConIn, hConOut> := hWF (xs, ys) hmem
  refine LamEntDepth.funTok (s := {(xs, ys)}) hu hA hConIn hConOut
    (fun p hp => by
      have hp' : p = (xs, ys) := Finset.mem_singleton.mp hp
      exact hp' |> hmem) hF ?_ ?_ ?_
* intro hxsF
  have hIn : lamInputUnion {(xs, ys)} = listToFinset xs := by
    rw [lamInputUnion_singleton]
  rw [hIn, hxsF]
* intro p hp z hz
  have hp' : p = (xs, ys) := Finset.mem_singleton.mp hp
  subst hp'
  refine ih (Nat.max (finsetMaxDepth (listToFinset xs)) (rawDepth z))
    ?_ (listToFinset xs) z rfl hConIn hz
  have htok : .funTok xs ys in u := mem_lamFunFinset.1 hmem
  have ht_lt : rawDepth z < finsetMaxDepth u :=
    Nat.lt_of_lt_of_le (rawDepth_funTok_gt_left ((mem_listToFinset).1 hz))
      (le_finsetMaxDepth_of_mem htok)
  have hxs_lt : finsetMaxDepth (listToFinset xs) < rawDepth (.funTok xs ys) :=
    finsetMaxDepth_list_lt_funTok xs ys
  rw [ <- hM]
  refine Nat.max_lt.2 <
    Nat.lt_of_lt_of_le hxs_lt (Nat.le_max_right _ _),
    Nat.lt_of_lt_of_le ht_lt (Nat.le_max_left _ _)>
* intro z hz
  rw [lamOutputUnion_singleton]
  refine ih (Nat.max (finsetMaxDepth (listToFinset ys)) (rawDepth z))
    ?_ (listToFinset ys) z rfl hConOut hz
  have ht_lt : rawDepth z < rawDepth (.funTok xs ys) :=
    rawDepth_funTok_gt_right ((mem_listToFinset).1 hz)
  have hys_lt : finsetMaxDepth (listToFinset ys) < rawDepth (.funTok xs ys) :=
    finsetMaxDepth_list_lt_funTok_out xs ys
  rw [ <- hM]
  refine Nat.max_lt.2 <
    Nat.lt_of_lt_of_le hys_lt (Nat.le_max_right _ _),
    Nat.lt_of_lt_of_le ht_lt (Nat.le_max_right _ _)>

/-- Inserting bot preserves depth-Con. -/
theorem LamConDepth_insert_bot {u : Finset (RawLamToken a)}
  (hu : LamConDepth A u) :
  LamConDepth A (insert (.bot : RawLamToken a) u) := by
haveI : DecidableEq (Finset (List (RawLamToken a) x List (RawLamToken a))) :=
  decidableEq_finset
by_cases hF : lamFunFinset u = {}
* have hF' : lamFunFinset (insert (.bot : RawLamToken a) u) = {} := by
  rw [lamFunFinset_insert_bot]; exact hF
  exact (LamConDepth_of_fun_empty A hF').2 (by
    rw [lamAtomFinset_insert_bot]
    exact (LamConDepth_of_fun_empty A hF).1 hu)
* have hu' := (LamConDepth_of_fun_nonempty A hF).1 hu
  rcases hu' with <hA, hWF, hFun>
  have hF' : lamFunFinset (insert (.bot : RawLamToken a) u) != {} := by
    rw [lamFunFinset_insert_bot]; exact hF
  refine (LamConDepth_of_fun_nonempty A hF').2 <?_, ?_, ?_>

```

```

* rw [lamAtomFinset_insert_bot]; exact hA
* intro p hp
  rw [lamFunFinset_insert_bot] at hp
  exact hWF p hp
* intro s hs hIn
  refine hFun s (by rw [ <- lamFunFinset_insert_bot u]; exact hs) hIn

/-- Inserting an entailed atom preserves depth-Con. -/
theorem LamConDepth_insert_atom {u : Finset (RawLamToken a)} {x : a}
  (ht : LamEntDepth A u (.atom x)) :
  LamConDepth A (insert (.atom x) u) := by
haveI : DecidableEq (Finset (List (RawLamToken a) x List (RawLamToken a))) :=
  decidableEq_finset
cases ht with
| atom hu hF _hne hEnt =>
  have hF' : lamFunFinset (insert (.atom x) u) = {} := by
    rw [lamFunFinset_insert_atom]; exact hF
  exact (LamConDepth_of_fun_empty A hF').2 (by
    rw [lamAtomFinset_insert_atom]
    exact A.ent_con hEnt)

/-- Weaken entailment when the larger set stays in the function world (`atom = {}`). -/
theorem LamEntDepth_weaken_fun {u v : Finset (RawLamToken a)} {t : RawLamToken a}
  (huv : u subseq v) (hv : LamConDepth A v) (hAv : lamAtomFinset v = {})
  (ht : LamEntDepth A u t) : LamEntDepth A v t := by
cases ht with
| bot _ => exact LamEntDepth.bot hv
| atom _ _ hne _ =>
  obtain <x, hx> := Finset.nonempty_of_ne_empty hne
  exact False.elim (Finset.notMem_empty x (by
    have := lamAtomFinset_mono huv hx; rwa [hAv] at this))
| funTok _ _ hConIn hConOut hs hWorld hInCompat hEntIn hEntOut =>
  exact LamEntDepth.funTok hv hAv hConIn hConOut
    (fun p hp => lamFunFinset_mono huv (hs hp))
    (fun hFv => hWorld (Finset.Subset.antisymm
      (fun p hp => by
        have := lamFunFinset_mono huv hp
        rw [hFv] at this; exact False.elim (Finset.notMem_empty p this))
      (Finset.empty_subset _)))
    hInCompat hEntIn hEntOut

/-- Weaken atom-entailment when the larger set stays in the atom world (`fun = {}`). -/
theorem LamEntDepth_weaken_atom {u v : Finset (RawLamToken a)} {x : a}
  (huv : u subseq v) (hv : LamConDepth A v) (hFv : lamFunFinset v = {})
  (ht : LamEntDepth A u (.atom x)) : LamEntDepth A v (.atom x) := by
cases ht with
| atom hu hF hne hEnt =>
  have hsubA := lamAtomFinset_mono huv
  have hne' : lamAtomFinset v != {} := by
    intro hAv
    obtain <y, hy> := Finset.nonempty_of_ne_empty hne
    exact Finset.notMem_empty y (by have := hsubA hy; rwa [hAv] at this)
  exact LamEntDepth.atom hv hFv hne'
    (A.ent_trans ((LamConDepth_of_fun_empty A hFv).1 hv)
      ((LamConDepth_of_fun_empty A hF).1 hu)
      (fun y hy => A.ent_refl ((LamConDepth_of_fun_empty A hFv).1 hv) (hsubA hy))
      hEnt)

/-! ### Funion / filter helpers + weaken_insert + funion_of_entSet + insert_of_ent -/

private theorem funion_empty_right_lam (u : Finset (RawLamToken a)) :
  u U' ({} : Finset (RawLamToken a)) = u := by
  ext x; simp only [mem_funion, Finset.notMem_empty, or_false]

```

```

private theorem funion_insert_lam (u : Finset (RawLamToken a)) (a : RawLamToken a)
  (t : Finset (RawLamToken a)) : u U ' insert a t = insert a (u U ' t) := by
  ext x
  constructor
  * intro hx
    rcases mem_funion.mp hx with hu | hins
    * exact Finset.mem_insert_of_mem (mem_funion.mpr (Or.inl hu))
    * rcases Finset.mem_insert.mp hins with ha | ht
      * exact Finset.mem_insert.mpr (Or.inl ha)
      * exact Finset.mem_insert_of_mem (mem_funion.mpr (Or.inr ht))
  * intro hx
    rcases Finset.mem_insert.mp hx with ha | h
    * exact mem_funion.mpr (Or.inr (Finset.mem_insert.mpr (Or.inl ha)))
    * rcases mem_funion.mp h with hu | ht
      * exact mem_funion.mpr (Or.inl hu)
      * exact mem_funion.mpr (Or.inr (Finset.mem_insert.mpr (Or.inr ht)))

private theorem insert_filter_ne_lam
  {p : List (RawLamToken a) x List (RawLamToken a)}
  {t : List (List (RawLamToken a) x List (RawLamToken a))}
  (hp : p in t) : insert p (t.filter (* != p)) = t := by
  ext q
  constructor
  * intro hq
    rcases Finset.mem_insert.mp hq with rfl | hq
    * exact hp
    * exact (Finset.mem_filter.mp hq).1
  * intro hq
    if h : q = p then
      exact Finset.mem_insert.mpr (Or.inl h)
    else
      exact Finset.mem_insert.mpr (Or.inr (Finset.mem_filter.mpr <hq, h>))

private theorem filter_ne_subset_of_subset_insert_lam
  {p : List (RawLamToken a) x List (RawLamToken a)}
  {t w : Finset (List (RawLamToken a) x List (RawLamToken a))}
  (ht : t subseq insert p w) : t.filter (* != p) subseq w := by
  intro q hq
  have <hqt, hne> := Finset.mem_filter.mp hq
  have : q in insert p w := ht hqt
  rcases Finset.mem_insert.mp this with rfl | hqW
  * exact False.elim (hne rfl)
  * exact hqW

theorem lamAtomFinset_funion (u v : Finset (RawLamToken a)) :
  lamAtomFinset (u U ' v) = lamAtomFinset u U ' lamAtomFinset v := by
  ext x
  constructor
  * intro hx
    have : .atom x in u U ' v := mem_lamAtomFinset.1 hx
    rcases mem_funion.1 this with hu | hv
    * exact mem_funion.2 (Or.inl (mem_lamAtomFinset.2 hu))
    * exact mem_funion.2 (Or.inr (mem_lamAtomFinset.2 hv))
  * intro hx
    rcases mem_funion.1 hx with hu | hv
    * exact mem_lamAtomFinset.2 (mem_funion.2 (Or.inl (mem_lamAtomFinset.1 hu)))
    * exact mem_lamAtomFinset.2 (mem_funion.2 (Or.inr (mem_lamAtomFinset.1 hv)))

theorem lamFunFinset_funion (u v : Finset (RawLamToken a)) :
  lamFunFinset (u U ' v) = lamFunFinset u U ' lamFunFinset v := by
  ext p
  constructor

```

```

* intro hp
  have : .funTok p.1 p.2 in u U ' v := mem_lamFunFinset.1 hp
  rcases mem_funion.1 this with hu | hv
  * exact mem_funion.2 (Or.inl (mem_lamFunFinset.2 hu))
  * exact mem_funion.2 (Or.inr (mem_lamFunFinset.2 hv))
* intro hp
  rcases mem_funion.1 hp with hu | hv
  * exact mem_lamFunFinset.2 (mem_funion.2 (Or.inl (mem_lamFunFinset.1 hu)))
  * exact mem_lamFunFinset.2 (mem_funion.2 (Or.inr (mem_lamFunFinset.1 hv)))

/-- Weaken Ent across inserting another token entailed by the same set. -/
theorem LamEntDepth_weaken_insert {u : Finset (RawLamToken a)}
  {a x : RawLamToken a}
  (hu' : LamConDepth A (insert a u))
  (hx : LamEntDepth A u x) (ha : LamEntDepth A u a) :
  LamEntDepth A (insert a u) x := by
  haveI : DecidableEq (Finset (List (RawLamToken a) x List (RawLamToken a))) :=
    decidableEq_finset
  have huv : u subseqeq insert a u := Finset.subset_insert a u
  match x with
  | .bot => exact LamEntDepth.bot hu'
  | .atom z =>
    cases decidableEq_finset (lamFunFinset (insert a u)) {} with
    | isTrue hF =>
      exact LamEntDepth_weaken_atom A huv hu' hF hx
    | isFalse hF =>
      have hAt := ((LamConDepth_of_fun_nonempty A hF).1 hu').1
      cases hx with
      | atom _ _ hne _ =>
        obtain <y, hy> := Finset.nonempty_of_ne_empty hne
        exact False.elim (Finset.notMem_empty y (by
          have := lamAtomFinset_mono huv hy; rwa [hAt] at this))
      | .funTok xs ys =>
        cases decidableEq_finset (lamFunFinset (insert a u)) {} with
        | isFalse hF =>
          have hAv := ((LamConDepth_of_fun_nonempty A hF).1 hu').1
          exact LamEntDepth_weaken_fun A huv hu' hAv hx
        | isTrue hF =>
          cases ha with
          | bot _ =>
            cases hx with
            | funTok hu hA hConIn hConOut hs hWorld hInCompat hEntIn hEntOut =>
              have hAv : lamAtomFinset (insert (.bot : RawLamToken a) u) = {} := by
                rw [lamAtomFinset_insert_bot]; exact hA
              exact LamEntDepth.funTok hu' hAv hConIn hConOut
                (by rw [lamFunFinset_insert_bot]; exact hs)
                (by rw [lamFunFinset_insert_bot]; exact hWorld)
                hInCompat hEntIn hEntOut
          | atom _ _ hne _ =>
            cases hx with
            | funTok _ hA _ _ _ _ _ =>
              obtain <y, hy> := Finset.nonempty_of_ne_empty hne
              exact False.elim (Finset.notMem_empty y (by rwa [hA] at hy))
            | .funTok _ _ _ _ _ _ _ =>
              -- `a` refined to funTok, so fun (insert a u) != {}, contradicting hF
              simp only [lamFunFinset_insert_funTok] at hF
              exact False.elim (Finset.notMem_empty _ (hF |> Finset.mem_insert_self _ _))

private theorem finsetMaxDepth_insert_le (u : Finset (RawLamToken a)) (a : RawLamToken a) :
  finsetMaxDepth (insert a u) <= Nat.max (rawDepth a) (finsetMaxDepth u) := by
  refine foldr_max_le_of_forall ?_
  intro x hx

```



```

have hx' : x in insert a u := Finset.mem_def.2 hx
rcases Finset.mem_insert.mp hx' with rfl | hu
* exact Nat.le_max_left _ _
* exact Nat.le_trans (le_finsetMaxDepth_of_mem hu) (Nat.le_max_right _ _)

/-- Prop 2.3(ii) via insert-first on `v`. -/
theorem LamConDepth_funion_of_entSet (M : Nat)
  (ihIns : forall u t, Nat.max (finsetMaxDepth u) (rawDepth t) < M ->
    LamEntDepth A u t -> LamConDepth A (insert t u))
  {u v : Finset (RawLamToken a)}
  (hu : LamConDepth A u)
  (hEnt : forall t in v, LamEntDepth A u t)
  (hbound : forall t in v, Nat.max (finsetMaxDepth u) (rawDepth t) < M) :

```

Lean 4 source (lines 1601–2000)

```

LamConDepth A (u U ' v) := by
haveI : DecidableEq (Finset (List (RawLamToken a) x List (RawLamToken a))) :=
  decidableEq_finset
suffices forall (v u : Finset (RawLamToken a)),
  LamConDepth A u ->
    (forall t in v, LamEntDepth A u t) ->
    (forall t in v, Nat.max (finsetMaxDepth u) (rawDepth t) < M) ->
    LamConDepth A (u U ' v) from
  this v u hu hEnt hbound
intro v
induction v using Finset.induction_on with
| empty =>
  intro u hu _ _
  simp [funion_empty_right_lam] using hu
| insert a t ha ih =>
  intro u hu hEnt hbound
  have mem_a : a in insert a t := Finset.mem_insert_self a t
  have han : Nat.max (finsetMaxDepth u) (rawDepth a) < M := hbound a mem_a
  have hEnt_a : LamEntDepth A u a := hEnt a mem_a
  have hu_ins : LamConDepth A (insert a u) := ihIns u a han hEnt_a
  have hEnt_t : forall x in t, LamEntDepth A (insert a u) x := by
    intro x hx
    exact LamEntDepth_weaken_insert A hu_ins
    (hEnt x (Finset.mem_insert_of_mem hx)) hEnt_a
  have hbound_t : forall x in t, Nat.max (finsetMaxDepth (insert a u)) (rawDepth x) < M :=
    by
      intro x hx
      have hxu : Nat.max (finsetMaxDepth u) (rawDepth x) < M :=
        hbound x (Finset.mem_insert_of_mem hx)
      have hDi := finsetMaxDepth_insert_le u a
      have h1 : Nat.max (finsetMaxDepth (insert a u)) (rawDepth x) <=
        Nat.max (Nat.max (rawDepth a) (finsetMaxDepth u)) (rawDepth x) :=
        max_le_max_nat hDi le_rfl
      refine Nat.lt_of_le_of_lt h1 ?_
      have h2 : Nat.max (Nat.max (rawDepth a) (finsetMaxDepth u)) (rawDepth x) =
        Nat.max (rawDepth a) (Nat.max (finsetMaxDepth u) (rawDepth x)) := by
        ac_rfl
      rw [h2]
      exact Nat.max_lt.2 <Nat.lt_of_le_of_lt (Nat.le_max_right _ _) han, hxu>
  have hunion : LamConDepth A (insert a u U ' t) :=
    ih (insert a u) hu_ins hEnt_t hbound_t
  have heq : insert a u U ' t = u U ' insert a t := by
    ext x
    constructor
    * intro hx
      rcases mem_funion.1 hx with h | h
      * rcases Finset.mem_insert.mp h with rfl | hu'
        * exact mem_funion.2 (Or.inr (Finset.mem_insert_self _ _))

```

```

    * exact mem_funion.2 (Or.inl hu')
    * exact mem_funion.2 (Or.inr (Finset.mem_insert_of_mem h))
  * intro hx
  rcases mem_funion.1 hx with hu' | h
  * exact mem_funion.2 (Or.inl (Finset.mem_insert_of_mem hu'))
  * rcases Finset.mem_insert.mp h with rfl | ht'
    * exact mem_funion.2 (Or.inl (Finset.mem_insert_self _ _))
    * exact mem_funion.2 (Or.inr ht')
  rwa [heq] at hunion

private theorem funion_assoc_lam (a b c : Finset (RawLamToken a)) :
  (a U ' b) U ' c = a U ' (b U ' c) := by
  ext x
  constructor
  * intro hx
  rcases mem_funion.mp hx with h | h
  * rcases mem_funion.mp h with h | h
    * exact mem_funion.mpr (Or.inl h)
    * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inl h)))
    * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inr h)))
  * intro hx
  rcases mem_funion.mp hx with h | h
  * exact mem_funion.mpr (Or.inl (mem_funion.mpr (Or.inl h)))
  * rcases mem_funion.mp h with h | h
    * exact mem_funion.mpr (Or.inl (mem_funion.mpr (Or.inr h)))
    * exact mem_funion.mpr (Or.inr h)

private theorem funion_comm_lam (a b : Finset (RawLamToken a)) : a U ' b = b U ' a := by
  ext x
  constructor
  * intro hx
  rcases mem_funion.mp hx with h | h
  * exact mem_funion.mpr (Or.inr h)
  * exact mem_funion.mpr (Or.inl h)
  * intro hx
  rcases mem_funion.mp hx with h | h
  * exact mem_funion.mpr (Or.inr h)
  * exact mem_funion.mpr (Or.inl h)

private theorem finsetMaxDepth_funion_le (u v : Finset (RawLamToken a)) :
  finsetMaxDepth (u U ' v) <= Nat.max (finsetMaxDepth u) (finsetMaxDepth v) := by
  refine foldr_max_le_of_forall ?_
  intro t ht
  have ht' : t in u U ' v := Finset.mem_def.2 ht
  rcases mem_funion.1 ht' with hu' | hv
  * exact Nat.le_trans (le_finsetMaxDepth_of_mem hu) (Nat.le_max_left _ _)
  * exact Nat.le_trans (le_finsetMaxDepth_of_mem hv) (Nat.le_max_right _ _)

/-- Prop 2.1(iii): inserting an entailed token preserves depth-Con. -/
theorem LamConDepth_insert_of_ent {u : Finset (RawLamToken a)} {t : RawLamToken a}
  (ht : LamEntDepth A u t) : LamConDepth A (insert t u) := by
  haveI : DecidableEq (Finset (List (RawLamToken a) x List (RawLamToken a))) :=
    decidableEq_finset
  suffices forall M u t, Nat.max (finsetMaxDepth u) (rawDepth t) = M ->
    LamEntDepth A u t -> LamConDepth A (insert t u) from
    this _ _ rfl ht
  intro M
  induction M using Nat.strong_induction_on with
  | h M ih =>
    intro u t hM ht
    have ihIns : forall u' t', Nat.max (finsetMaxDepth u') (rawDepth t') < M ->
      LamEntDepth A u' t' -> LamConDepth A (insert t' u') :=
      fun u' t' hlt hEnt => ih _ hlt u' t' rfl hEnt

```

```

match t with
| .bot => exact LamConDepth_insert_bot A (LamEntDepth_con A ht)
| .atom x => exact LamConDepth_insert_atom A ht
| .funTok xs ys =>
  cases ht with
| @funTok u xs ys s hu hA hConIn hConOut hs hWorld hInCompat hEntIn hEntOut =>
  have hF' : lamFunFinset (insert (.funTok xs ys) u) != {} := by
    rw [lamFunFinset_insert_funTok]
    exact Finset.insert_ne_empty _ _
  refine (LamConDepth_of_fun_nonempty A hF').2 <?_, ?_, ?_>
  * rw [lamAtomFinset_insert_funTok]; exact hA
  * intro p hp
    rw [lamFunFinset_insert_funTok] at hp
    rcases Finset.mem_insert.mp hp with rfl | hp
    * exact <hConIn, hConOut>
    * exact ((LamConDepth_of_fun_nonempty A hWorld).1 hu).2.1 p hp
  * intro sel hsel hIn
    if hp : (xs, ys) in sel then
      let t' := sel.filter (* != (xs, ys))
      have ht_eq : insert (xs, ys) t' = sel := insert_filter_ne_lam hp
      have ht'sub : t' subseteq lamFunFinset u :=
        filter_ne_subset_of_subset_insert_lam (by
          intro q hq; have := hsel hq; rwa [lamFunFinset_insert_funTok] at this)
      have hin' : LamConDepth A (listToFinset xs U ' lamInputUnion t') := by
        rwa [ <- ht_eq, lamInputUnion_insert ] at hIn
      have hsub : s U ' t' subseteq lamFunFinset u :=
        funion_subset_iff.mpr <hs, ht'sub>
      have hEnt_in_s : forall z in lamInputUnion s,
        LamEntDepth A (listToFinset xs) z := by
        intro z hz
        rcases (mem_lamInputUnion).1 hz with <p, hp', hz'>
        exact hEntIn p hp' z hz'
      have hEnt_big : forall z in lamInputUnion s,
        LamEntDepth A (listToFinset xs U ' lamInputUnion t') z := by
        intro z hz
        have hzx := hEnt_in_s z hz
        have hsubxs : listToFinset xs subseteq listToFinset xs U ' lamInputUnion t' :=
          subset_funion_left _ _
        cases decidableEq_finset
          (lamFunFinset (listToFinset xs U ' lamInputUnion t')) {} with
        | isFalse hFn =>
          have hAv := ((LamConDepth_of_fun_nonempty A hFn).1 hin').1
          exact LamEntDepth_weaken_fun A hsubxs hin' hAv hzx
        | isTrue hFhin =>
          match z with
          | .bot => exact LamEntDepth.bot hin'
          | .atom x =>
            exact LamEntDepth_weaken_atom A hsubxs hin' hFhin hzx
          | .funTok xs1 ys1 =>
            cases decidableEq_finset (lamFunFinset (listToFinset xs)) {} with
            | isTrue hxsF =>
              have hNoFun : lamFunFinset (lamInputUnion s) = {} := hInCompat hxsF
              have hzmem : .funTok xs1 ys1 in lamInputUnion s := hz
              have : (xs1, ys1) in lamFunFinset (lamInputUnion s) :=
                mem_lamFunFinset.2 hzmem
              rw [hNoFun] at this
              exact False.elim (Finset.notMem_empty _ this)
            | isFalse hxsF =>
              have : lamFunFinset (listToFinset xs U ' lamInputUnion t') != {} := by
                intro h
                have : lamFunFinset (listToFinset xs) = {} :=
                  Finset.Subset.antisymm
                    (fun p hp => by

```

```

      have hp' : p in lamFunFinset
        (listToFinset xs U ' lamInputUnion t') := by
          rw [lamFunFinset_funion]
          exact mem_funion.2 (Or.inl hp)
      rw [h] at hp'; exact False.elim (Finset.notMem_empty p hp')
    (Finset.empty_subset _)
    exact hxsF this
    exact False.elim (this hFhin)
have hDt : rawDepth (.funTok xs ys) <= M := by
  rw [← hM]; exact Nat.le_max_right _ _
have hDu : finsetMaxDepth u <= M := by
  rw [← hM]; exact Nat.le_max_left _ _
have hxs_lt : finsetMaxDepth (listToFinset xs) < M :=
  Nat.lt_of_lt_of_le (finsetMaxDepth_list_lt_funTok xs ys) hDt
have hys_lt : finsetMaxDepth (listToFinset ys) < M :=
  Nat.lt_of_lt_of_le (finsetMaxDepth_list_lt_funTok_out xs ys) hDt
have bound_in : forall z in lamInputUnion s,
  Nat.max (finsetMaxDepth (listToFinset xs U ' lamInputUnion t'))
    (rawDepth z) < M := by
  intro z hz
have hDuBig : finsetMaxDepth (listToFinset xs U ' lamInputUnion t') < M := by
  have hle := finsetMaxDepth_funion_le (listToFinset xs) (lamInputUnion t')
  refine Nat.lt_of_le_of_lt hle (Nat.max_lt.2 <hxs_lt, ?_>)
  by_cases ht'e : t' = {}
  * rw [ht'e, lamInputUnion_empty]
    exact Nat.lt_of_le_of_lt (Nat.zero_le _) hxs_lt
  * obtain <q, hq> := Finset.nonempty_of_ne_empty ht'e
    exact Nat.lt_of_lt_of_le
      (finsetMaxDepth_inputUnion_lt ht'sub hWorld) hDu
have hz_lt : rawDepth z < M := by
  rcases (mem_lamInputUnion).1 hz with <p, hp', hz'>
  have htok : .funTok p.1 p.2 in u := mem_lamFunFinset.1 (hs hp')
  exact Nat.lt_of_lt_of_le
    (Nat.lt_of_lt_of_le
      (rawDepth_funTok_gt_left ((mem_listToFinset).1 hz'))
      (le_finsetMaxDepth_of_mem htok))
    hDu
  exact Nat.max_lt.2 <hDuBig, hz_lt>
have hbig : LamConDepth A
  ((listToFinset xs U ' lamInputUnion t') U ' lamInputUnion s) :=
  LamConDepth_funion_of_entSet A M ihIns hin' hEnt_big bound_in
have hin_st' : LamConDepth A (lamInputUnion s U ' lamInputUnion t') := by
  have heq :
    (listToFinset xs U ' lamInputUnion t') U ' lamInputUnion s =
    listToFinset xs U ' (lamInputUnion s U ' lamInputUnion t') := by
    rw [funion_assoc_lam, funion_comm_lam (lamInputUnion t')]
  have hbig' : LamConDepth A
    (listToFinset xs U ' (lamInputUnion s U ' lamInputUnion t')) := by
    rwa [heq] at hbig
  exact LamConDepth_mono A (subset_funion_right _ _) hbig'
have hin_union : LamConDepth A (lamInputUnion (s U ' t')) := by
  rwa [lamInputUnion_funion]
have hout_union : LamConDepth A
  (lamOutputUnion s U ' lamOutputUnion t') := by
  have hout : LamConDepth A (lamOutputUnion (s U ' t')) :=
    ((LamConDepth_of_fun_nonempty A hWorld).1 hu).2.2 (s U ' t') hsub hin_union
  rwa [lamOutputUnion_funion] at hout
have hEnt_out' : forall z in listToFinset ys,
  LamEntDepth A (lamOutputUnion s U ' lamOutputUnion t') z := by
  intro z hz
  have hzx := hEntOut z hz
  have hsub0 : lamOutputUnion s subsepeq
    lamOutputUnion s U ' lamOutputUnion t' := subset_funion_left _ _

```

```

cases decidableEq_finset
  (lamFunFinset (lamOutputUnion s U ' lamOutputUnion t')) {} with
| isFalse hFn =>
  have hAv := ((LamConDepth_of_fun_nonempty A hFn).1 hout_union).1
  exact LamEntDepth_weaken_fun A hsub0 hout_union hAv hzx
| isTrue hFo =>
  match z with
  | .bot => exact LamEntDepth.bot hout_union
  | .atom x =>
    exact LamEntDepth_weaken_atom A hsub0 hout_union hFo hzx
  | .funTok xs1 ys1 =>
    -- Ent out_s funTok needs fun out_s != {}, but out_s subsesteq atom-world
union
cases hzx with
| funTok _ _ _ _ hs0 hWo _ _ _ =>
  have hOsF : lamFunFinset (lamOutputUnion s) = {} :=
    Finset.Subset.antisymm
      (fun p hp => by
        have : p in lamFunFinset
          (lamOutputUnion s U ' lamOutputUnion t') := by
            rw [lamFunFinset_funion]
            exact mem_funion.2 (Or.inl hp)
        rw [hFo] at this
        exact False.elim (Finset.notMem_empty p this))
      (Finset.empty_subset _)
  exact False.elim (hWo hOsF)
have bound_out : forall z in listToFinset ys,
  Nat.max (finsetMaxDepth (lamOutputUnion s U ' lamOutputUnion t'))
    (rawDepth z) < M := by
intro z hz
have hDuOut : finsetMaxDepth
  (lamOutputUnion s U ' lamOutputUnion t') < M := by
have hle := finsetMaxDepth_funion_le (lamOutputUnion s) (lamOutputUnion t')
refine Nat.lt_of_le_of_lt hle (Nat.max_lt.2 <?_, ?_>)
* by_cases hs0 : s = {}
  * rw [hs0, lamOutputUnion_empty]
    exact Nat.lt_of_le_of_lt (Nat.zero_le _) hys_lt
  * exact Nat.lt_of_lt_of_le
    (finsetMaxDepth_outputUnion_lt hs hWorld) hDu
* by_cases ht'e : t' = {}
  * rw [ht'e, lamOutputUnion_empty]
    exact Nat.lt_of_le_of_lt (Nat.zero_le _) hys_lt
  * exact Nat.lt_of_lt_of_le
    (finsetMaxDepth_outputUnion_lt ht'sub hWorld) hDu
have hz_lt : rawDepth z < M :=
  Nat.lt_of_lt_of_le
    (rawDepth_funTok_gt_right ((mem_listToFinset).1 hz)) hDt
exact Nat.max_lt.2 <hDuOut, hz_lt>
have hfinal : LamConDepth A
  ((lamOutputUnion s U ' lamOutputUnion t') U ' listToFinset ys) :=
  LamConDepth_funion_of_entSet A M ihIns hout_union hEnt_out' bound_out
have hout_t : LamConDepth A
  (listToFinset ys U ' lamOutputUnion t') := by
  have heq :
    (lamOutputUnion s U ' lamOutputUnion t') U ' listToFinset ys =
      lamOutputUnion s U ' (listToFinset ys U ' lamOutputUnion t') := by
    rw [funion_assoc_lam, funion_comm_lam (lamOutputUnion t')]
  have hfinal' : LamConDepth A
    (lamOutputUnion s U ' (listToFinset ys U ' lamOutputUnion t')) := by
    rwa [heq] at hfinal
  exact LamConDepth_mono A (subset_funion_right _ _) hfinal'
show LamConDepth A (lamOutputUnion sel)
rwa [ <- ht_eq, lamOutputUnion_insert]

```

```

else
  have hsel' : sel subseq lamFunFinset u := by
    intro p hp'
    have := hsel hp'
    rw [lamFunFinset_insert_funTok] at this
    rcases Finset.mem_insert.mp this with rfl | h
    * exact False.elim (hp hp')
    * exact h
  exact ((LamConDepth_of_fun_nonempty A hWorld).1 hu).2.2 sel hsel' hIn

/-! ### Scott (8)-(11): `ent_trans` (Def 2.1) -/

/-- Def 2.1 `ent_trans`: cases on the RHS token (Scott's turnstile advice). -/
theorem LamEntDepth_trans {u v : Finset (RawLamToken a)} {c : RawLamToken a}
  (hv : LamConDepth A v) (hu : LamConDepth A u)
  (hEnts : forall t in u, LamEntDepth A v t) (ht : LamEntDepth A u c) :
  LamEntDepth A v c := by
  haveI : DecidableEq (Finset (List (RawLamToken a) x List (RawLamToken a))) :=
    decidableEq_finset
  suffices forall M u v c,
    Nat.max (finsetMaxDepth v) (Nat.max (finsetMaxDepth u) (rawDepth c)) = M ->
    LamConDepth A v -> LamConDepth A u ->
    (forall t in u, LamEntDepth A v t) -> LamEntDepth A u c -> LamEntDepth A v c from
    this _ u v c rfl hv hu hEnts ht
  intro M
  induction M using Nat.strong_induction_on with
  | h M ih =>
    intro u v c hM hv hu hEnts ht
    have hleV : finsetMaxDepth v <= M := by rw [← hM]; exact Nat.le_max_left _ _
    have hleU : finsetMaxDepth u <= M := by
      rw [← hM]; exact Nat.le_trans (Nat.le_max_left _ _) (Nat.le_max_right _ _)
    have hleC : rawDepth c <= M := by
      rw [← hM]; exact Nat.le_trans (Nat.le_max_right _ _) (Nat.le_max_right _ _)
    -- (8) bot
    match c with
    | .bot => exact LamEntDepth.bot hv
    -- (10) atom
    | .atom x =>
      cases ht with
      | @atom u x hu' hF hne hEnt =>
        obtain <y0, hy0> := Finset.nonempty_of_ne_empty hne
        have hEy0 := hEnts _ (mem_lamAtomFinset.1 hy0)
        have hFv : lamFunFinset v = {} := by
          cases hEy0 with
          | atom _ hFv' _ => exact hFv'
        have hneV : lamAtomFinset v != {} := by
          cases hEy0 with
          | atom _ _ hneV _ => exact hneV
        have hAtomEnt : forall y in lamAtomFinset u, A.Ent (lamAtomFinset v) y := by
          intro y hy
          have hEy := hEnts _ (mem_lamAtomFinset.1 hy)
          cases hEy with
          | atom _ _ hE => exact hE
        exact LamEntDepth.atom hv hFv hneV
        (A.ent_trans ((LamConDepth_of_fun_empty A hFv).1 hv)
          ((LamConDepth_of_fun_empty A hF).1 hu') hAtomEnt hEnt)
    -- (11) funTok
    | .funTok xs ys =>
      cases ht with
      | @funTok u xs ys s hu' hA hConIn hConOut hs hWorld hInCompat hEntIn hEntOut =>

```

```

obtain <p0, hp0> := Finset.nonempty_of_ne_empty hWorld
have hEp0 := hEnts _ (mem_lamFunFinset.1 hp0)
have hAv : lamAtomFinset v = {} := by
  cases hEp0 with
  | funTok _ hAv' _ _ _ _ => exact hAv'
have hFv : lamFunFinset v != {} := by
  cases hEp0 with
  | funTok _ _ _ _ hFv' _ _ => exact hFv'
have hWit : forall p in s, exists s_p subseq lamFunFinset v,
  LamConDepth A (listToFinset p.1) /\ LamConDepth A (listToFinset p.2) /\
  (forall r in s_p, forall z in listToFinset r.1, LamEntDepth A (listToFinset p
.1) z) /\
  (forall z in listToFinset p.2, LamEntDepth A (lamOutputUnion s_p) z) := by
  intro p hp
  have hEp := hEnts _ (mem_lamFunFinset.1 (hs hp))
  cases hEp with
  | @funTok _ _ _ s_p _ _ hCi hCo hs_p _ _ hEi hEo =>
    exact <s_p, hs_p, hCi, hCo, hEi, hEo>
-- Combined witness by Finset.induction_on (choice-free; avoid erase/card)
have hComb : forall (s' : Finset (List (RawLamToken a) x List (RawLamToken a))),
  s' subseq s ->
  exists S subseq lamFunFinset v,
  (forall r in S, forall z in listToFinset r.1, LamEntDepth A (listToFinset
xs) z) /\
  (forall z in lamOutputUnion s', LamEntDepth A (lamOutputUnion S) z) /\
  LamConDepth A (lamOutputUnion S) := by
  intro s'
  induction s' using Finset.induction_on with
  | empty =>
    intro _
    refine <({} : Finset (List (RawLamToken a) x List (RawLamToken a))),
      Finset.empty_subset _, ?_, ?_, ?_>
    * intro _ hr; exact False.elim (Finset.notMem_empty _ hr)
    * intro z hz
      rw [lamOutputUnion_empty] at hz
      exact False.elim (Finset.notMem_empty z hz)
    * rw [lamOutputUnion_empty]; exact LamConDepth_empty A
  | insert q sR hq ihS =>
    intro hs'
    have hsR : sR subseq s := fun p hp => hs' (Finset.mem_insert_of_mem hp)
    have hqS : q in s := hs' (Finset.mem_insert_self q sR)

```

Lean 4 source (lines 2001–2400)

```

obtain <S', hS'sub, hS'in, hS'out, hConS'> := ihS hsR
obtain <Sq, hSqsub, hConQIn, hConQOut, hSqin, hSqout> := hWit q hqS
have hSsub : S' U ' Sq subseq lamFunFinset v :=
  funion_subset_iff.mpr <hS'sub, hSqsub>
have hEntAll : forall t in lamInputUnion (S' U ' Sq),
  LamEntDepth A (listToFinset xs) t := by
  intro t ht
  rcases (mem_lamInputUnion).1 ht with <r, hr, ht'>
  rcases mem_funion.1 hr with hr | hr
  * exact hS'in r hr t ht'
  * refine ih _ ?_ (listToFinset q.1) (listToFinset xs) t rfl
    hConIn hConQIn (fun w hw => hEntIn q hqS w hw) (hSqin r hr t ht')
  have htok : .funTok q.1 q.2 in u := mem_lamFunFinset.1 (hs hqS)
  have htokR : .funTok r.1 r.2 in v :=
    mem_lamFunFinset.1 (hSsub (mem_funion.2 (Or.inr hr)))
  have hzt := rawDepth_funTok_gt_left (v := r.2) ((mem_listToFinset).1 ht')
  have hDq := (finsetMaxDepth_of_mem_funTok htok).1
  have hDxs := finsetMaxDepth_list_lt_funTok xs ys
  exact Nat.max_lt.2 <Nat.lt_of_lt_of_le hDxs hleC,
    Nat.max_lt.2 <Nat.lt_of_lt_of_le hDq hleU,

```

```

    Nat.lt_of_lt_of_le
      (Nat.lt_of_lt_of_le hzt (le_finsetMaxDepth_of_mem htokR))
      hleV>>
have hinU : LamConDepth A (lamInputUnion (S' U ' Sq)) :=
  LamConDepth_mono A (subset_funion_right _ _)
  (LamConDepth_funion_of_entSet A M
    (fun _ _ _ hE => LamConDepth_insert_of_ent A hE)
    hConIn hEntAll (fun t ht => by
      rcases (mem_lamInputUnion).1 ht with <r, hr, ht'>
      have hDxs := finsetMaxDepth_list_lt_funTok xs ys
      have htok : .funTok r.1 r.2 in v := mem_lamFunFinset.1 (hSsub hr)
      exact Nat.max_lt.2 <Nat.lt_of_lt_of_le hDxs hleC,
        Nat.lt_of_lt_of_le
          (Nat.lt_of_lt_of_le
            (rawDepth_funTok_gt_left (v := r.2) ((mem_listToFinset).1 ht'))
            (le_finsetMaxDepth_of_mem htok)) hleV>))
have hConU : LamConDepth A (lamOutputUnion (S' U ' Sq)) :=
  ((LamConDepth_of_fun_nonempty A hFv).1 hv).2.2 (S' U ' Sq) hSsub hinU
have hConU' : LamConDepth A (lamOutputUnion S' U ' lamOutputUnion Sq) := by
  rw [← lamOutputUnion_funion]; exact hConU
refine <S' U ' Sq, hSsub, ?_, ?_, hConU>
* intro r hr z hz
  rcases mem_funion.1 hr with hr | hr
  * exact hS'in r hr z hz
  * exact hEntAll _ ((mem_lamInputUnion).2 <r, mem_funion.2 (Or.inr hr), hz>)
* intro z hz
  rw [lamOutputUnion_funion]
  have hz'' : z in listToFinset q.2 U ' lamOutputUnion sR := by
    rwa [lamOutputUnion_insert] at hz
  rcases mem_funion.1 hz'' with hzQ | hzR
  * have hzx := hSqout z hzQ
    have hsub := subset_funion_right (lamOutputUnion S') (lamOutputUnion Sq)
    cases decidableEq_finset
      (lamFunFinset (lamOutputUnion S' U ' lamOutputUnion Sq)) {} with
    | isFalse hFn =>
      exact LamEntDepth_weaken_fun A hsub hConU'
      ((LamConDepth_of_fun_nonempty A hFn).1 hConU').1 hzx
    | isTrue hFo =>
      match z with
      | .bot => exact LamEntDepth.bot hConU'
      | .atom _ =>
        exact LamEntDepth_weaken_atom A hsub hConU' hFo hzx
      | .funTok _ _ =>
        cases hzx with
        | funTok _ _ _ _ hWo _ _ =>
          exact False.elim (hWo (Finset.Subset.antisymm
            (fun p hp => by
              have : p in lamFunFinset
                (lamOutputUnion S' U ' lamOutputUnion Sq) := by
                  rw [lamFunFinset_funion]
                  exact mem_funion.2 (Or.inr hp)
                  exact False.elim (Finset.notMem_empty p (hFo |> this)))
                (Finset.empty_subset _)))
          * have hzx := hS'out z hzR
            have hsub := subset_funion_left (lamOutputUnion S') (lamOutputUnion Sq)
            cases decidableEq_finset
              (lamFunFinset (lamOutputUnion S' U ' lamOutputUnion Sq)) {} with
            | isFalse hFn =>
              exact LamEntDepth_weaken_fun A hsub hConU'
              ((LamConDepth_of_fun_nonempty A hFn).1 hConU').1 hzx
            | isTrue hFo =>
              match z with
              | .bot => exact LamEntDepth.bot hConU'

```



```

| .atom _ =>
  exact LamEntDepth_weaken_atom A hsub hConU' hFo hzx
| .funTok _ _ =>
  cases hzx with
| funTok _ _ _ _ hWo _ _ _ =>
  exact False.elim (hWo (Finset.Subset.antisymm
    (fun p hp => by
      have : p in lamFunFinset
        (lamOutputUnion S' U ' lamOutputUnion Sq) := by
          rw [lamFunFinset_funion]
          exact mem_funion.2 (Or.inl hp)
          exact False.elim (Finset.notMem_empty p (hFo |> this)))
        (Finset.empty_subset _)))
obtain <S, hSsub, hSin, hSoutS, hConS> :=
  hComb s (Finset.Subset.refl _)
have hConOutS : LamConDepth A (lamOutputUnion s) :=
  ((LamConDepth_of_fun_nonempty A hWorld).1 hu').2.2 s hs
  (LamConDepth_mono A (subset_funion_right _ _))
  (LamConDepth_funion_of_entSet A M
    (fun _ _ hE => LamConDepth_insert_of_ent A hE)
    hConIn
    (fun t ht => by
      rcases (mem_lamInputUnion).1 ht with <p, hp, ht'>
      exact hEntIn p hp t ht')
    (fun t ht => by
      rcases (mem_lamInputUnion).1 ht with <p, hp, ht'>
      have hDxs := finsetMaxDepth_list_lt_funTok xs ys
      have htok : .funTok p.1 p.2 in u := mem_lamFunFinset.1 (hs hp)
      exact Nat.max_lt.2 <Nat.lt_of_lt_of_le hDxs hleC,
        Nat.lt_of_lt_of_le
          (Nat.lt_of_lt_of_le
            (rawDepth_funTok_gt_left ((mem_listToFinset).1 ht'))
            (le_finsetMaxDepth_of_mem htok)) hleU>)))
have hSout : forall z in listToFinset ys, LamEntDepth A (lamOutputUnion S) z := by
  intro z hz
  refine ih _ ?_ (lamOutputUnion s) (lamOutputUnion S) z rfl
  hConS hConOutS (fun t ht => hSoutS t ht) (hEntOut z hz)
  exact Nat.max_lt.2 <
    Nat.lt_of_lt_of_le (finsetMaxDepth_outputUnion_lt hSsub hFv) hleV,
    Nat.max_lt.2 <
      Nat.lt_of_lt_of_le (finsetMaxDepth_outputUnion_lt hs hWorld) hleU,
      Nat.lt_of_lt_of_le
        (rawDepth_funTok_gt_right ((mem_listToFinset).1 hz)) hleC>>
have hInCompat' : lamFunFinset (listToFinset xs) = {} ->
  lamFunFinset (lamInputUnion S) = {} := by
  intro hxsF
  ext p
  constructor
  * intro hp
    have hmem : .funTok p.1 p.2 in lamInputUnion S := mem_lamFunFinset.1 hp
    rcases (mem_lamInputUnion).1 hmem with <r, hr, hr'>
    cases hSin r hr _ hr' with
    | funTok _ _ _ _ hWx _ _ _ => exact False.elim (hWx hxsF)
  * intro hp; exact False.elim (Finset.notMem_empty _ hp)
exact LamEntDepth.funTok hv hAv hConIn hConOut hSsub hFv hInCompat' hSin hSout

/--! ### Phase B -- `lambdaSystem` (Def 2.1) on `LamConDepth` -/

/-- Well-formed tokens under depth Con (Scott (1)-(3) for the InfoSys instance).
Early staged `IsLamToken` / `LamToken` remain scaffolding only. -/
def IsLamWF : RawLamToken a -> Prop
| .bot | .atom _ => True
| .funTok xs ys => LamConDepth A (listToFinset xs) /\ LamConDepth A (listToFinset ys)

```

```

/-- Carrier of `lambdaSystem`. -/
def LamDToken : Type _ := { t : RawLamToken a // IsLamWF A t }

instance : DecidableEq (LamDToken A) := Subtype.instDecidableEq

def lamDBot : LamDToken A := <.bot, trivial>

def lamDForget (t : LamDToken A) : RawLamToken a := t.val

private def lamDForgetInsert :
  LamDToken A -> Finset (RawLamToken a) -> Finset (RawLamToken a) :=
  fun t => insert (lamDForget A t)

private instance : LeftCommutative (lamDForgetInsert (A := A)) :=
  <fun p q s => insert_comm' (lamDForget A p) (lamDForget A q) s>

def lamDForgetFinset (u : Finset (LamDToken A)) : Finset (RawLamToken a) :=
  Multiset.foldr (lamDForgetInsert (A := A)) {} u.1

private theorem mem_foldr_lamDForget (s : Multiset (LamDToken A)) (t : RawLamToken a) :
  t in Multiset.foldr (lamDForgetInsert (A := A)) {} s <-> exists p in s, p.val = t :=
  by
  refine Multiset.induction_on s ?_ ?_
  * constructor
  * intro ht; exact False.elim (Finset.notMem_empty t ht)
  * rintro <_, hp, _>; exact False.elim (by cases hp)
  * intro p rest ih
  simp only [Multiset.foldr_cons, lamDForgetInsert, Finset.mem_insert, ih, Multiset.mem_cons]
  constructor
  * rintro (rfl | <q, hq, hq'>)
  * exact <p, Or.inl rfl, rfl>
  * exact <q, Or.inr hq, hq'>
  * rintro <q, hq, hq'>
  rcases hq with rfl | hq
  * exact Or.inl hq'.symm
  * exact Or.inr <q, hq, hq'>

theorem mem_lamDForgetFinset {u : Finset (LamDToken A)} {t : RawLamToken a} :
  t in lamDForgetFinset A u <-> exists p in u, p.val = t := by
  simpa [lamDForgetFinset] using mem_foldr_lamDForget A u.1 t

theorem lamDForgetFinset_empty : lamDForgetFinset A ({ } : Finset (LamDToken A)) = { } :=
  rfl

theorem lamDForgetFinset_singleton (p : LamDToken A) :
  lamDForgetFinset A {p} = {p.val} := by
  ext t
  constructor
  * intro ht
  rcases (mem_lamDForgetFinset A).1 ht with <q, hq, hq'>
  have : q = p := Finset.mem_singleton.mp hq
  subst this; exact Finset.mem_singleton.mpr hq'.symm
  * intro ht
  exact (mem_lamDForgetFinset A).2
    <p, Finset.mem_singleton_self p, (Finset.mem_singleton.mp ht).symm>

theorem lamDForgetFinset_insert (u : Finset (LamDToken A)) (p : LamDToken A) :
  lamDForgetFinset A (insert p u) = insert p.val (lamDForgetFinset A u) := by
  ext t
  constructor
  * intro ht
  rcases (mem_lamDForgetFinset A).1 ht with <q, hq, rfl>

```

```

    rcases Finset.mem_insert.mp hq with rfl | hq
    * exact Finset.mem_insert_self _ _
    * exact Finset.mem_insert_of_mem ((mem_lamDForgetFinset A).2 <q, hq, rfl>)
* intro ht
    rcases Finset.mem_insert.mp ht with rfl | ht
    * exact (mem_lamDForgetFinset A).2 <p, Finset.mem_insert_self p u, rfl>
    * rcases (mem_lamDForgetFinset A).1 ht with <q, hq, rfl>
      exact (mem_lamDForgetFinset A).2 <q, Finset.mem_insert_of_mem hq, rfl>

theorem lamDForgetFinset_mono {u v : Finset (LamDToken A)} (h : v ⊆ u) :
  lamDForgetFinset A v ⊆ lamDForgetFinset A u := by
  intro t ht
  rcases (mem_lamDForgetFinset A).1 ht with <p, hp, rfl>
  exact (mem_lamDForgetFinset A).2 <p, h hp, rfl>

/-- Singleton funTok is Con when components are (Scott (7) on a singleton). -/
theorem LamConDepth_singleton_funTok (xs ys : List (RawLamToken a))
  (hxs : LamConDepth A (listToFinset xs)) (hys : LamConDepth A (listToFinset ys)) :
  LamConDepth A ({.funTok xs ys} : Finset (RawLamToken a)) := by
  haveI : DecidableEq (Finset (List (RawLamToken a) × List (RawLamToken a))) :=
    decidableEq_finset
  have hF : lamFunFinset ({.funTok xs ys} : Finset (RawLamToken a)) = {(xs, ys)} :=
    lamFunFinset_singleton_funTok xs ys
  have hA : lamAtomFinset ({.funTok xs ys} : Finset (RawLamToken a)) = {} :=
    lamAtomFinset_singleton_funTok xs ys
  have hne : lamFunFinset ({.funTok xs ys} : Finset (RawLamToken a)) ≠ {} := by
    rw [hF]; exact Finset.singleton_ne_empty _
  refine (LamConDepth_of_fun_nonempty A hne).2 <hA, ?_, ?_>
* intro p hp
  have : p = (xs, ys) := Finset.mem_singleton.mp (hF |> hp)
  subst this; exact <hxs, hys>
* intro s hs hIn
  rw [hF] at hs
  by_cases hmem : (xs, ys) ∈ s
  * have hsin : s = {(xs, ys)} := by
    ext p
    constructor
    * intro hp
      exact Finset.mem_singleton.mpr (Finset.mem_singleton.mp (hs hp))
    * intro hp
      have : p = (xs, ys) := Finset.mem_singleton.mp hp
      subst this; exact hmem
  subst hsin
  rw [lamOutputUnion_singleton]
  exact hys
* have hempty : s = {} := by
  ext p
  constructor
  * intro hp
    exact False.elim (hmem (by
      have : p = (xs, ys) := Finset.mem_singleton.mp (hs hp)
      subst this; exact hp))
    * intro hp; exact False.elim (Finset.notMem_empty p hp)
  subst hempty
  rw [lamOutputUnion_empty]
  exact LamConDepth_empty A

theorem IsLamWF_of_mem {u : Finset (RawLamToken a)} (hu : LamConDepth A u)
  {t : RawLamToken a} (ht : t ∈ u) : IsLamWF A t := by
  haveI : DecidableEq (Finset (List (RawLamToken a) × List (RawLamToken a))) :=
    decidableEq_finset
  match t with
  | .bot | .atom _ => trivial

```

```

| .funTok xs ys =>
  have hmem : (xs, ys) in lamFunFinset u := mem_lamFunFinset.2 ht
  by_cases hF : lamFunFinset u = {}
  * exact False.elim (Finset.notMem_empty _ (hF |> hmem))
  * exact ((LamConDepth_of_fun_nonempty A hF).1 hu).2.1 _ hmem

/-- Bundled consistency on `LamDToken` finsets. -/
def LamDCon (u : Finset (LamDToken A)) : Prop :=
  LamConDepth A (lamDForgetFinset A u)

/-- Bundled entailment on `LamDToken`. -/
def LamDEnt (u : Finset (LamDToken A)) (p : LamDToken A) : Prop :=
  LamEntDepth A (lamDForgetFinset A u) p.val

theorem LamDCon_empty : LamDCon A ({}) : Finset (LamDToken A)) := by
  simpa [LamDCon, lamDForgetFinset_empty] using LamConDepth_empty A

theorem LamDCon_subset {u v : Finset (LamDToken A)}
  (hu : LamDCon A u) (hv : v subteq u) : LamDCon A v :=
  LamConDepth_mono A (lamDForgetFinset_mono A hv) hu

theorem LamDCon_singleton (p : LamDToken A) : LamDCon A {p} := by
  rw [LamDCon, lamDForgetFinset_singleton]
  match p.val, p.property with
  | .bot, _ => exact LamConDepth_singleton_bot A
  | .atom x, _ => exact LamConDepth_singleton_atom A x
  | .funTok xs ys, <hxs, hys> => exact LamConDepth_singleton_funTok A xs ys hxs hys

theorem LamDEnt_bot {u : Finset (LamDToken A)} (hu : LamDCon A u) :
  LamDEnt A u (lamDBot A) :=
  LamEntDepth.bot hu

theorem LamDEnt_of_mem {u : Finset (LamDToken A)} {p : LamDToken A}
  (hu : LamDCon A u) (hp : p in u) : LamDEnt A u p :=
  LamEntDepth_of_mem A hu ((mem_lamDForgetFinset A).2 <p, hp, rfl>)

theorem LamDCon_insert_of_ent {u : Finset (LamDToken A)} {p : LamDToken A}
  (ht : LamDEnt A u p) : LamDCon A (insert p u) := by
  rw [LamDCon, lamDForgetFinset_insert]
  exact LamConDepth_insert_of_ent A ht

theorem LamDEnt_trans {u v : Finset (LamDToken A)} {c : LamDToken A}
  (hv : LamDCon A v) (hu : LamDCon A u)
  (hEnts : forall t in u, LamDEnt A v t) (ht : LamDEnt A u c) :
  LamDEnt A v c :=
  LamEntDepth_trans A hv hu
  (fun t ht' => by
    rcases (mem_lamDForgetFinset A).1 ht' with <p, hp, rfl>
    exact hEnts p hp)
  ht

/-- **Definition 2.1 packaging.** The fun -calculus information system `D`. -/
def lambdaSystem : InfoSys (LamDToken A) where
  bot := lamDBot A
  Con := {u | LamDCon A u}
  Ent := LamDEnt A
  con_subset := fun hu hv => LamDCon_subset A hu hv
  con_sing := LamDCon_singleton A
  ent_con := fun h => LamDCon_insert_of_ent A h
  ent_bot := fun hu => LamDEnt_bot A hu
  ent_refl := fun hu hp => LamDEnt_of_mem A hu hp
  ent_trans := fun hv hu hEnts hEntc => LamDEnt_trans A hv hu hEnts hEntc

```

```

/-! ### Phase C -- domain equation `D ~ A + (D -> D)` -/

/-- Lift a raw Con set to a finset of depth-WF tokens. -/
def listToLamDFinset :
  (xs : List (RawLamToken a)) ->
    (forall t in xs, IsLamWF A t) -> Finset (LamDToken A)
| [], _ => {}
| t :: ts, h =>
  insert <t, h t List.mem_cons_self>
    (listToLamDFinset ts fun w hw => h w (List.mem_cons_of_mem t hw))

theorem mem_listToLamDFinset {xs : List (RawLamToken a)}
  {h : forall t in xs, IsLamWF A t} {p : LamDToken A} :
  p in listToLamDFinset A xs h <-> p.val in xs := by
  induction xs with
  | nil =>
    constructor
    * intro hp; exact False.elim (Finset.notMem_empty _ hp)
    * intro hp; cases hp
  | cons t ts ih =>
    constructor
    * intro hp
      rcases Finset.mem_insert.mp hp with hp | hp
      * subst hp; exact List.mem_cons_self
      * exact List.mem_cons_of_mem _ (ih.1 hp)
    * intro hp
      rcases List.mem_cons.mp hp with rfl | hp
      * exact Finset.mem_insert_self _ _
      * exact Finset.mem_insert_of_mem (ih.2 hp)

theorem lamDForget_listToLamDFinset (xs : List (RawLamToken a))
  (h : forall t in xs, IsLamWF A t) :
  lamDForgetFinset A (listToLamDFinset A xs h) = listToFinset xs := by
  induction xs with
  | nil => rfl
  | cons t ts ih =>
    rw [listToLamDFinset, lamDForgetFinset_insert, listToFinset, ih]

theorem LamDCon_listToLamDFinset (xs : List (RawLamToken a))
  (hCon : LamConDepth A (listToFinset xs)) :
  LamDCon A (listToLamDFinset A xs fun _t ht =>
    IsLamWF_of_mem A hCon ((mem_listToFinset).2 ht)) := by
  rw [LamDCon, lamDForget_listToLamDFinset]
  exact hCon

/-- Unfold a fun -token into the sum-of-function-space carrier (Scott's matching). -/
def lamUnfold (t : LamDToken A) :
  SumToken a (FunToken (lambdaSystem A) (lambdaSystem A)) :=
  match t with
  | <.bot, _> => .bot
  | <.atom x, _> => .left x
  | <.funTok xs ys, <hxs, hys>> =>
    let u := listToLamDFinset A xs fun _w hw =>
      IsLamWF_of_mem A hxs ((mem_listToFinset).2 hw)
    let v := listToLamDFinset A ys fun _w hw =>
      IsLamWF_of_mem A hys ((mem_listToFinset).2 hw)
    .right <(u, v), <LamDCon_listToLamDFinset A xs hxs,
      LamDCon_listToLamDFinset A ys hys>>

theorem lamUnfold_bot : lamUnfold A (lamDBot A) = SumToken.bot := rfl

theorem lamUnfold_atom (x : a) :
  lamUnfold A <.atom x, trivial> = SumToken.left x := rfl

```

```

/-- Right-hand side of the domain equation: `A + (D -> D)`. -/
def lamRhs : InfoSys (SumToken a (FunToken (lambdaSystem A) (lambdaSystem A))) :=

```

Lean 4 source (lines 2401–2405)

```

    sumSystem A (functionSystem (lambdaSystem A) (lambdaSystem A))

theorem lamRhs_eq_sum_function :
  lamRhs A = sumSystem A (functionSystem (lambdaSystem A) (lambdaSystem A)) :=
  rfl

```

A.39 Scott1982/Factoid83.lean

612 lines.

Lean 4 source (lines 1–400)

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Product

/-!
# Factoid 8.3 -- universal domain via `V` (with top) and `U` (top removed)

**Scott 1982, S8.** Construct `V` with bottom `Delta` and top `?`, close under product
flavours `(X, Delta)` / `(Delta, Y)`, take all finite sets consistent, and define entailment
by Scott's clauses (3)–(6). Then form `U` by deleting `?` and retaining only those
finite sets that do not entail `?` (clauses (7)–(10)).

The retract/universality theorem (every countably based domain is a retract of `U`)
is stated by Scott without proof here; we package `vSystem` / `uSystem` and the
domain equation  $V \simeq V \times V$  on the product-shaped tokens.
-/

namespace Scott1982

open Scott1982.Constructive

namespace InfoSys

set_option linter.unusedSectionVars false

/-! ## Tokens of `V` (Scott (1)–(2)) -/

/-- Data objects of `V`: bottom `Delta`, top `?`, and product left/right copies. -/
inductive VToken where
  | bot : VToken
  | top : VToken
  | pairL : VToken -> VToken
  | pairR : VToken -> VToken
  deriving DecidableEq

/-- Bottom `Delta_V`. -/
def vBot : VToken := .bot

/-- Top / rogue `?`. -/
def vTop : VToken := .top

```

```

/-! ## Projections `{X | (X,Delta) in u}` and `{Y | (Delta,Y) in u}` -/

private def pairLInsert : VToken -> Finset VToken -> Finset VToken
| .pairL t => insert t
| .bot | .top | .pairR _ => id

private def pairRInsert : VToken -> Finset VToken -> Finset VToken
| .pairR t => insert t
| .bot | .top | .pairL _ => id

private instance :
  LeftCommutative (pairLInsert : VToken -> Finset VToken -> Finset VToken) :=
    <fun p q s => by cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl>

private instance :
  LeftCommutative (pairRInsert : VToken -> Finset VToken -> Finset VToken) :=
    <fun p q s => by cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl>

/-- Left projection: `{X | (X, Delta) in u}`. -/
def vPairLFinset (u : Finset VToken) : Finset VToken :=
  Multiset.foldr pairLInsert ({ } : Finset VToken) u.1

/-- Right projection: `{Y | (Delta, Y) in u}`. -/
def vPairRFinset (u : Finset VToken) : Finset VToken :=
  Multiset.foldr pairRInsert ({ } : Finset VToken) u.1

private theorem mem_foldr_pairL (s : Multiset VToken) (t : VToken) :
  t in Multiset.foldr pairLInsert ({ } : Finset VToken) s <->
    exists p in s, p = .pairL t := by
  refine Multiset.induction_on s ?_ ?_
  * constructor
  * intro ht; exact False.elim (Finset.notMem_empty t ht)
  * rintro <_, hp, _>; exact False.elim (by cases hp)
  * intro p rest ih
  cases p with
  | pairL a =>
    simp only [Multiset.foldr_cons, pairLInsert, Finset.mem_insert, ih, Multiset.mem_cons]
    constructor
    * rintro (ht | <q, hq, hq'>)
    * exact <.pairL a, Or.inl rfl, congrArg VToken.pairL ht.symm>
    * exact <q, Or.inr hq, hq'>
    * rintro <q, hq, hq'>
    rcases hq with rfl | hq
    * injection hq' with ht; exact Or.inl ht.symm
    * exact Or.inr <q, hq, hq'>
  | bot | top | pairR _ =>
    simp only [Multiset.foldr_cons, pairLInsert, id_eq, ih, Multiset.mem_cons]
    constructor
    * rintro <q, hq, hq'>; exact <q, Or.inr hq, hq'>
    * rintro <q, hq, hq'>
    rcases hq with rfl | hq
    * exact False.elim (nomatch hq')
    * exact <q, hq, hq'>

private theorem mem_foldr_pairR (s : Multiset VToken) (t : VToken) :
  t in Multiset.foldr pairRInsert ({ } : Finset VToken) s <->
    exists p in s, p = .pairR t := by
  refine Multiset.induction_on s ?_ ?_
  * constructor
  * intro ht; exact False.elim (Finset.notMem_empty t ht)
  * rintro <_, hp, _>; exact False.elim (by cases hp)
  * intro p rest ih
  cases p with

```

```

| pairR a =>
  simp only [Multiset.foldr_cons, pairRInsert, Finset.mem_insert, ih, Multiset.mem_cons]
  constructor
  * rintro (ht | <q, hq, hq'>)
    * exact <.pairR a, Or.inl rfl, congrArg VToken.pairR ht.symm>
    * exact <q, Or.inr hq, hq'>
  * rintro <q, hq, hq'>
    rcases hq with rfl | hq
    * injection hq' with ht; exact Or.inl ht.symm
    * exact Or.inr <q, hq, hq'>
| bot | top | pairL _ =>
  simp only [Multiset.foldr_cons, pairRInsert, id_eq, ih, Multiset.mem_cons]
  constructor
  * rintro <q, hq, hq'>; exact <q, Or.inr hq, hq'>
  * rintro <q, hq, hq'>
    rcases hq with rfl | hq
    * exact False.elim (nomatch hq')
    * exact <q, hq, hq'>

theorem mem_vPairLFinset {u : Finset VToken} {t : VToken} :
  t in vPairLFinset u <-> .pairL t in u := by
  simpa [vPairLFinset] using mem_foldr_pairL u.1 t

theorem mem_vPairRFinset {u : Finset VToken} {t : VToken} :
  t in vPairRFinset u <-> .pairR t in u := by
  simpa [vPairRFinset] using mem_foldr_pairR u.1 t

theorem vPairLFinset_empty : vPairLFinset ({} : Finset VToken) = {} := rfl
theorem vPairRFinset_empty : vPairRFinset ({} : Finset VToken) = {} := rfl

theorem vPairLFinset_mono {u v : Finset VToken} (h : u subseq v) :
  vPairLFinset u subseq vPairLFinset v := by
  intro t ht
  exact mem_vPairLFinset.2 (h (mem_vPairLFinset.1 ht))

theorem vPairRFinset_mono {u v : Finset VToken} (h : u subseq v) :
  vPairRFinset u subseq vPairRFinset v := by
  intro t ht
  exact mem_vPairRFinset.2 (h (mem_vPairRFinset.1 ht))

theorem vPairLFinset_insert_pairL (u : Finset VToken) (t : VToken) :
  vPairLFinset (insert (.pairL t) u) = insert t (vPairLFinset u) := by
  ext s; simp only [Finset.mem_insert, mem_vPairLFinset]
  constructor
  * rintro (h | h)
    * exact Or.inl (VToken.pairL.inj h)
    * exact Or.inr h
  * rintro (rfl | h)
    * exact Or.inl rfl
    * exact Or.inr h

theorem vPairRFinset_insert_pairR (u : Finset VToken) (t : VToken) :
  vPairRFinset (insert (.pairR t) u) = insert t (vPairRFinset u) := by
  ext s; simp only [Finset.mem_insert, mem_vPairRFinset]
  constructor
  * rintro (h | h)
    * exact Or.inl (VToken.pairR.inj h)
    * exact Or.inr h
  * rintro (rfl | h)
    * exact Or.inl rfl
    * exact Or.inr h

theorem vPairLFinset_insert_bot (u : Finset VToken) :

```



```

    vPairLFinset (insert (.bot : VToken) u) = vPairLFinset u := by
ext t; simp only [mem_vPairLFinset, Finset.mem_insert]
constructor
* rintro (h | h); exacts [False.elim (nomatch h), h]
* exact Or.inr

theorem vPairRFinset_insert_bot (u : Finset VToken) :
  vPairRFinset (insert (.bot : VToken) u) = vPairRFinset u := by
ext t; simp only [mem_vPairRFinset, Finset.mem_insert]
constructor
* rintro (h | h); exacts [False.elim (nomatch h), h]
* exact Or.inr

theorem vPairLFinset_insert_top (u : Finset VToken) :
  vPairLFinset (insert (.top : VToken) u) = vPairLFinset u := by
ext t; simp only [mem_vPairLFinset, Finset.mem_insert]
constructor
* rintro (h | h); exacts [False.elim (nomatch h), h]
* exact Or.inr

theorem vPairRFinset_insert_top (u : Finset VToken) :
  vPairRFinset (insert (.top : VToken) u) = vPairRFinset u := by
ext t; simp only [mem_vPairRFinset, Finset.mem_insert]
constructor
* rintro (h | h); exacts [False.elim (nomatch h), h]
* exact Or.inr

theorem vPairLFinset_insert_pairR (u : Finset VToken) (t : VToken) :
  vPairLFinset (insert (.pairR t) u) = vPairLFinset u := by
ext s; simp only [mem_vPairLFinset, Finset.mem_insert]
constructor
* rintro (h | h); exacts [False.elim (nomatch h), h]
* exact Or.inr

theorem vPairRFinset_insert_pairL (u : Finset VToken) (t : VToken) :
  vPairRFinset (insert (.pairL t) u) = vPairRFinset u := by
ext s; simp only [mem_vPairRFinset, Finset.mem_insert]
constructor
* rintro (h | h); exacts [False.elim (nomatch h), h]
* exact Or.inr

/-- ## Entailment for `V` (Scott (3)-(6)); all finite sets consistent -/

/-- Entailment on `V` (Scott (3)-(6)). -/
inductive VEnt : Finset VToken -> VToken -> Prop where
| bot {u : Finset VToken} : VEnt u .bot
| top_mem {u : Finset VToken} (h : .top in u) : VEnt u .top
| top_both {u : Finset VToken}
  (hL : VEnt (vPairLFinset u) .top) (hR : VEnt (vPairRFinset u) .top) :
  VEnt u .top
| pairL_top {u : Finset VToken} {x : VToken} (h : .top in u) : VEnt u (.pairL x)
| pairL_proj {u : Finset VToken} {x : VToken}
  (h : VEnt (vPairLFinset u) x) : VEnt u (.pairL x)
| pairR_top {u : Finset VToken} {y : VToken} (h : .top in u) : VEnt u (.pairR y)
| pairR_proj {u : Finset VToken} {y : VToken}
  (h : VEnt (vPairRFinset u) y) : VEnt u (.pairR y)

/-- Scott: all finite subsets of `D_V` are consistent. -/
def VCon (u : Finset VToken) : Prop := True

theorem VCon_any (u : Finset VToken) : VCon u := trivial

/-- If `u |- ?`, then `u` entails every token (rogue top). -/

```

```

theorem VEnt_of_ent_top {u : Finset VToken} (h : VEnt u .top) (c : VToken) :
  VEnt u c := by
  match c with
  | .bot => exact VEnt.bot
  | .top => exact h
  | .pairL x =>
    cases h with
    | top_mem hm => exact VEnt.pairL_top hm
    | top_both hL _ => exact VEnt.pairL_proj (VEnt_of_ent_top hL x)
  | .pairR y =>
    cases h with
    | top_mem hm => exact VEnt.pairR_top hm
    | top_both _ hR => exact VEnt.pairR_proj (VEnt_of_ent_top hR y)

theorem VEnt_mono {u v : Finset VToken} {c : VToken}
  (huv : u subseteq v) (h : VEnt u c) : VEnt v c := by
  induction h generalizing v with
  | bot => exact VEnt.bot
  | top_mem hm => exact VEnt.top_mem (huv hm)
  | top_both _ _ ihL ihR =>
    exact VEnt.top_both (ihL (vPairLFinset_mono huv)) (ihR (vPairRFinset_mono huv))
  | pairL_top hm => exact VEnt.pairL_top (huv hm)
  | pairL_proj _ ih => exact VEnt.pairL_proj (ih (vPairLFinset_mono huv))
  | pairR_top hm => exact VEnt.pairR_top (huv hm)
  | pairR_proj _ ih => exact VEnt.pairR_proj (ih (vPairRFinset_mono huv))

theorem VEnt_of_mem {u : Finset VToken} {c : VToken}
  (hc : c in u) : VEnt u c := by
  induction c generalizing u with
  | bot => exact VEnt.bot
  | top => exact VEnt.top_mem hc
  | pairL x ih =>
    exact VEnt.pairL_proj (ih (mem_vPairLFinset.2 hc))
  | pairR y ih =>
    exact VEnt.pairR_proj (ih (mem_vPairRFinset.2 hc))

/-- Empty set does not entail top (no derivation bottoms out). -/
theorem not_VEnt_empty_top : not VEnt ({} : Finset VToken) .top := by
  have aux : forall {u : Finset VToken} {c : VToken},
    VEnt u c -> c = .top -> u = {} -> False := by
    intro u c h
    induction h with
    | bot => intro hc; exact nomatch hc
    | top_mem hm =>
      intro _ hu; exact Finset.notMem_empty _ (hu |> hm)
    | top_both _ _ ihL _ =>
      intro _ hu
      exact ihL rfl (by rw [hu]; exact vPairLFinset_empty)
    | pairL_top => intro hc; exact nomatch hc
    | pairL_proj => intro hc; exact nomatch hc
    | pairR_top => intro hc; exact nomatch hc
    | pairR_proj => intro hc; exact nomatch hc
  exact fun h => aux h rfl rfl

/-- From `v |- (X, Delta)`, either `?` in `v` or the left projection entails `X`. -/
theorem VEnt_pairL_inv {v : Finset VToken} {x : VToken}
  (h : VEnt v (.pairL x)) : .top in v \ / VEnt (vPairLFinset v) x := by
  cases h with
  | pairL_top hm => exact Or.inl hm
  | pairL_proj h' => exact Or.inr h'

theorem VEnt_pairR_inv {v : Finset VToken} {y : VToken}
  (h : VEnt v (.pairR y)) : .top in v \ / VEnt (vPairRFinset v) y := by

```

```

cases h with
| pairR_top hm => exact Or.inl hm
| pairR_proj h' => exact Or.inr h'

/-- Push an `EntSet` through the left projection (or absorb via top). -/
theorem VEnt_pairLFinset_of_EntSet {u v : Finset VToken}
  (hEnts : forall y in u, VEnt v y) :
  .top in v \ / (forall y in vPairLFinset u, VEnt (vPairLFinset v) y) := by
by_cases htop : .top in v
* exact Or.inl htop
* refine Or.inr ?_
  intro y hy
  have hy' := hEnts _ (mem_vPairLFinset.1 hy)
  rcases VEnt_pairL_inv hy' with hm | hproj
  * exact False.elim (htop hm)
  * exact hproj

theorem VEnt_pairRFinset_of_EntSet {u v : Finset VToken}
  (hEnts : forall y in u, VEnt v y) :
  .top in v \ / (forall y in vPairRFinset u, VEnt (vPairRFinset v) y) := by
by_cases htop : .top in v
* exact Or.inl htop
* refine Or.inr ?_
  intro y hy
  have hy' := hEnts _ (mem_vPairRFinset.1 hy)
  rcases VEnt_pairR_inv hy' with hm | hproj
  * exact False.elim (htop hm)
  * exact hproj

theorem VEnt_trans {u v : Finset VToken} {c : VToken}
  (hEnts : forall y in u, VEnt v y) (h : VEnt u c) : VEnt v c := by
induction h generalizing v with
| bot => exact VEnt.bot
| top_mem hm => exact hEnts _ hm
| top_both _ _ ihL ihR =>
  rcases VEnt_pairLFinset_of_EntSet hEnts with hm | hL
  * exact VEnt_of_ent_top (VEnt.top_mem hm) .top
  * rcases VEnt_pairRFinset_of_EntSet hEnts with hm | hR
    * exact VEnt_of_ent_top (VEnt.top_mem hm) .top
    * exact VEnt.top_both (ihL hL) (ihR hR)
| pairL_top hm => exact VEnt_of_ent_top (hEnts _ hm) _
| pairL_proj _ ih =>
  rcases VEnt_pairLFinset_of_EntSet hEnts with hm | hL
  * exact VEnt.pairL_top hm
  * exact VEnt.pairL_proj (ih hL)
| pairR_top hm => exact VEnt_of_ent_top (hEnts _ hm) _
| pairR_proj _ ih =>
  rcases VEnt_pairRFinset_of_EntSet hEnts with hm | hR
  * exact VEnt.pairR_top hm
  * exact VEnt.pairR_proj (ih hR)

/-- **Scott (1)-(6).** The information system `V` (all finite sets consistent). -/
def vSystem : InfoSys VToken where
  bot := vBot
  Con := {u | VCon u}
  Ent := VEnt
  con_subset := fun _ _ => trivial
  con_sing := fun _ => trivial
  ent_con := fun _ => trivial
  ent_bot := fun _ => VEnt.bot
  ent_refl := fun _ hp => VEnt_of_mem hp
  ent_trans := fun _ _ hEnts h => VEnt_trans hEnts h

```

```

/-! ## Domain equation `V` ~ `V x V` (product-shaped tokens) -/

/-- Unfold product-shaped tokens into `V x V`. Top `?` has no single product-token
image (it generates the top *element*); we send it to `(?, Delta)` as a left witness. -/
def vUnfold (t : VToken) : ProdToken vSystem vSystem :=
  match t with
  | .bot => prodBot vSystem vSystem
  | .top => <(.top, .bot), Or.inr rfl>
  | .pairL x => <(x, .bot), Or.inr rfl>
  | .pairR y => <(.bot, y), Or.inl rfl>

theorem vUnfold_bot : vUnfold .bot = prodBot vSystem vSystem := rfl

theorem vUnfold_pairL (x : VToken) :
  vUnfold (.pairL x) = <(x, .bot), Or.inr rfl> := rfl

theorem vUnfold_pairR (y : VToken) :
  vUnfold (.pairR y) = <(.bot, y), Or.inl rfl> := rfl

/-- Right-hand side of Scott's equation `V` ~ `V x V`. -/
def vRhs : InfoSys (ProdToken vSystem vSystem) :=
  productSystem vSystem vSystem

theorem vRhs_eq_product : vRhs = productSystem vSystem vSystem := rfl

/-! ## The universal domain `U` (Scott (7)-(10)) -/

/-- Tokens of `U`: `D_V` without `?` (Scott (7)). -/
def IsUToken (t : VToken) : Prop := t != .top

instance : DecidablePred IsUToken := fun t =>
  if h : t = .top then isFalse fun ht => ht h
  else isTrue h

def UToken : Type := { t : VToken // IsUToken t }

instance : DecidableEq UToken := Subtype.instDecidableEq

/-- Bottom `Delta_U = Delta_V` (Scott (8)). -/

```

Lean 4 source (lines 401–612)

```

def uBot : UToken := <.bot, by decide>

def uForget (t : UToken) : VToken := t.val

private def uForgetInsert : UToken -> Finset VToken -> Finset VToken :=
  fun t => insert (uForget t)

private instance : LeftCommutative uForgetInsert :=
  <fun p q s => insert_comm' (uForget p) (uForget q) s>

def uForgetFinset (u : Finset UToken) : Finset VToken :=
  Multiset.foldr uForgetInsert {} u.1

private theorem mem_foldr_uForget (s : Multiset UToken) (t : VToken) :
  t in Multiset.foldr uForgetInsert {} s <-> exists p in s, p.val = t := by
  refine Multiset.induction_on s ?_ ?_
  * constructor
  * intro ht; exact False.elim (Finset.notMem_empty t ht)
  * rintro <_, hp, _>; exact False.elim (by cases hp)
  * intro p rest ih
  simp only [Multiset.foldr_cons, uForgetInsert, Finset.mem_insert, ih, Multiset.mem_cons]
  constructor

```

```

* rintro (rfl | <q, hq, hq'>)
* exact <p, Or.inl rfl, rfl>
* exact <q, Or.inr hq, hq'>
* rintro <q, hq, hq'>
rcases hq with rfl | hq
* exact Or.inl hq'.symm
* exact Or.inr <q, hq, hq'>

theorem mem_uForgetFinset {u : Finset UToken} {t : VToken} :
  t in uForgetFinset u <-> exists p in u, p.val = t := by
  simpa [uForgetFinset] using mem_foldr_uForget u.1 t

theorem uForgetFinset_empty : uForgetFinset ({ } : Finset UToken) = { } := rfl

theorem uForgetFinset_singleton (p : UToken) :
  uForgetFinset {p} = {p.val} := by
  ext t
  constructor
  * intro ht
  rcases (mem_uForgetFinset).1 ht with <q, hq, hq'>
  have : q = p := Finset.mem_singleton.mp hq
  subst this; exact Finset.mem_singleton.mpr hq'.symm
  * intro ht
  exact (mem_uForgetFinset).2
    <p, Finset.mem_singleton_self p, (Finset.mem_singleton.mp ht).symm>

theorem uForgetFinset_insert (u : Finset UToken) (p : UToken) :
  uForgetFinset (insert p u) = insert p.val (uForgetFinset u) := by
  ext t
  constructor
  * intro ht
  rcases (mem_uForgetFinset).1 ht with <q, hq, rfl>
  rcases Finset.mem_insert.mp hq with rfl | hq
  * exact Finset.mem_insert_self _ _
  * exact Finset.mem_insert_of_mem ((mem_uForgetFinset).2 <q, hq, rfl>)
  * intro ht
  rcases Finset.mem_insert.mp ht with rfl | ht
  * exact (mem_uForgetFinset).2 <p, Finset.mem_insert_self p u, rfl>
  * rcases (mem_uForgetFinset).1 ht with <q, hq, rfl>
  exact (mem_uForgetFinset).2 <q, Finset.mem_insert_of_mem hq, rfl>

theorem uForgetFinset_mono {u v : Finset UToken} (h : v ⊆ u) :
  uForgetFinset v ⊆ uForgetFinset u := by
  intro t ht
  rcases (mem_uForgetFinset).1 ht with <p, hp, rfl>
  exact (mem_uForgetFinset).2 <p, h hp, rfl>

theorem not_mem_uForget_top (u : Finset UToken) : .top ⊄ uForgetFinset u := by
  intro ht
  rcases (mem_uForgetFinset).1 ht with <p, _, hp>
  exact p.property hp

/-- Consistency for `U`: finite `u ⊆ D_U` with `u ⊆ V` (Scott (9)). -/
def UCon (u : Finset UToken) : Prop :=
  not VEnt (uForgetFinset u) .top

/-- Entailment for `U` (Scott (10)). -/
def UEnt (u : Finset UToken) (p : UToken) : Prop :=
  UCon u /\ VEnt (uForgetFinset u) p.val

theorem UCon_empty : UCon ({ } : Finset UToken) := by
  simpa [UCon, uForgetFinset_empty] using not_VEnt_empty_top

```

```

/-- Singleton of a non-top token does not entail top. -/
theorem not_VEnt_singleton_ne_top {t : VToken} (ht : t != .top) :
  not VEnt ({t} : Finset VToken) .top := by
  intro h
  match t, h with
  | .bot, .top_mem hm =>
    exact False.elim (nomatch Finset.mem_singleton.mp hm)
  | .bot, .top_both hL _ =>
    have hL' : vPairLFinset ({.bot} : Finset VToken) = {} := by
      ext x; constructor
      * intro hx; exact False.elim (nomatch mem_vPairLFinset.1 hx)
      * intro hx; exact False.elim (Finset.notMem_empty x hx)
    exact not_VEnt_empty_top (by rwa [hL'] at hL)
  | .top, _ => exact ht rfl
  | .pairL x, .top_mem hm =>
    exact False.elim (nomatch Finset.mem_singleton.mp hm)
  | .pairL x, .top_both _ hR =>
    have hR' : vPairRFinset ({.pairL x} : Finset VToken) = {} := by
      ext y; constructor
      * intro hy; exact False.elim (nomatch mem_vPairRFinset.1 hy)
      * intro hy; exact False.elim (Finset.notMem_empty y hy)
    exact not_VEnt_empty_top (by rwa [hR'] at hR)
  | .pairR y, .top_mem hm =>
    exact False.elim (nomatch Finset.mem_singleton.mp hm)
  | .pairR y, .top_both hL _ =>
    have hL' : vPairLFinset ({.pairR y} : Finset VToken) = {} := by
      ext x; constructor
      * intro hx; exact False.elim (nomatch mem_vPairLFinset.1 hx)
      * intro hx; exact False.elim (Finset.notMem_empty x hx)
    exact not_VEnt_empty_top (by rwa [hL'] at hL)

theorem UCon_singleton (p : UToken) : UCon {p} := by
  rw [UCon, uForgetFinset_singleton]
  exact not_VEnt_singleton_ne_top p.property

theorem UCon_subset {u v : Finset UToken}
  (hu : UCon u) (hv : v subseq u) : UCon v :=
  fun h => hu (VEnt_mono (uForgetFinset_mono hv) h)

/-- If inserting a non-top `y` creates a top-entailment and `u |- y`, then already `u |-
  ?`. -/
theorem VEnt_insert_top_imp {u : Finset VToken} {y : VToken}
  (hy : y != .top) (hEnt : VEnt u y) (hIns : VEnt (insert y u) .top) :
  VEnt u .top := by
  induction y generalizing u with
  | bot =>
    cases hIns with
    | top_mem hm =>
      rcases Finset.mem_insert.mp hm with h | h
      * exact False.elim (nomatch h)
      * exact VEnt.top_mem h
    | top_both hL hR =>
      rw [vPairLFinset_insert_bot] at hL
      rw [vPairRFinset_insert_bot] at hR
      exact VEnt.top_both hL hR
  | top => exact False.elim (hy rfl)
  | pairL x ih =>
    cases hIns with
    | top_mem hm =>
      rcases Finset.mem_insert.mp hm with h | h
      * exact False.elim (nomatch h)
      * exact VEnt.top_mem h
    | top_both hL hR =>

```

```

    rw [vPairLFinset_insert_pairL] at hL
    rw [vPairRFinset_insert_pairL] at hR
    rcases VEnt_pairL_inv hEnt with htop | hx
    * exact VEnt.top_mem htop
    * by_cases hxTop : x = .top
      * subst hxTop; exact VEnt.top_both hx hR
      * exact VEnt.top_both (ih hxTop hx hL) hR
| pairR z ih =>
  cases hIns with
  | top_mem hm =>
    rcases Finset.mem_insert.mp hm with h | h
    * exact False.elim (nomatch h)
    * exact VEnt.top_mem h
  | top_both hL hR =>
    rw [vPairLFinset_insert_pairR] at hL
    rw [vPairRFinset_insert_pairR] at hR
    rcases VEnt_pairR_inv hEnt with htop | hz
    * exact VEnt.top_mem htop
    * by_cases hzTop : z = .top
      * subst hzTop; exact VEnt.top_both hL hz
      * exact VEnt.top_both hL (ih hzTop hz hR)

/-- Adding an entailed non-top token cannot create a top-entailment. -/
theorem not_VEnt_insert_top_of_ent {u : Finset VToken} {y : VToken}
  (hy : y != .top) (hEnt : VEnt u y) (hNot : not VEnt u .top) :
  not VEnt (insert y u) .top :=
  fun hIns => hNot (VEnt_insert_top_imp hy hEnt hIns)

theorem UCon_insert_of_ent {u : Finset UToken} {p : UToken}
  (h : UEnt u p) : UCon (insert p u) := by
  rcases h with <hCon, hEnt>
  rw [UCon, uForgetFinset_insert]
  exact not_VEnt_insert_top_of_ent p.property hEnt hCon

theorem UEnt_bot {u : Finset UToken} (hu : UCon u) : UEnt u uBot :=
  <hu, VEnt.bot>

theorem UEnt_of_mem {u : Finset UToken} {p : UToken}
  (hu : UCon u) (hp : p in u) : UEnt u p :=
  <hu, VEnt_of_mem ((mem_uForgetFinset).2 <p, hp, rfl>)>

theorem UEnt_trans {u v : Finset UToken} {c : UToken}
  (hv : UCon v) (_hu : UCon u)
  (hEnts : forall y in u, UEnt v y) (h : UEnt u c) : UEnt v c :=
  <hv, VEnt_trans
    (fun y hy => by
      rcases (mem_uForgetFinset).1 hy with <p, hp, rfl>
      exact (hEnts p hp).2)
  h.2>

/-- **Scott (7)-(10).** The universal-domain information system `U`. -/
def uSystem : InfoSys UToken where
  bot := uBot
  Con := {u | UCon u}
  Ent := UEnt
  con_subset := fun hu hv => UCon_subset hu hv
  con_sing := UCon_singleton
  ent_con := fun h => UCon_insert_of_ent h
  ent_bot := fun hu => UEnt_bot hu
  ent_refl := fun hu hp => UEnt_of_mem hu hp
  ent_trans := fun hv hu hEnts h => UEnt_trans hv hu hEnts h

```

```

end InfoSys

```

end Scott1982

A.40 Scott1982/Factoid84.lean

317 lines.

Lean 4 source

```
/-
Copyright (c) 2026  Lars Warren Ericson.  All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github:  https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Proposition55
import Scott1982.Factoid34
import Scott1982.Factoid35

/-!
# Factoid 8.4 -- a domain of domains (Scott?s sketch)

**Scott 1982, S8 (final remark).** Scott is deliberately sketchy: finite information
about a domain is packaged as an approximable map  $r : P \rightarrow P$  on the powerset
domain  $P$  of the non-negative integers (with  $0$  as  $\Delta$  and  $1$  as a reserved
rogue like  $?$ ). The maps satisfying (1)-(5) below are exactly the elements of
the domain of domains; the token-level InfoSys is said to arise from
 $(P \rightarrow P)$  by a construction ?similar to?  $U$  from  $V$ , with details deferred.

We formalize  $P$ , state (1)-(5) on  $\text{ApproximableMap } P \ P$ , package
 $\text{DomainApprox}$ , record Scott?s consistency remark, and exhibit one inhabitant
(the ?flat? map that treats  $1$  as rogue). Functors-as-approximable-maps and
powerdomains are left external, as in the source.
-/

namespace Scott1982

open Scott1982.Constructive
open InfoSys
open ApproximableMap

namespace Factoid84

/-! ## The powerset domain  $P$  (Scott) -/

/-- Minimal entailment on  $\text{Nat}$  with  $0 = \Delta$ :  $u \mid- n$  iff  $n = 0$  or  $n$  in  $u$ . -/
def PEnt (u : Finset Nat) (n : Nat) : Prop :=
  n = 0  $\vee$  n in u

/-- **Powerset information system.** Tokens  $\text{Nat}$ , bot  $0$ , every finite set
consistent, minimal entailment. -/
def P : InfoSys Nat where
  bot := 0
  Con := Set.univ
  Ent := PEnt
  con_subset := fun _ _ => trivial
  con_sing := fun _ => trivial
  ent_con := fun _ => trivial
  ent_bot := fun _ => Or.inl rfl
  ent_refl := fun _ ha => Or.inr ha
  ent_trans := by
    intro u v c _ _ hEnts hc
```



```

    rcases hc with rfl | hc
    * exact Or.inl rfl
    * exact hEnts c hc

theorem P_AllConsistent : P.AllConsistent := fun _ => trivial

/-- Scott's shorthand  $\bar{n} = \{n\}$  as a finite set of tokens. -/
def bar (n : Nat) : Finset Nat := {n}

/-- Finite element  $\bar{n}$  (entailment closure of  $\{n\}$ ). -/
def barElement (n : Nat) : P.Element :=
  P.closure (bar n) trivial

theorem mem_barElement (n x : Nat) :
  x in (barElement n).carrier <=> x = 0 \/\ x = n := by
  constructor
  * intro hx
    have : PEnt (bar n) x := (P.mem_closure_iff (by trivial : bar n in P.Con)).1 hx
    rcases this with rfl | hx'
    * exact Or.inl rfl
    * exact Or.inr (Finset.mem_singleton.mp hx')
  * intro h
    rcases h with rfl | hn
    * exact (P.mem_closure_iff (by trivial : bar n in P.Con)).2 (Or.inl rfl)
    * rw [hn]
      exact (P.mem_closure_iff (by trivial : bar n in P.Con)).2
        (Or.inr (Finset.mem_singleton_self n))

/-- Bottom element  $\bar{\text{False}}$  of  $|P|$  (closure of  $\{\}$ ). -/
def botElement : P.Element :=
  P.closure {} trivial

theorem mem_botElement (x : Nat) : x in botElement.carrier <=> x = 0 := by
  constructor
  * intro hx
    have : PEnt {} x := (P.mem_closure_iff (by trivial : ({ } : Finset Nat) in P.Con)).1 hx
    rcases this with rfl | hx'
    * rfl
    * exact False.elim (Finset.notMem_empty _ hx')
  * rintro rfl
    exact (P.mem_closure_iff (by trivial : ({ } : Finset Nat) in P.Con)).2 (Or.inl rfl)

/-- Top element  $\bar{\text{top}}$  of  $|P|$  (all tokens). -/
def topElement : P.Element :=
  P.topElement P_AllConsistent

theorem mem_topElement (x : Nat) : x in topElement.carrier := trivial

theorem barElement_zero_eq_bot : barElement 0 = botElement := by
  apply le_antisymm
  * intro x hx
    rcases (mem_barElement 0 x).1 hx with rfl | rfl
    * exact (mem_botElement 0).2 rfl
    * exact (mem_botElement 0).2 rfl
  * intro x hx
    have : x = 0 := (mem_botElement x).1 hx
    subst this
    exact (mem_barElement 0 0).2 (Or.inl rfl)

/-! ## Scott's conditions (1)-(5) on  $r : P \rightarrow P$  -/

/-- (1)  $\bar{I}_P \text{ subseq } r$  (reflexivity of the induced entailment). -/
def ReflApprox (r : ApproximableMap P P) : Prop :=

```

```

forall {u v : Finset Nat}, (idMap P).rel u v -> r.rel u v

/-- (2) `r` comp `r` = `r` (transitivity / idempotence). -/
def IdemApprox (r : ApproximableMap P P) : Prop :=
  ApproximableMap.comp r r = r

/-- (3)  $\bar{0}$  subseq r(False) -- `0` plays the role of `Delta`. -/
def BotApprox (r : ApproximableMap P P) : Prop :=
  (barElement 0).carrier subseq (r.toElement botElement).carrier

/-- (4)  $\top$  = r( $\bar{1}$ ) -- `1` plays the role of the rogue `?`. -/
def RogueApprox (r : ApproximableMap P P) : Prop :=
  r.toElement (barElement 1) = topElement

/-- (5)  $\bar{1} ? r(\bar{0})$  -- `1` is excluded as inconsistent at the bottom. -/
def ConsistentApprox (r : ApproximableMap P P) : Prop :=
  not ((barElement 1).carrier subseq (r.toElement (barElement 0)).carrier)

/-- Scott (1)-(5): approximable maps that present entailment relations / domains. -/
structure IsDomainApprox (r : ApproximableMap P P) : Prop where
  refl : ReflApprox r
  idem : IdemApprox r
  bot : BotApprox r
  rogue : RogueApprox r
  consistent : ConsistentApprox r

/-- Elements of the domain of domains (Scott's assertion after (1)-(5)). -/
def DomainApprox : Type :=
  { r : ApproximableMap P P // IsDomainApprox r }

/-- Scott's consistency remark: for such an `r`, a finite `u` is ?consistent?
when  $\bar{1} ? r(\bar{u})$ . -/
def scottCon (r : ApproximableMap P P) (u : Finset Nat) : Prop :=
  not ((barElement 1).carrier subseq
    (r.toElement (P.closure u (by trivial))).carrier)

theorem scottCon_empty_iff_consistent (r : ApproximableMap P P) :
  scottCon r {} <=> ConsistentApprox r := by
  simp only [scottCon, ConsistentApprox, barElement_zero_eq_bot]
  rfl

/-! ## (3) is automatic for any approximable map -/

theorem BotApprox_any (r : ApproximableMap P P) : BotApprox r := by
  intro x hx
  have hx0 : x = 0 := by
    rcases (mem_barElement 0 x).1 hx with rfl | rfl <|> rfl
  subst hx0
  refine <{}, fun a ha => False.elim (Finset.notMem_empty a (Finset.mem_coe.1 ha)), ?_>
  have hempty := r.empty_rel
  have hbot : P.EntSet {} ({0} : Finset Nat) := by
    intro z hz
    rw [Finset.mem_singleton] at hz; subst hz
    exact Or.inl rfl
  exact r.mono hempty (P.entSet_empty {}) hbot trivial trivial

/-! ## Example: flat domain map (treat `1` as rogue) -/

/-- If `1` in `u`, accept any `v`; else require `v` subseq `u U {0}` and `1 notin v`. -/
def flatRel (u v : Finset Nat) : Prop :=
  (1 in u) \ / ((forall n in v, n = 0 \ / n in u) /\ 1 notin v)

theorem flatRel_of_mem_one {u v : Finset Nat} (h : 1 in u) : flatRel u v :=

```

```

Or.inl h

theorem flatRel_empty : flatRel {} {} :=
  Or.inr <fun _ hn => False.elim (Finset.notMem_empty _ hn), Finset.notMem_empty _>

def flatMap : ApproximableMap P P where
  rel := flatRel
  rel_dom := fun _ => trivial
  rel_cod := fun _ => trivial
  empty_rel := flatRel_empty
  union_right := by
    intro u v v' hv hv'
    rcases hv with h1 | <hv, hv1>
    * exact Or.inl h1
    * rcases hv' with h1 | <hv', hv1'>
    * exact Or.inl h1
    * refine Or.inr <?_, ?_>
    * intro n hn
    rcases mem_funion.1 hn with h | h
    * exact hv n h
    * exact hv' n h
    * intro h1
    rcases mem_funion.1 h1 with h | h
    * exact hv1 h
    * exact hv1' h
  mono := by
    intro u u' v v' hrel hEntu hEntv _ _
    rcases hrel with h1 | <hv, hv1>
    * -- 1 in u; show 1 in u' from EntSet
    have h := hEntu 1 h1
    rcases h with h0 | h1'
    * exact False.elim (by cases h0)
    * exact Or.inl h1'
    * refine Or.inr <?_, ?_>
    * intro n hn
    have hn' := hEntv n hn
    rcases hn' with rfl | hn'
    * exact Or.inl rfl
    * have hn'' := hv n hn'
    rcases hn'' with rfl | hn''
    * exact Or.inl rfl
    * have hu' := hEntu n hn''
    rcases hu' with rfl | hu'
    * exact Or.inl rfl
    * exact Or.inr hu'
    * intro h1v
    have h := hEntv 1 h1v
    rcases h with h0 | h1v'
    * exact False.elim (by cases h0)
    * exact hv1 h1v'

theorem flatMap_refl : ReflApprox flatMap := by
  intro u v <_, _, hEnt>
  by_cases h1 : 1 in u
  * exact Or.inl h1
  * refine Or.inr <?_, ?_>
    * intro n hn; exact hEnt n hn
    * intro h1v
    have h := hEnt 1 h1v
    rcases h with h0 | h
    * exact absurd h0 (by decide)
    * exact h1 h

```

```

theorem flatMap_toElement_of_mem_one {x : P.Element} (h1 : 1 in x.carrier) :
  flatMap.toElement x = topElement := by
  apply le_antisymm
  * intro _ _; exact mem_topElement _
  * intro n _
    refine <{1}, ?_, Or.inl (Finset.mem_singleton_self 1)>
    intro a ha
    have : a = 1 := Finset.mem_singleton.mp (Finset.mem_coe.1 ha)
    subst this; exact h1

theorem flatMap_rogue : RogueApprox flatMap :=
  flatMap_toElement_of_mem_one ((mem_barElement 1 1).2 (Or.inr rfl))

theorem flatMap_consistent : ConsistentApprox flatMap := by
  intro hsub
  have h1 : 1 in (barElement 1).carrier := (mem_barElement 1 1).2 (Or.inr rfl)
  have h1' : 1 in (flatMap.toElement (barElement 0)).carrier := hsub h1
  rcases (mem_toElement flatMap (barElement 0)).1 h1' with <u, hu, hrel>
  rcases hrel with h1u | <_, hv1>
  * have : 1 in (barElement 0).carrier := hu (Finset.mem_coe.2 h1u)
    rcases (mem_barElement 0 1).1 this with h | h
    * exact absurd h (by decide)
    * exact absurd h (by decide)
  * exact hv1 (Finset.mem_singleton_self 1)

theorem flatMap_idem : IdemApprox flatMap := by
  refine ApproximableMap.ext fun u v => ?_
  constructor
  * rintro <w, hf, hg>
    rcases hf with h1u | <hw, hw1>
    * exact Or.inl h1u
    * rcases hg with h1w | <hv, hv1>
      * exact False.elim (hw1 h1w)
      * refine Or.inr <?_, hv1>
        * intro n hn
          have hn' := hv n hn
          rcases hn' with rfl | hn'
          * exact Or.inl rfl
          * exact hw n hn'
  * intro h
    rcases h with h1u | <hv, hv1>
    * exact <insert 1 v, Or.inl h1u, Or.inl (Finset.mem_insert_self 1 v)>
    * exact <v, Or.inr <hv, hv1>, Or.inr <fun n hn => Or.inr hn, hv1>>

theorem flatMap_isDomain : IsDomainApprox flatMap where
  refl := flatMap_refl
  idem := flatMap_idem
  bot := BotApprox_any flatMap
  rogue := flatMap_rogue
  consistent := flatMap_consistent

/-- Witness that Scott?s (1)-(5) are inhabited. -/
def flatDomainApprox : DomainApprox :=
  <flatMap, flatMap_isDomain>

/-! ## Identity fails (4): `I_P` is not a domain map -/

theorem not_RogueApprox_idMap : not RogueApprox (idMap P) := by
  intro h
  have h2 : 2 in ((idMap P).toElement (barElement 1)).carrier := by
    rw [h]; exact mem_topElement 2
  rcases (mem_toElement (idMap P) (barElement 1)).1 h2 with <u, hu, hrel>
  have hEnt : P.EntSet u {2} := hrel.2.2

```

```

have h2u := hEnt 2 (Finset.mem_singleton_self 2)
rcases h2u with h0 | h2u
* exact absurd h0 (by decide)
* have : 2 in (barElement 1).carrier := hu (Finset.mem_coe.2 h2u)
  rcases (mem_barElement 1 2).1 this with h0 | h1'
  * exact absurd h0 (by decide)
  * exact absurd h1' (by decide)

end Factoid84

end Scott1982

```

A.41 Scott1982/DomainEquation.lean

33 lines.

Lean 4 source

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott1982
-/

import Scott1982.Factoid81
import Scott1982.Factoid82

/-!
# Domain equations -- Section 8

Thin re-export of the first two S8 constructions:

* Factoid 8.1 (`Factoid81.lean`): `treeSystem` solves  $T \approx A + (T \times T)$ .
* Factoid 8.2 (`Factoid82.lean`): `lambdaSystem` solves  $D \approx A + (D \rightarrow D)$ .

Factoids 8.3-8.4 (`Factoid83.lean`, `Factoid84.lean`) are independent modules
(imported from the library root `Scott1982.lean`); they do not depend on this file.
-/

namespace Scott1982

namespace InfoSys

-- Factoid 8.1: see Factoid81.lean (`treeSystem`, `treeUnfold`, `treeRhs`)
-- Factoid 8.2: see Factoid82.lean (`lambdaSystem`, `lamUnfold`, `lamRhs`)
-- Factoid 8.3-8.4: see Factoid83.lean / Factoid84.lean (not re-exported here)

end InfoSys

end Scott1982

```